教程资料



ELFBOARD

进阶篇之-ELF 2(RK3588)

开发板软件系统开发教程

ARM Cortex-A55 ARM Cortex-A76

Embedded Learning Fans Board

Instruction Manual

Rev. 1.0

2024/11/09

Forlinx Embedded Technology Co. Ltd. www.elfboard.com

前言

本手册为系统开发手册，主要讲解了系统开发过程中会涉及到的一些知识。学习完本手册后能对开发板接口的适配，简单驱动的开发有一定的了解，并且能够掌握一些 uboot 阶段的调试方法和 Buildroot 的开发流程。

本手册一共分为四部分：

- 第一部分 Linux 内核移植，讲解了 SDK 的目录结构及编译方法和开发板的适配；
- 第二部分 驱动开发，讲解了一些简单的字符驱动；
- 第三部分 Uboot 介绍，讲解了 Uboot 的目录结构，编译方法及常用的命令；
- 第四部分 Buildroot 开发，讲解了 Buildroot 的目录结构及开发方法。

本手册中一些符号及格式的相关说明：

表现形式	含义
⚠	注意或者是需要特别关注的信息，一定要仔细阅读
🕮	对测试章节做的相关说明
🗁	表示相关路径
灰底蓝色	指在命令行输入的命令，需要手动输入
灰底黑色字体	输入命令后的串口输出信息
灰底黑色加粗	串口输出信息中的关键信息
//	对输入指令或输出信息的解释内容
用户名@主机名	root@elf2-buildroot: 开发板串口登录账户信息 elf@elf2-buildroot：开发板远程登录账户信息 elf@ubuntu:~$：开发环境 Linux 账户信息 用户可通过该信息确定功能操作的环境

例：插入 TF 卡后，通过 ls 指令查看挂载目录。

```
root@elf2-buildroot:~# ls /mnt                    //列出/mnt 目录下的文件
external_sd  sdcard  storage  udisk  usb_storage
```

- root@elf2-buildroot：用户名为 root，主机名为 elf2-buildroot，表示在开发板上使用 root 用户进行操作。
- // ：对指令操作或打印信息的解释内容，不需要输入。
- ls /mnt：灰底蓝色字体，表示需要手动输入的相关命令。
- external_sd sdcard storage udisk usb_storage：灰底黑色字体为输入命令后的输出信息，加粗字体为关键信息，在此处表示 TF 卡的挂载目录。

目录

前言 - 1 -
目录 - 2 -
第一章 Linux 内核移植 - 5 -
1.1 内核简介 - 5 -
1.2 SDK 的目录结构和文件说明 - 6 -
1.2.1 SDK 工程目录介绍 - 7 -
1.2.2 内核目录介绍 - 8 -
1.3 SDK 编译 - 9 -
1.3.1 初次编译 - 9 -
1.3.2 编译后生成文件说明 - 11 -
1.3.3 单独编译内核 - 12 -
1.4 烧录流程介绍 - 15 -
1.4.1 OTG 烧录流程 - 15 -
1.4.2 TF 卡烧录系统 - 25 -
1.5 Makefile 介绍 - 27 -
1.6 Kconfig 介绍 - 33 -
1.6.1 Kconfig 作用 - 33 -
1.6.2 Kconfig 语法 - 33 -
1.7 初识设备树 - 39 -
1.7.1 设备树的起源 - 39 -
1.7.2 设备树组成和结构及 dts、dtb、dtsi - 40 -
1.7.3 设备组织架构 - 40 -
1.7.4 设备树 Makefile 修改 - 45 -
1.7.5 开发板使用的设备树 - 46 -
1.8 IOMUX 介绍 - 46 -
1.9 git 管理源码 - 48 -
1.9.1 git 是什么？ - 48 -
1.9.2 为什么要用 git？ - 49 -
1.9.3 git 安装和初次使用 - 49 -
1.9.4 本地仓库管理 - 53 -
1.10 开发板适配 - 62 -
1.10.1 LED - 62 -
1.10.2 PWM - 67 -
1.10.3 UART - 70 -
1.10.4 SPI - 73 -
1.10.5 GMAC - 76 -
1.10.6 Sound - 80 -
1.10.7 MIPI-CSI - 87 -
1.10.8 SDMMC - 95 -
1.10.9 SARADC - 98 -
1.10.10 USB 控制器 - 101 -
1.10.11 MIPI-DSI - 113 -

1.10.12 HDMI ..... - 121 -
1.10.13 DP ..... - 127 -
1.10.14 RTC ..... - 127 -
1.10.15 PCIe ..... - 129 -
1.10.16 Wifi&蓝牙 ..... - 132 -
第二章 驱动开发 ..... - 136 -
2.1 驱动基本介绍 ..... - 136 -
2.1.1 Linux 设备驱动的分类 ..... - 136 -
2.1.2 Linux 设备驱动与整个软硬件系统的关系 ..... - 137 -
2.2 Linux 驱动模块 ..... - 137 -
2.2.1 helloworld 驱动 ..... - 137 -
2.2.2 带参数的驱动模块 ..... - 140 -
2.3 字符驱动 ..... - 142 -
2.3.1 字符驱动描述 ..... - 142 -
2.3.2 字符驱动框架描述 ..... - 143 -
2.3.3 测试示例 ..... - 143 -
2.3.4 自动创建节点 ..... - 151 -
2.4 内核空间与用户空间的数据拷贝 ..... - 156 -
2.4.1 数据拷贝介绍 ..... - 156 -
2.4.2 获取内核空间数据 ..... - 157 -
2.4.3 获取用户空间数据 ..... - 162 -
2.5 Pinctrl 和 GPIO 子系统 ..... - 167 -
2.5.1 Pinctrl 子系统 ..... - 167 -
2.5.2 GPIO 子系统 ..... - 171 -
2.5.3 ioctl 介绍 ..... - 171 -
2.5.4 LED 驱动 ..... - 172 -
2.6 Linux 系统中的中断 ..... - 181 -
2.6.1 Linux 中断介绍 ..... - 181 -
2.6.2 按键中断驱动 ..... - 182 -
2.7 Platform 总线 ..... - 189 -
2.7.1 platform 总线简介 ..... - 189 -
2.7.2 platform 总线驱动简单示例 ..... - 189 -
2.7.3 platform 驱动控制 LED ..... - 192 -
第三章 Uboot 介绍 ..... - 201 -
3.1 Uboot 简介 ..... - 201 -
3.1.1 BootLoder ..... - 201 -
3.1.2 U-Boot 版本 ..... - 201 -
3.2 Uboot 目录结构介绍 ..... - 202 -
3.3 U-Boot 编译 ..... - 204 -
3.3.1 SDK 目录下编译 ..... - 204 -
3.3.2 编译后生成的文件 ..... - 204 -
3.4 uboot 常用命令 ..... - 205 -
3.4.1 查看命令 ..... - 205 -
3.4.2 环境变量命令 ..... - 207 -

3.4.3 常用操作命令 ..... - 209 -
3.5 uboot 启动流程分析 ..... - 212 -
第四章 Buildroot ..... - 215 -
4.1 Buildroot 简介 ..... - 215 -
4.2 目录结构介绍 ..... - 215 -
4.3 Buildroot 编译 ..... - 216 -
4.3.1 编译系统 ..... - 216 -
4.3.2 单独编译 package ..... - 217 -
4.4 Buildroot 定制 ..... - 220 -
4.4.1 修改编译选项 ..... - 220 -
4.4.2 添加软件包 ..... - 221 -
4.5 文件系统常用指令 ..... - 225 -
4.5.1 系统管理与设置常用命令 ..... - 225 -
4.5.2 磁盘管理与维护常用命令 ..... - 226 -
声明 ..... - 230 -
更多帮助 ..... - 231 -

第一章 Linux 内核移植

1.1 内核简介

学到本章节，大家应该对 Linux 操作系统都有了一定的了解，但可能还不知道我们拿到手的内核源码都经历了什么。

linux 有一个庞大的开源社区，每个人都可以向开源社区提交代码。由于 linux 内核十分庞大所以每个模块都有对应的维护人员。维护人员的工作就是审核人们提交的代码是否正确，如果没有问题，就会合并到主分支上。这样就会使 linux 内核不断完善和更新。

接下来就是芯片原厂例如恩智浦，开发人员会不定期在 linux 官网下载稳定版本的内核，将其适配到各个芯片上，芯片原厂也会对内核进行维护形成自己的维护版本。

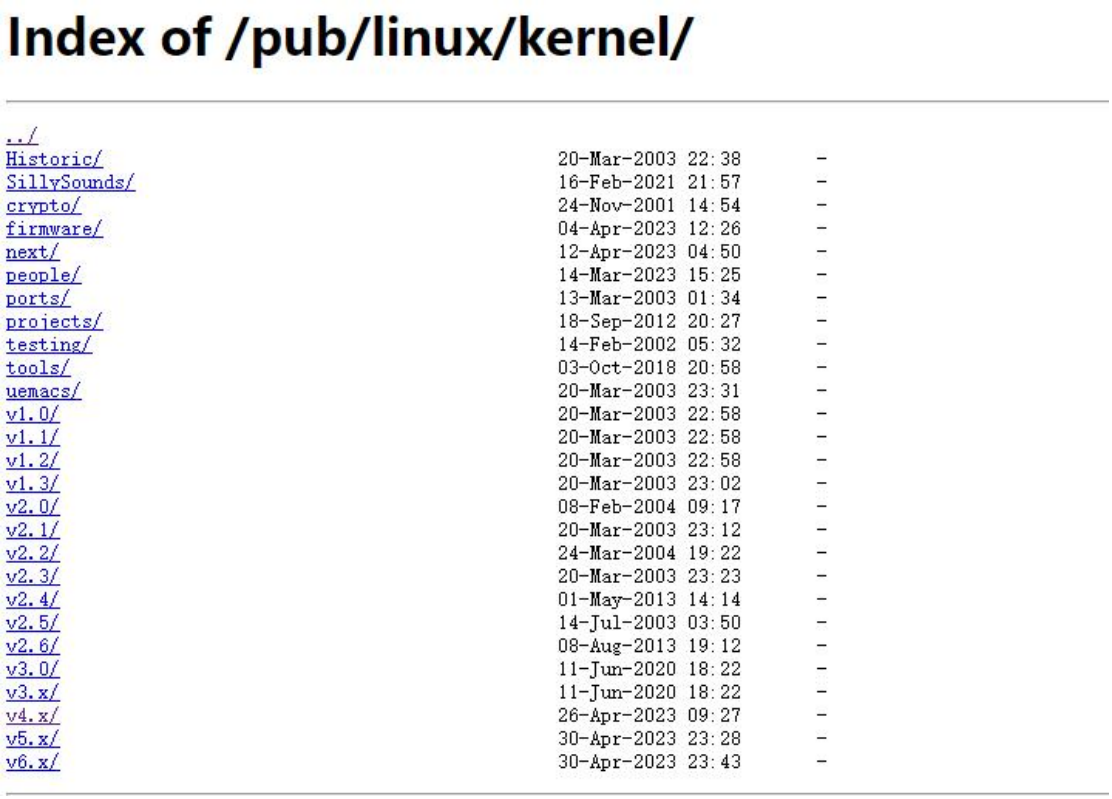
最后是板卡厂商，会到芯片原厂获取已经适配好的稳定版本的内核，在其基础上进行各个外设接口的适配，然后给用户提供移植好的板级开发包。板卡厂商也会对移植好的内核版本进行维护，例如一些 BUG 修复或者物料替换。

接下来讲一下获取这三种源码的方法：

1.获取 linux 官网源码

地址:https://mirrors.edge.kernel.org/pub/linux/kernel/

如果不是在芯片原厂工作一般不会到 linux 官网下载源码，但我们有必要了解这种获取源码的方法。因为 linux 官网对一些 bug 或者驱动更新的比较及时，在遇到一些内核问题时就可以到 linux 官网的源码中查一下是不是有相关的提交记录。

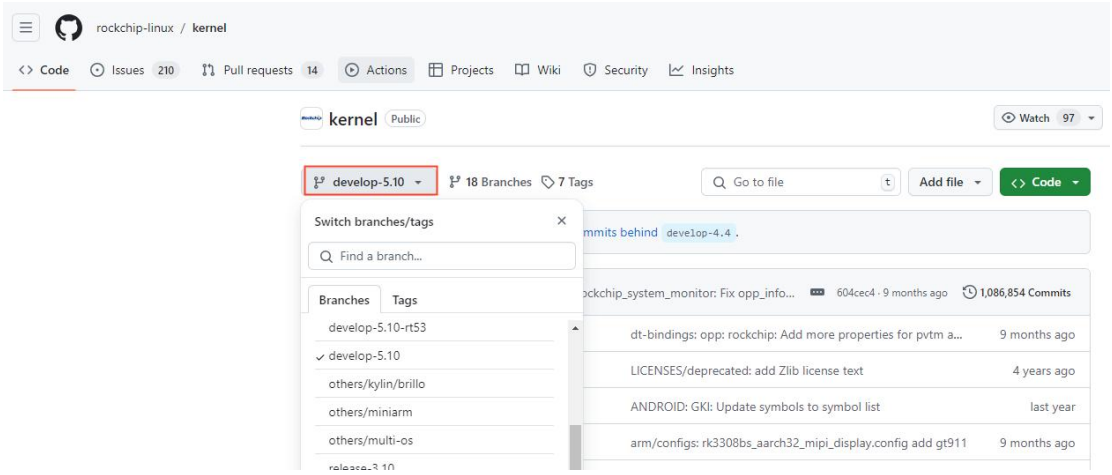


2.获取芯片原厂的 linux 源码

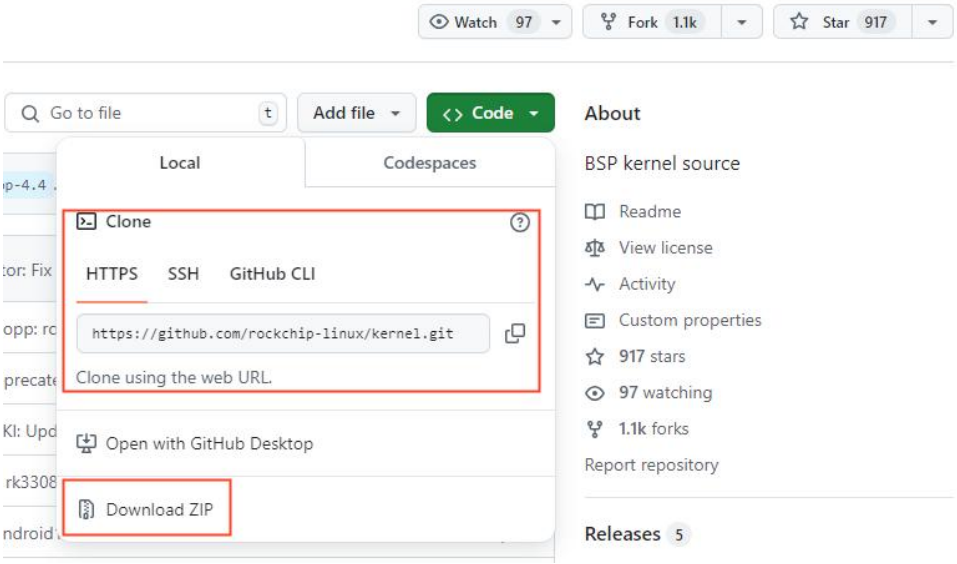
每个芯片原厂都有自己的 git 仓库，他们会把已经开发完成的内核源码放到 git 仓库中，供用户查看和下载。下面以 rockchip 的仓库为例进行讲解。

地址: https://github.com/rockchip-linux/kernel

1）切换到对应的版本，因为开发板是在 linux5.10 的上进行的适配，所以讲解均以 5.10 版本内核为例。



2）可以选择使用 git 克隆，也可以选择下载 ZIP 压缩包。



3.获取 ELF2 开发板源码

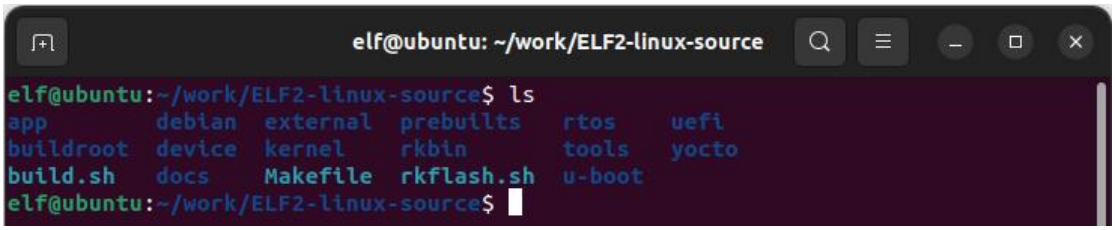
ELF2 的 SDK 源码在资料包当中的路径如下：ELF 2 开发板资料包\02-Linux 源代码\ELF2-linux-source.tar.bz2.0*

1.2 SDK 的目录结构和文件说明

将 ELF 2 开发板资料包\02-Linux 源代码\ELF2-linux-source.tar.bz2.0*SDK 源码压缩包复制到 Ubuntu 开发环境的/home/elf/work/目录下，解压缩：

```
elf@ubuntu:~/work$ cat ELF2-linux-source.tar.bz2.0* | tar vxj
```

解压完成之后使用 ls 命令查看 SDK 源码目录：

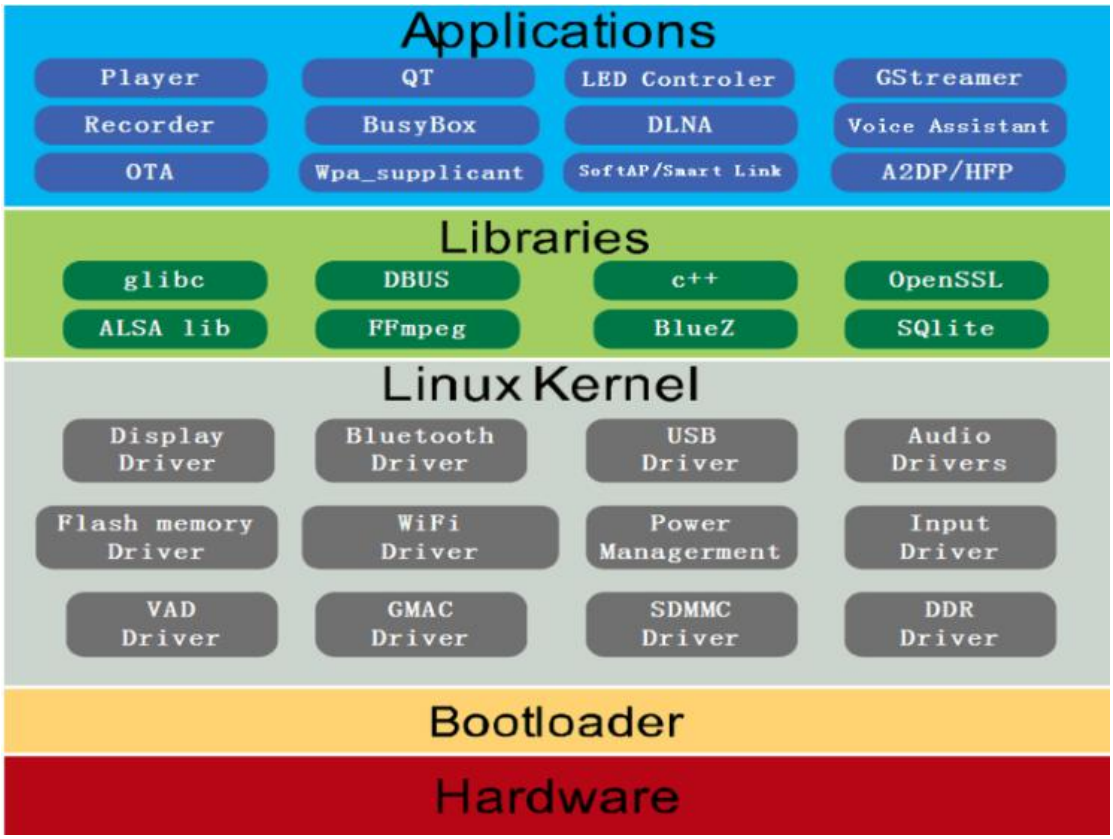


1.2.1 SDK 工程目录介绍

SDK 源码解压后会有多个目录，各个目录及主要的文件说明如下：

目录	目录说明
app	存放上层应用 app，主要是 qcamera/qfm/qplayer/settings 等一些应用程序。
buildroot	基于 buildroot (2021.11) 开发的根文件系统。
debian	基于 debian 10 开发的根文件系统，支持部分芯片。
device/rockchip/elf2	存放 elf2 配置和 Parameter文件，以及一些编译与打包固件的脚本和预备文件。
docs	存放芯片模块开发指导文档、平台支持列表、芯片平台相关文档、Linux 开发指南等。
IMAGE	存放每次生成编译时间、XML、补丁和固件目录。
external	存放第三方相关仓库,包括音频、视频、网络、recovery 等。
kernel	存放 kernel 5.10 开发的代码。
prebuilts	存放交叉编译工具链。
rkbin	存放 Rockchip 相关的 Binary 和工具。
rockdev	存放编译输出固件。
tools	存放 Linux 和 Windows 操作系统环境下常用工具。
u-boot	存放基于 v2017.09 版本进行开发的 uboot 代码。
yocto	基于 yocto gatesgarth 3.2 开发的根文件系统，支持部分芯片。

SDK 的软件框图如下：



从下至上分为 Bootloader、Linux Kernel、Libraries、Applications 四个层次。各层次内容如下：

- Bootloader 层主要提供底层系统支持包，如 Bootloader、U-Boot、ATF 相关支持。
- Kernel 层主要提供 Linux Kernel 的标准实现，Linux 也是一个开放的操作系统。Rockchip 平台的 Linux 核心为标准的 Linux4.4/4.19/5.10 内核，提供安全性，内存管理，进程管理，网络协议栈等基础支持；主要是通过 Linux 内核管理设备硬件资源，如 CPU 调度、缓存、内存、I/O 等。
- Libraries 层对应一般嵌入式系统，相当于中间件层次。包含了各种系统基础库，及第三方开源程序库支持，对应用层提供 API 接口，系统定制者和应用开发者可以基于 Libraries 层的 API 开发新的应用。
- Applications 层主要是实现具体的产品功能及交互逻辑，需要一些系统基础库及第三方程序库支持，开发者可以开发实现自己的应用程序，提供系统各种能力给到最终用户。

1.2.2 内核目录介绍

从上面的介绍中我们可以了解到，内核其实就是 SDK 源码中的一部分，接下来咱们来了解一下内核源码中一些比较常用的文件夹。

目录	目录说明
arch/	可支持的不同 CPU 架构下的核心代码。例如 arm 就是 arm 架构相关的代码，arm 目录下包括很多处理器平台，也包括了启动代码 boot、架构相关配置文件 configs、内核相关文件 kernel、内存管理 mm 和库 lib 等
block/	块设备相关通用函数
crypto/	常见的加密算法相关代码
Documentation/	说明文档，对每个目录和模块有详细说明
drivers/	设备驱动程序，其中每一个目录都代表一种设备驱动
fs/	可支持的文件系统相关代码
include/	通用的头文件

init/	内核初始化核心代码
ipc/	内核进程间通信相关代码
kernel/	内核核心代码，目录下实现了多数 linux 系统的内核函数
lib/	内核共用的函数库
mm/	内存管理相关代码
net/	网络相关代码
sample/	示例代码
scripts/	用于内核配置的脚本文件，用于实现内核配置的图形界面
security/	安全性相关，支持安全操作系统相关代码。包括 SELinux、Apparmor、Smack 和 TOMOYO Linux 安全模块。
sound	音频相关代码
tools/	常用工具代码
usr/	内核启动相关代码
virt/	内核虚拟化相关代码

1.3 SDK 编译

⚠**注意：**

初次解压 SDK 源码后，需要先对源码进行整体编译。

整体编译过后，可根据实际情况再进行单独编译。

1.3.1 初次编译

SDK 包中默认提供了一个编译脚本 build.sh，执行此脚本会自动设置编译所需的环境变量，完成自动编译。

在 SDK 目录下输入./build.sh -h 能够看到脚本的具体使用方法：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh -h
Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]          choose chip
defconfig[:<config>]              choose defconfig
  *_defconfig                         switch to specified defconfig
      available defconfigs:
      elf2_sdkbuild_defconfig
  olddefconfig                        resolve any unresolved symbols in .config
```

defconfig 选项为选择对应的板级配置文件，从上边的打印信息可以看出默认只有 elf2_sdkbuild_defconfig 这一个配置选项，所以这就是 elf2 开发板的配置文件，不需要额外选择。

```
kernel[:dry-run]                  build kernel
```

kernel 选项为单独编译内核源码，例如：./build.sh kernel

```
uboot[:dry-run]                   build u-boot
```

uboot 选项为单独编译 u-boot 源码，例如：./build.sh uboot

```
modules[:dry-run]                 build kernel modules
```

modules 选项为单独编译 modules 模块，例如：./build.sh modules

buildroot	build buildroot rootfs

buildroot 选项为单独编译 buildroot 文件系统，例如：./build.sh buildroot

all	build images

all 选项为编译所有镜像，包括 u-boot、kernel、rootfs、recovery，例如：./build.sh all

firmware	pack and check firmwares

firmware 选项为打包镜像，执行此指令会将生成的镜像打包到 rockdev 目录下，例如：./build.sh firmaware

cleanall	cleanup

cleanall 为清除指令，执行此指令会将前边编译生成的镜像及中间文件清除掉。例如：./build.sh cleanall

首先我们需要对 SDK 进行全编译，因为编译 buildroot 耗时比较长，所以默认不会编译 Buildroot，如果想要编译 Buildroot 查看下边 Buildroot 编译章节。

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh chip
Log colors: message notice warning error fatal

Log saved at /home/elf/work/ELF2-linux-source/output/sessions/2024-11-06_19-13-45
Pick a chip:

1. elf2
2. rk3588
Which would you like? [1]: 1
```

选择以后会出现两个选项：1. elf2_desktop_defconfig 和 2. elf2_sdkbuild_defconfig，咱们这里选择 2，编译 buildroot 系统。

```
Switching to chip: elf2
Pick a defconfig:

1. elf2_desktop_defconfig
2. elf2_sdkbuild_defconfig
Which would you like? [1]: 2
```

当执行./build.sh chip 脚本，需要选择编译目标，这里我们选择 1.elf2，然后再选择 2，此时会使用 elf2 的 elf2_sdkbuild_defconfig 文件生成对应的.config 文件。

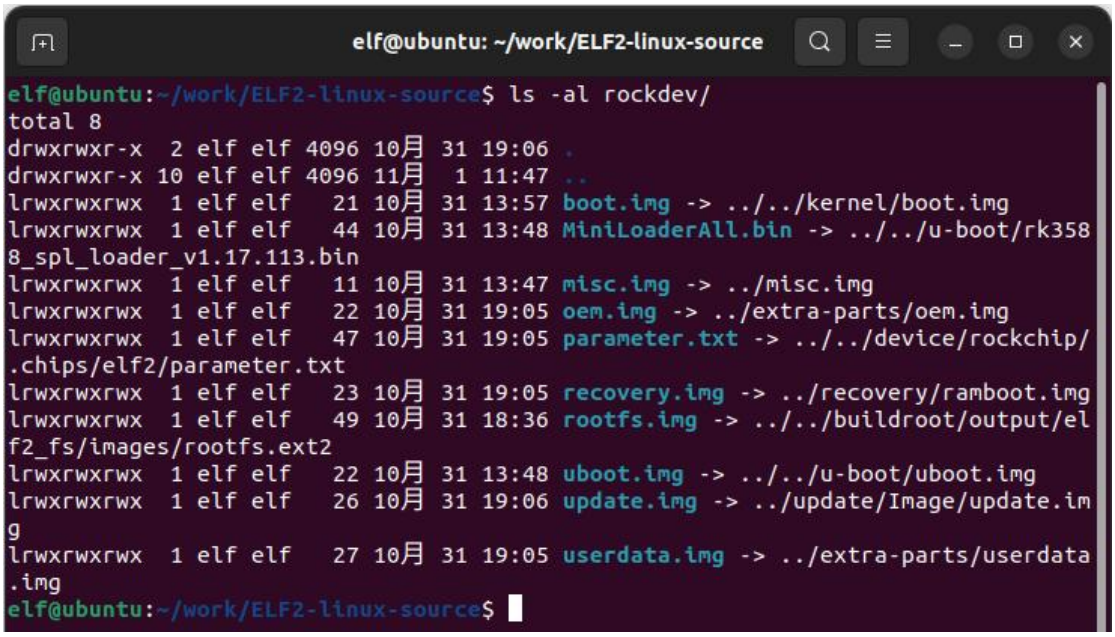
```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh
```

当执行./build.sh 脚本，后边不跟任何选项时，默认执行的就是./build.sh all。编译完成后打印如下：

```
Package: linux-headers-5.10-arm64
Source: linux-rockchip (5.10)
Version: 5.10-rockchip
Architecture: aarch64
Section: kernel
```

```
Priority: optional
Multi-Arch: foreign
Maintainer: Tao Huang <huangtao@rock-chips.com>
Homepage: https://www.kernel.org/
Description: Kbuild and headers for Rockchip Linux 5.10 arm64 configuration
Packing linux-headers-5.10-arm64_aarch64.deb...
Running mk-kernel.sh - linux-headers-aarch64 succeeded.
Running mk-kernel.sh - linux-headers succeeded.
Running 99-all.sh - build_all succeeded.
```

全编译完成后，在 rockdev 目录下会生成对应的镜像文件。



1.3.2 编译后生成文件说明

镜像	镜像说明
boot.img	内核部分镜像，包含 kernel 编译出的 Image 镜像和内核 DTB 镜像。
MiniLoaderAll.bin	设备初始引导程序(Bootloader)，它负责加载 uboot 程序，是设备上电时执行的第一段代码。
misc.img	存储了一些系统参数和标记，比如重启模式标志（Recovery、Bootloader 模式等）。这个文件很少直接使用，通常在系统更新或者设备切换启动模式时用到。
oem.img	给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
parameter.txt	描述了系统各个分区的布局和大小，告诉烧录工具如何将镜像写入到设备的存储器中。这个文件对分区的划分和镜像烧录位置非常重要。
recovery.img	包含 Recovery 模式下的系统映像。Recovery 模式通常用于执行恢复、更新系统、清除数据等操作。它是独立于主系统的一个小型操作环境。
rootfs.img	根文件系统，包括操作系统的主要组件、库和二进制文件。这个文件系统在设备启动后会被挂载为根目录。

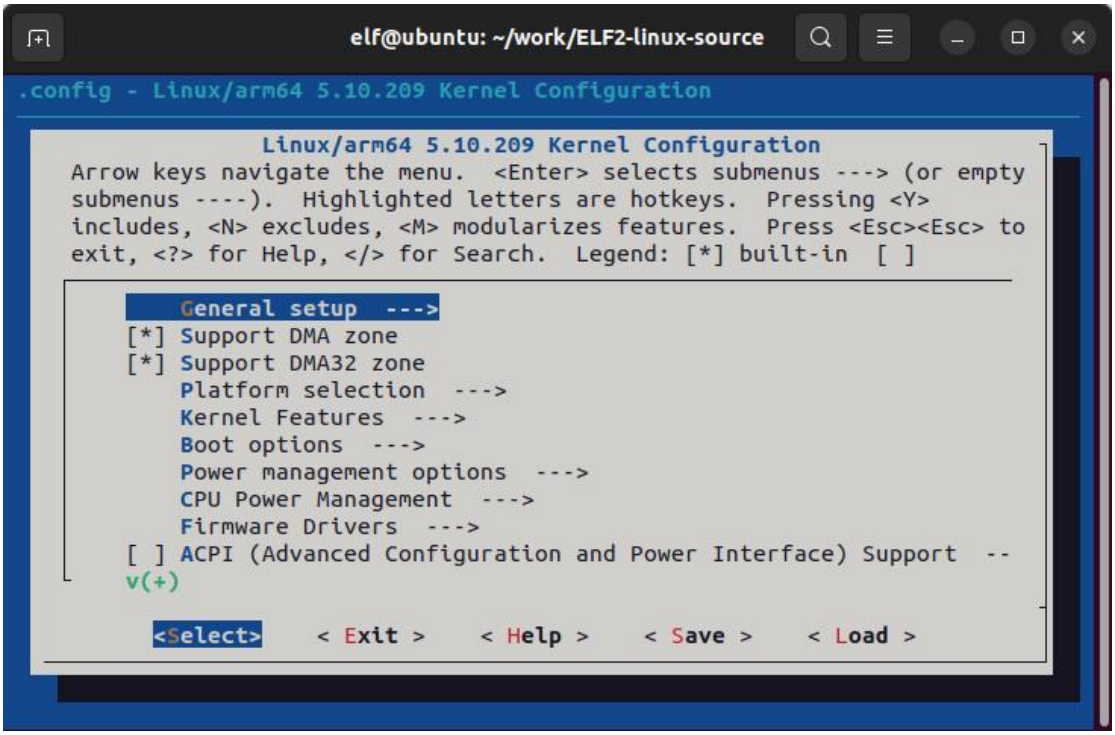
uboot.img	uboot 引导程序，它是 MiniLoaderAll.bin 运行完成后的第二阶段引导程序，uboot 负责初始化硬件设备并加载内核。
update.img	是所有其他镜像和文件的组合打包，用于通过烧录工具一次性将所有文件烧录到设备中。这是设备固件的完整镜像文件，用于系统更新或初次烧录。
userdata.img	包含用户数据分区的初始映像，通常是空的或者包含一些默认数据。当系统运行时，用户数据会存储在这个分区中，包括应用数据、用户配置等。

1.3.3 单独编译内核

1.更改内核配置

往往我们需要根据自己的需求进行内核配置，接下来我们看一下如何进行内核配置的更改。更改内核配置就是根据自己的需要，将内核中的驱动、软件包、组件等功能模块进行裁剪或者添加。可使用./build.sh kconfig 命令生成图形界面进行修改。

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kconfig
```



此图形界面所呈现的配置选项的菜单列表来源于 Kconfig 文件(下面章节会讲到)，这些配置选择，就是根据.config 读取出来的，我们也可以通过配置选项进行配置：

1）Y（将该功能编译进内核）

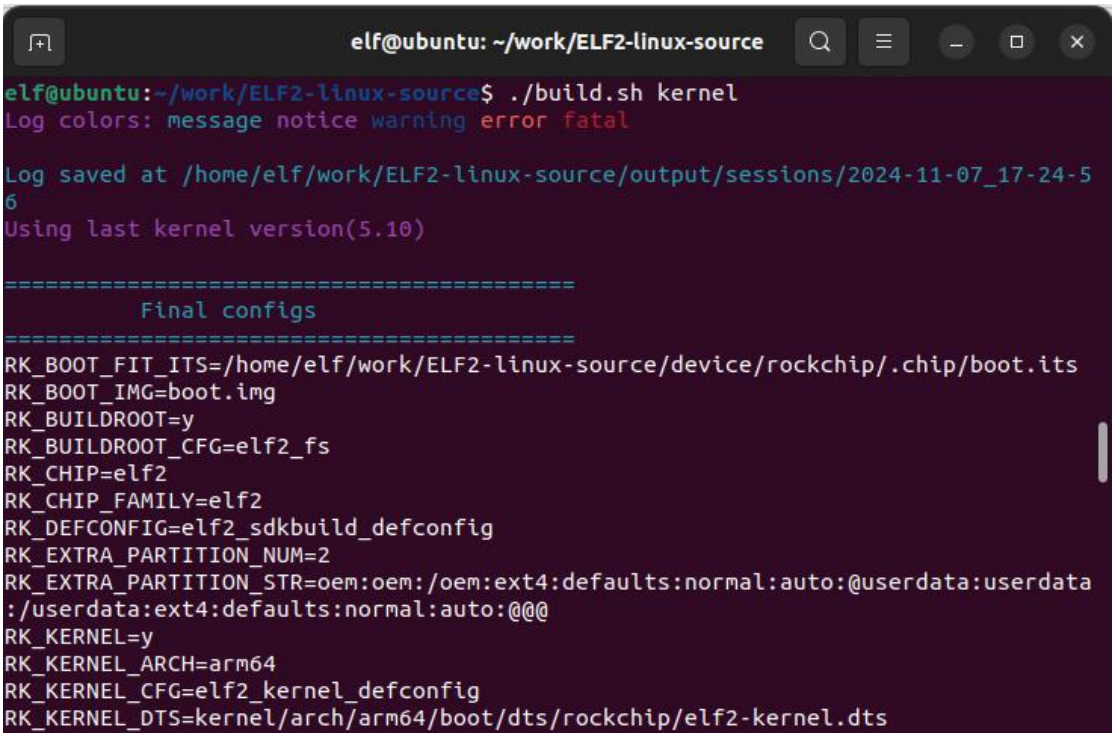
2）N（不将该功能编译进内核）

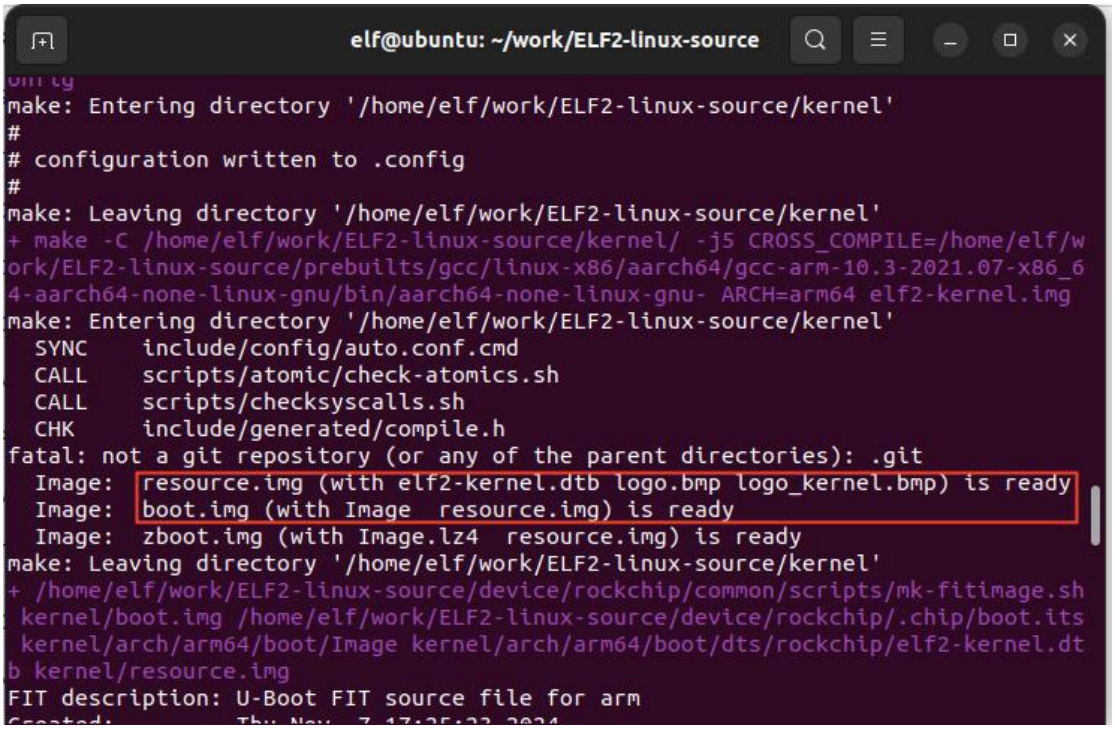
3）M（将该功能编译成可以在需要时动态插入到内核中的模块）

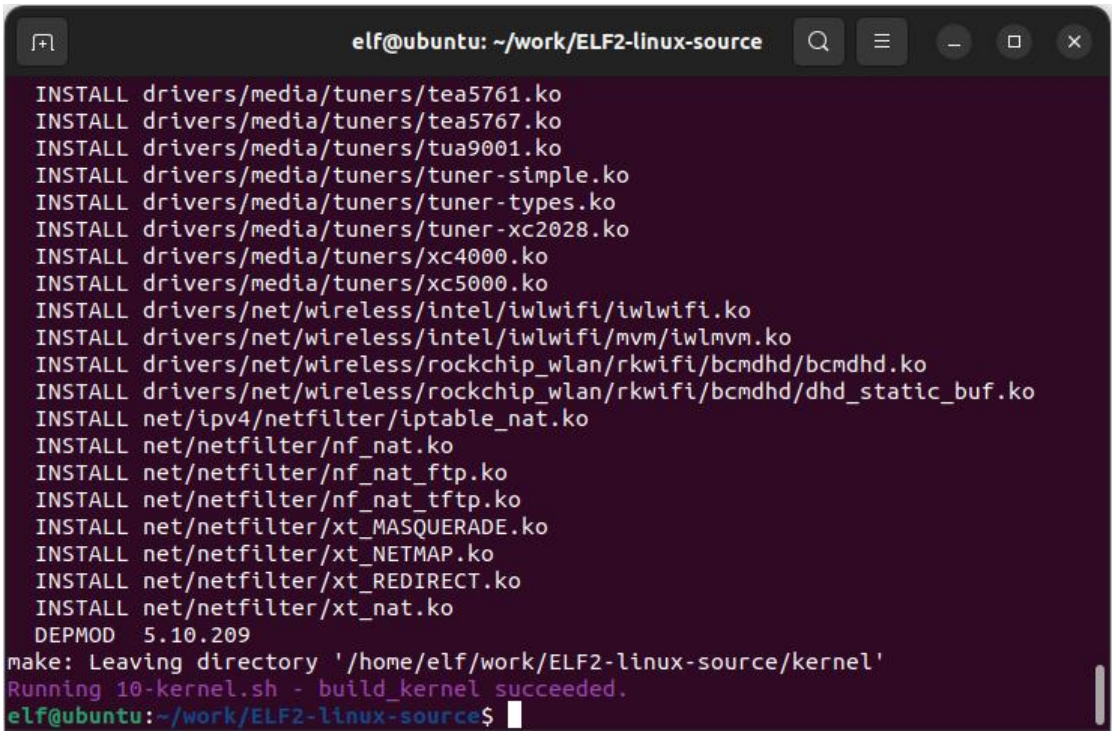
三种选择根据实际需求进行配置，最后在保存退出的时候，会把最新的配置更新到.config 中，同时也会对应修改 arch/arm64/configs/elf2_kernel_defconfig 文件。

2.单独编译内核

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```


elf@ubuntu: ~/work/ELF2-linux-source
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
Log colors: message notice warning error fatal
Log saved at /home/elf/work/ELF2-linux-source/output/sessions/2024-11-07_17-24-56
Using last kernel version(5.10)
Final configs
RK_BOOT_FIT_ITS=/home/elf/work/ELF2-linux-source/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=elf2_fs
RK_CHIP=elf2
RK_CHIP_FAMILY=elf2
RK_DEFCONFIG=elf2_sdkbuild_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=oem:oem:/oem:ext4:defaults:normal:auto:@userdata:userdata:/userdata:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=elf2_kernel_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/elf2-kernel.dts

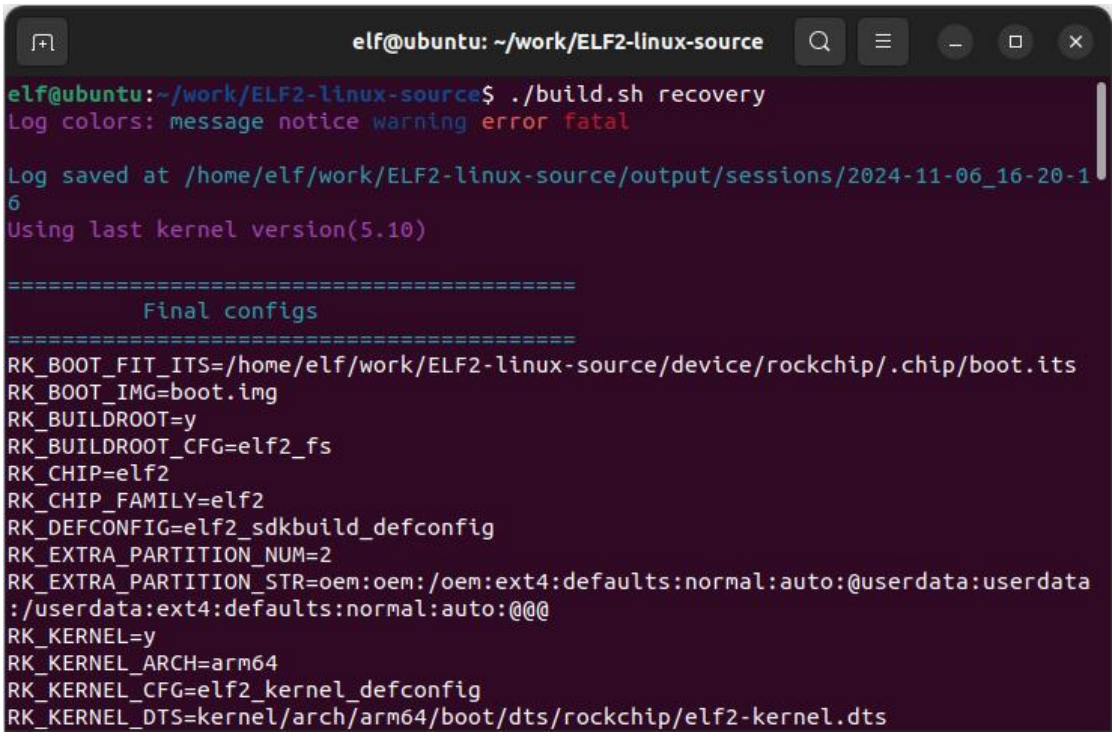

elf@ubuntu: ~/work/ELF2-linux-source
make: Entering directory '/home/elf/work/ELF2-linux-source/kernel'
#
configuration written to .config
#
make: Leaving directory '/home/elf/work/ELF2-linux-source/kernel'
+ make -C /home/elf/work/ELF2-linux-source/kernel/ -j5 CROSS_COMPILE=/home/elf/work/ELF2-linux-source/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu- ARCH=arm64 elf2-kernel.img
make: Entering directory '/home/elf/work/ELF2-linux-source/kernel'
SYNC include/config/auto.conf.cmd
CALL scripts/atomic/check-atomics.sh
CALL scripts/checksyscalls.sh
CHK include/generated/compile.h
fatal: not a git repository (or any of the parent directories): .git
Image: resource.img (with elf2-kernel.dtb logo.bmp logo_kernel.bmp) is ready
Image: boot.img (with Image resource.img) is ready
Image: zboot.img (with Image.lz4 resource.img) is ready
make: Leaving directory '/home/elf/work/ELF2-linux-source/kernel'
+ /home/elf/work/ELF2-linux-source/device/rockchip/common/scripts/mk-fitimage.sh kernel/boot.img /home/elf/work/ELF2-linux-source/device/rockchip/.chip/boot.its kernel/arch/arm64/boot/Image kernel/arch/arm64/boot/dts/rockchip/elf2-kernel.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm

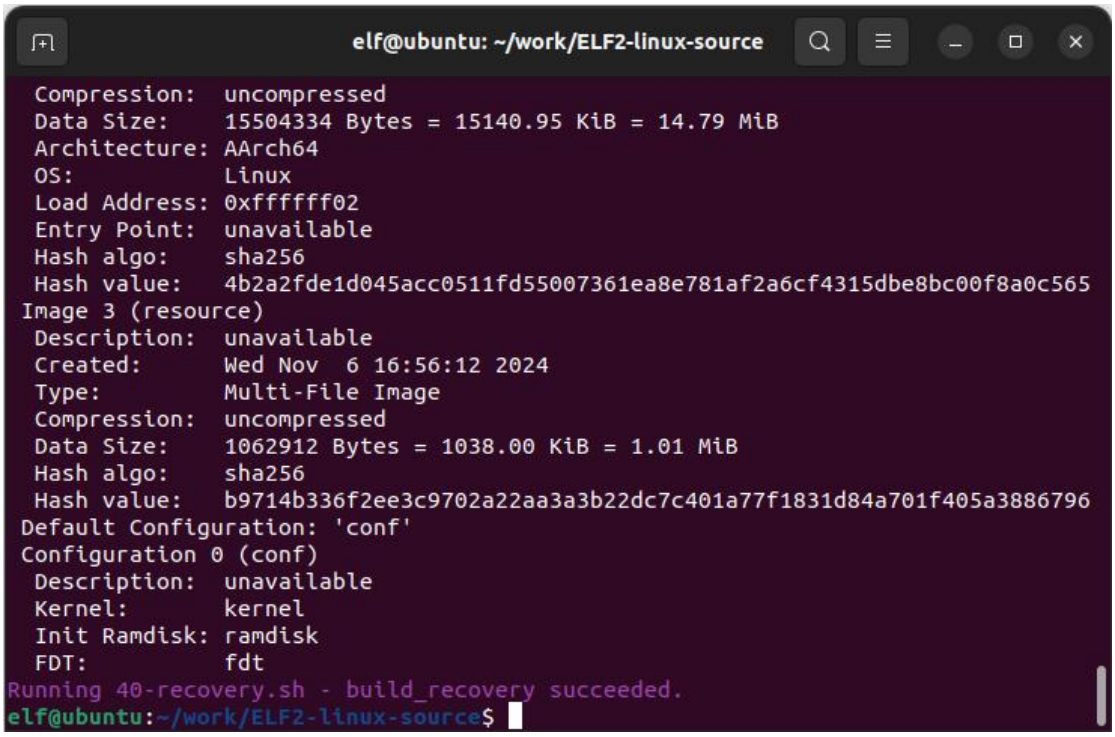


编译完成之后，显示 build_kernel succeeded，会在 output/firmware/目录下生成 boot.img 文件，从上图可以看到 boot.img 包含 Image 和 resource.img 镜像，其中 Image 为内核镜像。resource.img 中又包含 DTB 镜像，logo 图片。

因为 recovery.img 是包含内核的，可以把 recovery.img 看作是一个包含内核和文件系统的精简系统，主要是烧录时启动的，所以更改 kernel 后 recovery 是需要重新打包生成的。

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh recovery
```





1.4 烧录流程介绍

前边章节讲了如何编译生成镜像文件，生成自己专属的镜像之后，如何将这些镜像烧写到 ELF2 开发板上呢，这一节我们来介绍一下烧写流程，方便大家理解如何将镜像烧写到存储介质上。

ELF2 开发板提供了两种烧录接口：OTG 烧录和 TF 卡烧录。其中 OTG 烧录提供了三个工具。

工具	运行系统	描述
RKDevTool	Windows	瑞芯微开发工具，分步升级固件及整个 update 升级固件工具
FactoryTool	Windows	量产升级工具，支持 USB 一拖多烧录
Linux_Upgrade_tool	Linux	Linux 下开发的工具，支持固件的升级

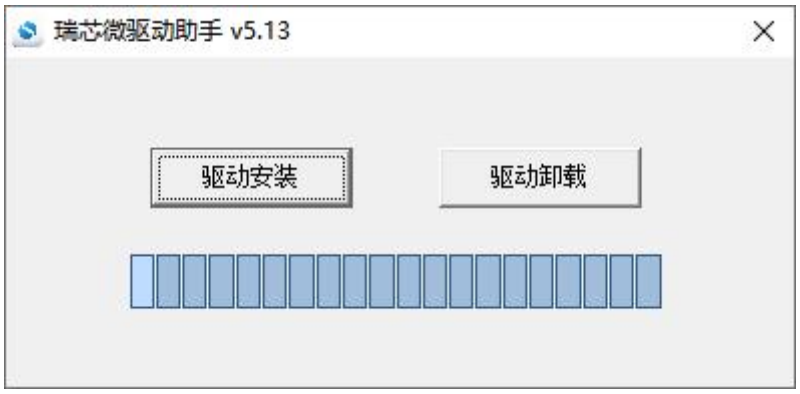
TF 卡烧录提供了一个制卡工具。

工具	运行系统	描述
SD_Firmware_Tool	Windows	瑞芯微开发工具，支持制作烧录卡和启动卡

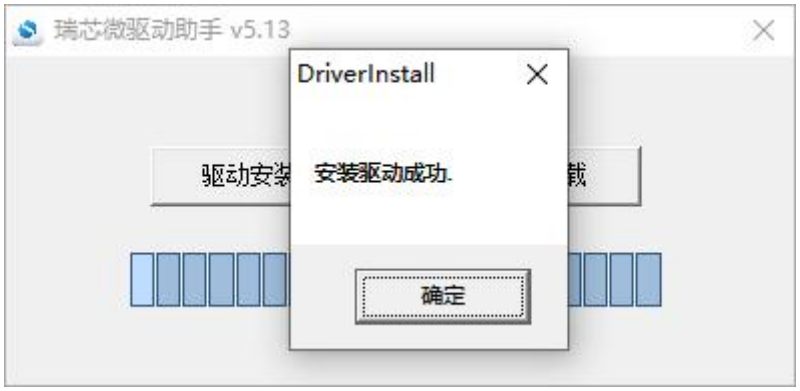
1.4.1 OTG 烧录流程

1.4.1.1 驱动安装

使用 OTG 烧录前首先需要在 Windows 系统上安装驱动，驱动路径在：ELF 2 开发板资料包\05-常用工具\03-烧写工具\OTG 烧写\DriverAssitant_v5.13.zip，将上述压缩包解压到任意路径，以管理员权限运行。打开 DriverInstall.exe 程序。



点击“驱动安装”。



1.4.1.2 RKDevTool 烧写

路径：ELF 2 开发板资料包\05-常用工具\03-烧写工具\OTG 烧写\RKDevTool_Release_v3.31.zip

这是瑞芯微提供的一款开发工具，支持分步烧录和整体烧录。USB OTG 烧写所需镜像在：ELF 2 开发板资料包\03-镜像\00-eMMC 镜像目录下

首次烧录时需要选择整体烧录，打开工具后选择“升级固件”。



点击“固件”选择 update.img，这个是打包后的完整升级包。



使用 12V 电源适配器连接 ELF 2 开发板 PWR 接口，用 Type-C 线连接 ELF 2 开发板 USB-DP 接口和主机，按住开发板的 RECOVERY 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVERY，烧录工具会显示“发现一个 LOADER 设备”，如果没有出现，需要检查一下是否识别到了虚拟机上。



点击升级，等待升级完成。



更新完成后显示“下载固件成功”。更新完成后系统会自动重启。

MASKROM 模式介绍

如果 loader 损坏无法进入 Loader 模式时，可以按住 BOOT 键然后按 RST 键进入 maskrom 模式

进行烧写。



此时系统将提示发现一个 maskrom 设备，烧写流程与 loader 模式一致，最好使用 update.img 烧写。

注意：maskrom 模式下不要点击“设备分区表”，为无效操作。

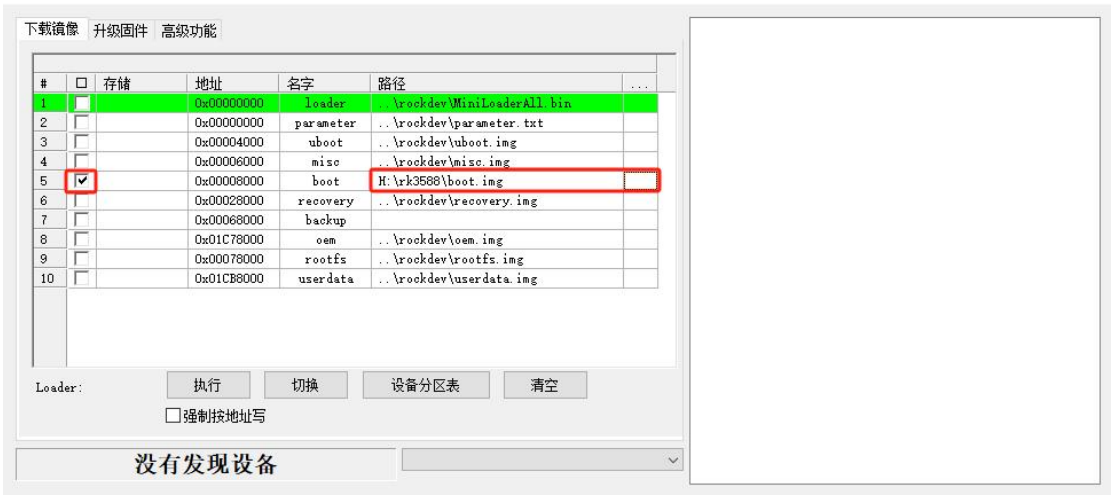
注意：maskrom 模式下单独烧写不会清除 UBOOT 环境变量。

除了整体更新之外，RKDevTool 还支持单步烧录，例如只更改了内核部分，只需要单独编译内核之后，将生成的 boot.img 更新到开发板中。

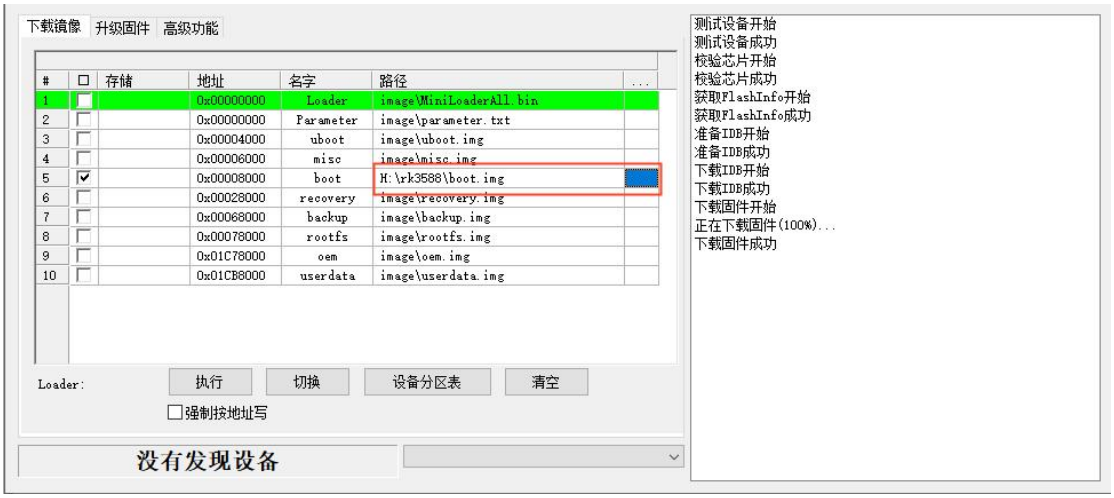
点击“下载镜像”按钮。



勾选烧录 boot 分区。

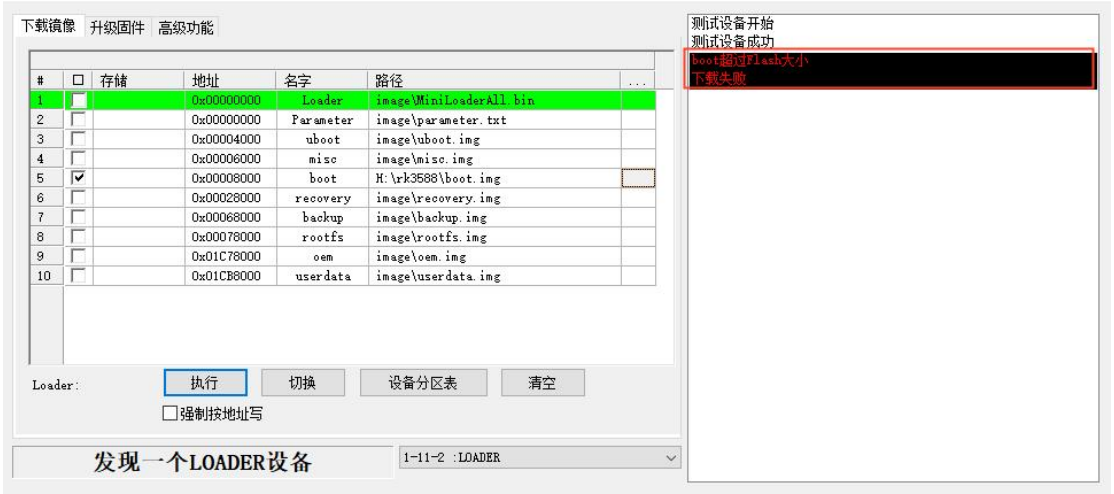


选择 boot.img 镜像。

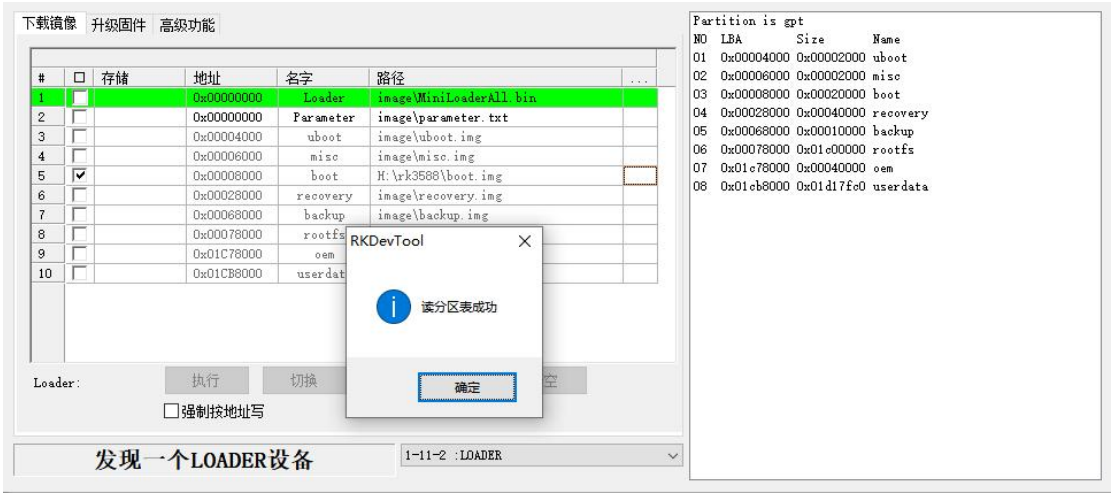


同样，用 Type-C 线连接开发板 USB-DP 口和主机，按住开发板的 RECOVERY 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVERY，烧录工具会显示“发现一个 LOADER 设备”，最后点击“执行”。

如果显示“boot 超过 Flash 大小，下载失败”。



需要点击“设备分区表”，重新识别设备分区表。

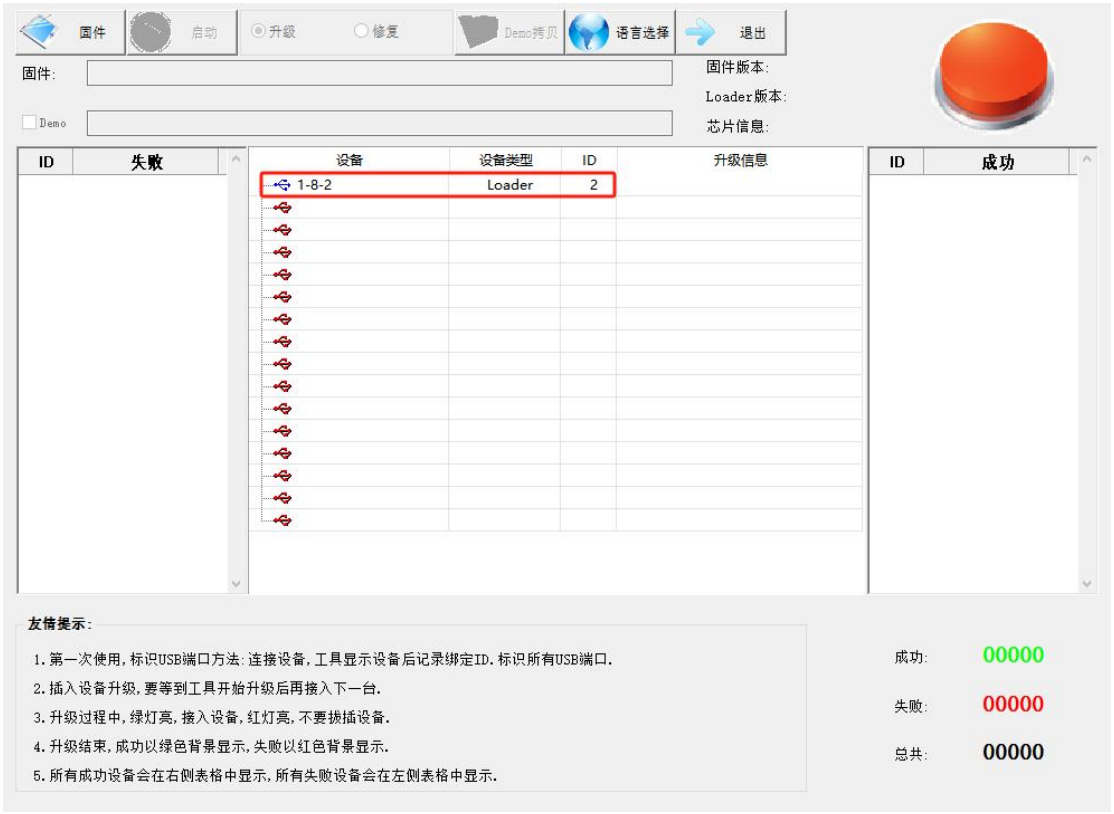


再次点击“执行”进行烧录。

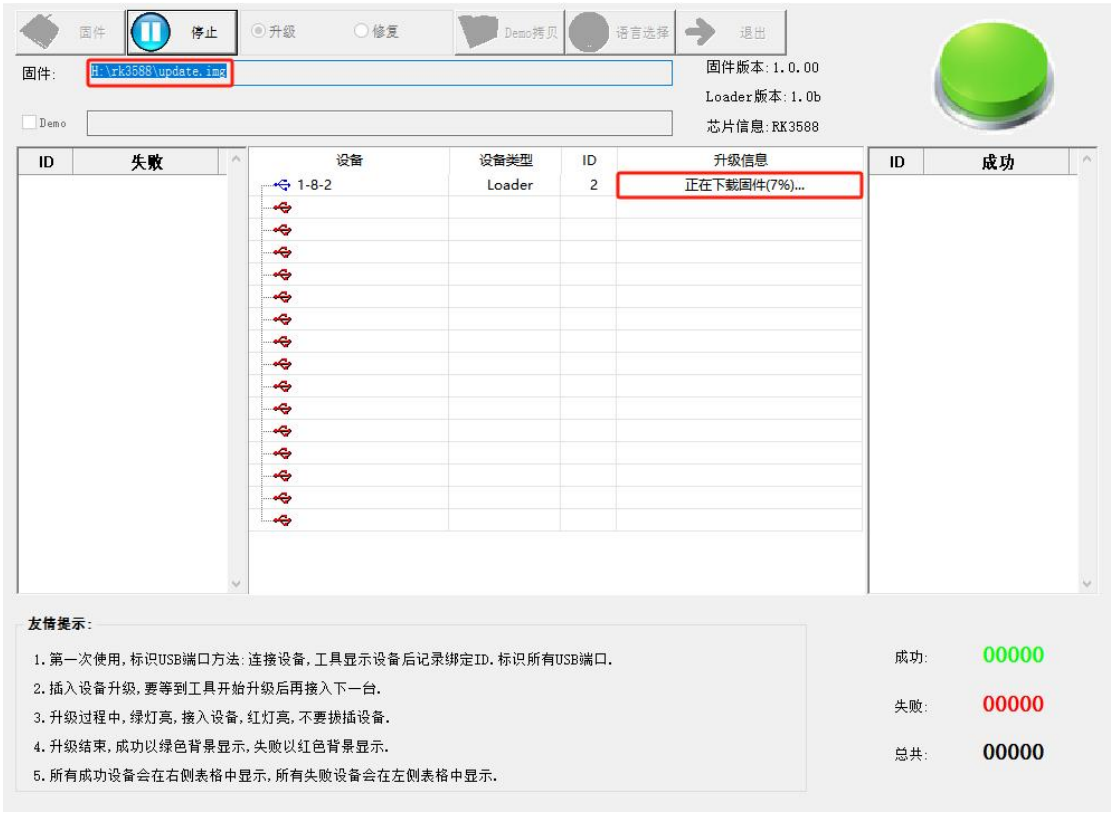


1.4.1.3 FactoryTool 烧写

FactoryTool 是工厂批量 OTG 烧写时使用到的工具，其不需要读取镜像，可以批量烧写，另外其可烧写某些较大的镜像文件，若 RKDevTool 兼容性不满足时，亦可尝试该方式，使用前将其解压到全英文路径下，用 Type-C 线连接开发板和主机，按住开发板的 RECOVER 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVER 键。瑞芯微开发工具上将提示发现 loader 设备。

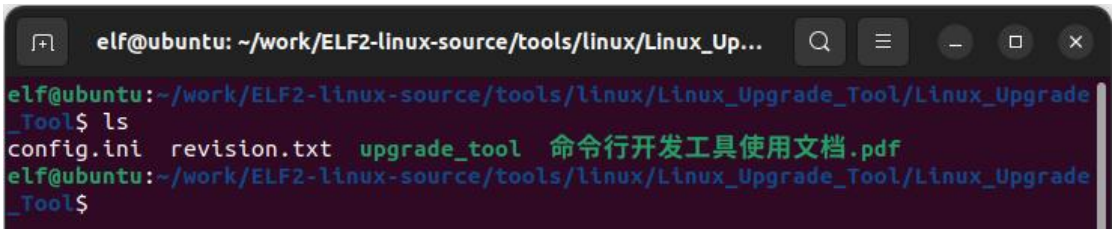


点击选择固件，点击启动，此时识别到 loader 设备将自动开始烧写。

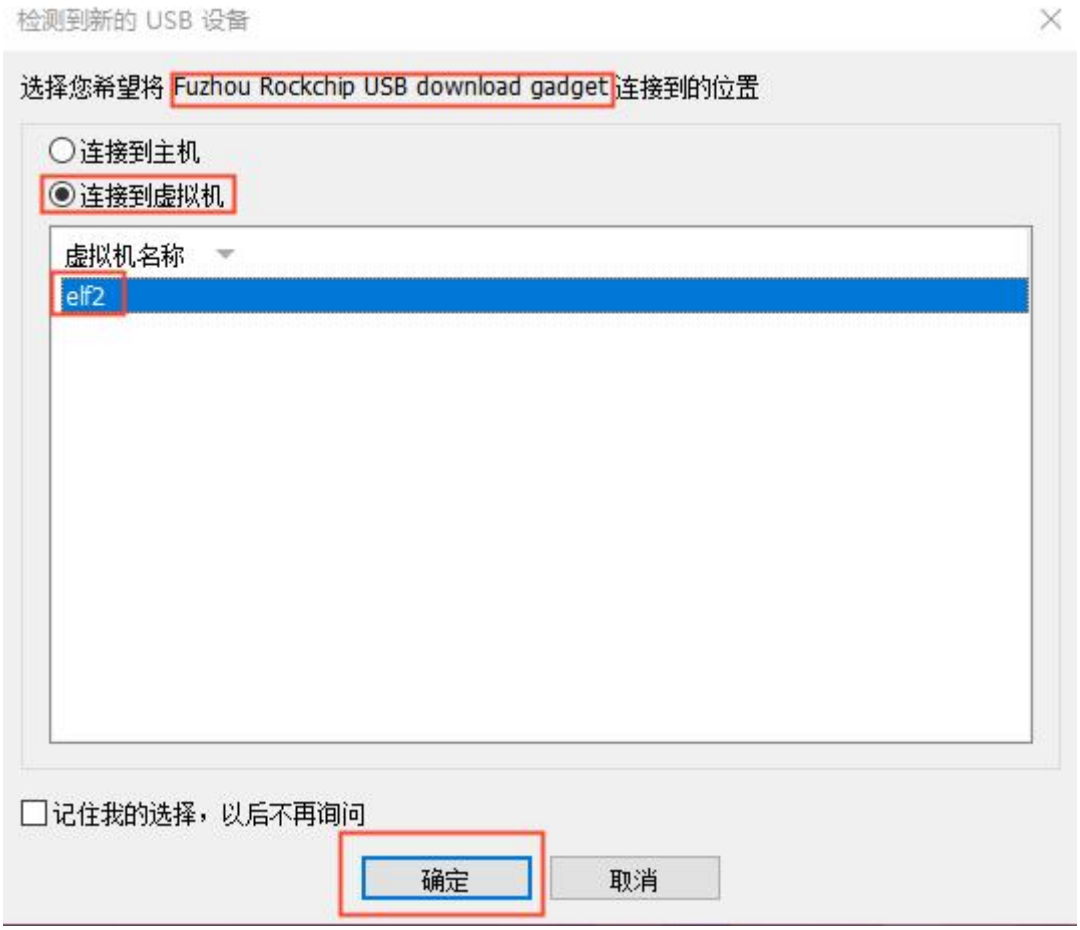


1.4.1.4 Linux_Upgrade_tool 烧写

Linux 下的烧写工具位于 SDK 中 tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool 目录下。



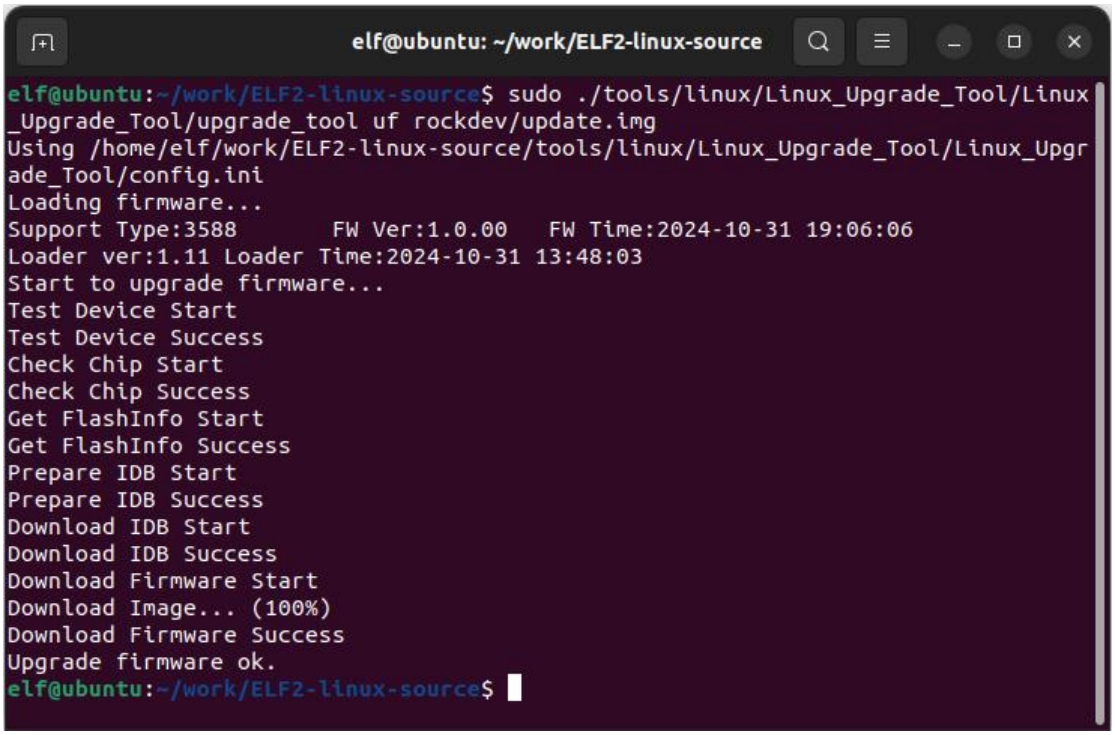
用 Type-C 线连接开发板 USB-DP 口和主机，按住开发板的 RECOVERY 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVERY，将 usb 设备识别到虚拟机中。



从上边的截图可以看出设备的名称是 Fuzhou Rockchip USB download 设备。

整体烧录：

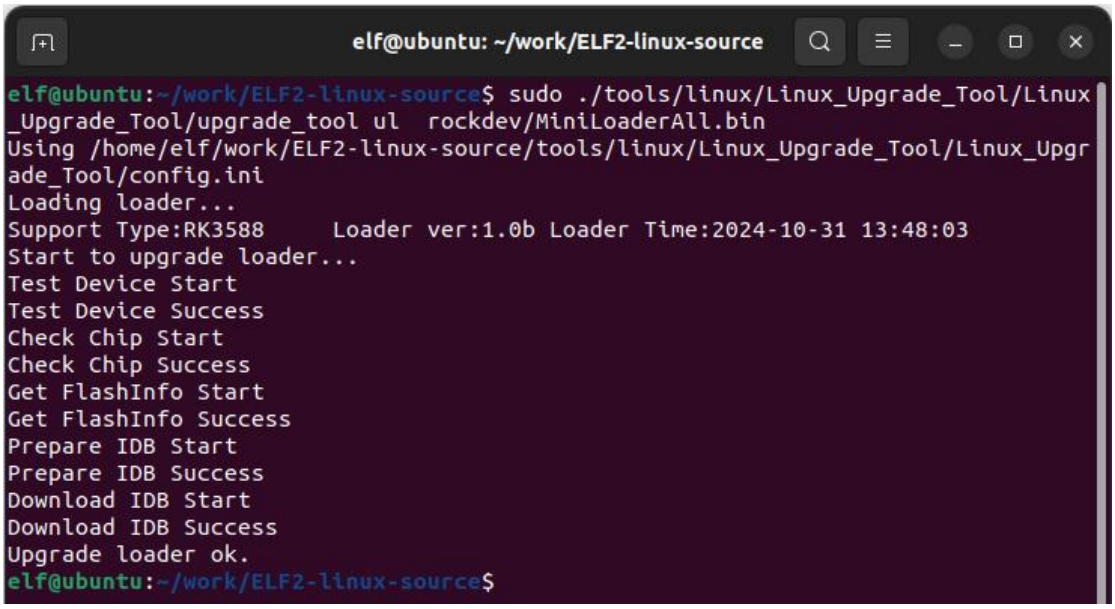
```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool uf rockdev/update.img
```



Linux_Upgrade_tool 同样支持单步烧录。

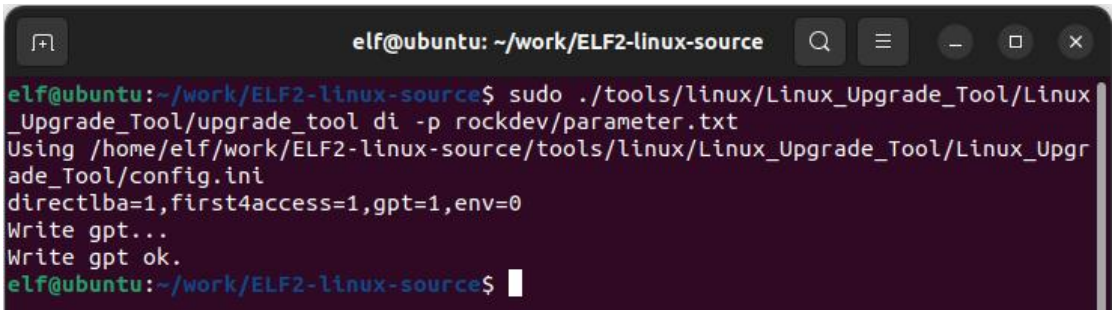
MiniLoaderAll.bin 为 rk3588 片上 ROM 里存放的引导程序，此部分瑞芯微不开源，一般不涉及到单独烧录，但是依然提供了单独烧录的接口。单独烧录 MiniLoaderAll.bin：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool ul  rockdev/MiniLoaderAll.bin
```



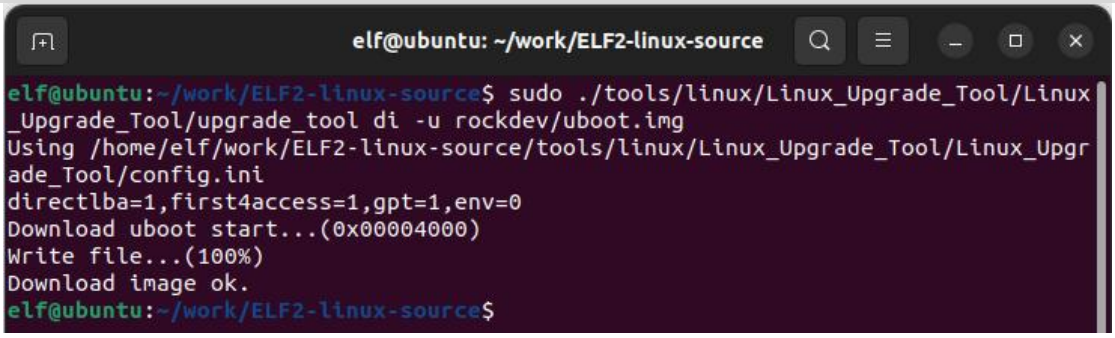
parameter.txt 为分区配置文件，如何修改分区大小或者添加分区需要修改此文件，单独烧录 parameter.txt：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -p rockdev/parameter.txt
```



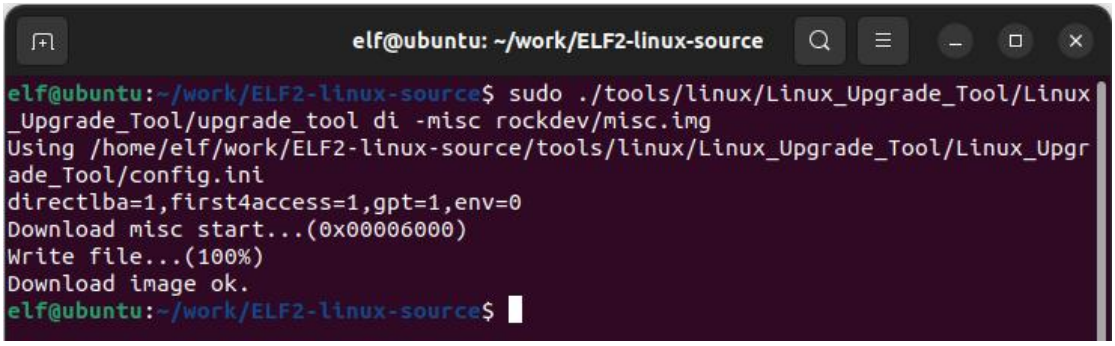
只修改了 uboot，可以选择单独烧录 u-boot.img：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -u rockdev/uboot.img
```



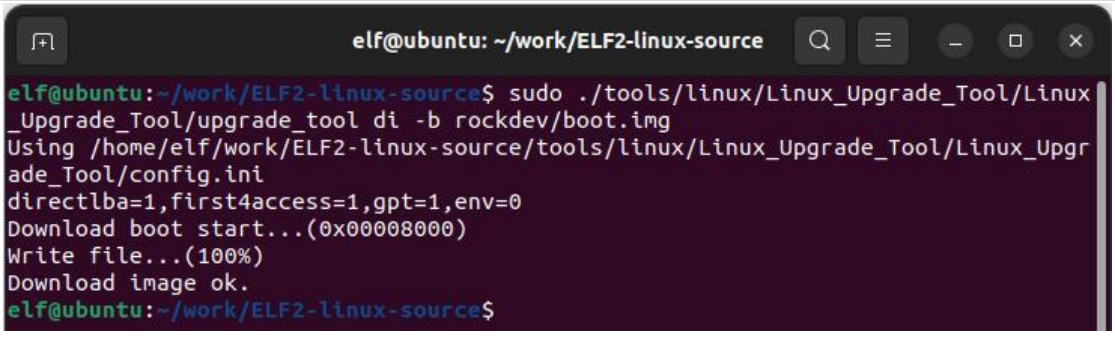
一般涉及单独烧录 misc.img：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -misc rockdev/misc.img
```



只修改了内核源码，可以选择单独烧录 boot.img：

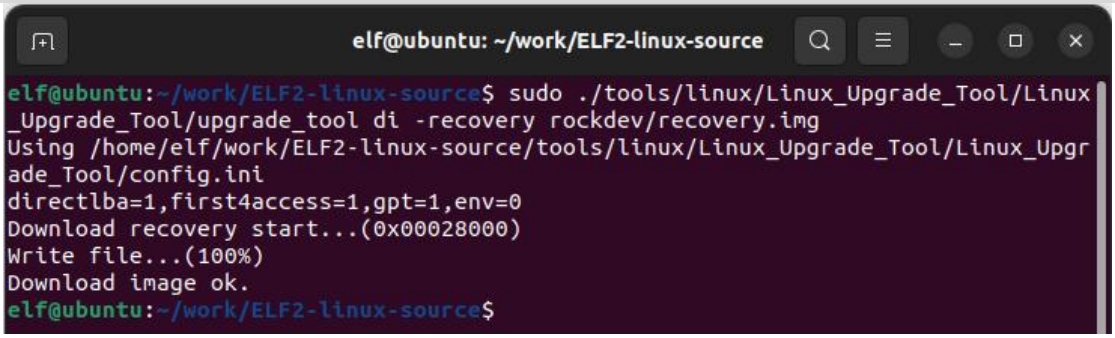
```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -b rockdev/boot.img
```



如果涉及到对烧录方式自定义修改时，修改 recovery 后可以单独烧录 recovery.img：

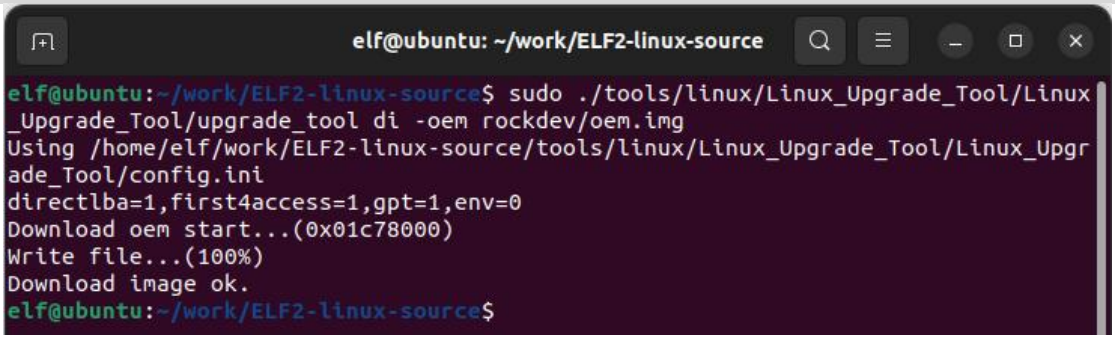
```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upg
```

```
rade_Tool/upgrade_tool di -recovery rockdev/recovery.img
```



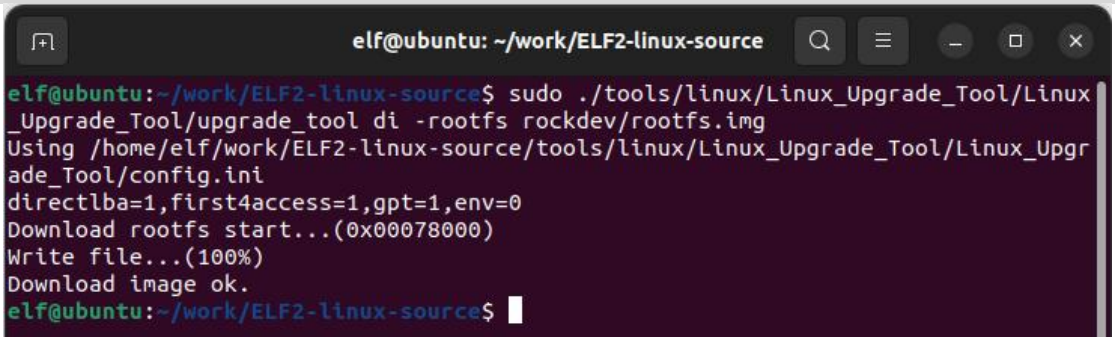
一般不涉及单独烧录 oem.img：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -oem rockdev/oem.img
```



单独烧录 rootfs.img：

```
elf@ubuntu:~/work/ELF2-linux-source$ sudo ./tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/upgrade_tool di -rootfs rockdev/rootfs.img
```



1.4.2 TF 卡烧录系统

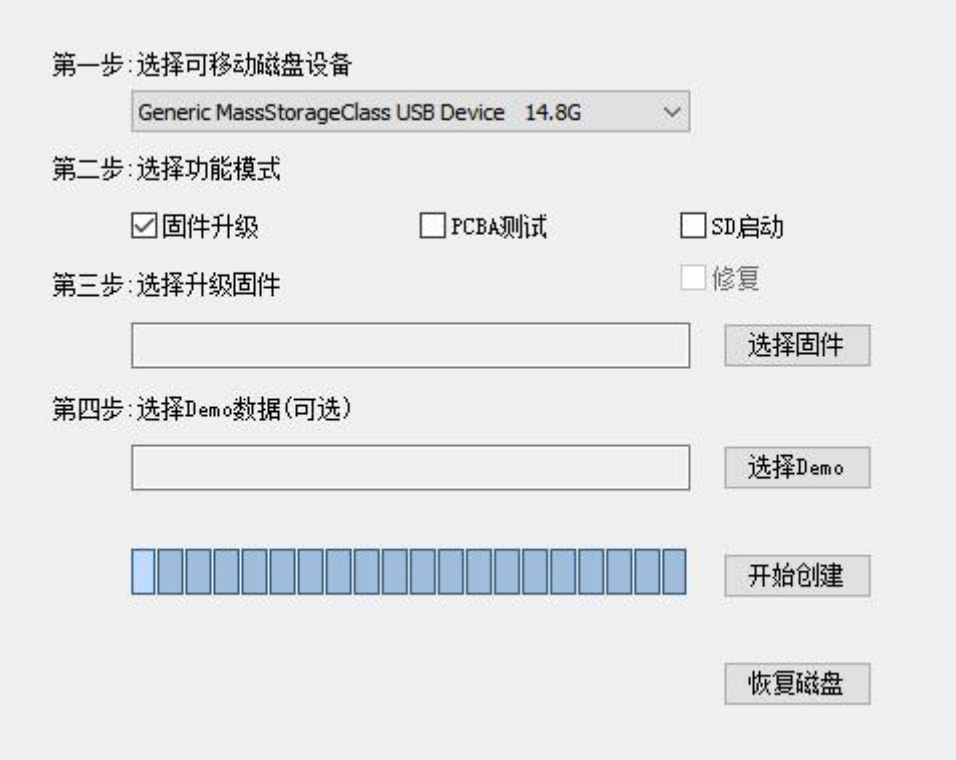
ELF2 开发板不仅可以通过 USB OTG 烧写镜像，还可以通过 TF 卡进行烧写，接下来我们对 TF 卡的烧录方法进行介绍。

1.4.2.1 制作烧录卡

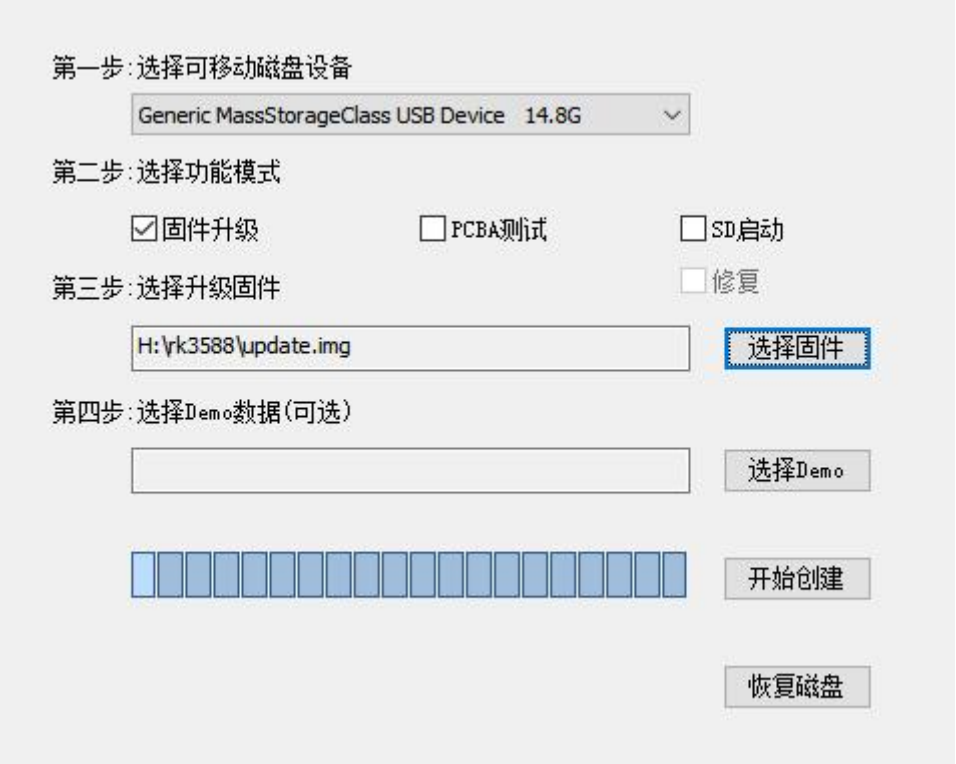
⚠**注意**：测试 TF 卡容量最大为 32G，使用 32G 以上 TF 卡可能会烧写失败。

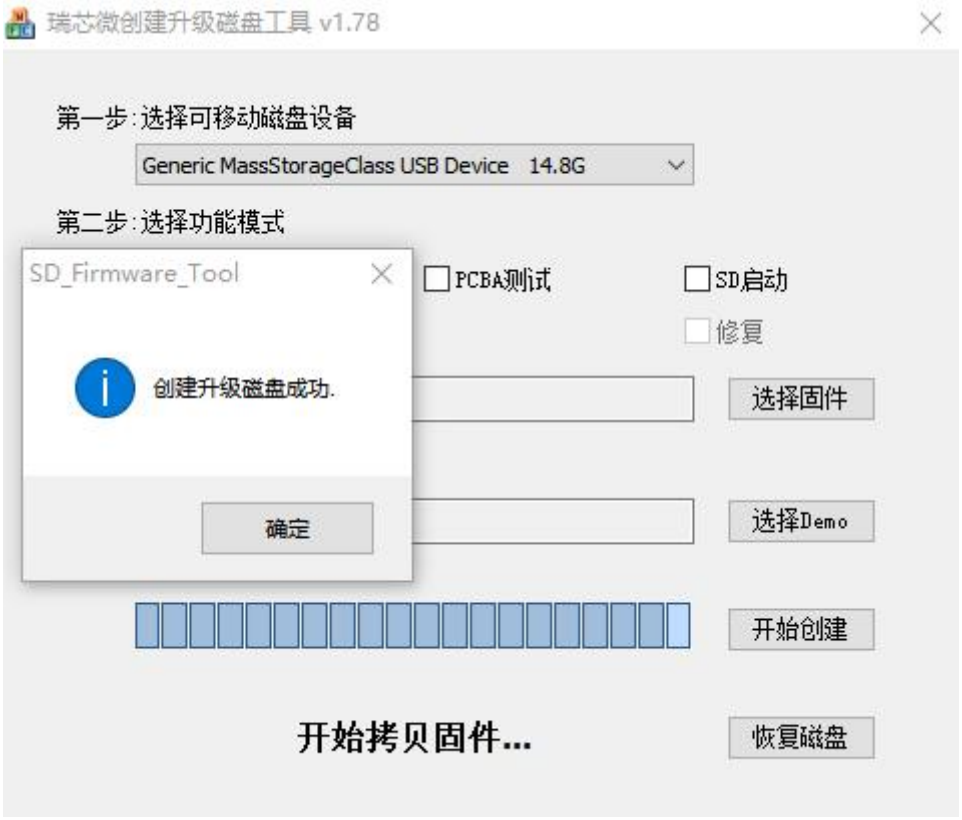
将ELF 2开发板资料包\05-常用工具\03-烧写工具\TF卡烧写\SDDiskTool_v1.78.zip 拷贝到 windows 任意目录。以管理员权限运行SD_Firmware_Tool.exe。

使用读卡器将 TF 卡插到 PC 机上，并且识别到 Windows 系统上。



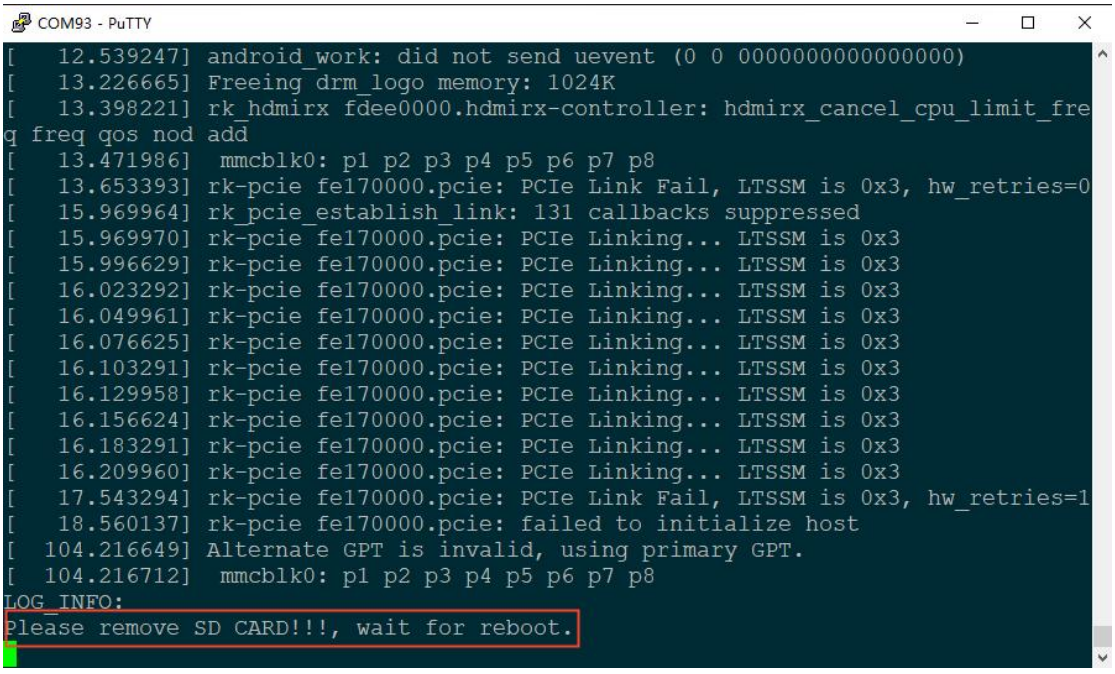
选择磁盘设备，勾选“固件升级”，并选择 update.img。点击创建。





1.4.2.2 烧写过程

将 TF 卡插入开发板并按 RST 键重启，系统将自动进入到烧录流程。烧写完成后 debug 口将提示：Please remove SD CARD!!!, wait for reboot.



此时将 TF 卡拔出，系统将自动重启。（请勿直接断电重启）

1.5 Makefile 介绍

前面我们编译内核时，那么多.c 文件最终生成一个 Image 镜像。其实是 make 工具通过解析 Makefile 文件进行一系列编译操作，最终生成我们想要的镜像文件。Makefile 文件中描述了各工程所有文件的编译

顺序、编译规则以及依赖关系，决定了工程中文件是否需要编译，以及这些文件的编译顺序。Make 工具还可以通过比较文件最后修改时间，来决定哪些文件需要更新哪些不需要更新，更改了某个文件之后，只对依赖此文件的目标文件进行重新编译更新，这就大大减少了编译时间。

打开源码目录可以看到，Makefile 文件不仅存在于源码根目录下，在其他的子目录下也基本都有 Makefile 文件。在执行编译时，Make 工具会解析根目录下 Makefile 文件进行编译，而根目录的 Makefile 会调用子目录下的 Makefile，子目录下又有子目录，层层调用。

Makefile 需要按照一定的格式语法规则进行书写。如果你是做 Linux 应用开发的人员，那么写 Makefile 就是必备技能，就需要深谙 Makefile 语法规则。对于我们 Linux 平台初级开发人员，很少需要我们去写一个复杂的 Makefile 文件，只是在做平台移植的时候可能需要去简单修改或者阅读 Makefile 文件，所以呢，下面我们就简单了解一下 Makefile 基本格式规则。

1.目标和依赖

目标就是我们需要生成的文件，依赖就是生成目标文件所需要的其他文件，称为依赖文件。基本语法规则如下：

```
targets … :dependent_files …
(tap)command
                …
```

我们举例说明。

举例一:

我们先创建一个简单的 app.c 文件:

```
elf@ubuntu:~/work/tmp$ touch app.c
```

在文件中输入如下代码:

```
#include <stdio.h>
int main(void)
{
        printf("my first app !!! \r\n");
        return 0;
}
```

按照基本规则建立一个简单的 Makefile 文件，app 为我们需要的目标文件，app.c 为所要生成的 app 的依赖文件，gcc –o app app.c 就是生成目标 app 需要执行的命令。

建立一个 Makefile 文件：

```
elf@ubuntu:~/work/tmp$ touch makefile
```

文件中输入如下内容：

```
app: app.c
        gcc -o app app.c
```

使用 make 命令，可以看到执行了命令 gcc -o app app.c，ls 命令查看文件，发现在该目录下生成了 app 目标文件：

```
elf@ubuntu:~/work/tmp$ make
gcc -o app app.c
elf@ubuntu:~/work/tmp$ ls
app app.c makefile
```

执行 app，即可看到 app.c 中我们写的打印信息：

```
elf@ubuntu:~/work/tmp$ ./app
my first app !!!
```

然后我们再执行 make 命令，会发现出现如下信息，‘app’ is up to date，说明 app 已经是最新的，没

必要再重新生成：

```
elf@ubuntu:~/work/tmp$ make
make: ‘app’ is up to date.
```

我们做一下稍微的改动，修改 app.c 文件中的内容或者使用 touch 命令更改一下 app.c 的时间属性，然后再进行 make 看看：

```
elf@ubuntu:~/work/tmp$ touch app.c
elf@ubuntu:~/work/tmp$ make
gcc -o app app.c
```

可以看到又重新执行了 gcc -o app app.c 命令，重新生成了 app。也就是说，make 工具会判断依赖文件的时间戳是否比目标文件时间戳更新，来决定是否重新生成目标文件。

举例二：

假如我们有多个文件需要编译，比如我们创建多个文 fun1.c,fun1.h,fun2.c,fun2.h,app.c。

fun1.c:

```
#include <stdio.h>
#include "fun1.h"
void fun1(char *s)
{
        printf("%s\r\n",s);
}
```

fun1.h:

```
#ifndef _FUN1_H
#define _FUN1_H
void fun1(char *s);
#endif
```

fun2.c:

```
#include "fun2.h"
int fun2(int x,int y)
{
        return x+y;
}
```

fun2.h:

```
#ifndef _FUN2_H
#define _FUN2_H
int fun2(int x,int y);
#endif
```

创建 app.c 文件，该文件引用 fun1.c 和 fun2.c 中定义的函数：

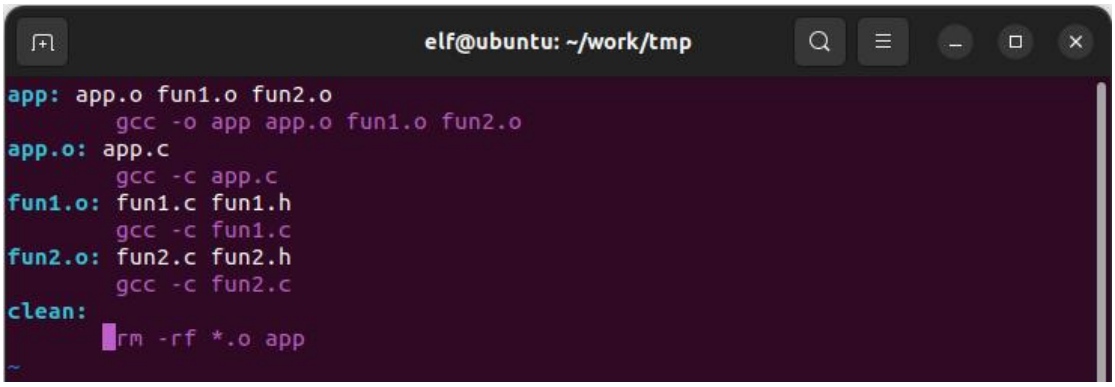
```
#include <stdio.h>
#include "fun1.h"
#include "fun2.h"
int main(void)
{
        fun1("I am fun1 !");
        printf("fun2 return value=%d \n",fun2(1,2));
        return 0;
```

```
}
```

最后创建 makefile 文件，其中 app 是我们的最终目标文件，app 的依赖文件为 app.o，fun1.o，fun2.o，这几个.o 文件称为中间目标文件，它们又有各自所依赖的.c 和.h 文件，最后的 clean 也是一个目标文件，但是这个目标没有任何依赖，也不会生成真正文件，只是执行一条命令，我们称之为伪目标。我们可以看到 clean 执行的是删除.o 和 app 的命令。

```
app: app.o fun1.o fun2.o
	gcc -o app app.o fun1.o fun2.o
app.o: app.c
	gcc -c app.c
fun1.o: fun1.c fun1.h
	gcc -c fun1.c
fun2.o: fun2.c fun2.h
	gcc -c fun2.c
clean:
	rm -rf *.o app
```

⚠**注意：**复制粘贴过程中可能会出现格式问题，参考下图进行调整。



创建完成之后，我们直接 make：

```
elf@ubuntu:~/work/tmp$ make
gcc -c app.c
gcc -c fun1.c
gcc -c fun2.c
gcc -o app app.o fun1.o fun2.o
elf@ubuntu:~/work/tmp$ ls
app   app.c   app.o   fun1.c   fun1.h   fun1.o   fun2.c   fun2.h   fun2.o   makefile
```

上图可以看到生成最终目标 app 和一些中间目标文件，这个过程经过了以下四个步骤：

```
gcc -c app.c
gcc -c fun1.c
gcc -c fun2.c
gcc -o app app.o fun1.o fun2.o
```

执行 app，可以看到执行成功：

```
elf@ubuntu:~/work/tmp$ ./app
I am fun1 !
fun2 return value=3
```

现在修改其中一个文件 fun2.h 后，比如更改一下 fun2.h 的时间属性，然后再次编译：

```
elf@ubuntu:~/work/tmp$ touch fun2.h
elf@ubuntu:~/work/tmp$ make
gcc -c fun2.c
gcc -o app app.o fun1.o fun2.o
```

可以看到只用了两个步骤，app 的其他依赖文件并没有被重新编译生成。

最后，执行 make clean 清除中间文件和 app：

```
elf@ubuntu:~/work/tmp$ make clean
rm -rf *.o app
elf@ubuntu:~/work/tmp$ ls
app.c  fun1.c  fun1.h  fun2.c  fun2.h  makefile
```

2.变量定义

变量是在 Makefile 中定义的名字，用来代替一个文本字符串，该文本字符串称为该变量的值。变量名是不包括“:”、“#”、“=”结尾空格的任何字符串。变量名是大小写敏感的。推荐在 Makefile 内部使用小写字母作为变量名。

变量赋值方式有以下几种：

1）递归赋值

变量的名称 = 变量值

变量的值将会是整个 Makefile 中最后被指定的值。

举例，建立一个 makefile 文件，定义两个变量 a 和 b，引用变量使用$()，使用@echo 进行打印：

```
a = 123
b = $(a) string
a = 678
target:
	@echo "a = $(a)"
	@echo "b = $(b)"
```

可以看到 b 的值为 678 string，而不是 123 string

```
elf@ubuntu:~/work/tmp$ make
a = 678
b = 678 string
```

2）简单赋值

变量的名称 := 变量值

变量的值决定于它在 Makefile 中的位置，而不是整个 Makefile 展开后最终的值。

```
a := 123
b := $(a) string
a := 678
target:
	@echo "a = $(a)"
	@echo "b = $(b)"
```

make 后可以看到 b 的值为 123 string：

```
elf@ubuntu:~/work/tmp$ make
a = 678
b = 123 string
```

3）条件赋值

变量的名称 ?= 变量值

如果变量未定义，则使用该变量值定义变量。如果该变量已经被定义赋值，则该赋值语句无效，使用原有值。

```
a ?= 123
b ?= $(a) string
a ?= 678
target:
        @echo "a = $(a)"
        @echo "b = $(b)"
```

make 后，a 的值是第一次被赋予的值：

```
elf@ubuntu:~/work/tmp$ make
a = 123
b = 123 string
```

4）追加赋值

变量的名称 += 变量值

举例：

```
a := 123
b := $(a) string
a += $(b)
target:
        @echo "a = $(a)"
        @echo "b = $(b)"
```

make 之后结果：

```
elf@ubuntu:~/work/tmp$ make
a = 123 123 string
b = 123 string
```

3.系统变量

系统变量或者叫环境变量，包含了常见编译器、汇编器的名称及其编译选项。下表列出了 Makefile 中常见系统变量及其部分默认值：

AR：库文件维护程序的名称，默认值为 ar；
AS：汇编程序的名称，默认值为 as；
CC：C 编译器的名称，默认值为 cc；
CPP：C 预编译器的名称，默认值为$(CC) –E；
CXX：C++编译器的名称，默认值为 g++；
FC：FORTRAN 编译器的名称，默认值为 f77；
RM：文件删除程序的名称，默认值为 rm –f；
ARFLAGS：库文件维护程序的选项，无默认值；
ASFLAGS：汇编程序的选项，无默认值；
CFLAGS：C 编译器的选项，无默认值；
CPPFLAGS：C 预编译的选项，无默认值；
CXXFLAGS：C++编译器的选项，无默认值；
FFLAGS：FORTRAN 编译器的选项，无默认值；

4.条件表达式的语法

1）比较 arg1 和 arg2 值是否相同，如果相同则执行 text-if-true，否则执行 text-if-false。

```
ifeq (<arg1>, <arg2> )
text-if-true
else
text-if-false
endif
```

比较 arg1 和 arg2 值是否相同，如果不同则执行 text-if-true，否则执行 text-if-false，与上面的 ifeq 相反。

```
ifneq (<arg1>, <arg2> )
text-if-true
else
text-if-false
endif
```

2）判断 variable 是否定义，如果定义则为真，执行 text-if-true，否则执行 text-if-false。

```
ifdef <variable >
text-if-true
else
text-if-false
endif
```

判断 variable 是否定义，如果没有定义则为真，执行 text-if-true，否则执行 text-if-false。

```
ifndef <variable >
text-if-true
else
text-if-false
endif
```

1.6 Kconfig 介绍

1.6.1 Kconfig 作用

之前章节中，讲解了使用 make menuconfig 图形化界面来配置内核，而界面中这些选项菜单列表主要是由 arch/arm/下 Kconfig 输出给 menuconfig 工具。我们将一个新的驱动程序添加进内核，只需要我们按照一定的格式将此驱动信息添加进该驱动所属的 Kconfig 中，menuconfig 工具就可以识别到这种格式，并将新的驱动信息呈现到配置选项菜单列表中，下面简单讲解一下 Kconfig 语法规则。

1.6.2 Kconfig 语法

Kconfig 文件描述了菜单选项，每行都是以关键字开头，我们逐一解释关键字意义：

1.config

```
config <symbol>
<config options>
```

config 是关键字，表示一个配置选项的开始；紧跟着的<symbol>是配置选项的名称。

```
config VXLAN
        tristate "Virtual eXtensible Local Area Network (VXLAN)"
        depends on INET
        select NET_UDP_TUNNEL
        select GRO_CELLS
        help
          This allows one to create vxlan virtual interfaces that provide
          Layer 2 Networks over Layer 3 Networks. VXLAN is often used
          to tunnel virtual network infrastructure in virtualized environments.
          For more information see:
            http://tools.ietf.org/html/draft-mahalingam-dutt-dcops-vxlan-02

          To compile this driver as a module, choose M here: the module
          will be called vxlan.
```

例如上图中：

config 是关键字，紧跟着的 VXLAN 是配置选项的名称，其实省略了前缀“CONFIG_”。

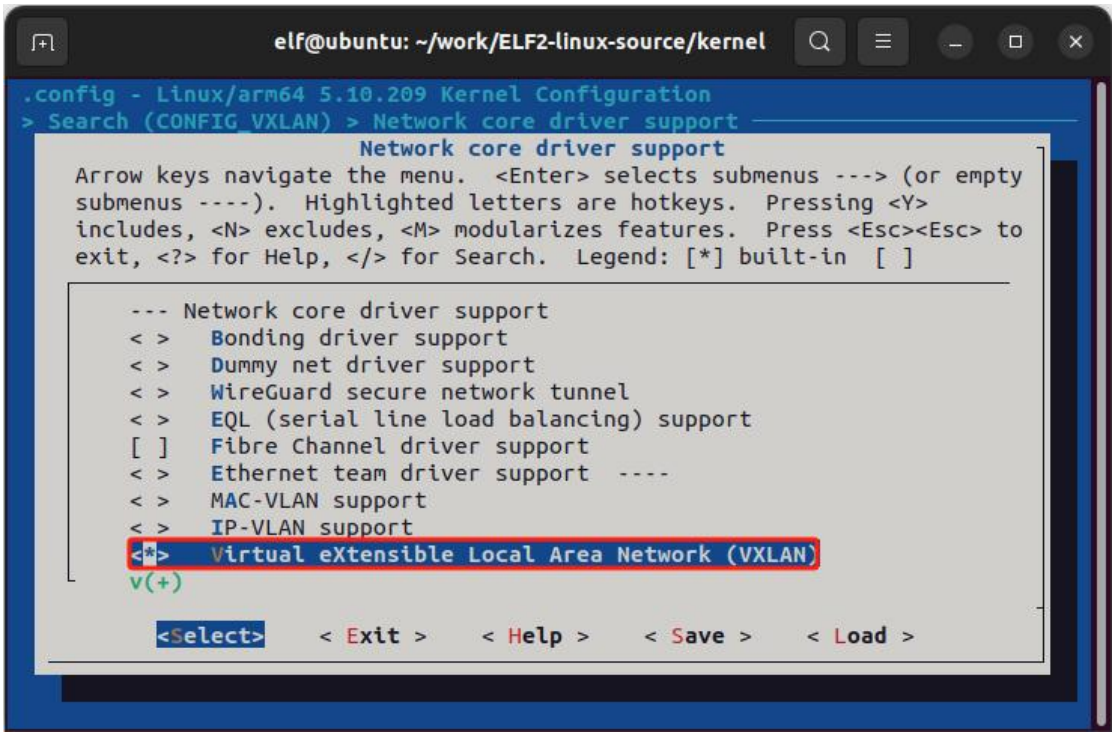
tristate 表示变量类型，即“CONFIG_VXLAN”的类型，有 5 种类型：bool，tristate，string，hex 和 int，其中 bool 类型变量的值：y 和 n；tristate 类型变量的值：y，n 和 m；string 变量的值：字符串；hex 类型变量的值：是一个十六进制的数据；int 类型变量的值：是一个整型的数据。

tristate 之后的字符串“Virtual eXtensible Local Area Network (VXLAN)”是提示信息，在配置界面中上下移动光标选中它时，就可以通过按空格或回车键来设置 CONFIG_VXLAN 的值。

depends on：表示依赖于 XXX，“depends on IENT”表示只有当 IENT 配置选项被选中时，当前配置选项的提示信息才会出现，才能设置当前配置选项。

select：当前选项选中后则 select 后指定的选项自动被选择。

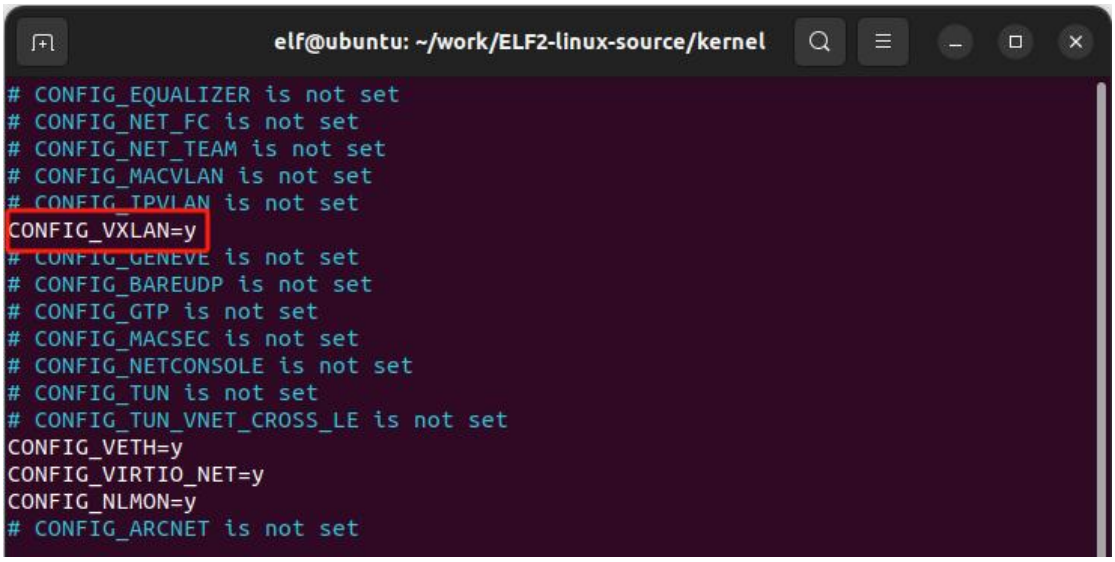
比如我们在图形界面中选中 CONFIG_VXLAN 这一项，CONFIG_VXLAN 类型为 tristate，所以可通过空格选择此项是 y，n，m：



然后在.config 文件中就多出一项：

```
#CONFIG_NET_TEAM is not set
#CONFIG_MACVLAN is not set
#CONFIG_IPVLAN is not set
```

```
CONFIG_VXLAN = y
#CONFIG_NETCONSOLE is not set
```

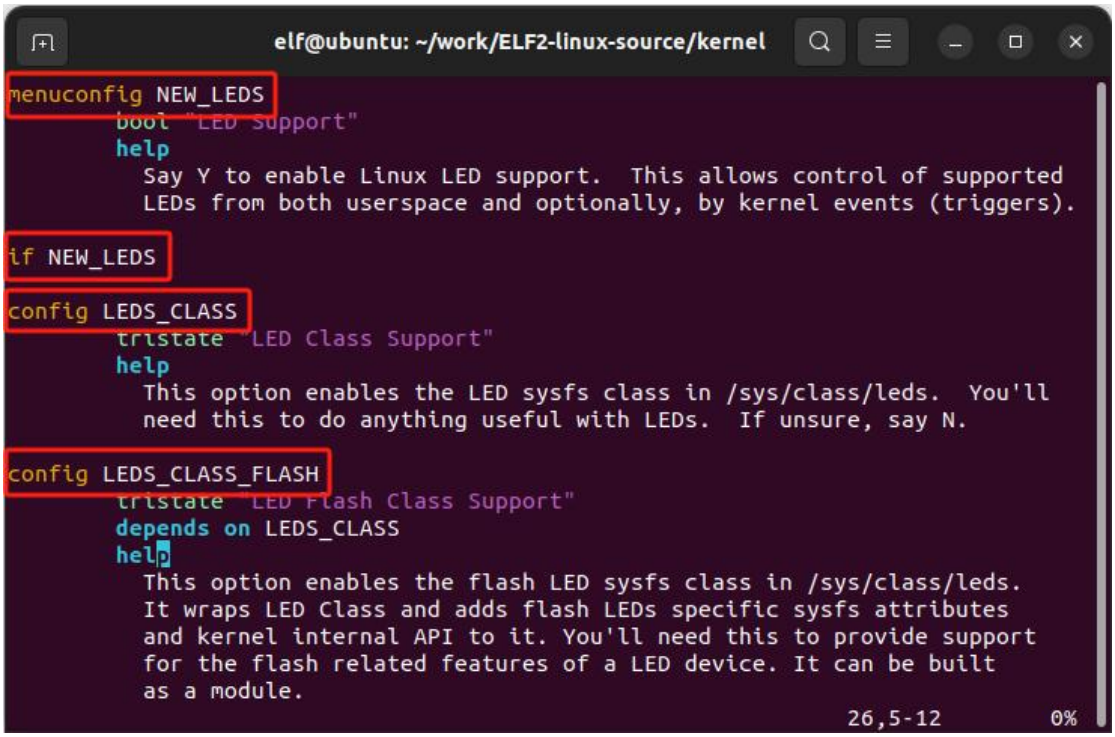


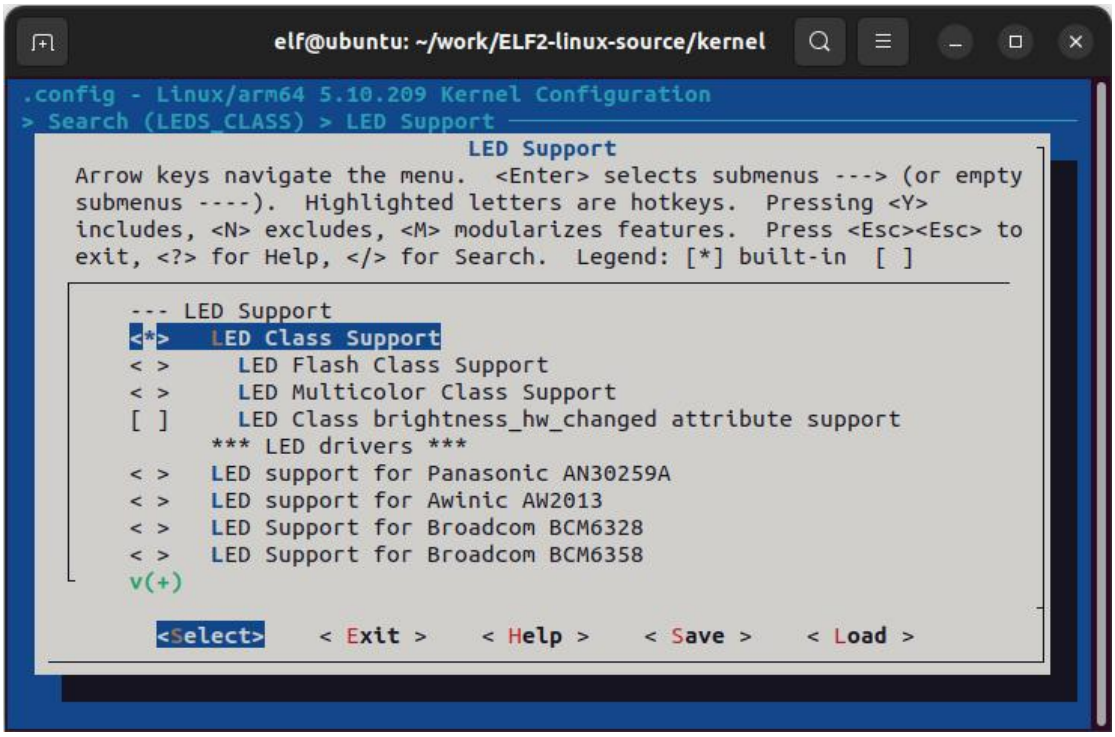
Makefile 文件会根据这项配置选择编译该功能驱动。

2.menuconfig

```
menuconfig <symbol>
<config options>
```

此关键字是一个菜单名称，该菜单名下有一系列的配置选项，只有选中该菜单项后，才会呈现该菜单下的配置选项。例如下图，kernel/drivers/leds/Kconfig 中使用 if/endif 语句判断 NEW_LEDS 选项是否被选中，如果被选中，则会出现 LEDS_CLASS，LEDS_CLASS_FLASH 等配置选项。

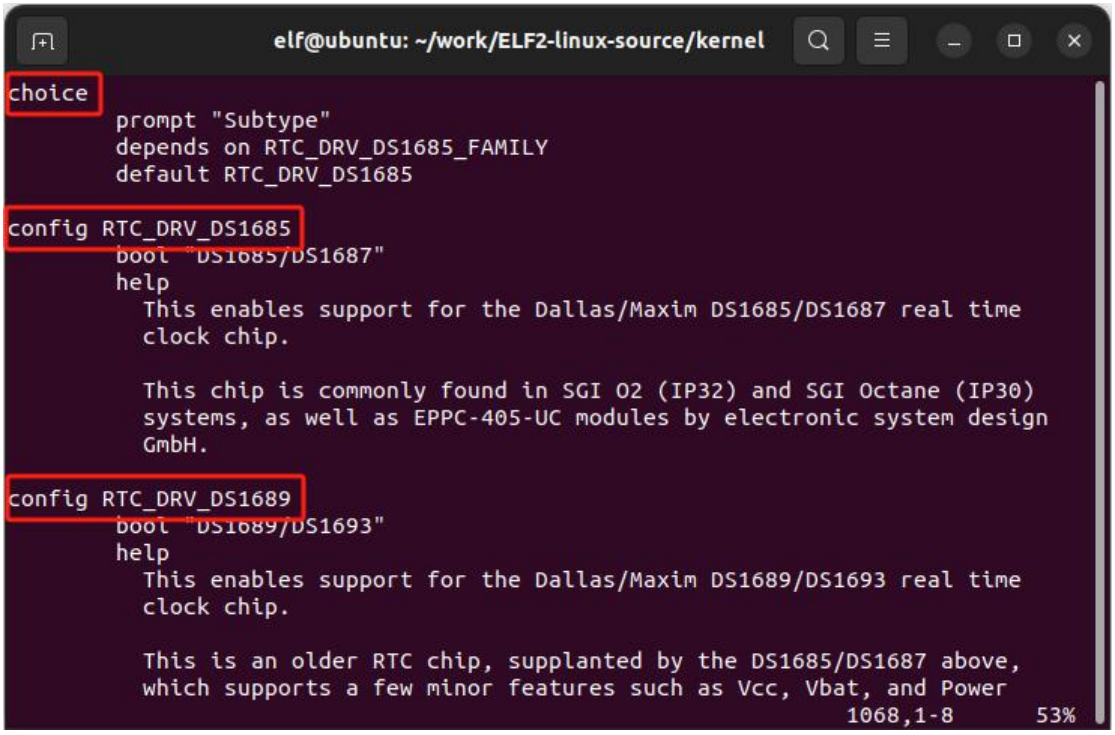


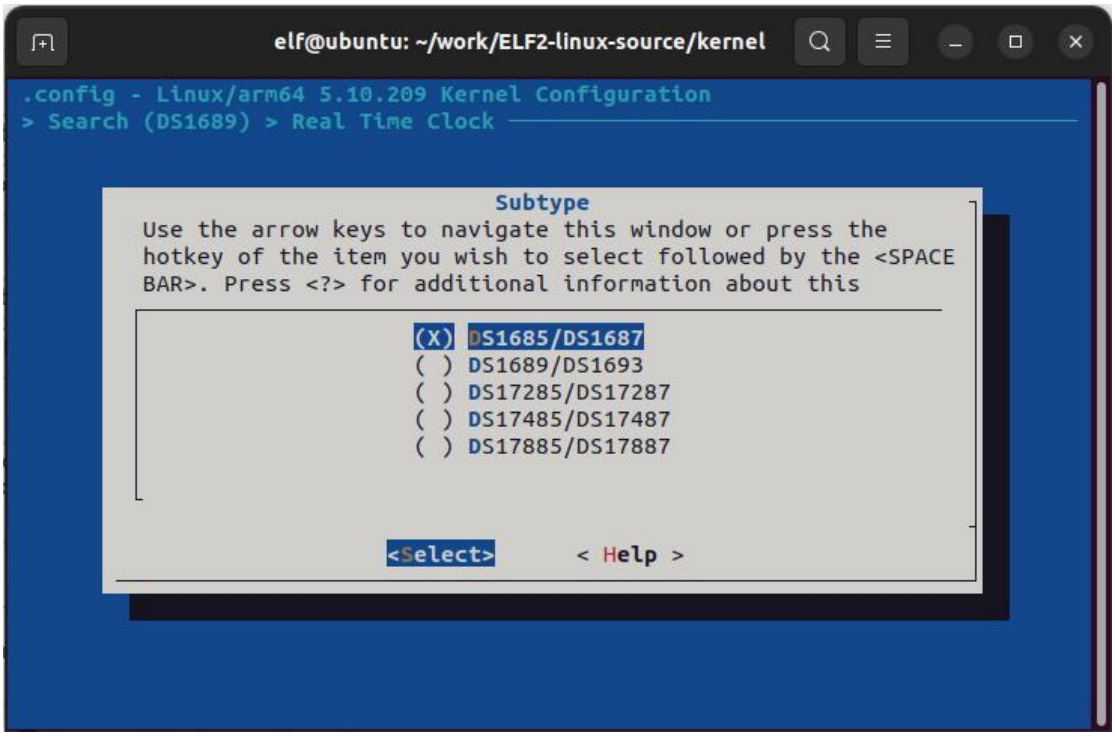


3.choices

```
choice
<choice options>
<choice block>
endchoice
```

该关键字定义了一组配置选择项，用户只能在这些配置项中选择一项。比如，RTC 支持多种型号时钟芯片，我们只能选择其中一种，kernel/drivers/rtc/Kconfig 中：



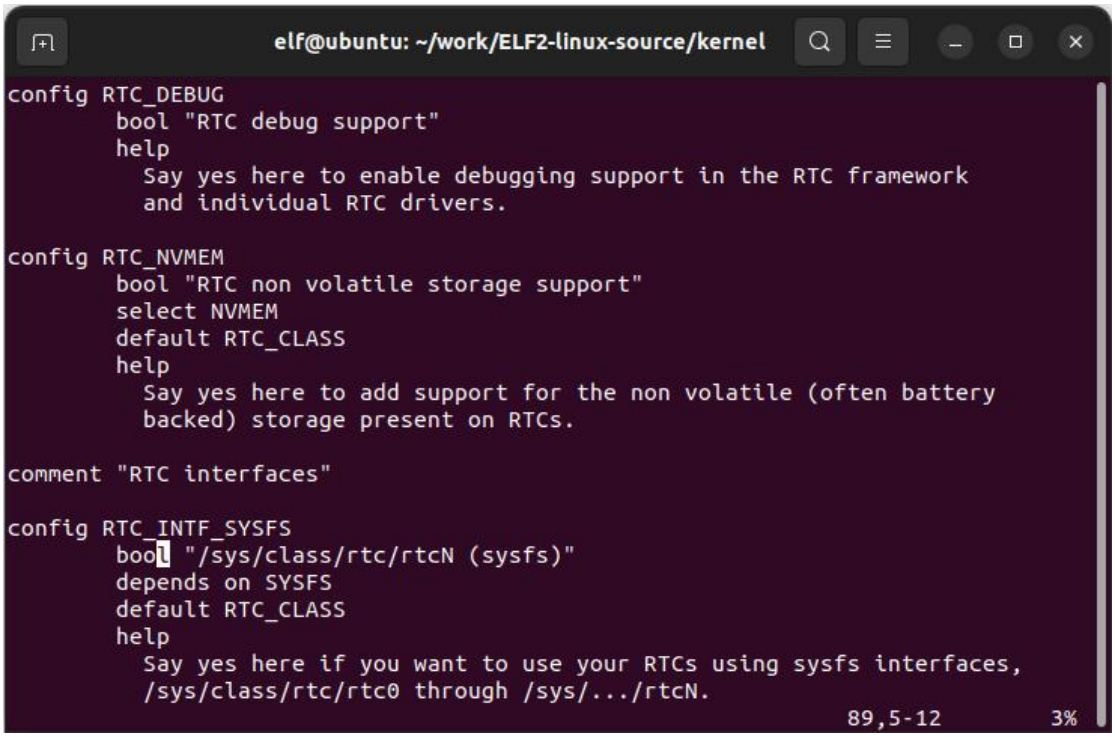


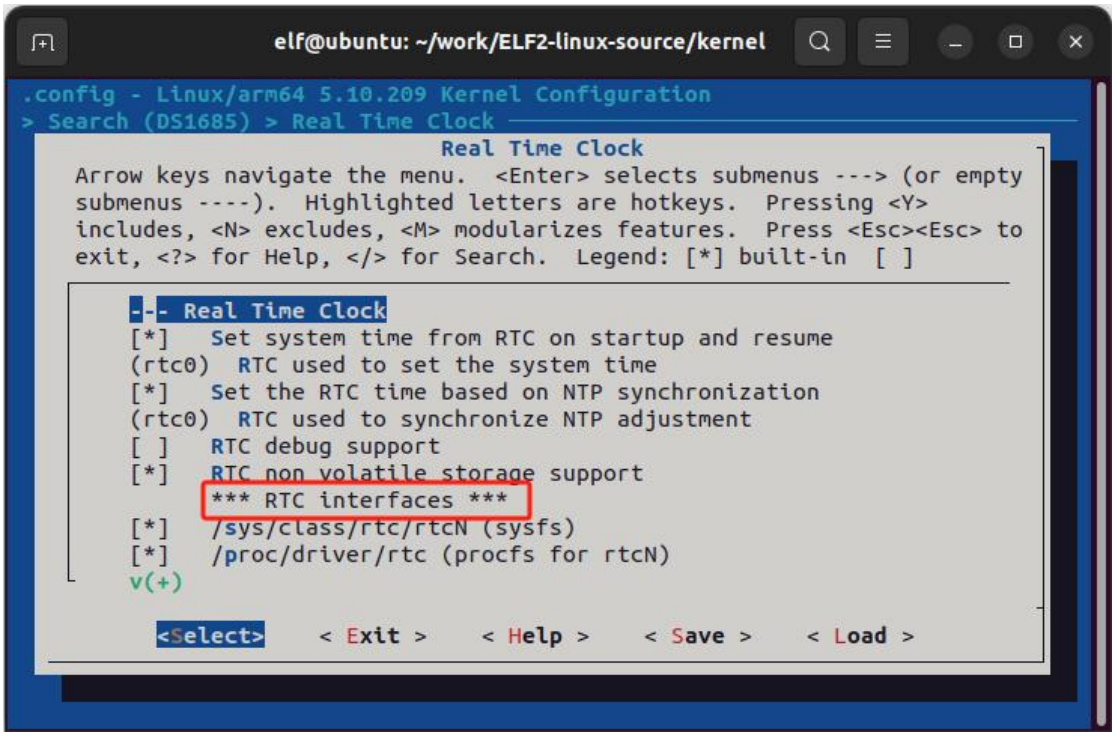
4.comment

```
comment <prompt>
<comment options>
```

这里定义了在配置过程中显示给用户的提示信息。

例如：



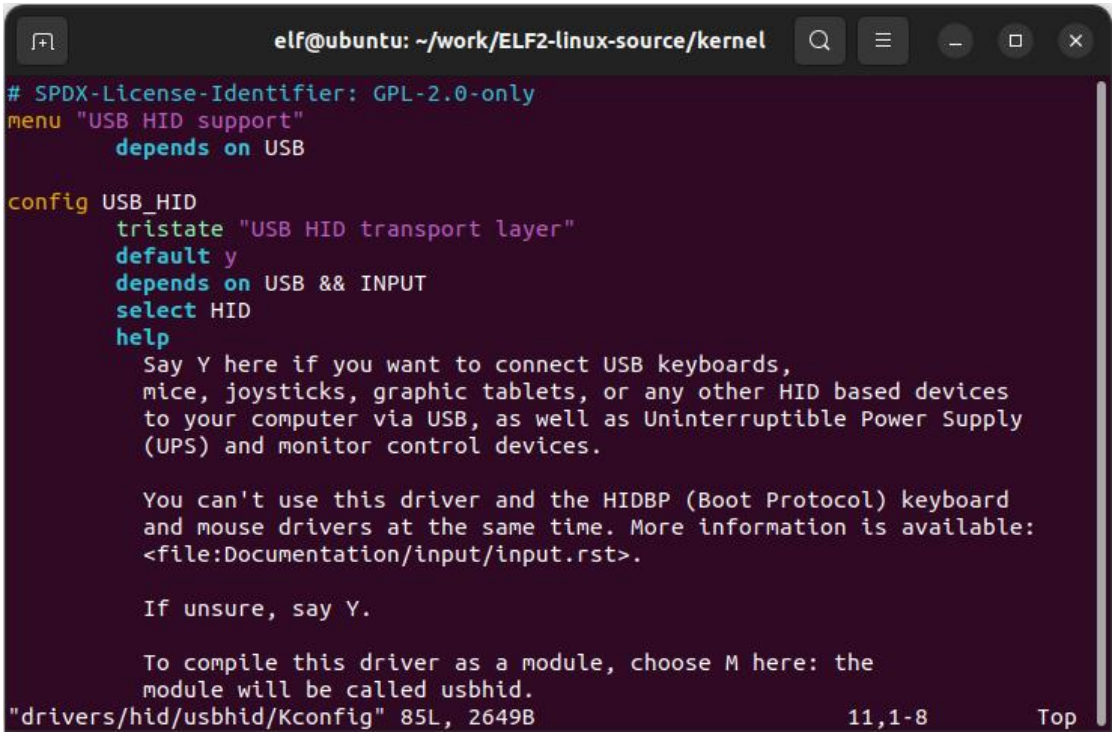


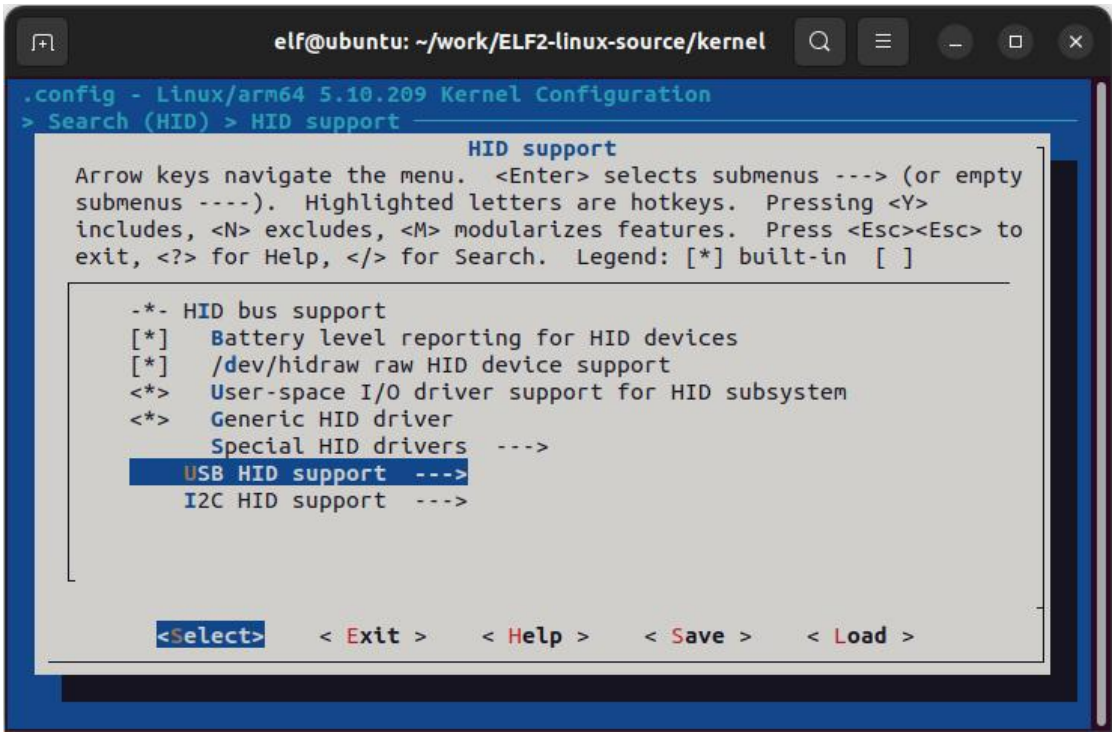
5.menu

```
menu <prompt>
<menu options>
<menu block>
endmenu
```

这里定义了一个简单的菜单名，与 menuconfig 关键字的菜单名不同，menuconfig 关键字的菜单名自身可配置，menu 关键字的菜单名不可配置。

如下图，kernel/drivers/hid/usbhid/Kconfig 中 menu 关键字之后是 USB HID support 菜单名。





6.source

source 用于引用读取其他目录中 Kconfig 文件，如：

```
menu " Device Drivers "
source " drivers/amba/Kconfig "
source " drivers/base/Kconfig "
source " drivers/bus/Kconfig "
source " drivers/connector/Kconfig "
source " drivers/mtd/Kconfig "
source " drivers/of/Kconfig "
source " drivers/parport / Kconfig "
source " drivers/pnp/Kconfig "
source " drivers/block/Kconfig "
```

1.7 初识设备树

前边讲解了内核编译选项的配置，这一节我们详细讲一下设备树。设备树的知识是 Linux 驱动开发人员必备的技能，我们现在使用的 Linux 内核，绝大多数都是支持设备树功能的，掌握设备树是软件嵌入式工程师必备的一项技能。

1.7.1 设备树的起源

设备树(Device Tree)是一种描述硬件资源的数据结构，它由 uboot 传递给 Linux 内核，被内核解析，内核根据设备树中的硬件描述信息加载利用相应驱动资源。在引入设备树之前，Linux 内核中充斥着大量的用于描述板级硬件信息的文件，拥有不同硬件资源的板卡 ，都有其对应的板级文件或者冗余文件，随着 Linux 内核支持的板卡越来越多，板级文件也越来越多。尤其是一些冗余的文件信息导致了内核的臃肿，为了解决这一问题 Linus Torvalds 在 2011 年 3 月 17 日的 ARM Linux 邮件列表宣称“this whole ARM thing is a fucking pain in the ass”，因此，ARM Linux 社区作出了回应，引入了设备树 Device Tree，这样一来许多硬件的细节可以直接透过设备树传递给 Linux 内核，而不再需要在内核中进行大量的冗余编码来适配

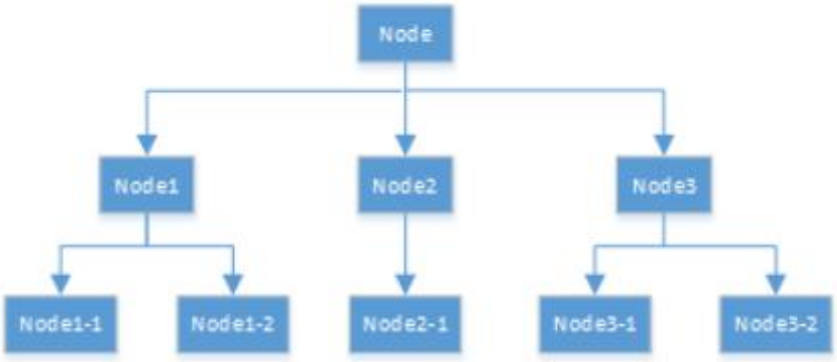
不同的板卡。

1.7.2 设备树组成和结构及 dts、dtb、dtsi

设备树 Device Tree 由一系列被命名的节点（node）和属性（property）组成，而节点本身可包含子节点。所谓属性，其实就是成对出现的 name 和 value。在设备树中，可描述的信息包括：

CPU 的数量和类别；
内存基地址和大小；
总线和桥；
外设连接；
中断控制器和中断使用情况；
GPIO 控制器和 GPIO 使用情况；
Clock 控制器和 Clock 使用情况；

这种以树状节点的方式描述一个设备的各种硬件信息：CPU、GPIO、时钟、中断、内存等，形成类似文本文件，很好的解决了这些问题。



它基本上就是画一棵电路板上 CPU、总线、设备组成的树，uboot 在启动的时候会将这棵树传递给内核，然后内核可以识别这棵树，并根据它展开出 Linux 内核中的 platform_device、i2c_client、spi_device 等设备，而这些设备用到的内存、IRQ 等资源，也被传递给了内核，内核会将这些资源绑定给展开的相应的设备。

通常由.dts 文件以文本方式对系统设备树进行描述，经过 Device Tree Compiler(dtc)将 dts 文件转换成二进制文件 binary device tree blob(dtb)，.dtb 文件可由 Linux 内核解析，有了 device tree 就可以在不改动 Linux 内核的情况下，对不同的平台实现无差异的支持，硬件有变动时不需要重新编译内核或驱动程序，只需更换相应的 dts 文件即可。dtsi 文件被 dts 文件包含引用，这类似于 C 语言 C 文件 include 包含.h 文件，对于同一个节点的设置情况，dts 中的配置会覆盖 dtsi 中的配置。

1.7.3 设备组织架构

下图是一个设备树文件的基本架构示例：

```
node@0 {
        a-string-property = "string";
        a-string-list-property = "first string", "second string";
        a-byte-data-property = [0x01 0x02 0x03 0x04];

        child-node@0 {
                first-child-property;
                second-child-property = <1>;
                a-reference-to-someting = <&node1>;
        };
        child-node@1 {
        };
};

node1: node@1 {
        an-empty-property;
        a-cell-property = <1 2 3 4>;

        child-node@0 {
        };
```

简单概括一下有这几个部分：

1.节点

节点的命名方式一般为 node-name@unit-address：

其中 node-name 为节点名称，unit-address 为节点地址。例如上示例图中 node@0 的 node 为节点名称；0 就是节点的地址，地址主要是为了区分其他的节点以保证节点的唯一性，节点地址是非必须的，可以省略。

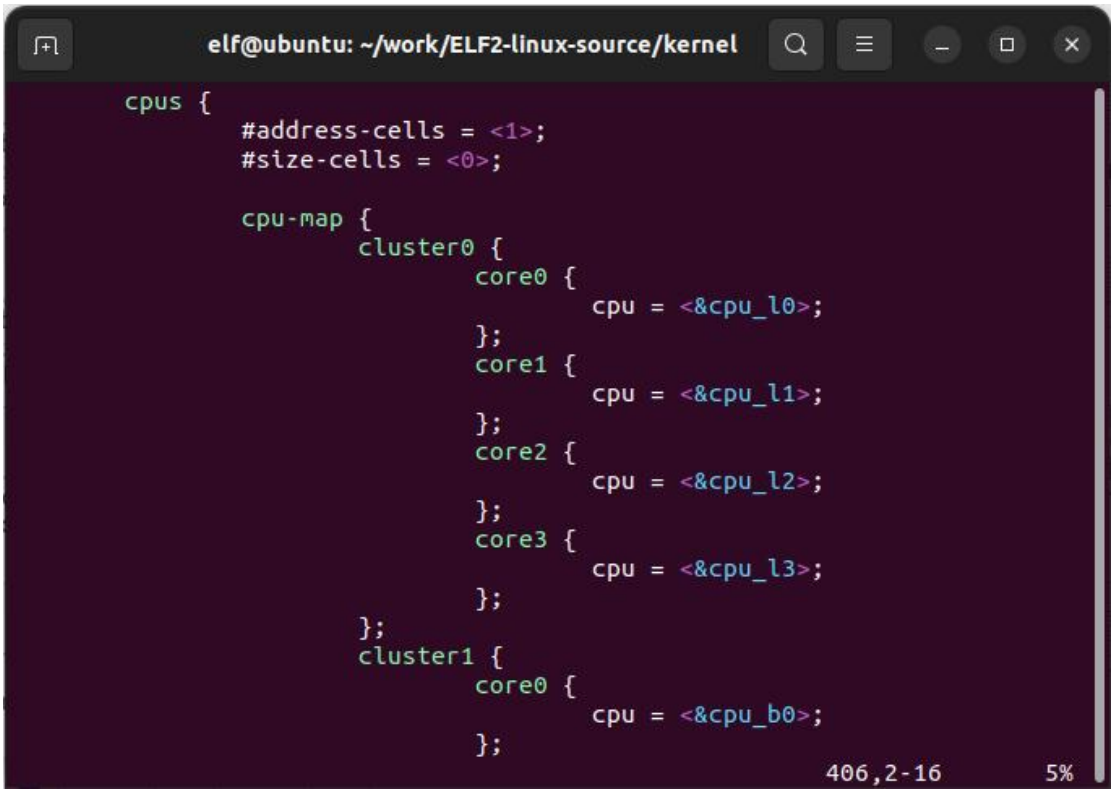
在一个节点下面可包含一系列的子节点，例如上示例图的 note@0 下面又可包含其自己的子节点 child-node@0 和 child-node@1。

下面是几种常见的节点：

1）根节点：设备树文件中都包含一个根节点，使用“/”表示。

2）CPU 节点

下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/rk3588s.dtsi



2.属性

每个节点都有不同属性，属性包括属性名称和属性值，属性值可以为空或任意的字节流属性，一般属性值有三类：

string 字符串型，使用双引号“”包含；

byte 数组型：使用方括号[]包含；

u32 型：使用尖括号<>包含；

如示例图中节点

node@0 的第一个属性名称是 a-string-property,属性值为字符串“string”；

node@0 的第三个属性名称为 a-byte-data-property，其属性值为 byte 数组：[0x01 0x02 0x03 0x04]，

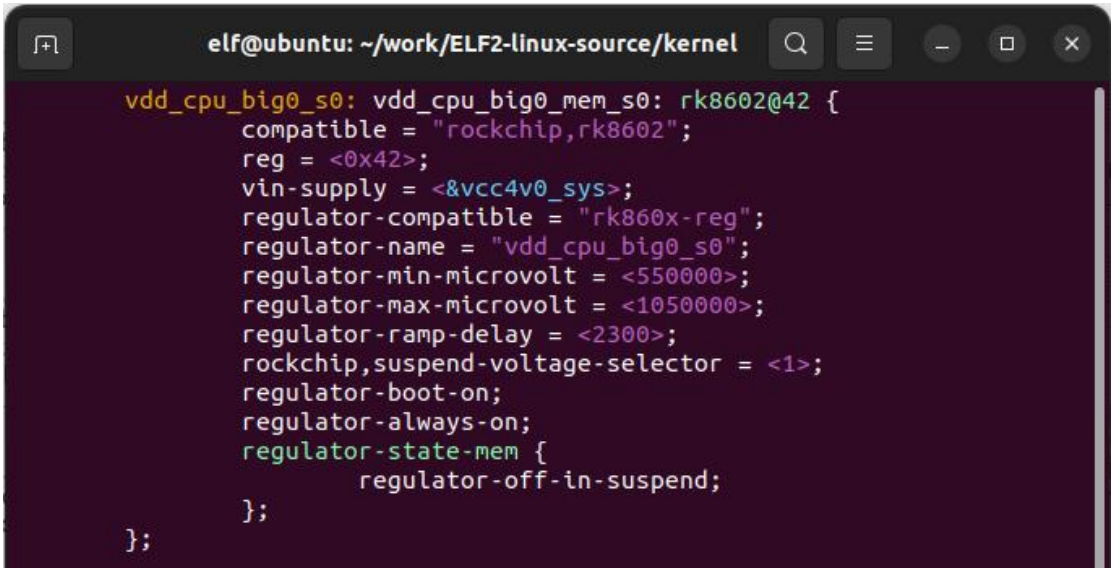
node1(node@1)节点的 a-cell-property 属性值为 u32 型<1 2 3 4>。

常见的属性：

1）compatible

compatible 是兼容的意思，即该设备可被一个或者多个驱动匹配。例如在

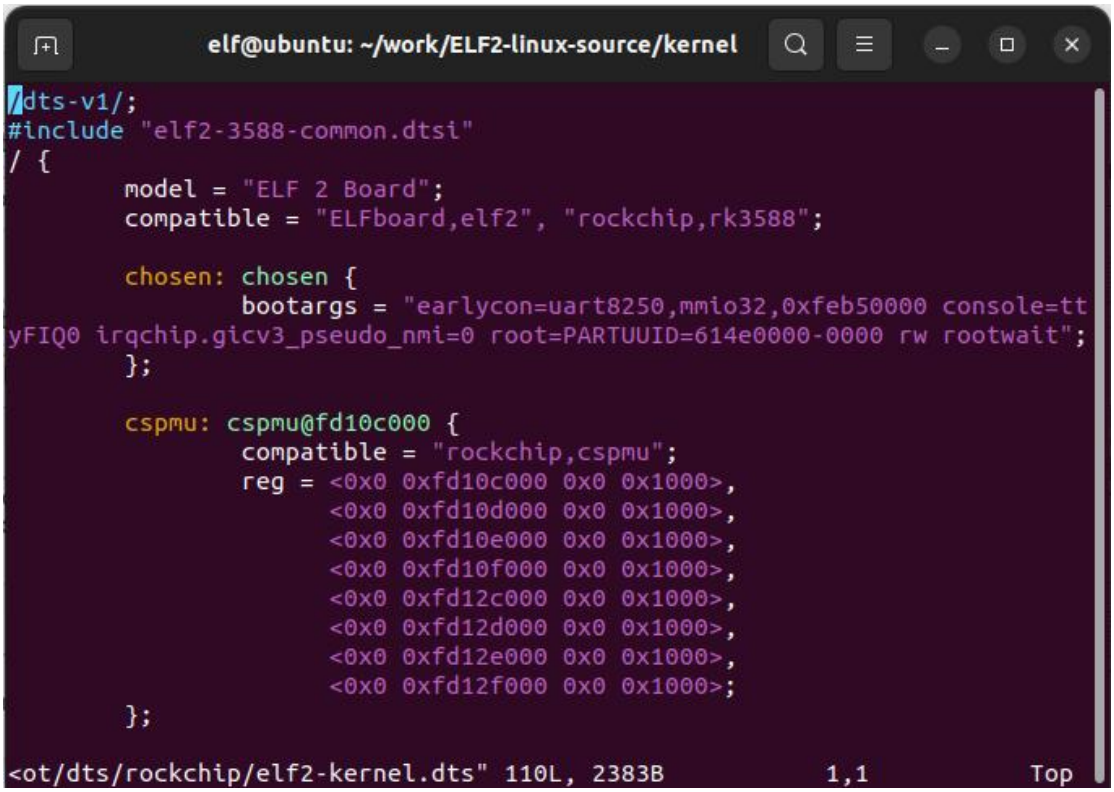
下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/elf2-3588.dtsi，vdd_cpu_big0_s0 节点：



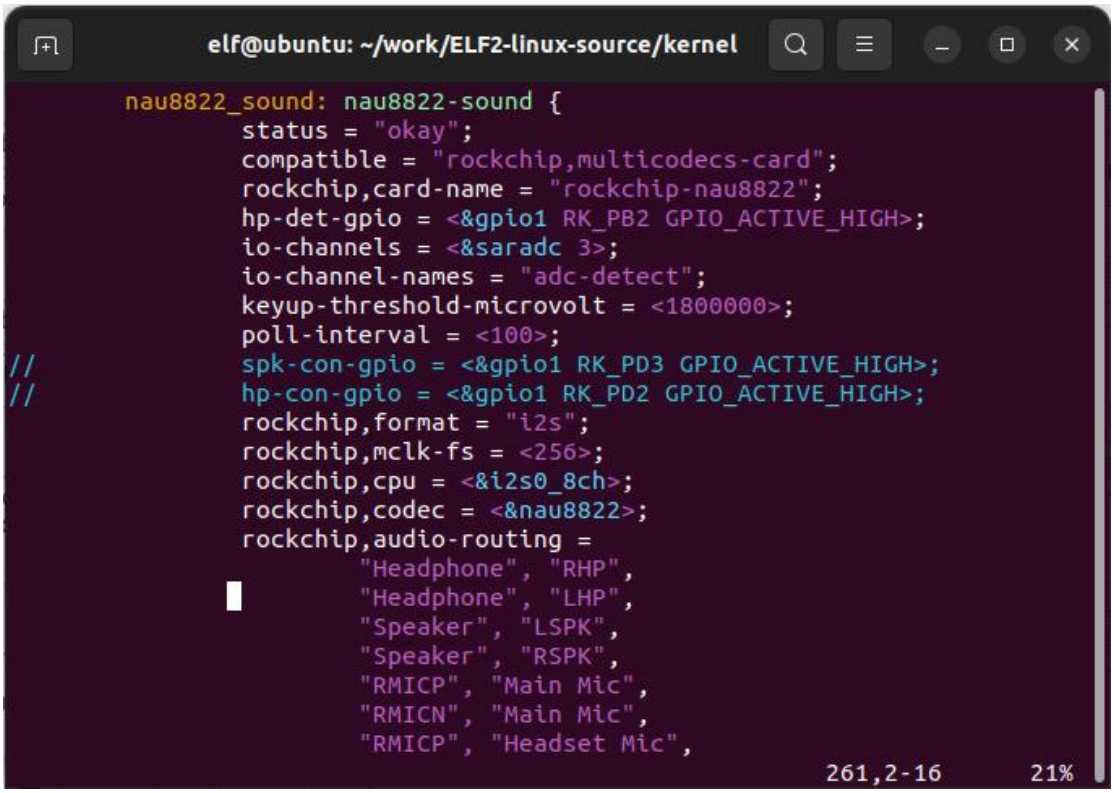
此节点的 compatible 的值为"rockchip,rk8602",内核在解析设备树之后，会首先根据“rockchip,rk8602”查找匹配驱动模块，其中“rockchip”是厂商名称，“rk8602”是驱动模块的名称。

2）model

model 属性用于表示设备型号信息。下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/elf2-kernel.dts，如 model = "ELF 2 Board"，表示该设备为 ELF 的 2 开发板。



下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/elf2-kernel.dts，如 sound 节点，设备型号信息为 rockchip-nau8822：



3）#address-cells，#size-cells，reg

reg 属性用来表示节点地址资源的，属性值一般是寄存器的起始地址及地址之后连续的空间长度。

#address-cells，#size-cells 分别表示子节点中 reg 的地址以及地址之后连续的空间长度用几个 32bit 数据表示。

下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi



如上图，#address-cells=<1>,表示其子节点 altmode 中 reg 的地址是用一个 32bit 数据表示，上图中的 0。#size-cells=<0>,说明其子节点 ocrams 中 reg 地址之后的空间长度是 0。

4）status

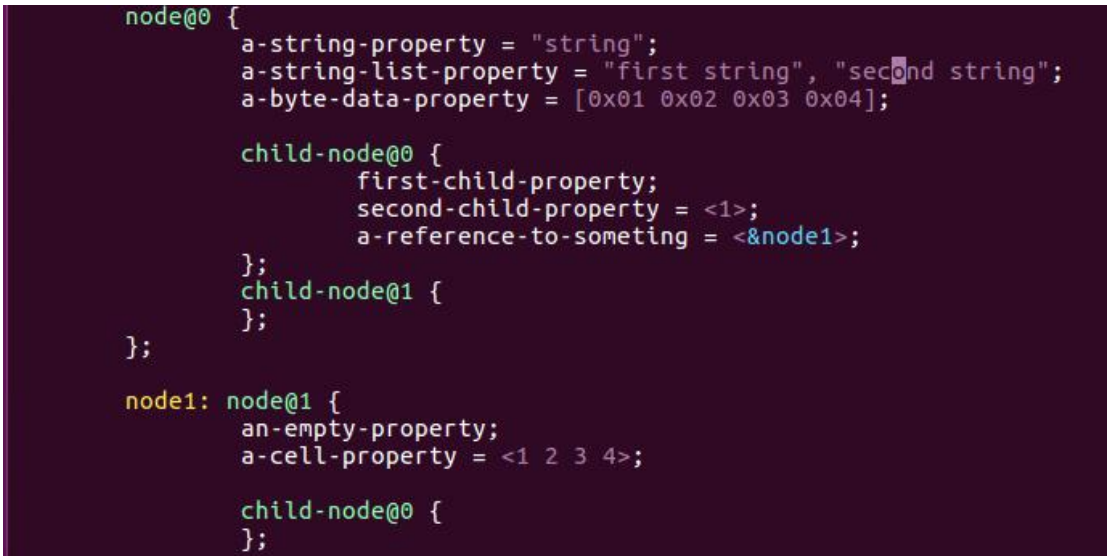
status 属性用来表示节点的状态，即相关硬件的状态，用字符串表示。“okay”表示硬件正常工作，“disabled”表示硬件当前不可用，“fail”表示因为出错不可用，“fail-sss”表示因为某种原因出错不可用，sss 表示具体的出错原因。实际中，基本只用“okay”和“disabled”。

下图查看的设备树文件路径：arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi



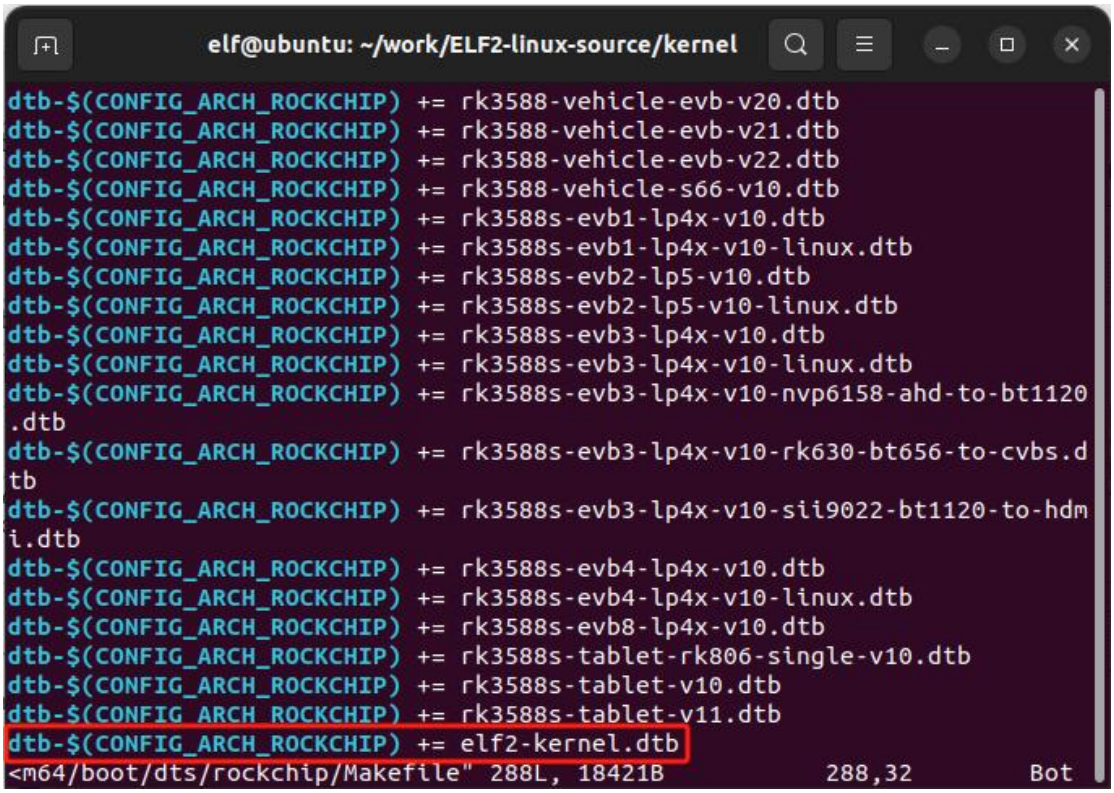
3.标签

图中的 node1： node@1 节点，其中 node1 就是本节点的一个标签，标签不是必须的，定义标签主要为了方便在其他地方对该节点进行引用的时候，不必写该节点的全路径，直接使用标签即可。如在 node@0 的子节点 child-node@0 的第三个属性就引用了 node1（&node1）。



1.7.4 设备树 Makefile 修改

在 arch/arm64/boot/dts/rockchip/目录下会有很多.dts，通过配置 Makefile 文件可以对这些 dts 文件进行选择性编译，打开 arch/arm64/boot/dts/rockchip/Makefile：



如果我们因为板子硬件改动或硬件外设配置不同而新增加了 dts，则需要在这个 Makefile 的这个位置添加上对应的.dtb 文件名，参与编译。

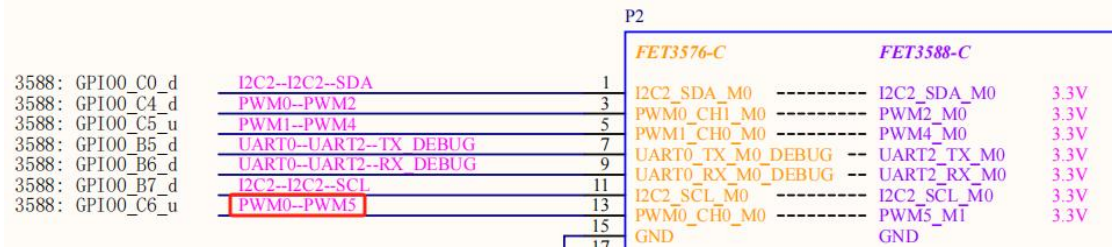
1.7.5 开发板使用的设备树

ELF2 使用的设备树命名为 elf2-kernel.dts。

1.8 IOMUX 介绍

IOMUX 是指引脚功能复用功能配置，MUX 是指 multiplex 多路复用的意思，由于芯片硬件引脚资源有限，但又想实现尽可能多的功能，于是芯片厂商，就实现了一个引脚可复用为多路功能的设计。不同厂家的 SOC 芯片有不同的配置方法，下面我们讲 rk3588 的引脚功能复用的配置方法。

下边以背光引脚为例进行讲解。



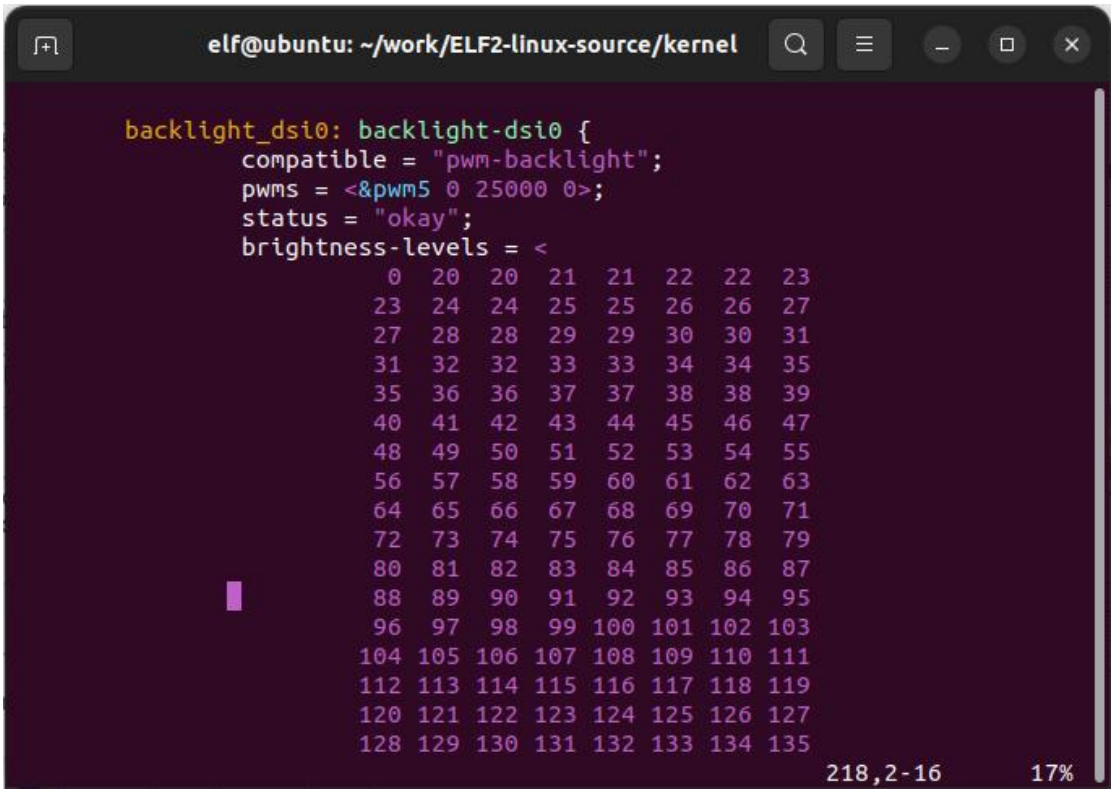
从原理图中我们可以看出屏幕背光用的是核心板上的 P2 的 13 引脚。

接下来我们查看硬件资料中的引脚复用表：

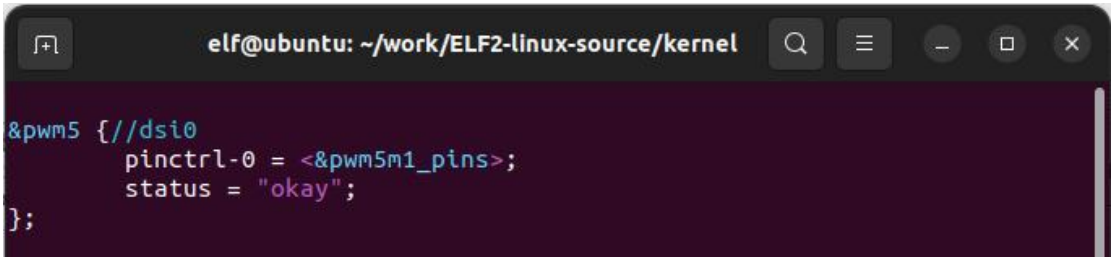


连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7	Alt8	Alt9
P2_13	NPU_AVS		SPI0_CLK_M0	PCIE30X4_CLKREQN_M0	I2S1_SDI1_M1		SATA_CP_POD	UART0_RTSN	PWM5_M1

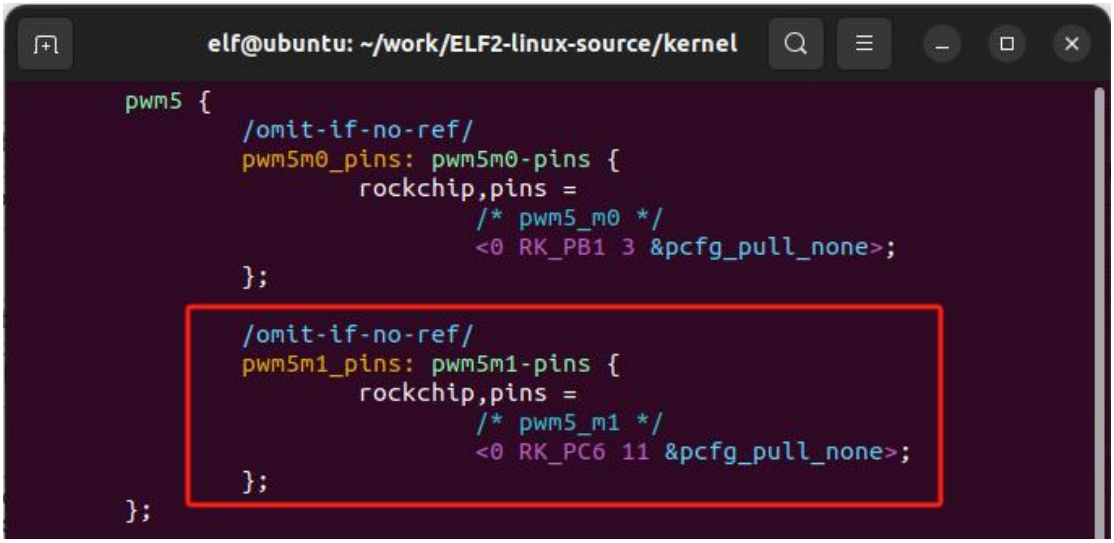
能够看出 P2 的 13 引脚可以复用成 GPIO0_C6，SPI0_CLK，PWM5，UART0_RTSN 等功能。与软件中是如何对应的呢，打开源码中 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi，其中背光节点 backlight 节点引用了 pwm5 标签。



接下来搜索 pwm5 标签，pinctrl-0 属性中定义了引脚的复用状态：



进一步搜索 pwm5m1_pins 标签(arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi)：



其中 rockchip,pins 描述的引脚复用及电气属性配置。“0 RK_PC6”描述的是引脚名称，说明使用的引脚是 GPIO0_PC6，“11”描述的是将此引脚复用成 pwm5 的功能。

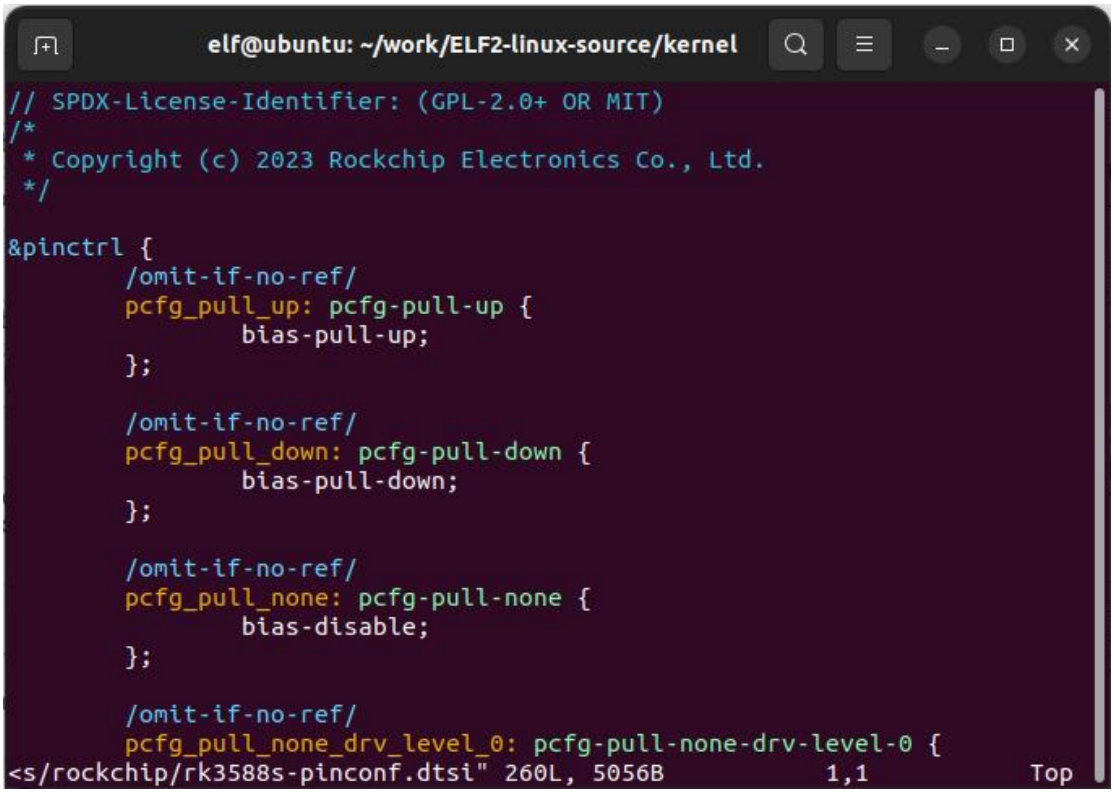
下表在（ELF 2开发板资料包\06-RK原厂资料）的《Rockchip RK3588 TRM V1.0-Part1-20220309.pdf》

中第 6.26.2 章节中找到：

Bit	Attr	Reset Value	Description
15:12	RW	0x0	gpio0c7_sel 4'h8: SPI0_MISO_M0 4'h9: I2C6_SDA_M0 4'ha: UART1_RTSN_M2 4'hb: PWM6_M0 4'hc: PCIE30X4_WAKEN_M0
11:8	RW	0x0	gpio0c6_sel 4'h8: SPI0_CLK_M0 4'hb: PWM5_M1 4'hc: PCIE30X4_CLKREQN_M0 4'hd: SATA_CP_POD

“&pcfg_pull_none”描述的是没有配置上下拉的功能。

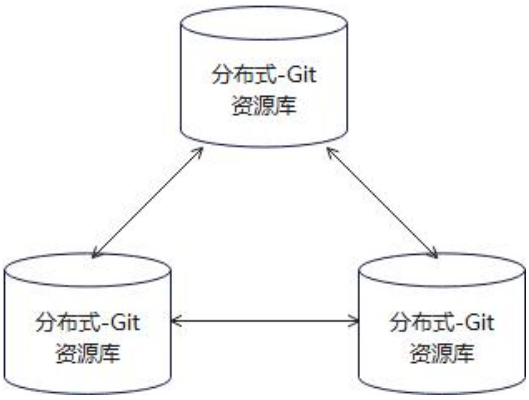
在 arch/arm64/boot/dts/rockchip/rk3588s-pinconf.dtsi 中可以看到还有其他的配置属性，包括上拉，下拉还有引脚驱动能力的配置。



1.9 git 管理源码

1.9.1 git 是什么？

git 是一个开源的分布式版本控制系统，可以有效、高速地处理从很小到非常大的项目版本管理，也是 Linus Torvalds 为了帮助管理 Linux 内核开发而开发的一个开放源码的版本控制软件。git 与常用的版本控制工具 SVN 等不同，它采用了分布式版本库的方式，不必服务器端软件支持。



简而言之，git 就是一个版本管理库，也是一个版本管理工具，它的作用就是帮助我们记录版本信息，以及修改内容。git 的结构是分布式的资源库，特点是没有严格的服务器概念，每个单体都可作为资源库。这个特点就让我们人人有自己的 git 仓库这件事变得容易实现。

1.9.2 为什么要用 git?

提到版本控制，总会想起大学毕业时写论文的场景，你电脑上的毕业论文一定出现过这番景象！

```
课设_初稿.doc
课设_修改 1.doc
课设_修改 2.doc
课设_修改 3.doc
课设_完整版 1.doc
课设_完整版 2.doc
课设_完整版 3.doc
课设_最终版 1.doc
课设_最终版 2.doc
课设_死也不改版.doc
...
```

这就是最原始的版本控制方法，这种方式的缺点也是显著的。

- 多个文件，保留所有版本时，需要为每个版本保存一个文件。
- 协同操作，多人协同操作时，需要将文件打包发来发去。
- 容易丢失，被删除意味着永远失去...

为了解决这个痛点，应运而生了一批版本管理工具，如：git、svn 等。其中 linux 当中最常用的当属 git，因此学会使用 git 是十分必要的。

1.9.3 git 安装和初次使用

git 仓库的构建分两种方式，一种是管理当前的代码，给当前代码创建一个 git 仓库，这种需要初始化新仓库；还有一种是用 clone 克隆一个远程仓库到自己当前目录下，这种适合一群人维护一个仓库，也是常见的方式。

本文只介绍第一种当前代码的管理。

1.Ubuntu 上安装 git

在/home/elf/work 目录下新建 example 目录，进入 example 目录，新建 hello 目录。

```
elf@ubuntu:~/work$ mkdir example
elf@ubuntu:~/work$ cd example
elf@ubuntu:~/work/example$ mkdir hello
```

```
elf@ubuntu:~/work/example$ cd hello
```

执行 sudo apt-get install git 进行 git 安装。

```
elf@ubuntu:~/work/example/hello$ sudo apt-get install git
```

2.创建版本库

在/home/elf/work/example/hello/目录下创建一个新的 hello.c 文件并添加以下内容：

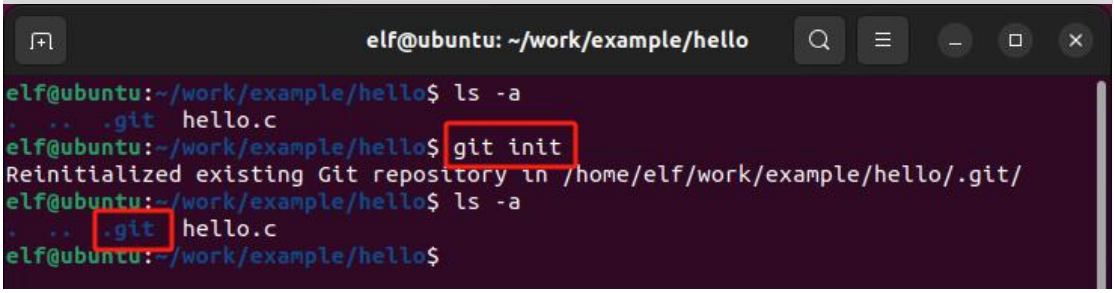
```
elf@ubuntu:~/work/example/hello$ vi hello.c
#include <stdio.h>
int main(void)
{
        printf("Hello World!\n");
        return 0;
}
```

以下以用 git 管理/home/elf/work/example/hello/hello.c 文件为示例，展示 git 的安装和最简单的流程。

1）git 初始化

git 初始化新仓库，执行 git init 后会在当前目录生成一个.git 目录。

```
elf@ubuntu:~/work/example/hello$ git init
```



2）设置用户名和邮箱

每当我们在一个新的 git 上，首先执行 git config --global user.name "Your Name"创建个人 git 账户，执行 git config --global user.email 创建账户邮箱。

以账户名为 elf_li，邮箱 elf_li@exmple.com 为例操作。

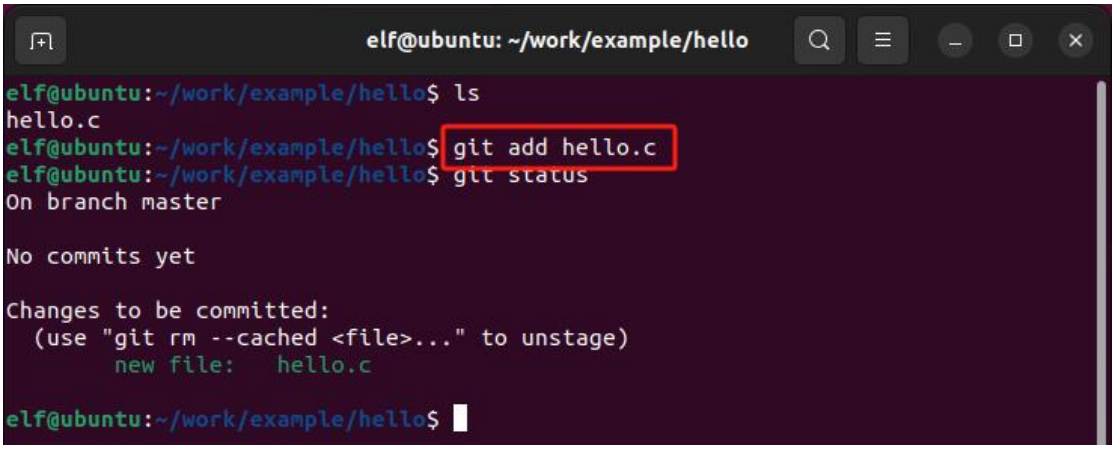
```
elf@ubuntu:~/work/example/hello$ git config --global user.name "elf_li"
elf@ubuntu:~/work/example/hello$ git config --global user.email "elf_li@exmple.com"
```

3）将当前需要跟踪的文件加到 git 仓库（将文件添加到暂存区域），执行 git add [filename]，以添加 hello.c 到仓库暂存区为例。

```
elf@ubuntu:~/work/example/hello$ git add hello.c
```

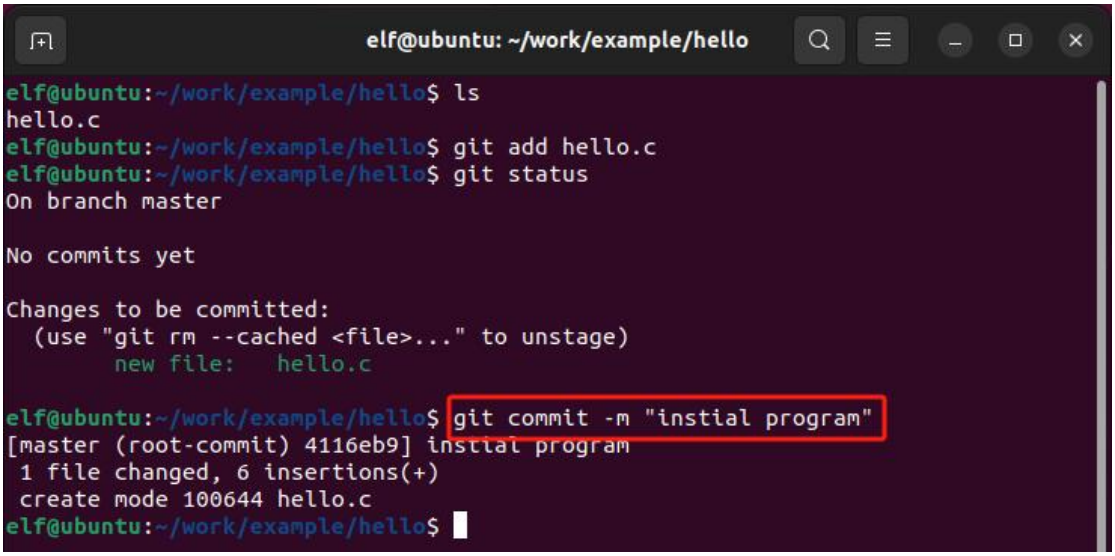
4）提交版本

提交暂存区域的所有修改到本地仓库（如果修改文件后未添加到暂存区，将不会被提交），用 git status 指令（在分支内操作时会做具体的讲解）可以看到相关信息，提示需要提交。

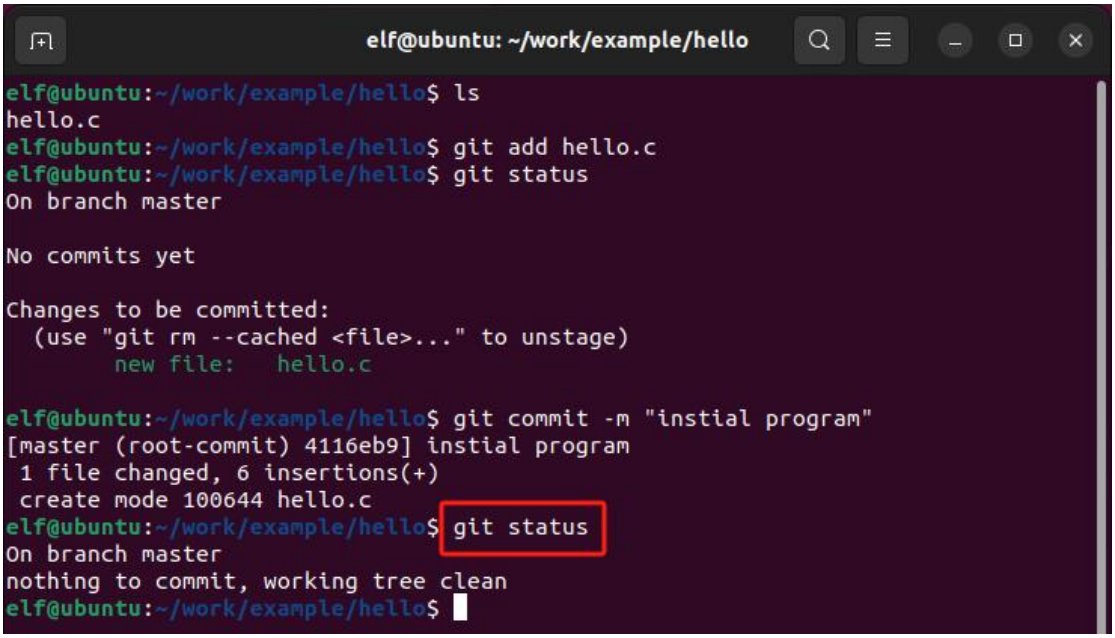


执行 git commit -m “xxx” 进行提交。

```
elf@ubuntu:~/work/example/hello$ git commit -m "instial program"
```



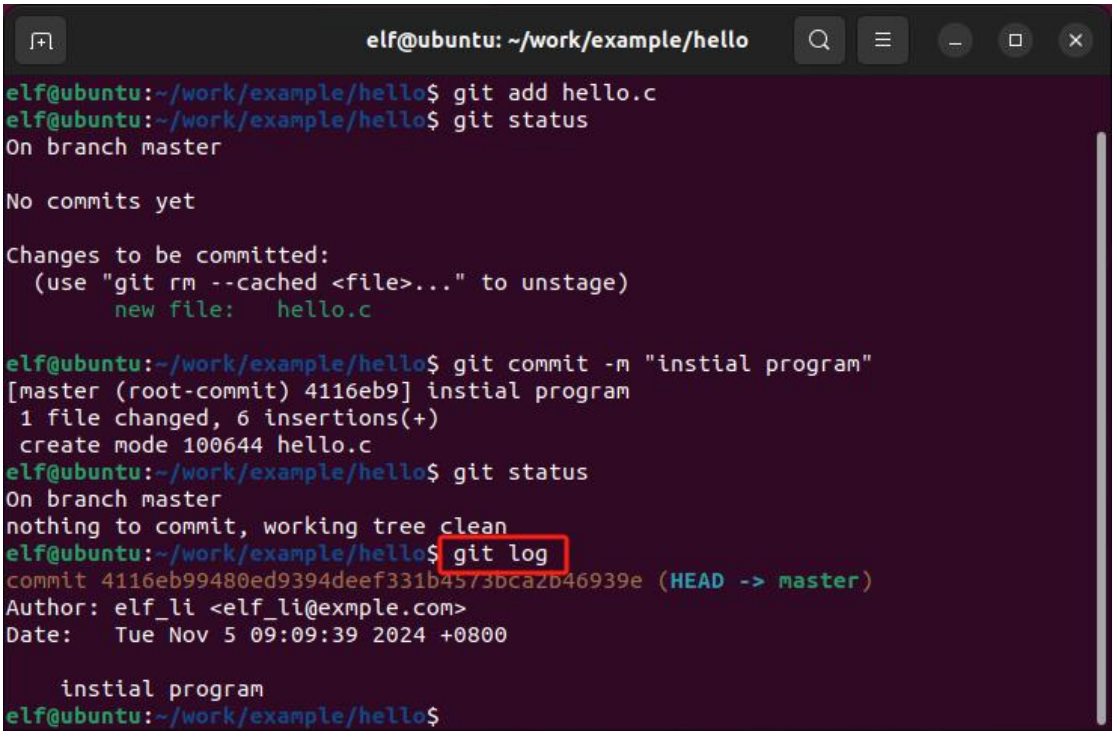
此时用 git status 查看，已经没有 hello.c 的提示了。



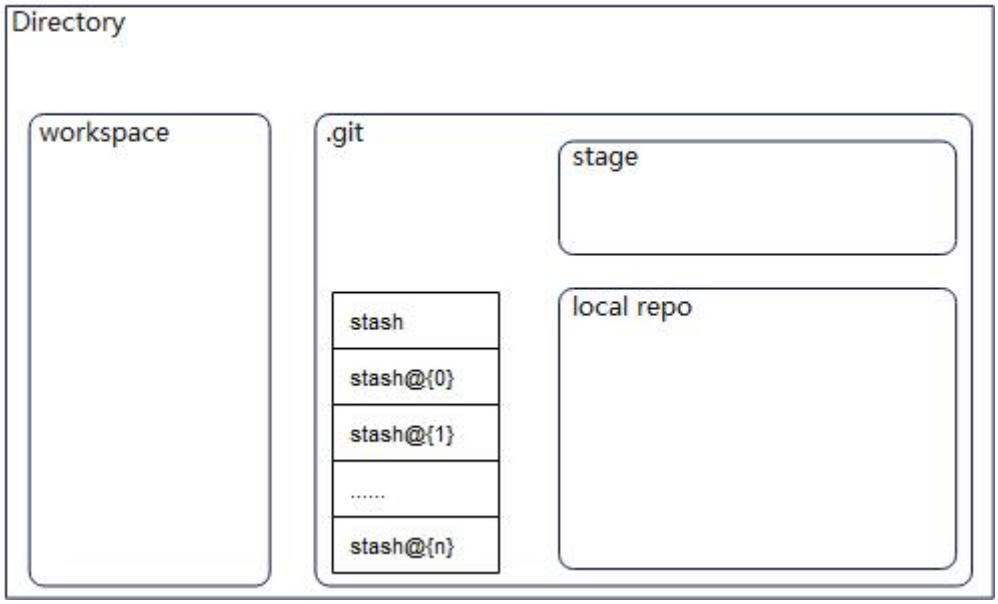
5）查看当前提交信息

有过提交操作后，可以使用 git log 查看提交记录信息。

```
elf@ubuntu:~/work/example/hello$ git log
```



完成以上操作，已经构建了一个本地资源库，主要包括以下部分：

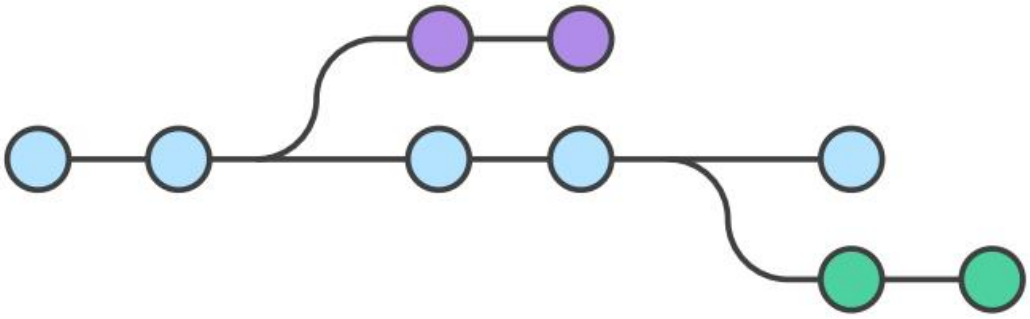


内容	说明
Directory	使用 git 管理的目录，包含工作空间和 git 管理空间。 即：/home/elf//work/example/hello 目录
workspace	工作区，平时存放项目代码的地方
.git	存放 git 管理信息的目录，git 初始化时创建
staging	暂存区，用于存放临时改动，事实上它只是一个文件，保存即将提交到文

	件列表信息
local repo	本地版本库，是安全存放数据的位置，含有提交的所有版本的数据。其中 HEAD 指向最新放入仓库的版本。
Stash	隐藏，是一个工作状态保存栈，用于保存/恢复 WorkSpace 中的临时状态

1.9.4 本地仓库管理

git 分支管理模型是开发过程中非常重要的一部分，可以防止协同人员无规则的随便提交。一个分支代表一条独立的开发线。使用分支意味着你可以从开发主线上分离开来，然后在不影响主线的同时继续工作。

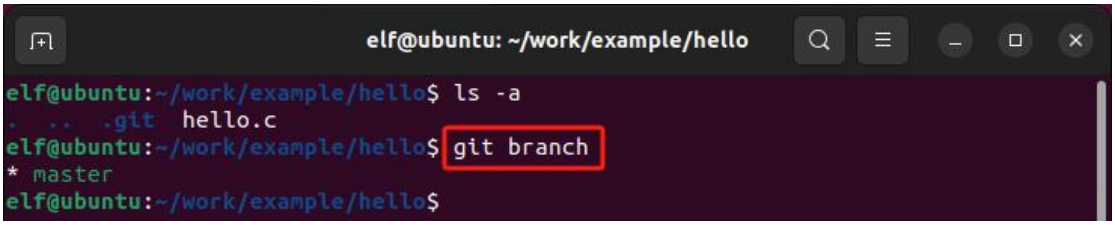


1.9.4.1 当前分支内的操作

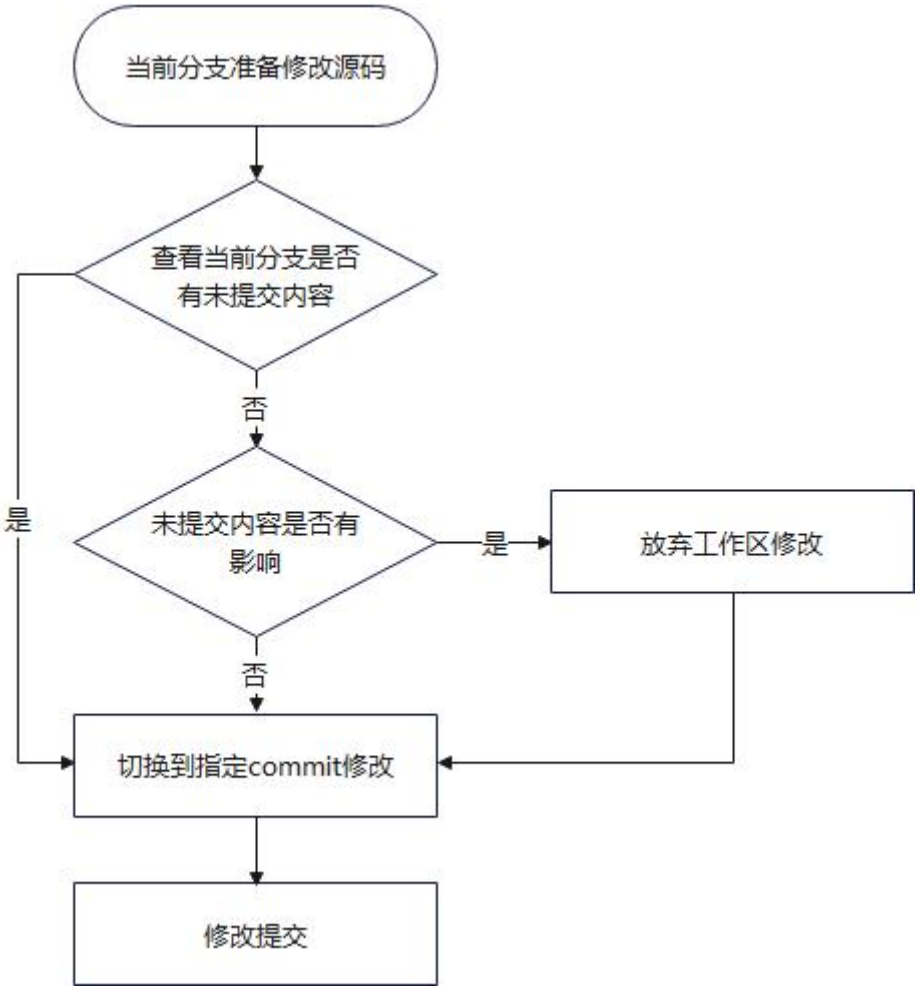
以刚搭建好的 git 仓库为例，刚搭建完的仓库只有 master 分支，使用 git branch 查看当前的分支情况，

```
elf@ubuntu:~/work/example/hello$ git branch
```

*所在分支为当前分支，即 master 分支



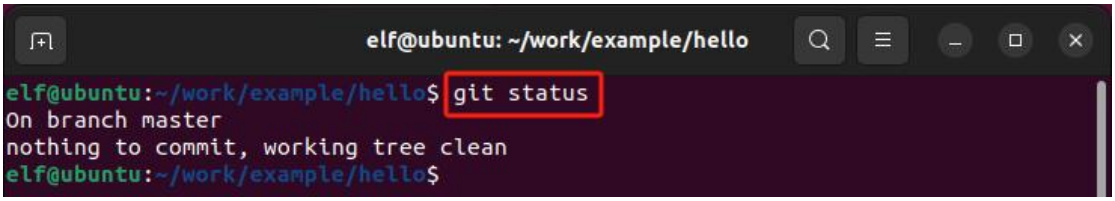
当前分支进行源码修改时简单流程图如下：



在当前分支上进行源码修改前，需要先查看当前分支工作区是否干净，干净则进行修改工作。如果之前在分支上做过修改且没有提交，需要进行清除，因此也需要养成改完及时提交的习惯。

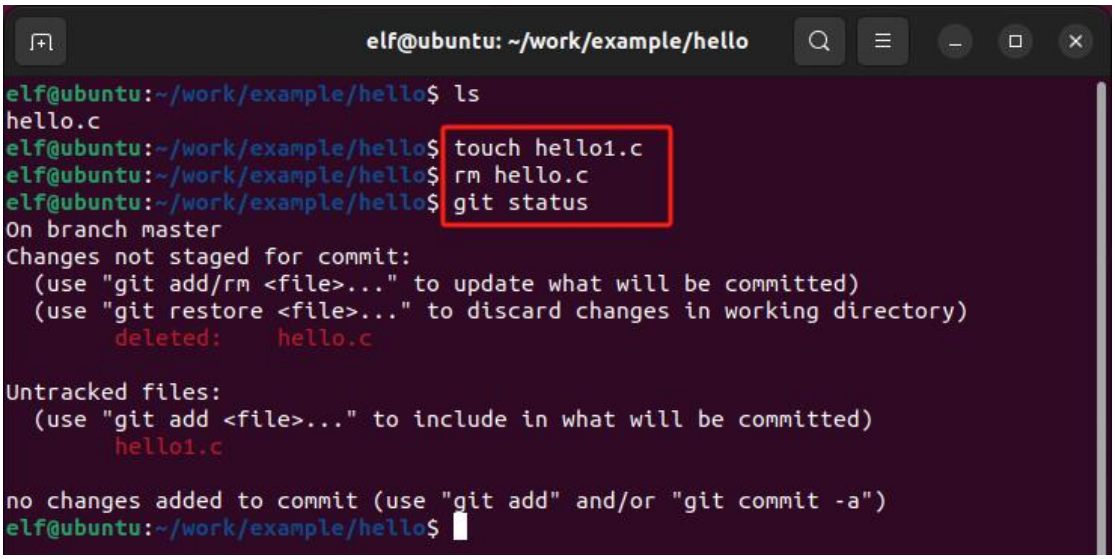
使用 git status 查看，该分支内没有未提交的内容，工作区是干净的，可以直接进行源码修改

```
elf@ubuntu:~/work/example/hello$ git status
```



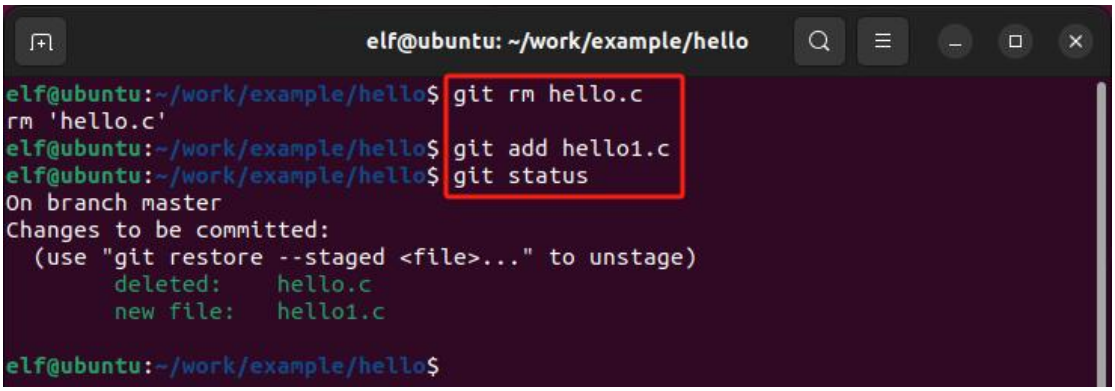
1.git 文件的添加、删除及提交

创建 hello1.c，删除 hello.c，此时是在工作区进行修改，并未提交到暂存区，使用 git status 查看工作区状态：



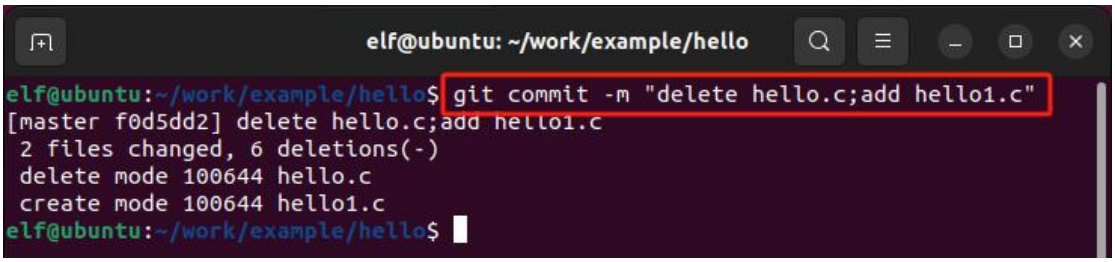
从状态中可以看到当前分支，未提交的更改为 hello.c 且前面有一个 delete 说明，hello.c 需要在 git 跟踪中被删除;还可以看到未被跟踪的文件为 hello1.c，该文件是新添加的，若需要添加到跟踪中，需要使用 git add 添加 ，操作如下

```
elf@ubuntu:~/work/example/hello$ git rm hello.c
elf@ubuntu:~/work/example/hello$ git add hello1.c
```



通过状态信息可看到，修改提交到暂存区，如果该修改最终要使用，需要提交版本。

```
elf@ubuntu:~/work/example/hello$ git commit -m "delete hello.c;add hello1.c"
```



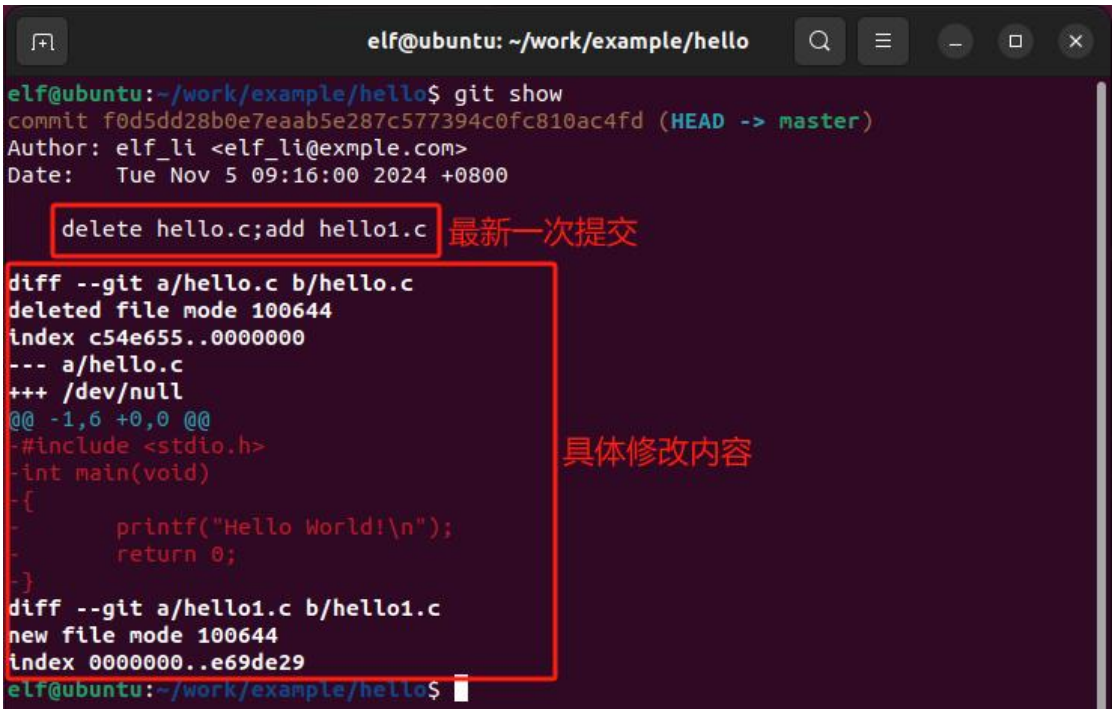
2.记录的查看

如果查看提交的修改，可以使用以下指令：

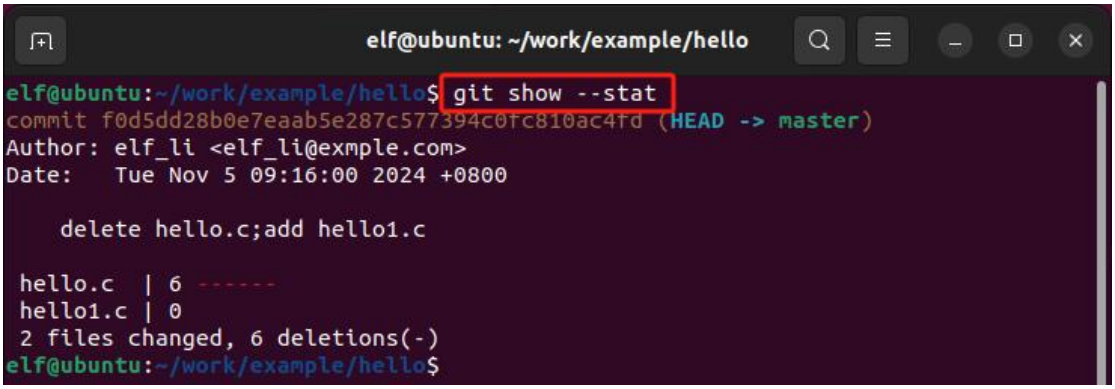
指令	说明
git show	查看最新一次提交的具体修改内容
git show [commitid]	查看某次提交的具体修改内容
git show --stat	查看最新一次提交修改过的文件

git show [commitid] --stat	查看某次提交修改过的文件
git show [filename]	查看最新一次提交的某个文件的更改内容
git show [commitid] [filename]	查看某次提交的某个文件的更改内容

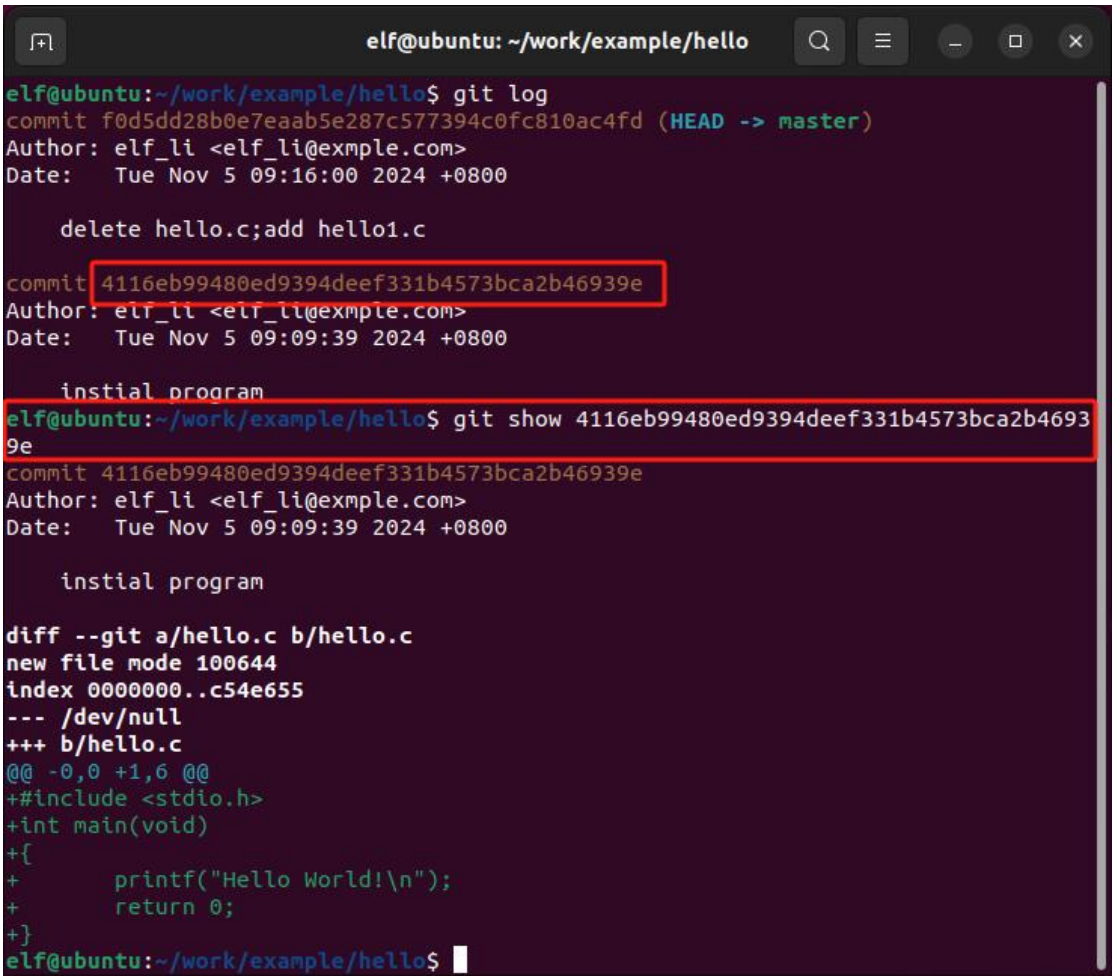
git show 查看最新一次提交的具体修改内容：



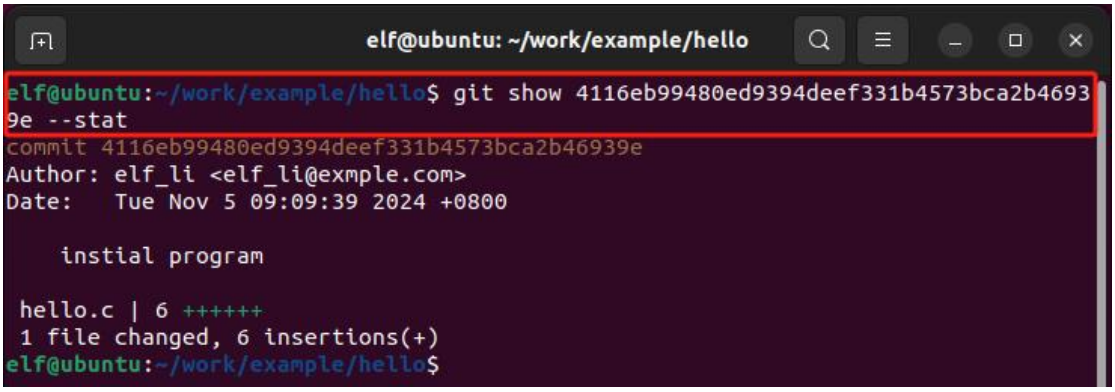
git show --stat 查看最新一次提交修改过的文件：



git show [commitid]查看某次提交的具体修改内容：

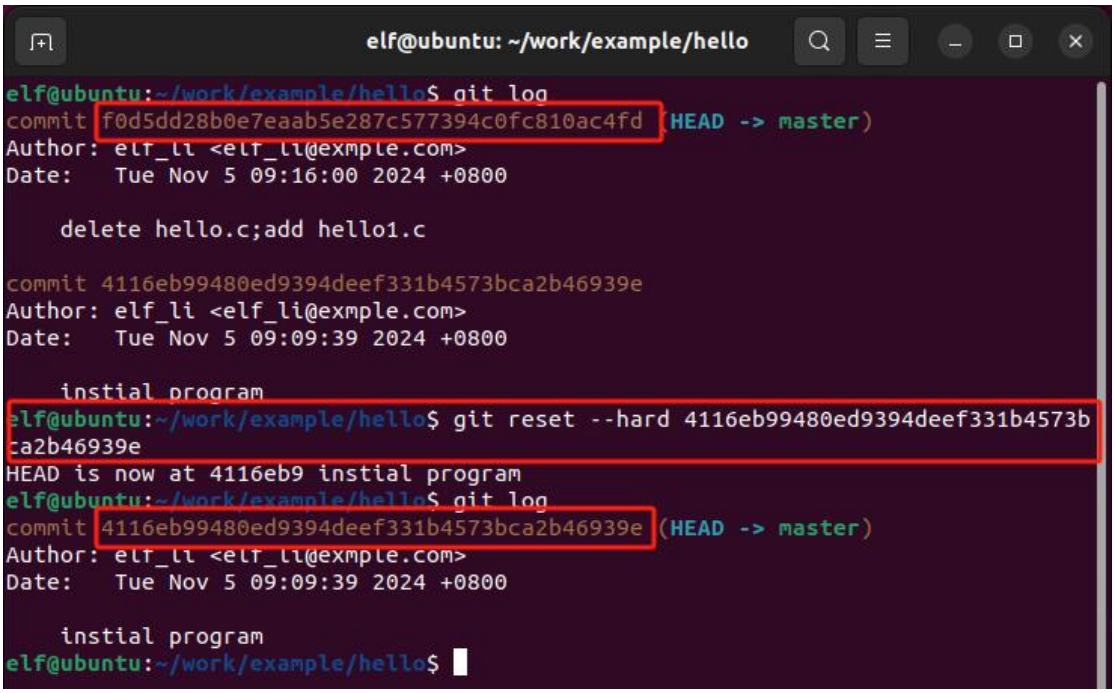


git show [commitid] --stat 查看某次提交修改过的文件：



3.commit 回退

commit 回退主要使用 git reset 命令，根据此命令后面参数的不一样，撤回的效果不通。执行 git reset --hard [commitid]，可以回退到指定的 commit：

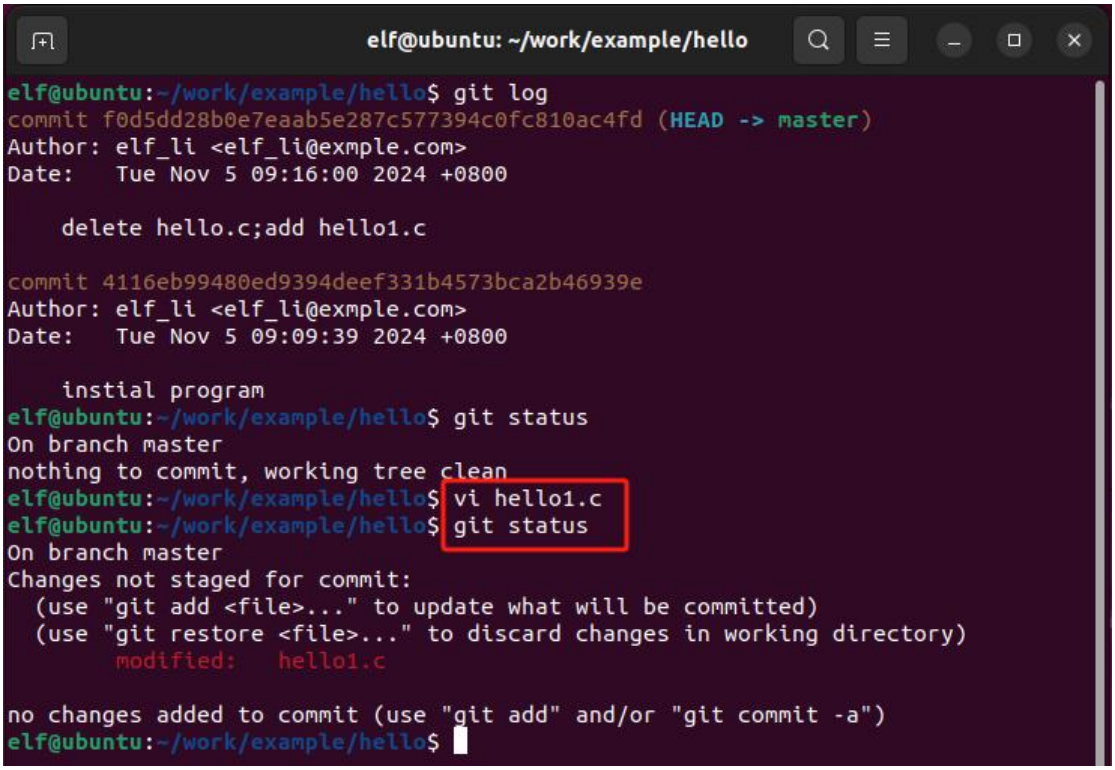


使用--hard 参数撤销工作区中所有未提交的修改内容，将暂存区与工作区都回到上一次版本，并删除之前的所有信息提交。当然也可以使用此命令，找到最新的 commit 号通过此命令再切换回来。

4.工作区的修改-----未提交到暂存区

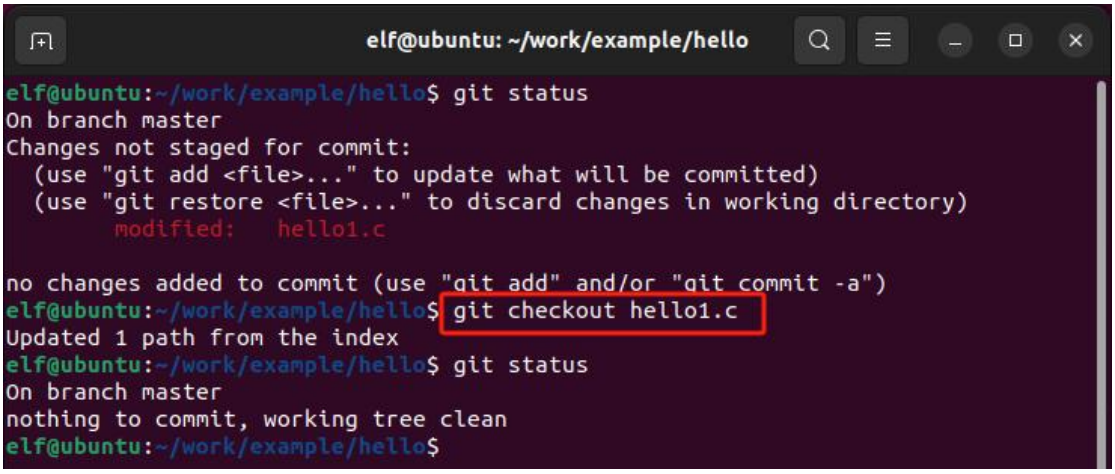
在工作区修改过程中，如果未提交到暂存区，需要放弃文件的修改可以如下操作。

如下图工作区的 hello1.c 做了修改，但未提交到暂存区：



此时如果放弃修改，可以使用 git checkout -- [filename]

```
elf@ubuntu:~/work/example/hello$ git checkout hello1.c
```

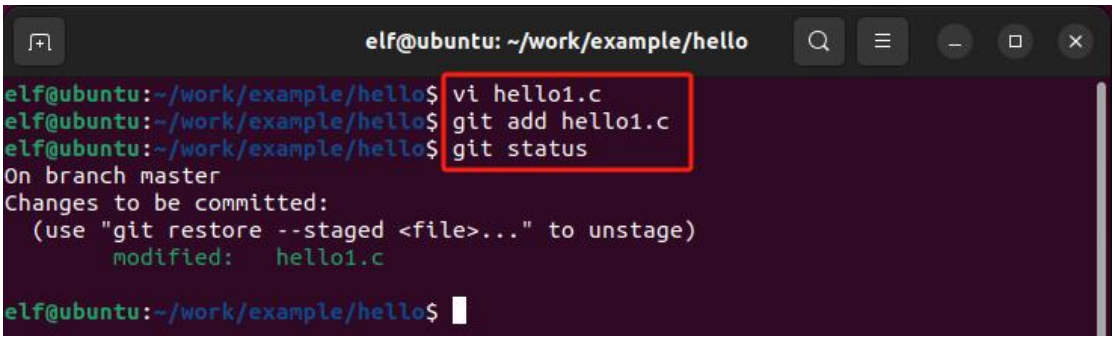


此外清除所有未提交的修改，可以使用 git checkout .

5.工作区的修改----已提交到暂存区

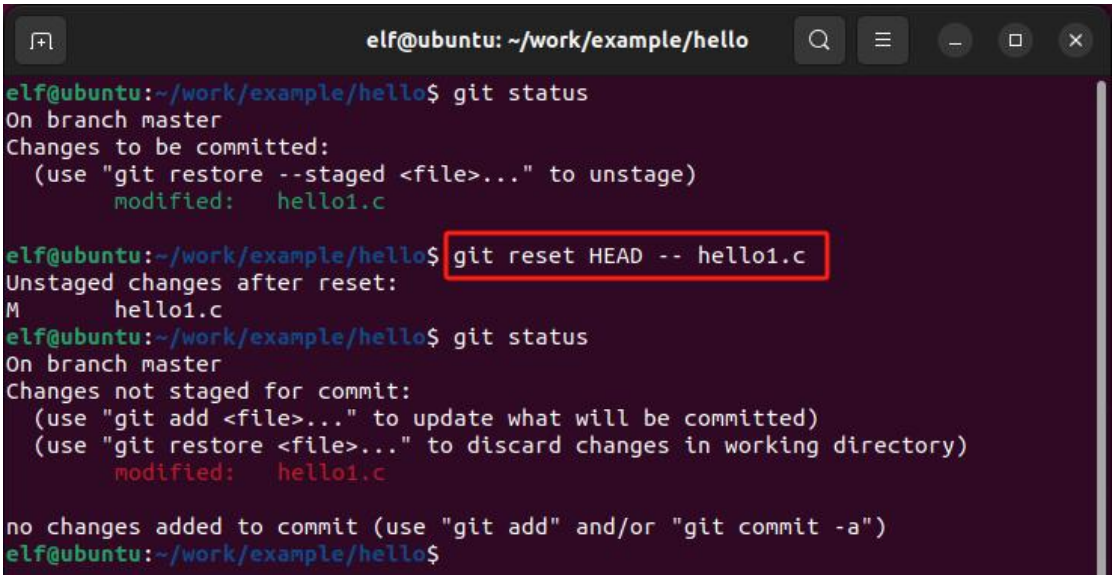
在工作区修改过程中，如果已提交到暂存区，需要放弃文件的修改可以如下操作。

如下图工作区的 hello1.c 做了修改，且已经提交到暂存区：



撤回提交到暂存区，需要执行 git reset HEAD -- [filename]，文件回到工作区可以重新更改再提交。

```
elf@ubuntu:~/work/example/hello$ git reset HEAD -- hello1.c
```



6.git 的 patch 打入打出

简单来说 patch 中存储的是你对代码的修改。在 git 中我们常用 git format-patch 和 git am 命令生成 patch 和打 patch，用此方法获得的 patch 包含 commit 里提交的 code 修改和 commit 信息，可以获取任意两个 commit 之间的 patch。

生成 patch 常用到的指令如下，其中<r1> <r2>代表具体 commit 号。

指令	说明
git format-patch HEAD^	生成最近的 1 次 commit 的 patch
git format-patch HEAD^^	生成最近的 2 次 commit 的 patch
git format-patch HEAD^^^	生成最近的 3 次 commit 的 patch
git format-patch HEAD^^^^	生成最近的 4 次 commit 的 patch
git format-patch -1 <r1>	生成<r1>这个 commit 的 patch
git format-patch <r1>	生成<r1>这个 commit 以来的修改 patch（不包含该 commit）
git format-patch --root <r1>	生成从根到 r1 提交的所有 patch

打入 patch 常用到的指令如下，xxx.patch 表示 patch 的名称：

指令	说明
git apply --stat xxx.patch	查看 patch 的情况
git apply --check xxx.patc	检查 patch 是否能够打上，如果没有任何输出，则说明无冲突，可以打上
git am xxx.patch	打入 xxx.patch
git am --signoff xxx.patch	添加-s 或者--signoff，会把自己的名字添加为 signed off by 信息，作用是注明打 patch 的人是谁，因为有时打 patch 的人并不是 patch 的作者
git am ~/patch-set/*.patch	将路径~/patch-set/*.patch 按照先后顺序打上

1.9.4.2 分支间的操作

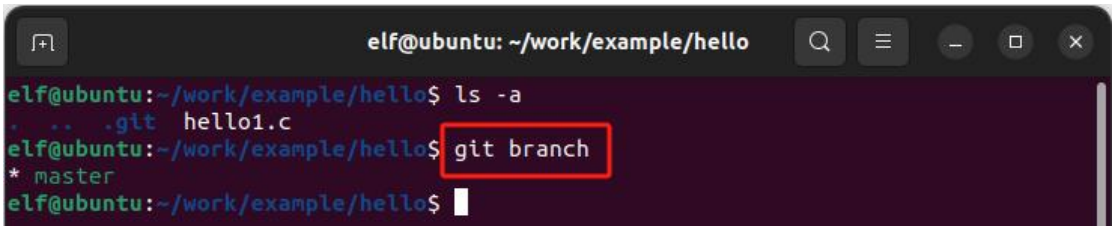
当工作区的文件可能在某一阶段用于两个项目，之后做不同修改时，就可以在该阶段切出新的分支，分别进行管理。

1.当前分支

以刚搭建好的 git 仓库为例，刚搭建完的仓库只有 master 分支，使用 git branch 查看当前的分支情况，

```
elf@ubuntu:~/work/example/hello$ git branch
```

*所在分支为当前分支，及 master 分支

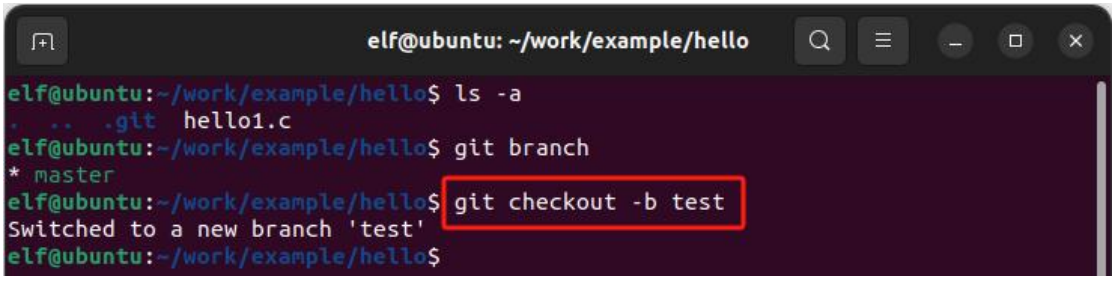


2.创建分支

前分支下创建新分支，需要将当前分支的工作区清干净，然后在创建新的分支。

git checkout -b 分支名，新建并切换到新的分支，已新建 test 分支为例

```
elf@ubuntu:~/work/example/hello$ git checkout -b test
```



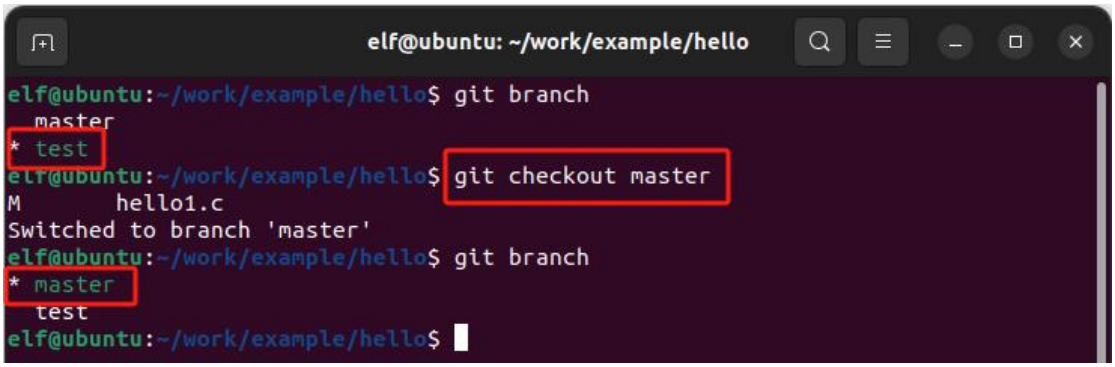
查看分支情况和工作区状态可知，新的分支的工作区状态保留之前分支的情况。

3.切换分支

在切换分支之前，需要保证工作区干净再进行切换，指令为 git checkout 分支名。

例当前在 test 分支，切换到 master。

```
elf@ubuntu:~/work/example/hello$ git checkout master
```



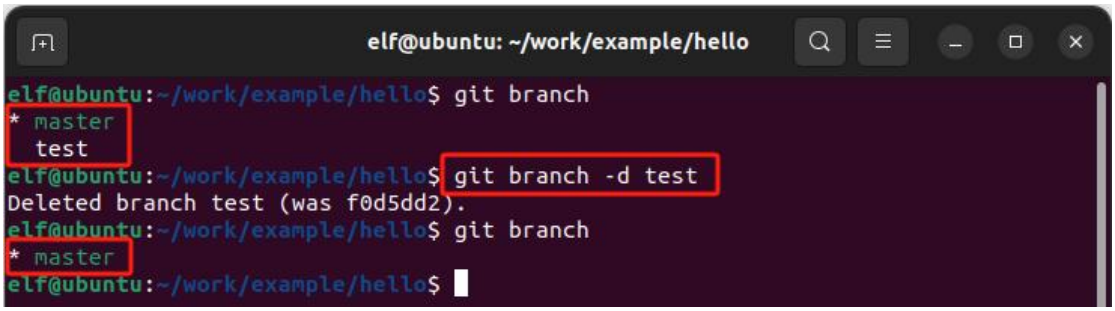
切换完再查看分支，可以看到当前分支已经切换到 master

4.删除本地分支

若创建分支名称有问题需要删除，可以用如下命令：git branch -d 分支名。

例如删除 test 分支

```
elf@ubuntu:~/work/example/hello$ git branch -d test
```



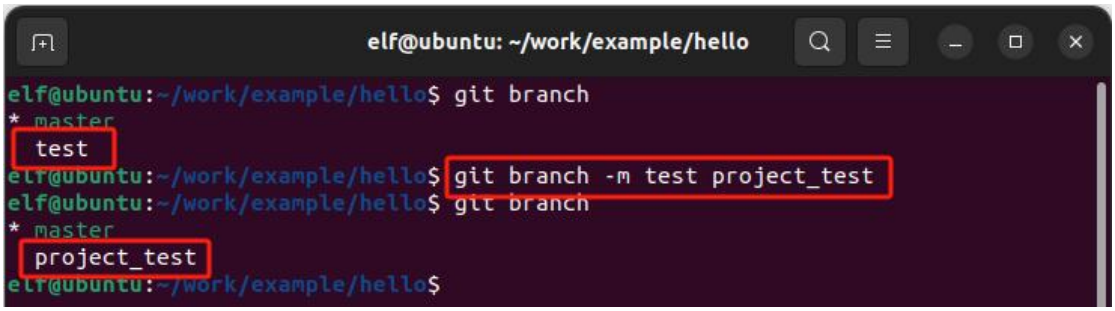
再次查看分支情况，分支已删除。

5.重命名本地分支

将本地分支 name1 更名为 name2，执行指令如下：git branch -m name1 name2

例如将分支名 test 变更为 project_test。

```
elf@ubuntu:~/work/example/hello$ git branch -m test project_test
```



6.分支对比

指令	说明
git diff	同一分支下暂存区和工作区做对比
git diff 分支 1 分支 2 --stat	查看两个分支间的文件差异
git diff 分支 1 分支 2 文件路径	查看分支间某个文件的差异

1.10 开发板适配

ELF2 开发板上有预留多种外设接口，包括：PWM、UART、SPI、GMAC、SDMMC 等等。这一章节我们主要讲解 ELF2 开发板上这些外设接口的适配。

1.10.1 LED

嵌入式开发工程师点亮一个 LED 灯，就类似软件工程师实现一个 hello world 程序。ELF2 开发板主控板上有红、蓝两个 LED 小灯，其中红色的为电源灯是不可以控制的，蓝色 LED 为可控的，默认是当作呼吸灯的。在 Linux 系统点亮一个 LED 方法很多，比如在驱动中通过操作相关引脚的寄存器实现，也可以借助 Linux 中 Pinctrl 子系统和 GPIO 子系统实现，Pinctrl 子系统主要根据设备树中引脚配置信息配置 IOMUX，GPIO 子系统则主要是提供控制 GPIO 的 API。本节直接使用 Linux 系统中自带的 gpio-leds 驱动实现，gpio-leds 通过调用 Pinctrl 子系统和 GPIO 子系统实现 LED 灯的控制。

1.10.1.1 硬件原理

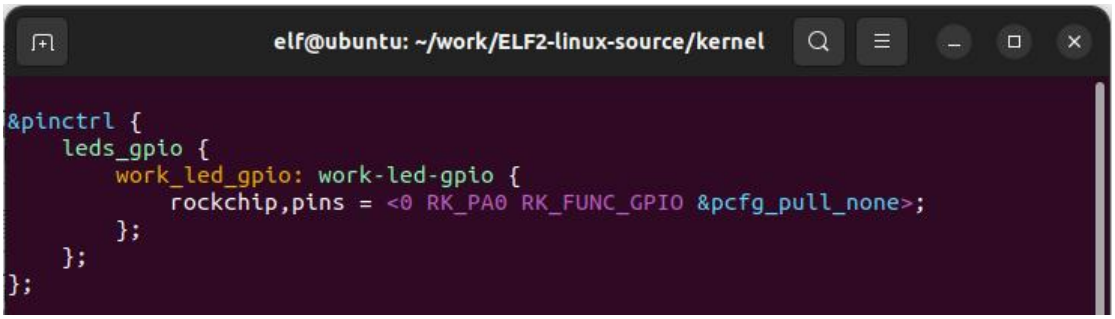
LED 灯在核心板上，由于核心板原理图暂时不开源，所以此处给出 LED 的引脚为 GPIO0_A0。

1.10.1.2 IOMUX 配置

在 arch/arm64/boot/dts/rockchip/elf2-3588.dtsi 中添加 pinctrl：

```
&pinctrl {
    leds_gpio {
        work_led_gpio: work-led-gpio {
            rockchip,pins = <0 RK_PA0 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};
```

效果如下：



⚠**注意：**在 IOMUX 配置环节，一定要确保一个引脚只被复用为一个功能，如果同一个引脚在其他地方被复用成另一个功能，很可能影响到我们期望的功能。

1.10.1.3 设备节点配置

在设备树中添加完 IOMUX 配置相关代码之后，接下来在设备树中添加供 GPIO 子系统使用的相关的配置。打开 arch/arm64/boot/dts/rockchip/elf2-3588.dtsi，在根节点下，添加子节点 leds：

```
leds: leds {
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 =<&work_led_gpio>;
        work_led: work {
                gpios = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>;
                linux,default-trigger = "heartbeat";
        };
};
```

⚠**注意：**一定要在根节点下添加，效果如下：



leds 的 compatible 属性值为“gpio-leds”，用于驱动匹配。

pinctrl-name 属性值为“default”，定义了 pincrtl 的名字为“default”

pinctrl-0 的属性值为<&work_led_gpio>，就是引用了之前在 IOMUX 中定义的 leds_gpio 也就是说，在此处对 IOMUX 的配置进行了引用，类似函数的调用。

接下来的子节点，对应具体的 LED 灯。

子节点的 lable 标签属性是可选的，可有可无。

gpios 属性值指定了具体的 GPIO 引脚以及引脚的极性配置。

如：

gpios = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>；

指定是 GPIO0_A0 引脚，极性是 GPIO_ACTIVE_HIGH，是指的高电平有效还是低电平有效，需要根据具体电路配合使用。

linux,default-trigger 属性值设置了 LED 初始默认功能，这里设置的是呼吸灯的功能。

1.10.1.4 配置 LED 驱动编译进内核

在 SDK 目录下执行./build.sh kconfig 指令进入到图形界面进行配置：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kconfig
```

确保 LED 相关的配置都编译进内核：

CONFIG_LEDS_GPIO

路径如下：

```
-> Device Drivers
    -> LED Support
```

CONFIG_LEDS_TRIGGERS

路径如下：

```
-> Device Drivers
    -> LED Support
        ->LED Trigger support
```

CONFIG_LEDS_TRIGGER_DEFAULT_ON

路径如下：

```
-> Device Drivers
    -> LED Support
        ->LED Trigger support
            -> LED Default ON Trigger
```

CONFIG_LEDS_TRIGGER_TIMER

路径如下：

```
-> Device Drivers
    -> LED Support
        ->LED Trigger support
            -> LED Timer Trigger
```

CONFIG_LEDS_TRIGGER_HEARTBEAT

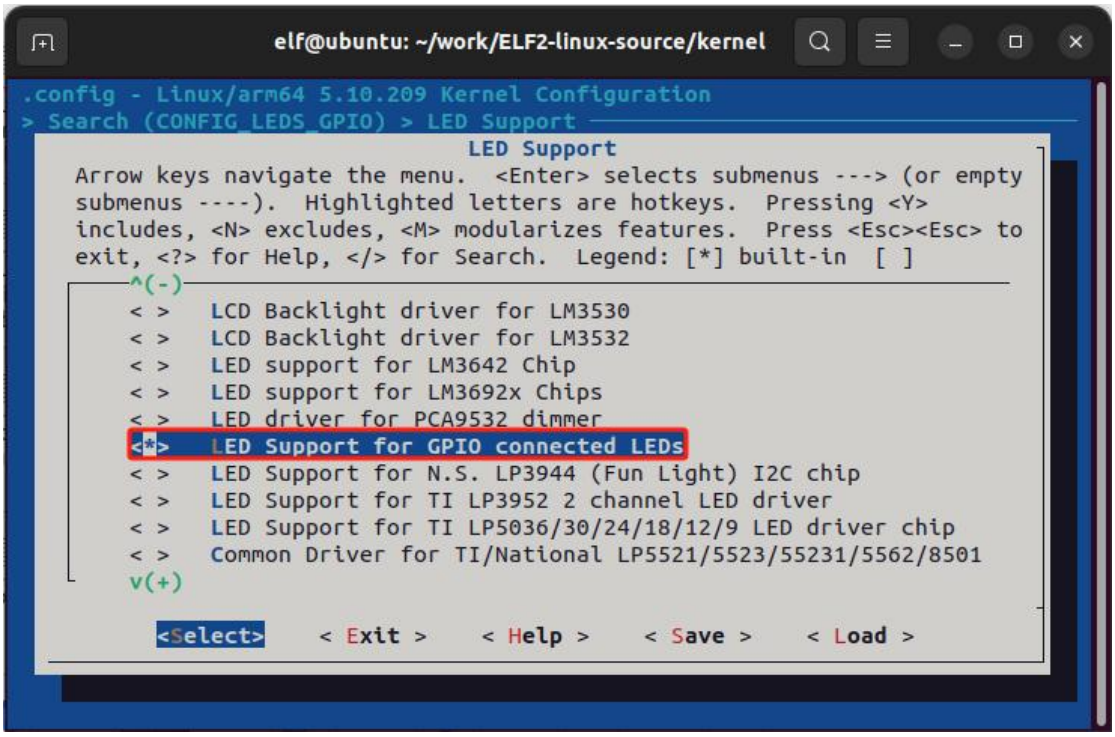
路径如下：

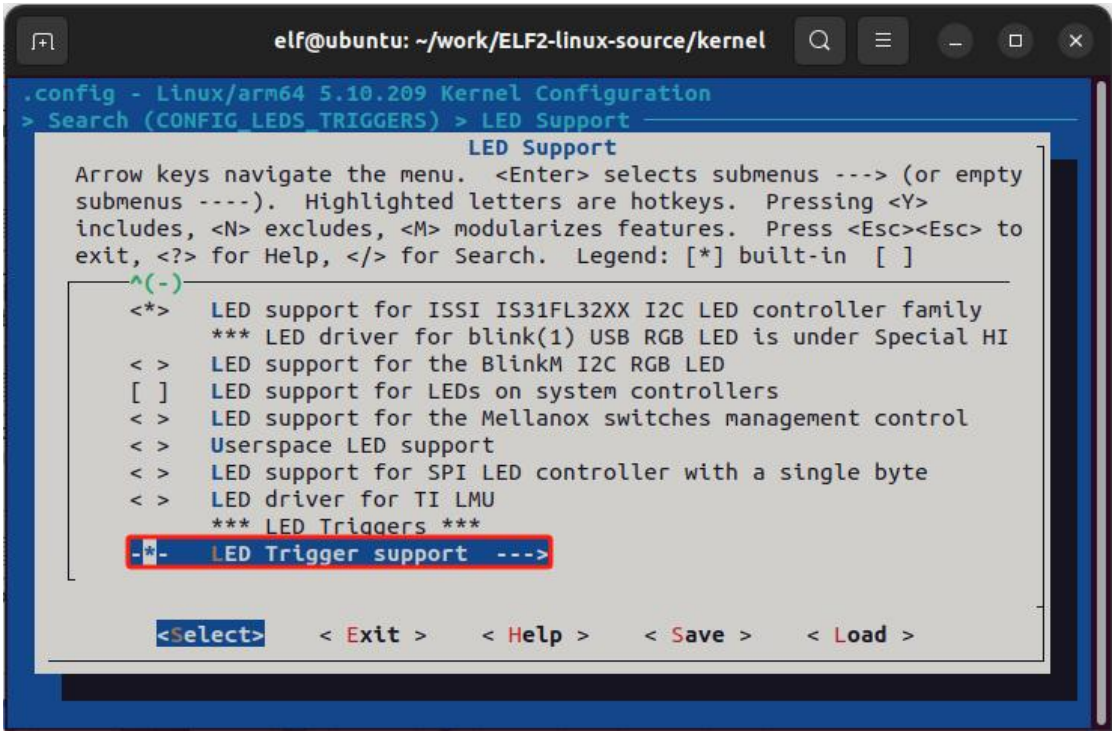
```
-> Device Drivers
    -> LED Support
        ->LED Trigger support
            -> LED Heartbeat Trigger
```

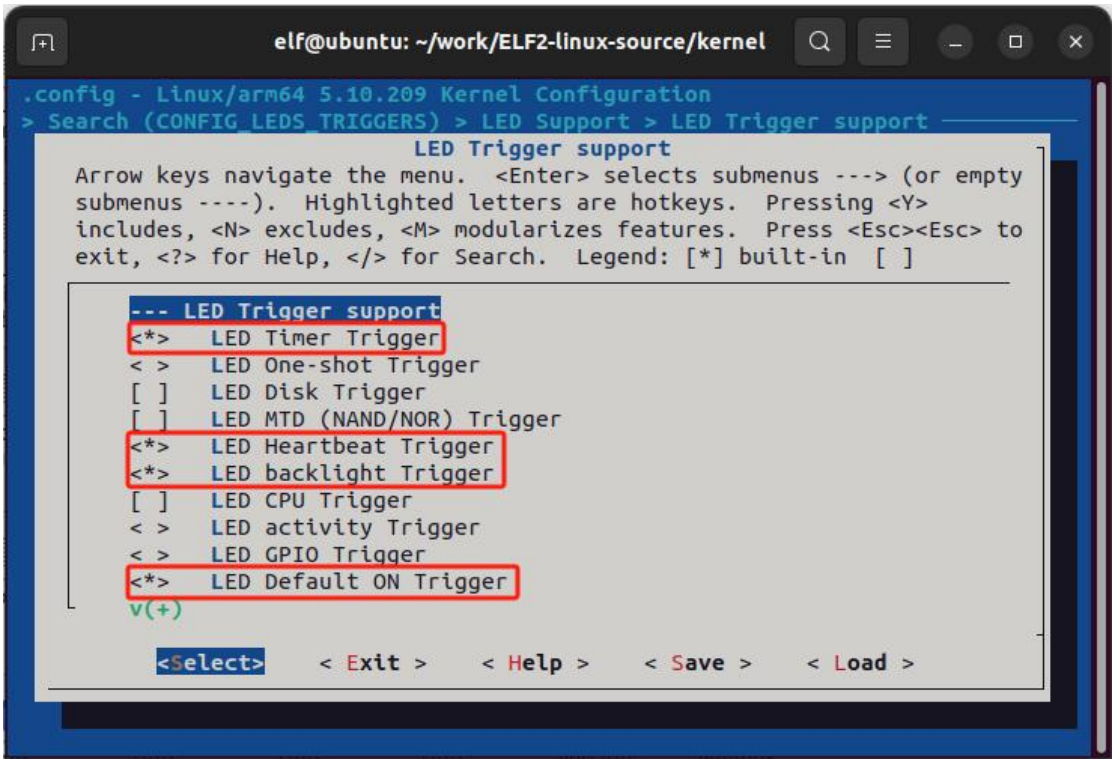
CONFIG_LEDS_TRIGGER_BACKLIGHT

路径如下：

```
-> Device Drivers
    -> LED Support
        ->LED Trigger support
            -> LED backlight Trigger
```


elf@ubuntu: ~/work/ELF2-linux-source/kernel
.config - Linux/arm64 5.10.209 Kernel Configuration
> Search (CONFIG_LEDS_GPIO) > LED Support
LED Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
^(-)
< > LCD Backlight driver for LM3530
< > LCD Backlight driver for LM3532
< > LED support for LM3642 Chip
< > LED support for LM3692x Chips
< > LED driver for PCA9532 dimmer
<*> LED Support for GPIO connected LEDs
< > LED Support for N.S. LP3944 (Fun Light) I2C chip
< > LED Support for TI LP3952 2 channel LED driver
< > LED Support for TI LP5036/30/24/18/12/9 LED driver chip
< > Common Driver for TI/National LP5521/5523/55231/5562/8501
v(+)
<Select> < Exit > < Help > < Save > < Load >


elf@ubuntu: ~/work/ELF2-linux-source/kernel
.config - Linux/arm64 5.10.209 Kernel Configuration
> Search (CONFIG_LEDS_TRIGGERS) > LED Support
LED Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
^(-)
<*> LED support for ISSI IS31FL32XX I2C LED controller family
*** LED driver for blink(1) USB RGB LED is under Special HI
< > LED support for the BlinkM I2C RGB LED
[ ] LED support for LEDs on system controllers
< > LED support for the Mellanox switches management control
< > Userspace LED support
< > LED support for SPI LED controller with a single byte
< > LED driver for TI LMU
*** LED Triggers ***
-*- LED Trigger support --->
<Select> < Exit > < Help > < Save > < Load >



1.10.1.5 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

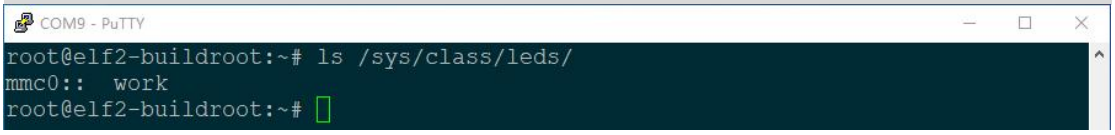
编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

重启后可以看到核心板上的蓝色 LED 为闪烁状态。

接下来我们还可以手动对这个 led 进行控制：

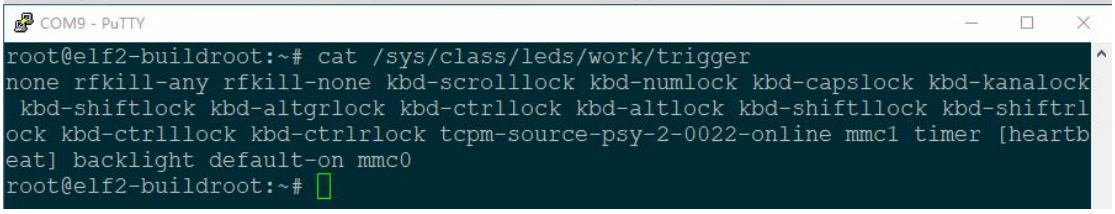
查看节点，可以看到在/sys/class/leds 下已经生成了在设备树中创建的 work 节点。

```
root@elf2-buildroot:~# ls /sys/class/leds/
mmc0::  work
```



查看 LED 触发模式，可以看到触发模式就是呼吸灯。

```
root@elf2-buildroot:~# cat /sys/class/leds/work/trigger
```



将触发模式设置为 none，此时 LED 灯会变为灭的状态。

```
root@elf2-buildroot:~#  echo none > /sys/class/leds/work/trigger
```

控制 LED 灯亮。

```
root@elf2-buildroot:~#  echo 1 > /sys/class/leds/work/brightness
```

控制 LED 灯灭。

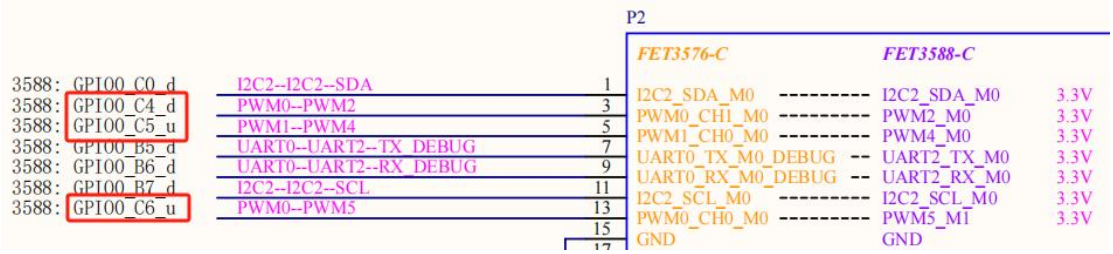
```
root@elf2-buildroot:~#  echo 0 > /sys/class/leds/work/brightness
```

1.10.2 PWM

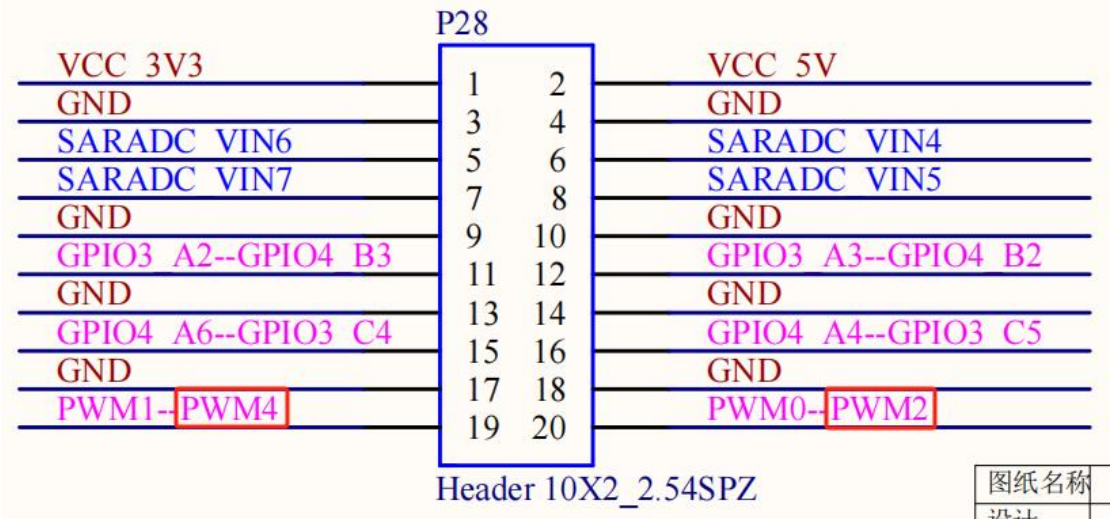
PWM 是 Pulse Width Modulation 的缩写，是指脉冲宽度调制，通过调节脉冲的宽度输出想要的波形。ELF2 开发板上默认配置了 3 个 PWM 功能的引脚。

1.10.2.1 硬件原理

板子上的 PWM 引脚分别为 GPIO0_C4、GPIO0_C5 和 GPIO0_C6。

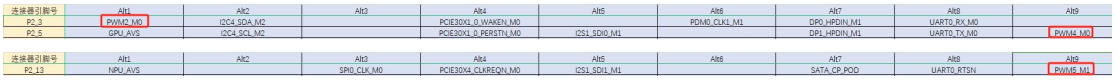


其中 GPIO0_C6 是用作了 MIPI 屏幕的背光，GPIO0_C4 和 GPIO0_C5 直接引到了 20Pin 的端子座上。



1.10.2.2 IOMUX 配置

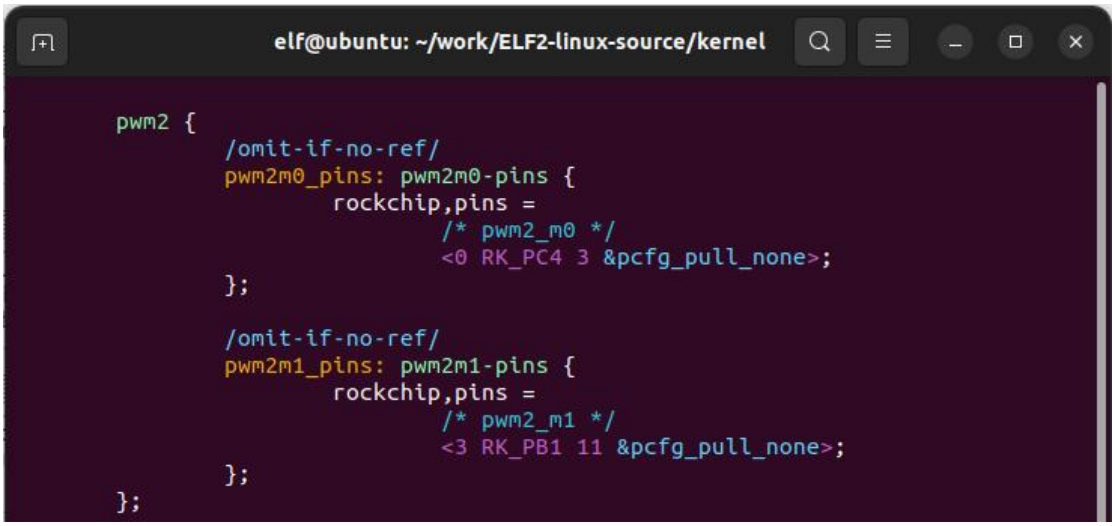
从引脚复用表中可以看出 P2 的 3 引脚可以复用成 PWM2 功能，P2 的 5 引脚可以复用成 PWM4 功能，P2 的 13 引脚可以复用成 PWM5 功能。

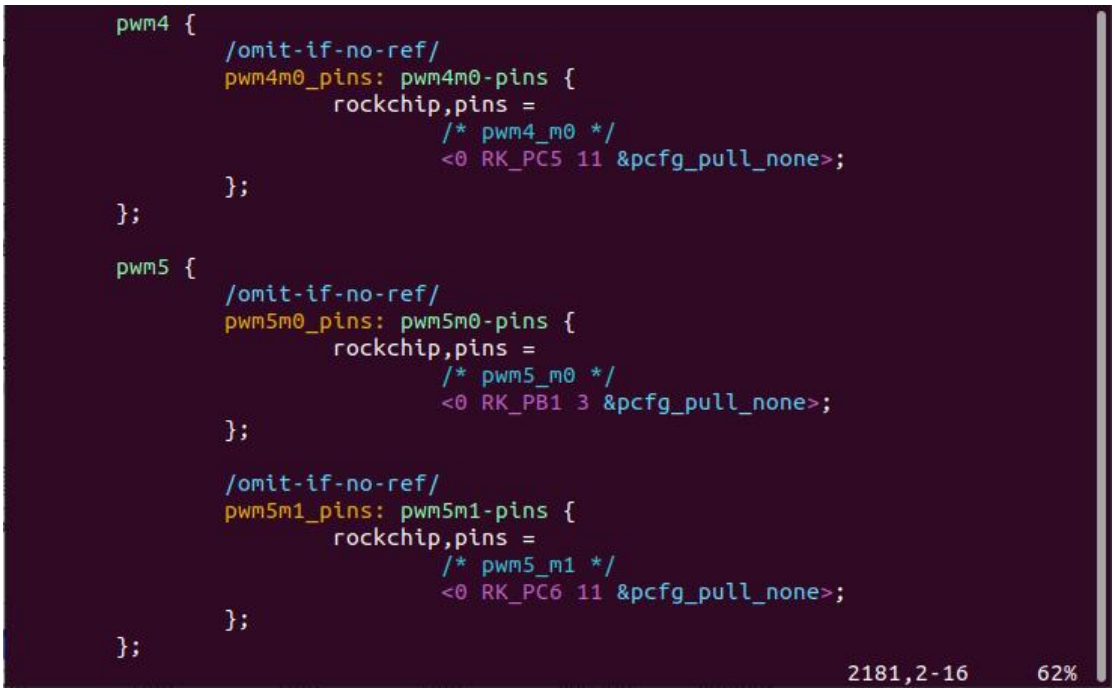


连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7	Alt8	Alt9
P2_3	PWM2_M0	I2C4_SDA_M2		PCIE30X1_0_WAKEN_M0		PDM0_CLK1_M1	DP0_HPDIN_M1	UART0_RX_M0	
P2_5	GPU_AVS	I2C4_SCL_M2		PCIE30X1_0_PERSTN_M0	ICS1_SDI0_M1		DP1_HPDIN_M1	UART0_TX_M0	PWM4_M0

连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7	Alt8	Alt9
P2_13	NPU_AVS		SPI0_CLK_M0	PCIE30X4_CLKREQN_M0	ICS1_SDI1_M1		SATA_CP_POD	UART0_RTSN	PWM5_M1

在 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 中找到它们的定义，如下：

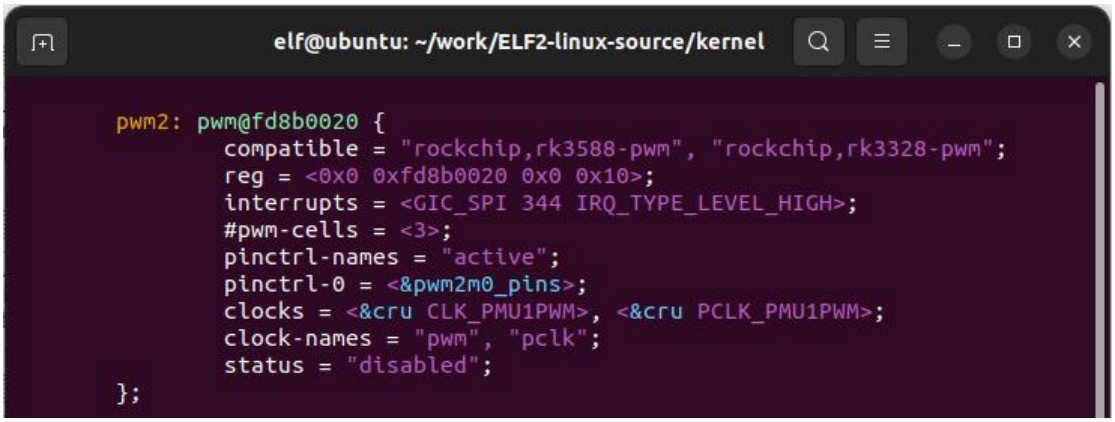


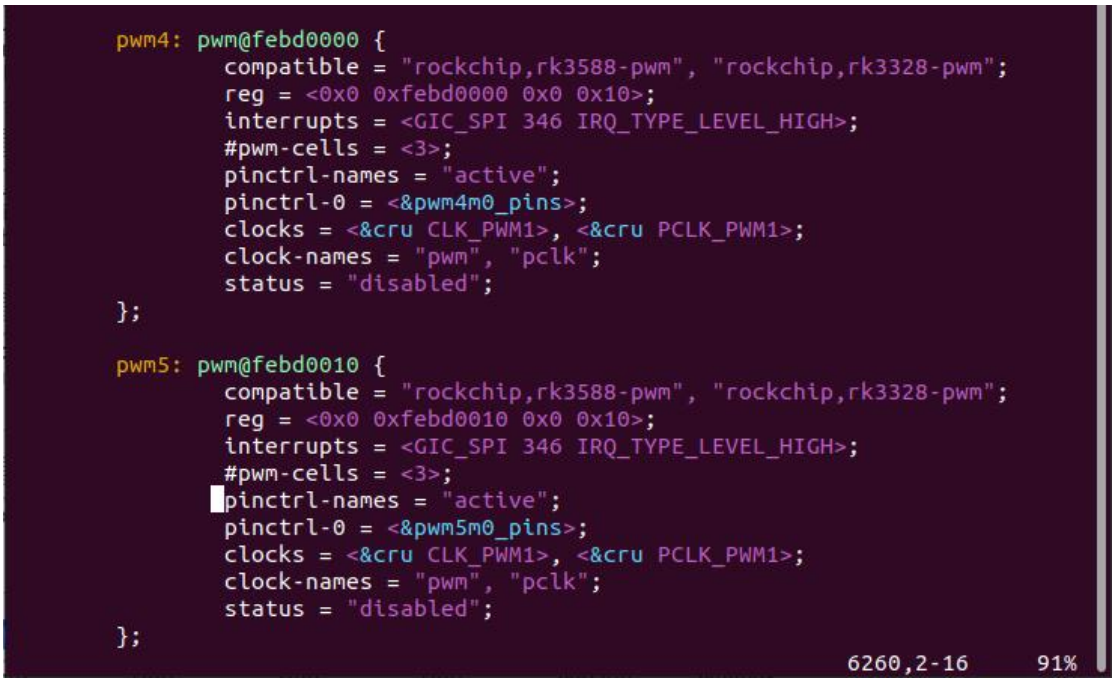


可以看到 pwm2m0_pins 描述的是 GPIO0_C4 复用成 pwm2。pwm4m0_pins 描述的是 GPIO0_C5 复用成 pwm4。pwm5m1_pins 描述的是 GPIO0_C6 复用成 pwm5。

1.10.2.3 设备节点配置

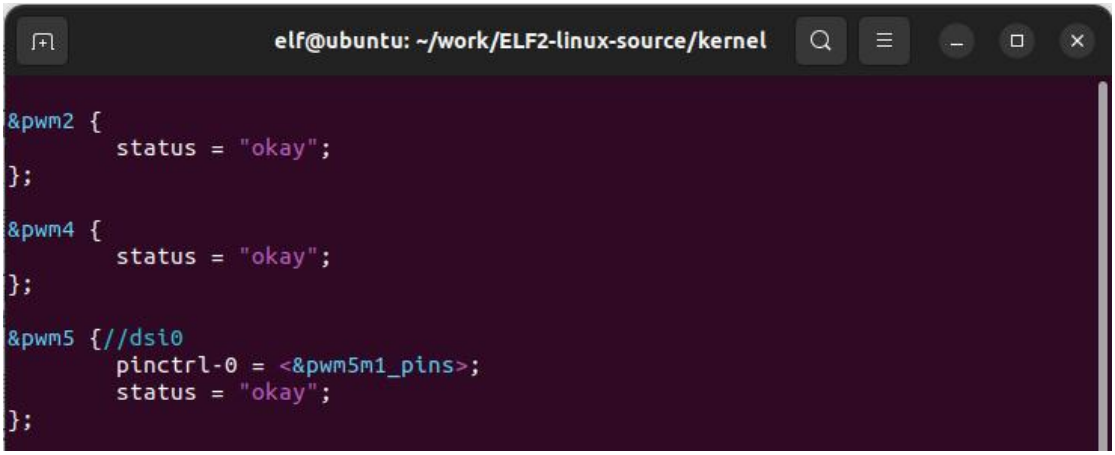
在设备树文件 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 中已经定义好 16 路 PWM 信号的相关节点和使用的引脚。





pwm2 默认使用的为 pwm2m0_pins，pwm4 默认使用的为 pwm4m0_pins，pwm5 默认使用的为 pwm5m0_pins。其中 pwm2 和 pwm4 默认使用的引脚与原理图中的一致。pwm5 使用的与原理图不一致。所以后续只需要追加 pwm5 的引脚描述即可。

打开 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 添加相关节点。



因为 pwm2 和 pwm4 使用的引脚与默认配置的一致，所以只需要引用节点，然后将 status 状态修改为“okay”即可。pwm5 默认使用的引脚与原理图不一致，所以要追加引脚描述为 pwm5m1_pins。

1.10.2.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

重启开发板，查看 pwm 节点：

```
root@elf2-buildroot:~# ls /sys/class/pwm/
pwmchip0    pwmchip1    pwmchip2
```

由于 pwm 驱动中生成的节点是依次向下枚举的，所以 pwmchip0 节点对应的是设备树中的 pwm2，pwmchip1 节点对应的是设备树中的 pwm4，pwmchip2 节点对应的是设备树中的 pwm5。

下面以测试 pwm2 为例，设置 pwm2 为输出：

```
root@elf2-buildroot:~# echo 0 > /sys/class/pwm/pwmchip0/export
```

设置 PWM 周期，单位为 ns：

```
root@elf2-buildroot:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/period
```

设置 PWM 占空比为 50%：

```
root@elf2-buildroot:~# echo 500000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
```

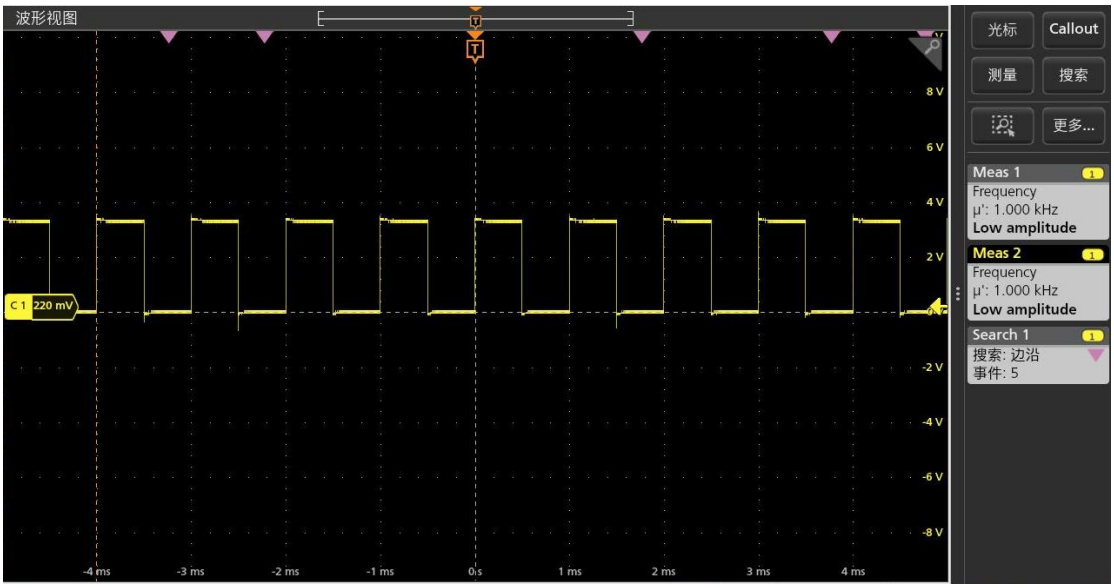
使能 PWM2 功能：

```
root@elf2-buildroot:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

失能 PWM2 功能：

```
root@elf2-buildroot:~# echo 0 > /sys/class/pwm/pwmchip0/pwm0/enable
```

使能 PWM2 功能后，使用示波器测量 P28 的 20 引脚，可以看出波形为 1kHz，占空比为 50%。



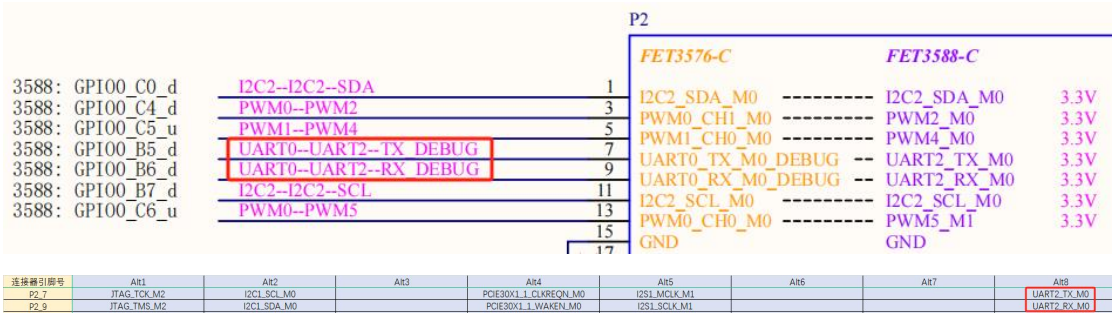
1.10.3 UART

UART(Universal Asynchronous Receiver/Transmitter)，通用异步收发器，是一个全双工的支持串行通信协议的接口。

ELF2 开发板一共引出 2 路 UART 接口，分别是 UART2、UART9，其中 UART2 用于调试，UART9 连接到了 P29 的 6 引脚和 P23 的 8 引脚上。下面我们对这几个接口逐一进行配置。

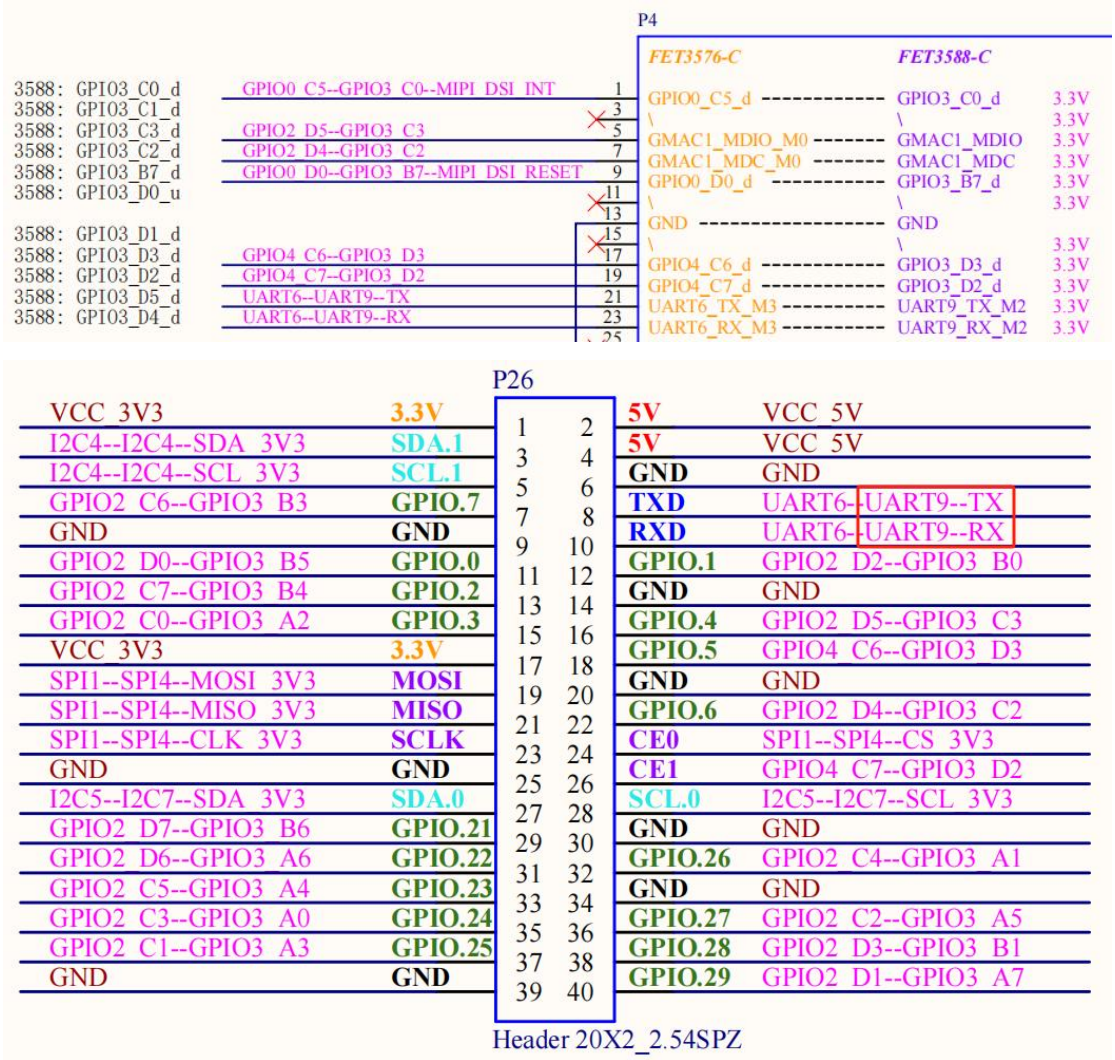
1.10.3.1 硬件原理

1.UART2



从原理图中可以得到我们使用的 UART2_TX 和 UART2_RX 为 debug 调试口。引脚对应的连接器编号分别为 P2 的 7 引脚和 P2 的 9 引脚。引脚分别为 GPIO0_B5 和 GPIO0_B6。

2.UART9



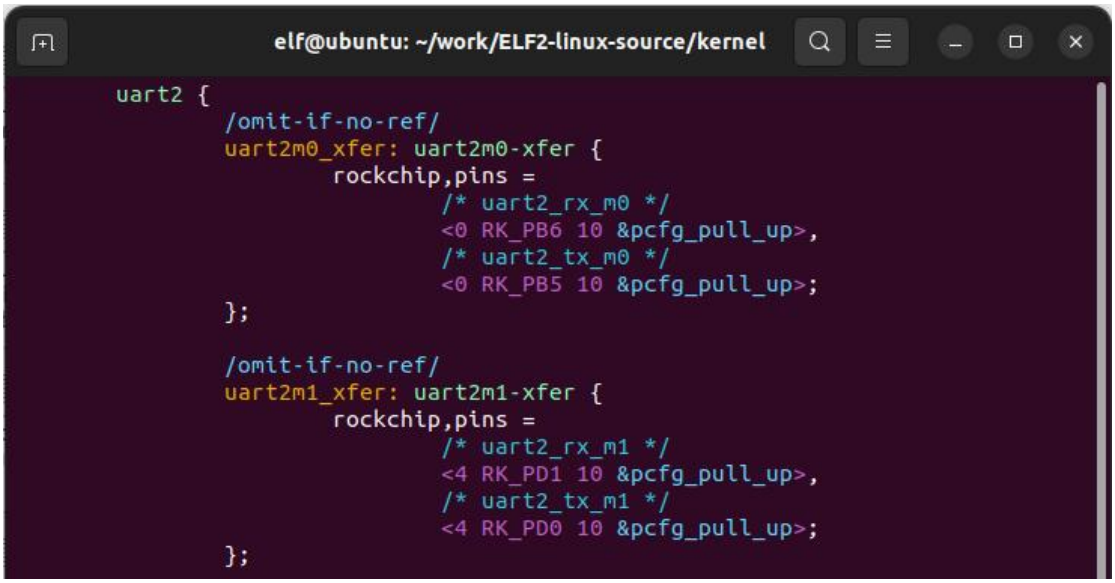
从原理图中可以得到 UART9_TX 和 UART9_RX 引脚对应的连接器编号分别为 P4 的 21 引脚和 P4 的 23 引脚。引脚分别为 GPIO3_D5 和 GPIO3_D4。

1.10.3.2 IOMUX 配置

确定好引脚之后，我们就可以在设备树中添加相关引脚的 IOMUX 配置。

1.UART2

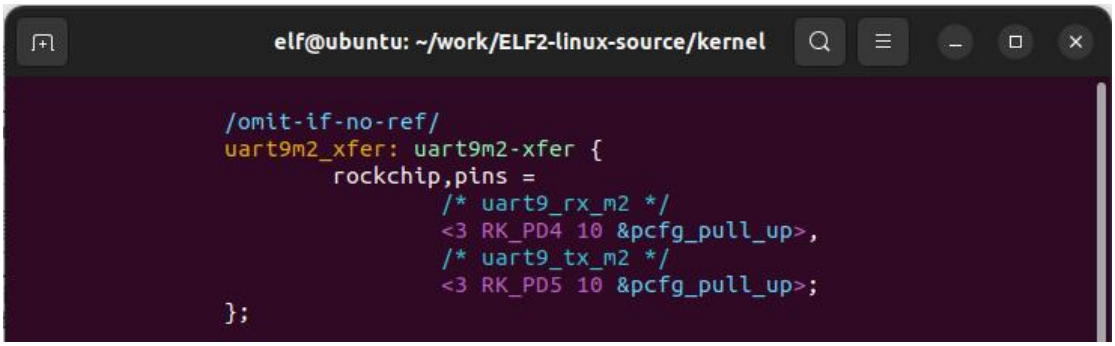
在 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 中找到 UART2 的定义，如下：



uart2m0_xfer 描述的就是 GPIO0_B5 和 GPIO0_B6 复用成 UART2。

2.UART9

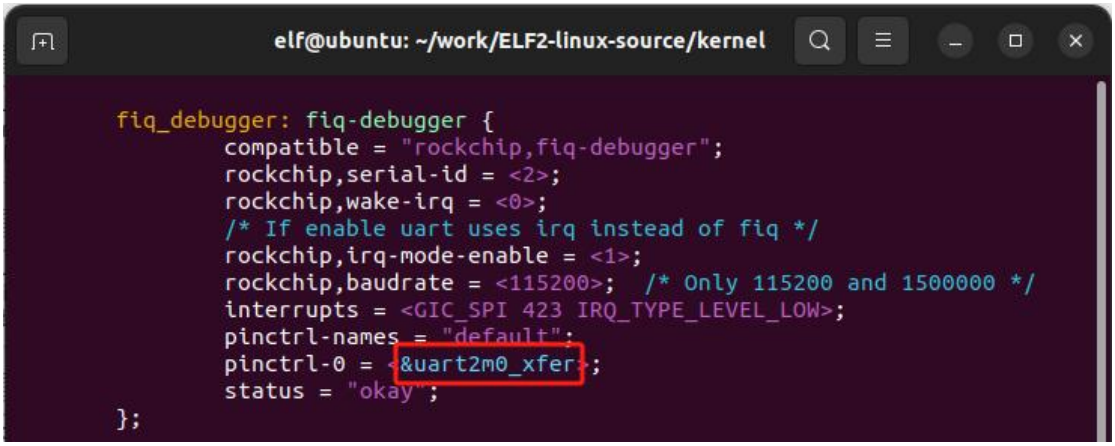
在 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 中找到 UART9 的定义，如下：



uart9m2_xfer 描述的就是 GPIO3_D4 和 GPIO3_D5 复用成 UART9。

1.10.3.3 设备节点配置

打开 arch/arm64/boot/dts/rockchip/elf2-kernel.dts 文件，将 fiq_debugger 中引用的引脚，写成 uart2m0_xfer。



1.fiq_debugger：

是该设备的节点名称。

2.compatible = "rockchip,fiq-debugger";

这个字段指定了设备的兼容属性，是用来匹配驱动的。

3.rockchip,serial-id = <2>;

指定 Serial ID，这里值为 2，因为使用的是 UART2，所以这里为 2。

4.rockchip,wake-irq = <0>;

表示唤醒中断的 ID，这里设置为 0，通常表示没有特定的唤醒中断。

5.rockchip,irq-mode-enable = <1>;

表示启用 IRQ 模式（中断请求模式)。<1> 代表启用，表示此设备将使用 IRQ 代替 FIQ。

6.rockchip,baudrate = <115200>;

设置串行通信的波特率，指定为 115200。由于下方的注释，表明这个设备只支持波特率 115200 和 1500000。

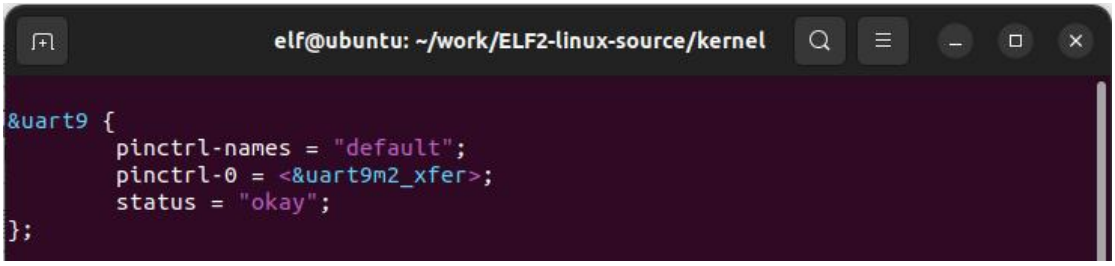
7.interrupts = <GIC_SPI 423 IRQ_TYPE_LEVEL_LOW>;

指定该设备使用的中断配置。GIC_SPI 表示该中断使用了通用中断控制器的 SPI（共享中断），423 是中断的编号，IRQ_TYPE_LEVEL_LOW 表示中断类型为低电平触发模式。

8.pinctrl-0 = <&uart2m0_xfer>;

这里指定了使用的引脚就是 uart2m0_xfer 描述的引脚。

打开 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 文件，添加引用 uart9 节点，并且将 pinctrl-0 中引用的引脚，写成 uart9m2_xfer。



这样设备节点就添加完成了。

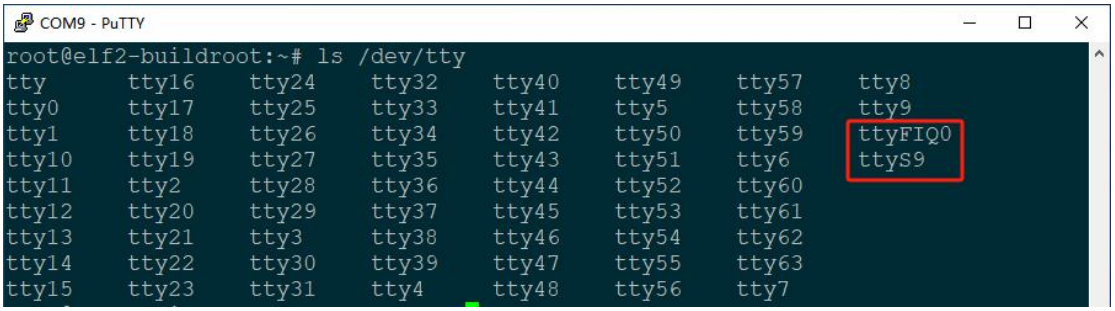
1.10.3.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

启动之后，可以看到在/dev 目录下生成节点 ttyFIQ0 和 ttyS9。

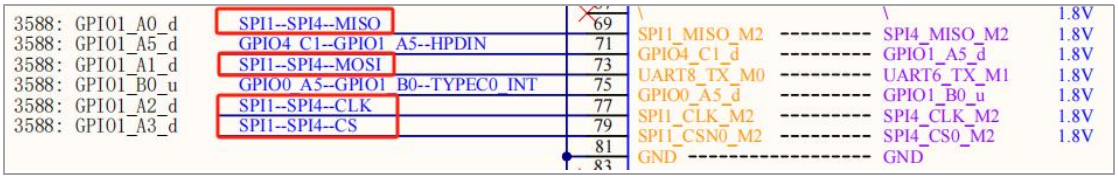


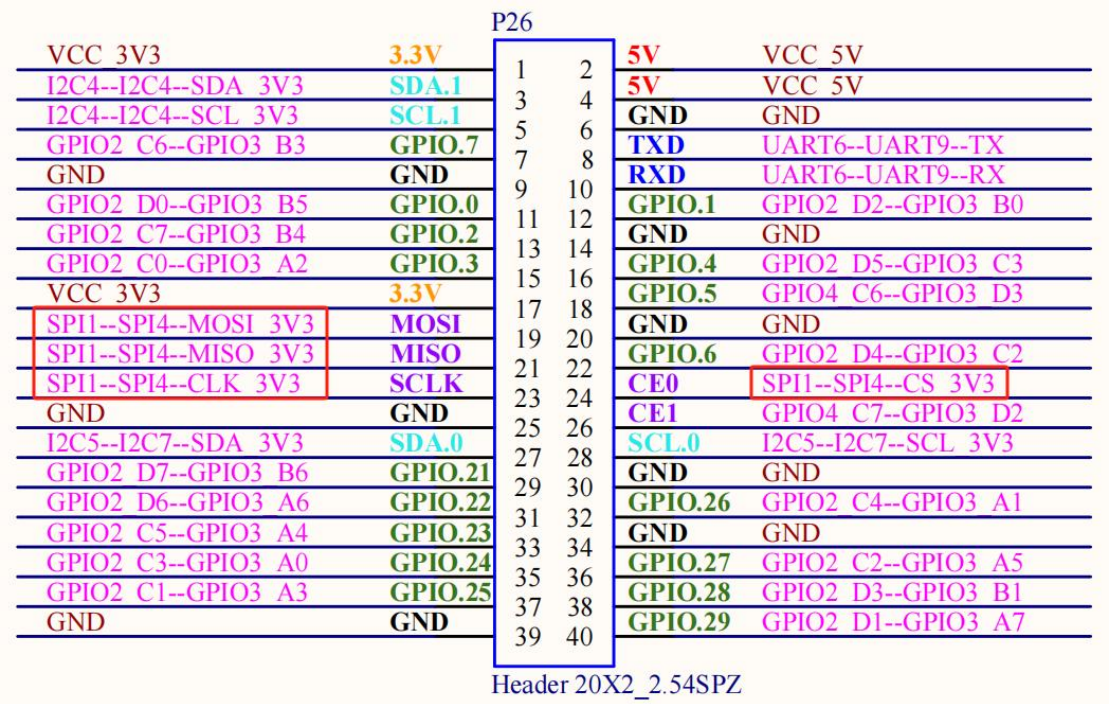
1.10.4 SPI

SPI 是串行外设接口（Serial Peripheral Interface）的缩写，是一种高速的，全双工，同步的通信总

线。SPI 的通信以主从方式工作，一般需要 4 根线，MISO（主设备数据输入）、MOSI（主设备数据输出）、SCLK（时钟）、CS（片选）。ELF2 开发板使用了 SPI4，没有连接任何子设备，而是配置了 spidev 驱动，可以实现在不编译内核驱动程序的情况下，通过标准的文件操作方式与 SPI 设备进行通信。

1.10.4.1 硬件原理

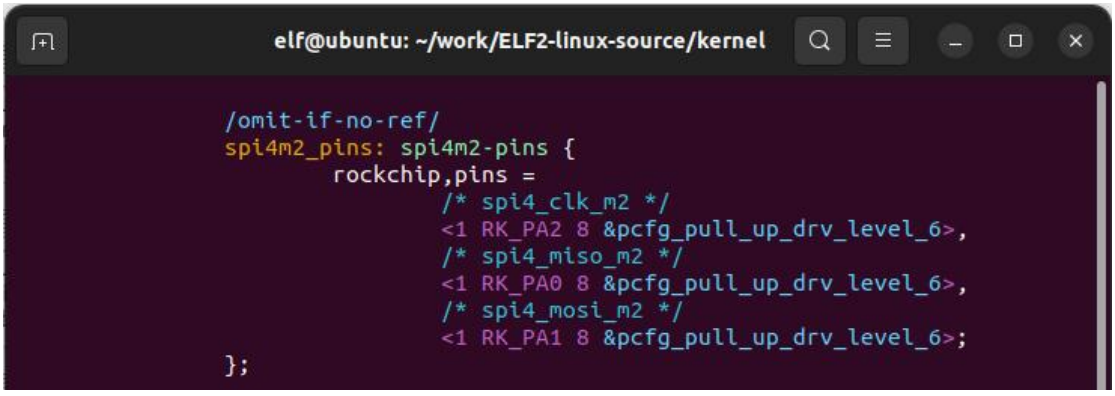




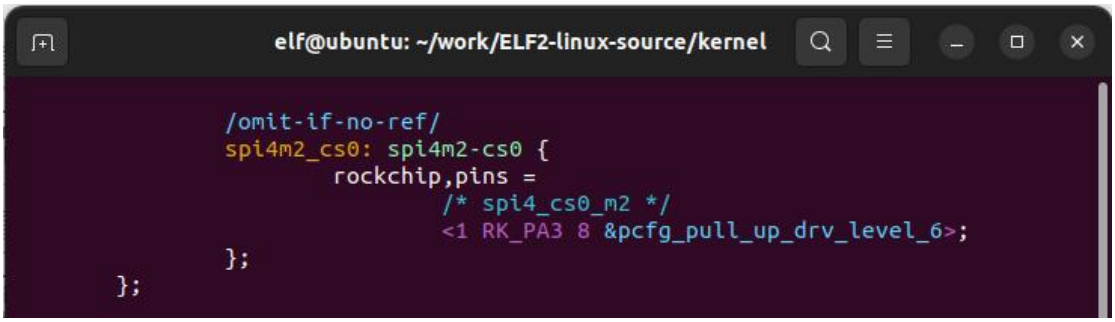
从原理图中可以看到，SPI 引脚直接从核心板引到了 P26 的端子座上。

1.10.4.2 IOMUX 配置

在 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 中找到引脚相关描述：



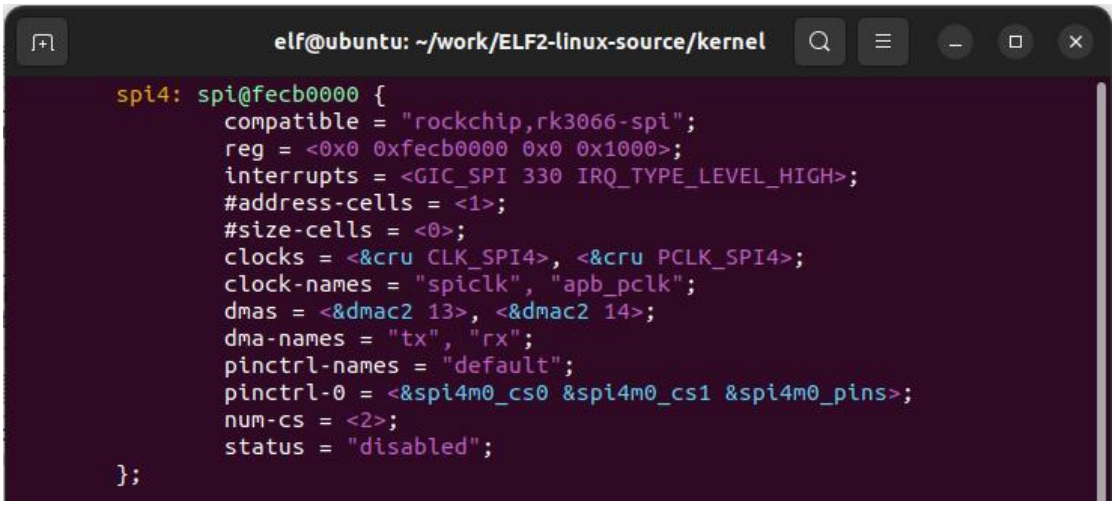
从原理图中可以看出使用的引脚分别为：GPIO1_A0、GPIO1_A1 和 GPIO1_A2，所以相关的引脚描述为 spi4m2_pins，描述了 spi4_clk、spi4_miso 和 spi4_mosi。



GPIO1_A3 为片选引脚，相关的引脚描述为 spi4m2_cs0。

1.10.4.3 设备节点配置

在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 中已经存在了 spi4 接口的相关节点 spi4：



compatible：定义了驱动的匹配属性。

reg：表示该设备的寄存器空间。

interrupts：定义了设备的中断属性。

clocks：定义该设备所依赖的时钟源。

clock-names：为每个时钟资源指定名称。

dmas：指定 SPI 控制器的 DMA 通道，用于高效数据传输。

dma-names：为每个 DMA 通道指定名称，"tx" 表示传输通道，"rx" 表示接收通道。

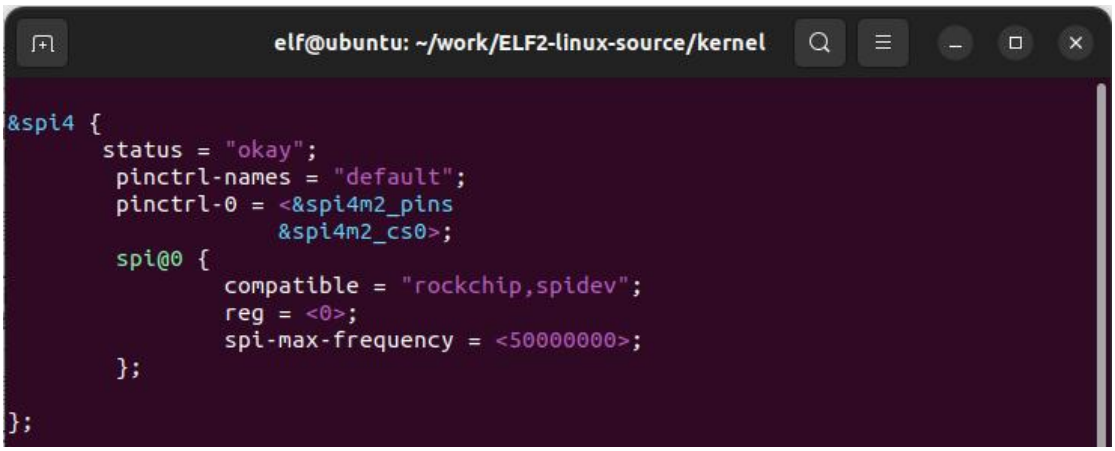
pinctrl-names：定义设备的引脚控制状态，"default" 表示设备的默认引脚配置。

pinctrl-0：引用引脚控制器节点，配置 SPI 接口使用的引脚，包括 cs0（片选 0）、cs1（片选 1）以及其他 SPI 数据和时钟引脚。

pinctrl-0：引用引脚控制器节点，配置 SPI 接口使用的引脚，包括 cs0（片选 0）、cs1（片选 1）以及其他 SPI 数据和时钟引脚。

status：定义设备的启用状态，"disabled" 表示默认情况下该设备处于禁用状态。

在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中修正 spi4 节点，因为默认使用的引脚与原理图不一致，并且添加 spidev 节点，用于用户空间操作 spi 设备。



将 status 状态修改为“okay”，将 pinctrl-0 中的引脚修改为原理图中实际使用的引脚，也就是上接 IOMUX 配置中提到的 spi4m2_pins 和 spi4m2_cs0。

最后在 spi4 节点下添加了子节点 spi，用于启用 spidev，节点会匹配“rockchip,spidev”驱动，会在用户空间生成 spidev 设备节点，通常是/dev/spidevX.Y 设备文件，用户程序可以通过此设备文件访问 SPI 设备。

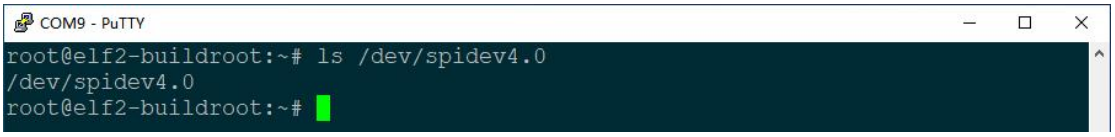
1.10.4.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

启动之后，可以看到在/dev 目录下生成节点 spidev4.0。



短接 MOSI (PIN19)与 MISO (PIN21)，使用下面的命令进行测试：

```
root@elf2-buildroot:~# cmddemo_spi spidev4.0
[/dev/spidev4.0 nwrite=38] 0xff  0xff  0xff  0xff  0xff  0xff  0x40  0x00  0x00
0x00  0x00  0x95  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xde  0xad  0xbe  0xef  0xba
0xad  0xf0  0x0d
[/dev/spidev4.0 nread=38 ] 0xff  0xff  0xff  0xff  0xff  0xff  0x40  0x00  0x00
0x00  0x00  0x95  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xde  0xad  0xbe  0xef  0xba
0xad  0xf0  0x0d
 0xff
```

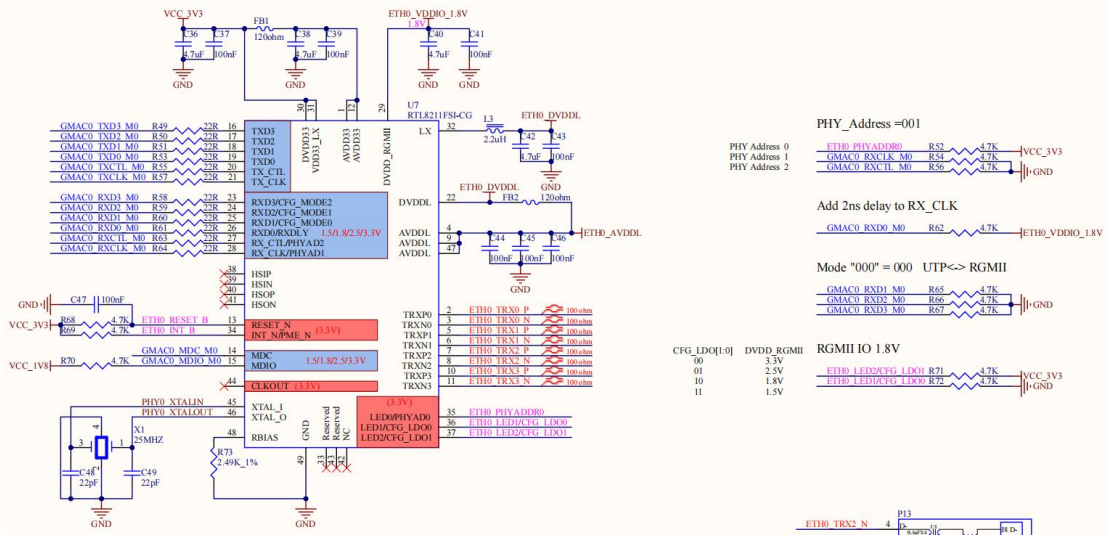
因为是短接回环测试，所以接收到的数据和写入的数据是一致的。

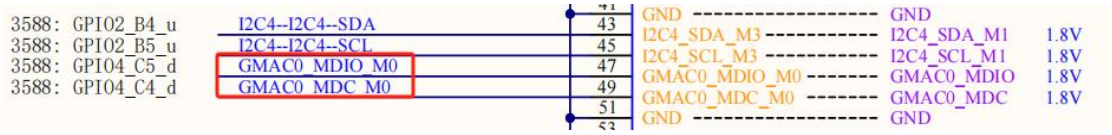
1.10.5 GMAC

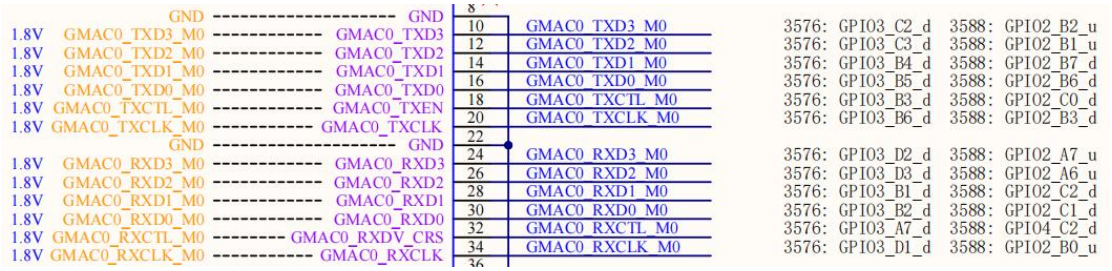
GMAC 是用于实现以太网通信的控制器模块，它支持千兆位速率的以太网传输。

RK3588 支持两路 GMAC（千兆以太网控制器），这意味着它能够同时配置和使用两个独立的千兆以太网接口。这两个 GMAC 可以通过不同的接口模式（如 RGMII 或 SGMII）与外部的物理层器件（PHY）连接，实现多种网络连接方式。其中 ELF2 开发板上只使用了一路 GMAC0。

1.10.5.1 硬件原理

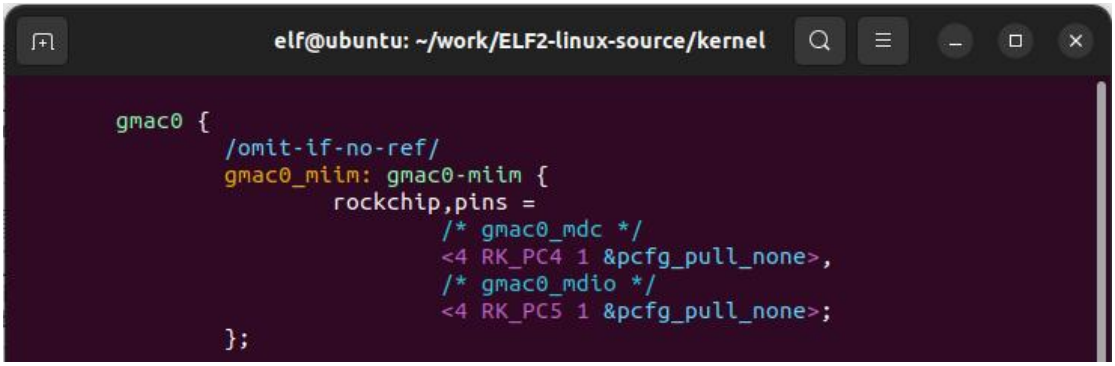






1.10.5.2 IOMUX 配置

GMAC0 相关的引脚描述在 arch/arm64/boot/dts/rockchip/rk3588-vccio3-pinctrl.dtsi 中。



```
/omit-if-no-ref/
gmac0_rx_bus2: gmac0-rx-bus2 {
        rockchip,pins =
                /* gmac0_rxd0 */
                <2 RK_PC1 1 &pcfg_pull_none>,
                /* gmac0_rxd1 */
                <2 RK_PC2 1 &pcfg_pull_none>,
                /* gmac0_rxdv_crs */
                <4 RK_PC2 1 &pcfg_pull_none>;
};

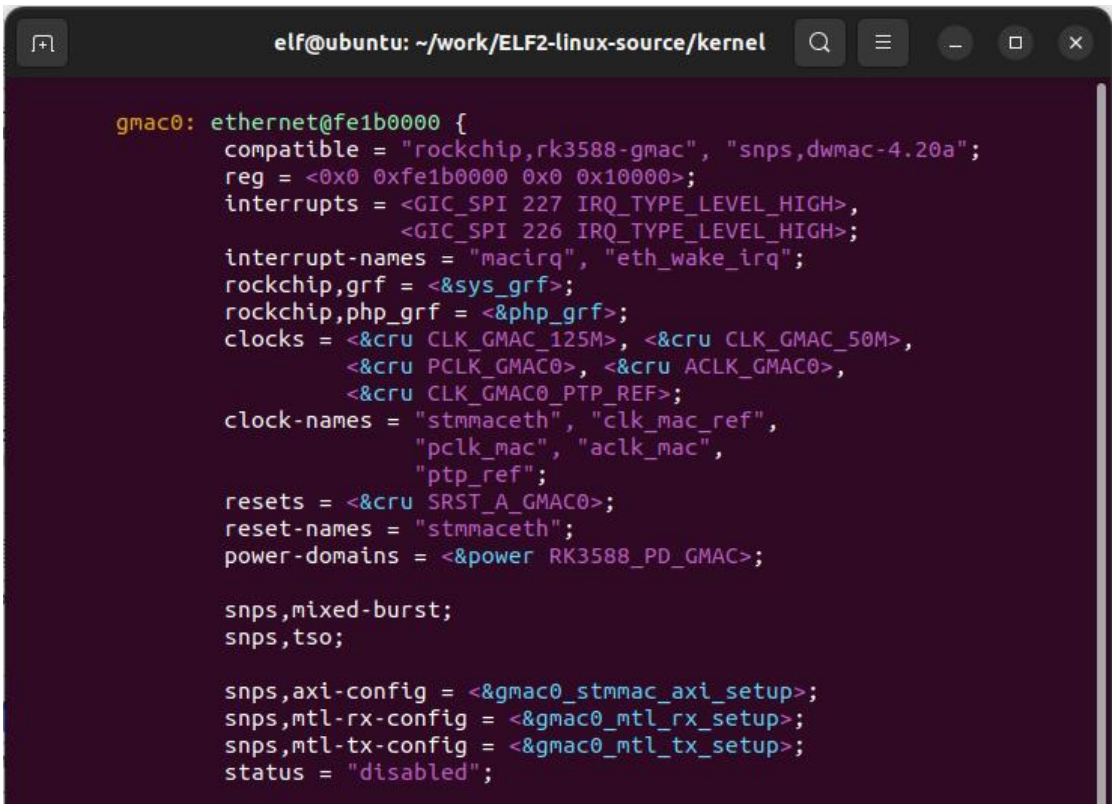
/omit-if-no-ref/
gmac0_tx_bus2: gmac0-tx-bus2 {
        rockchip,pins =
                /* gmac0_txd0 */
                <2 RK_PB6 1 &pcfg_pull_none>,
                /* gmac0_txd1 */
                <2 RK_PB7 1 &pcfg_pull_none>,
                /* gmac0_txen */
                <2 RK_PC0 1 &pcfg_pull_none>;
};
                                                92,3-24       14%
```

```
/omit-if-no-ref/
gmac0_rgmii_clk: gmac0-rgmii-clk {
        rockchip,pins =
                /* gmac0_rxclk */
                <2 RK_PB0 1 &pcfg_pull_none>,
                /* gmac0_txclk */
                <2 RK_PB3 1 &pcfg_pull_none>;
};

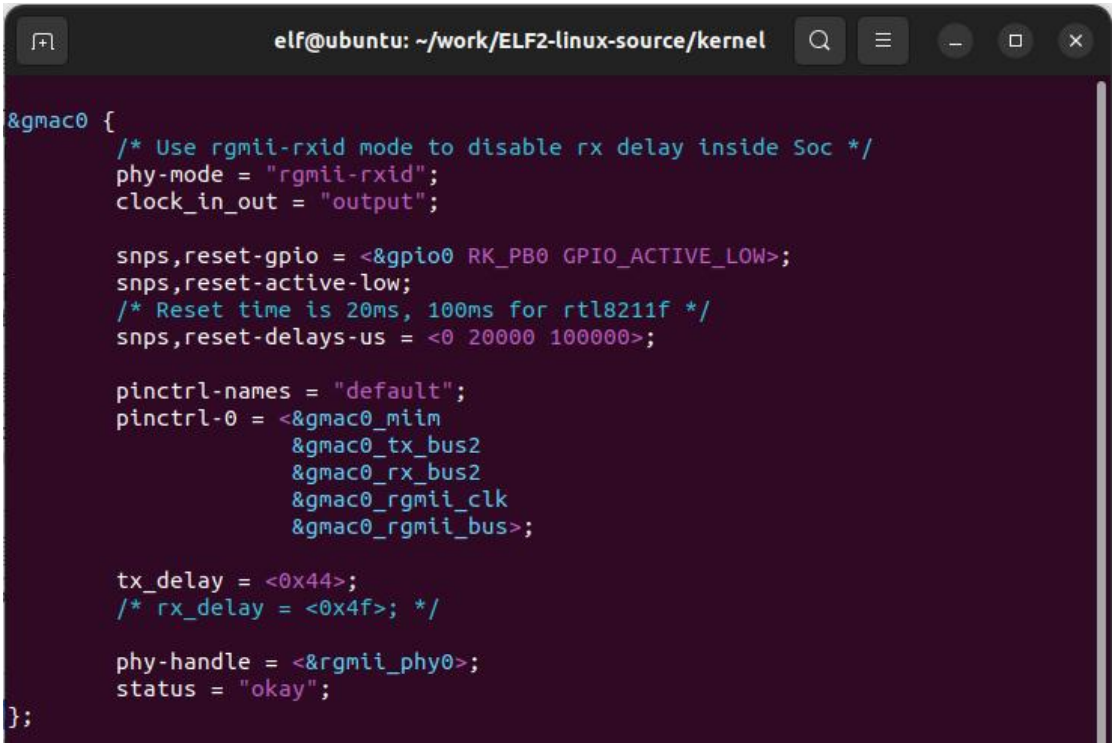
/omit-if-no-ref/
gmac0_rgmii_bus: gmac0-rgmii-bus {
        rockchip,pins =
                /* gmac0_rxd2 */
                <2 RK_PA6 1 &pcfg_pull_none>,
                /* gmac0_rxd3 */
                <2 RK_PA7 1 &pcfg_pull_none>,
                /* gmac0_txd2 */
                <2 RK_PB1 1 &pcfg_pull_none>,
                /* gmac0_txd3 */
                <2 RK_PB2 1 &pcfg_pull_none>;
};
                                                114,3-24      19%
```

1.10.5.3 设备节点配置

arch/arm64/boot/dts/rockchip/rk3588.dtsi 中默认定义了 gmac0 的节点：



另外我们需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中追加节点的相关配置：



phy-mode = "rgmii-rxid";：指定 GMAC 使用 RGMII（Reduced Gigabit Media Independent Interface）协议，并启用 rxid 模式。硬件上 GMAC0_RXD0_M0 接了上拉，所以启用了 PHY 的 RX delay，此处模式要修改为”rgmii-rxid”。

clock_in_out = "output";：这个属性指定时钟信号的方向为输出，表明 GMAC 将输出所需的时钟信号给外部 PHY 设备。

snps,reset-gpio = <&gpio0 RK_PB0 GPIO_ACTIVE_LOW>;：这是一个 GPIO 配置，用于控制 GMAC 的复位引脚。这里指定 gpio0 控制器的引脚 RK_PB0 被用作复位，低电平有效。

snps,reset-delays-us = <0 20000 100000>;：这个属性定义了复位过程中的时间延迟（以微秒为单位）。第一个值 0 表示无延迟（复位触发后立即进行设置）。第二个值 20000 是复位保持时间的延迟（20ms）。第三个值 100000 是复位释放后的延迟（100ms）用于某些 PHY，像 rtl8211f。

pinctrl-0 ：指定所使用的引脚描述。

tx_delay = <0x44>;：指定发送时延，用于改善传输数据的时序，这里的值 0x44（十六进制）表示特定的时延设置。

phy-handle = <&rgmii_phy0>;：指向连接到 GMAC0 的 PHY 芯片的设备树节点，通常用于指定实际的网络物理层组件。

1.10.5.4 测试

将开发板网口通过网线连接到电脑上，同时将 IP 设置成与电脑同一网段：

```
root@elf2-buildroot:~# ifconfig eth0 172.20.2.160
```

设置完成后 ping 电脑的 IP 地址：

```
root@elf2-buildroot:~# ping 172.20.2.60
PING 172.20.2.60 (172.20.2.60) 56(84) bytes of data.
64 bytes from 172.20.2.60: icmp_seq=1 ttl=128 time=0.938 ms
64 bytes from 172.20.2.60: icmp_seq=2 ttl=128 time=0.480 ms
64 bytes from 172.20.2.60: icmp_seq=3 ttl=128 time=0.479 ms
```

1.10.6 Sound

在设备树中音频相关的节点主要包含三部分，分别是 DAI、Codec 和 Machine。

1.DAI（Digital Audio Interface）

DAI 是数字音频接口的简称，它是处理器与外部音频设备（如 Codec）之间传输音频数据的接口。DAI 主要描述了音频信号的传输方式，常见的 DAI 类型包括 I²S、TDM、PDM、S/PDIF 等。

2.Codec（编解码器）

Codec 是音频编解码器的简称，通常用于将模拟信号转换为数字信号（ADC），或者将数字信号转换为模拟信号（DAC）。Codec 是音频系统中的一个重要组件，它既可以是一个独立的硬件模块，也可以内置在处理器中。

3.Machine Driver（机器驱动）

Machine Driver 是 ASoC 框架中的一个特殊组件，它负责连接音频处理器的 DAI 和外部 Codec，同时配置音频系统的拓扑结构。Machine Driver 是音频系统的集成部分，定义了整个音频路径的工作方式。

在 ASoC（ALSA System on Chip）音频子系统中，DAI、Codec 和 Machine Driver 紧密协作以实现音频数据的处理和传输：

DAI 驱动 定义了处理器和外部设备之间的音频接口和传输方式（如 I²S、TDM）。

Codec 驱动 负责处理模拟和数字音频信号的编解码功能，并与 DAI 驱动进行数据交换。

Machine Driver 作为系统的协调者，将处理器的 DAI 驱动与 Codec 驱动连接起来，同时根据硬件平台配置音频路径和数据流向。

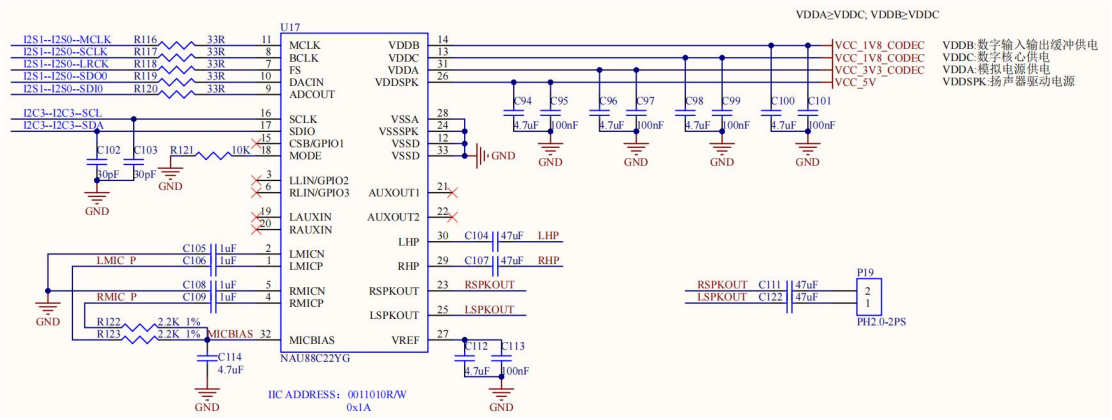
在 ELF2 开发板中各部分的作用如下：

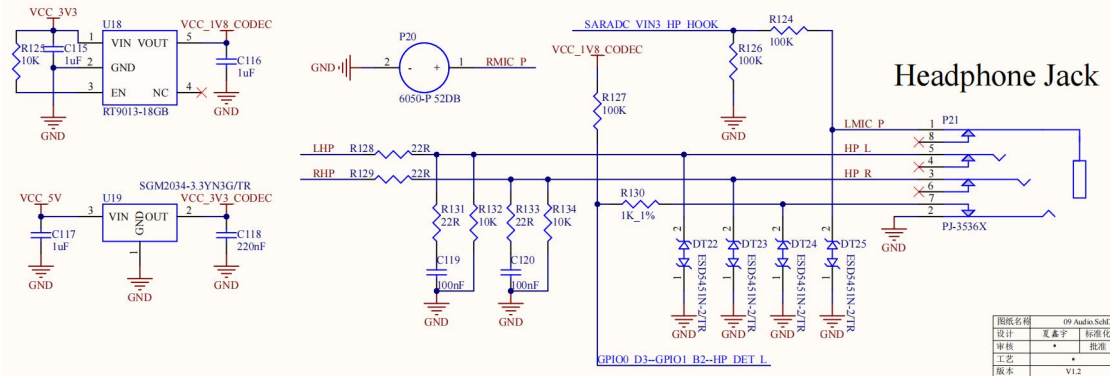
DAI 驱动：负责配置 RK3588 的 I²S 接口和 PDM 接口的音频数据传输参数。

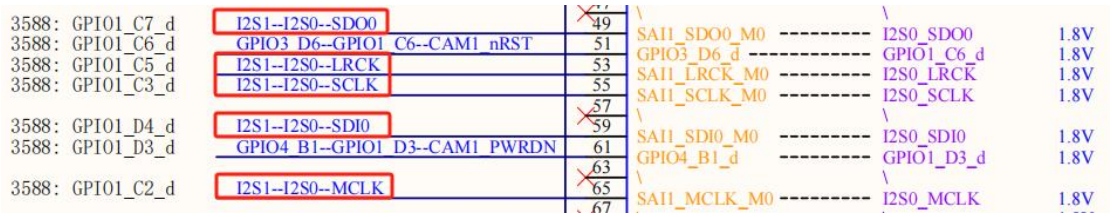
Codec 驱动：控制外部的音频编解码器，进行模拟和数字信号的转换。

Machine Driver：负责将 I²S 和 PDM 接口的 DAI 驱动与 Codec 驱动连接起来，同时根据实际需要配置音频流的路由。

1.10.6.1 硬件原理

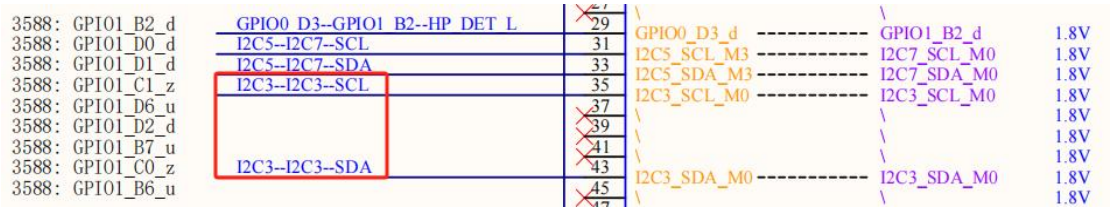






连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7
P2_49	I2S0_SDO0			I2C4_SCL_M4		UART4_CTSN	
P2_51			PDM0_CLK0_M0	I2C4_SDA_M4	PWM15_IR_M2		
P2_53	I2S0_LRCK			I2C2_SCL_M3		UART4_RTSN	
P2_55	I2S0_SCLK			I2C6_SCL_M1	PWM7_IR_M2	UART3_CTSN	SPI4_CS0_M0
P2_57			PDM0_SDI0_M0				SPI1_CS1_M2
P2_59	I2S0_SDI0						

连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7
P2_65	I2S0_MCLK			I2C6_SDA_M1	PWM3_IR_M2	UART3_RTSN	SPI4_CLK_M0



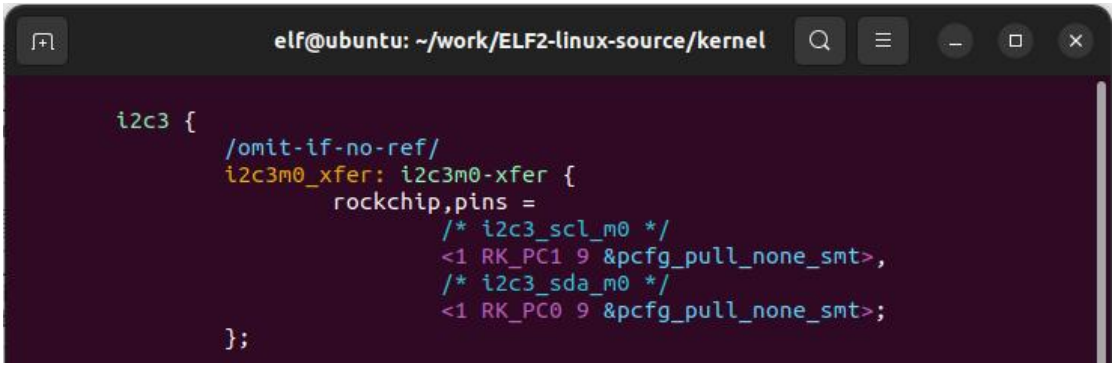
连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7
P2_43				I2C3_SDA_M0		UART3_RX_M0	SPI4_MISO_M0

连接器引脚号	Alt1	Alt2	Alt3	Alt4	Alt5	Alt6	Alt7
P2_35				I2C3_SCL_M0		UART3_TX_M0	SPI4_MOSI_M0

从原理图中可以看出 ELF2 开发板使用的是 I2S0 这一组引脚来传输数字音频信号，使用 I2C3 这一组引脚来对 Codec 音频芯片进行配置。

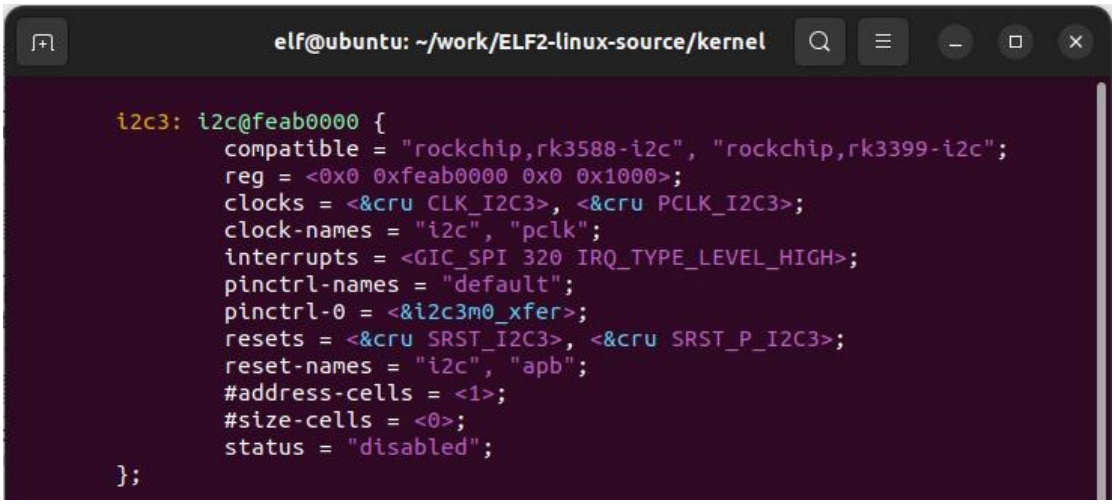
1.10.6.2 IOMUX 配置

确定完引脚，可以在设备树 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 中查找引脚相关描述：



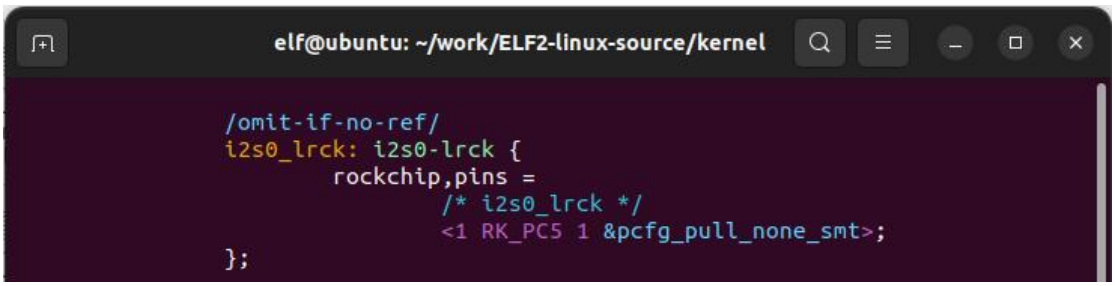
从原理图中可以看出 I2C3 使用的引脚为 GPIO1_C0 和 GPIO1_C1，对应的引脚描述为 i2c3m0_xfer。

在设备树 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 默认已经添加了 i2c3 的节点，并且默认配置的引脚就为 i2c3m0_xfer。

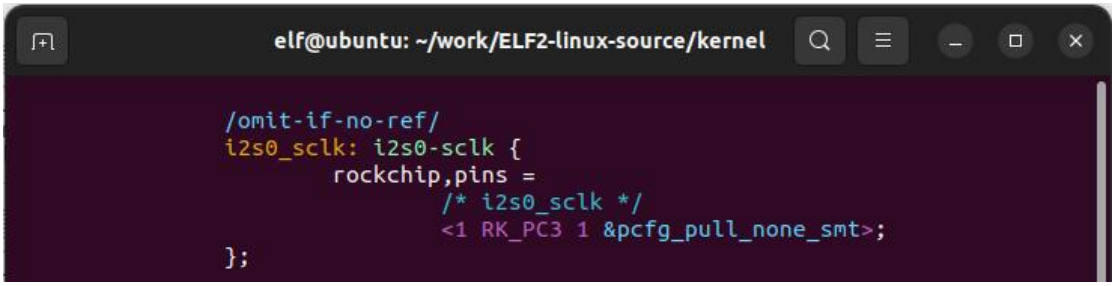


I2S0 的引脚从原理图中可以看出，使用的为：GPIO1_C5、GPIO1_C3、GPIO1_D4、GPIO1_C7 和 GPIO1_C2。

i2s0_lrck:



i2s0_sclk:



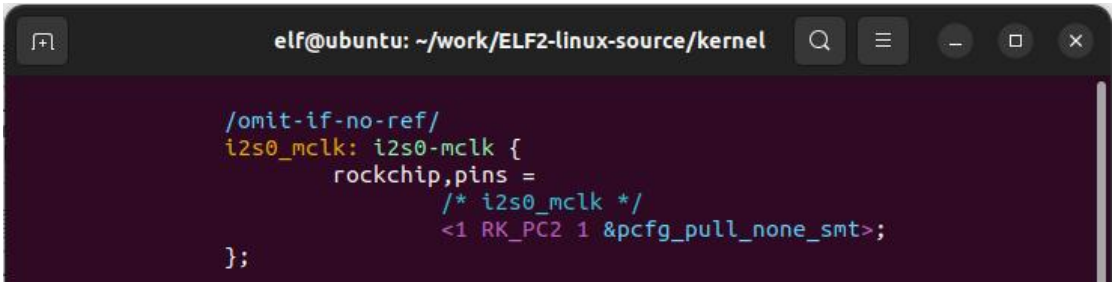
i2s0_sdi0:



i2s0_sdo0:

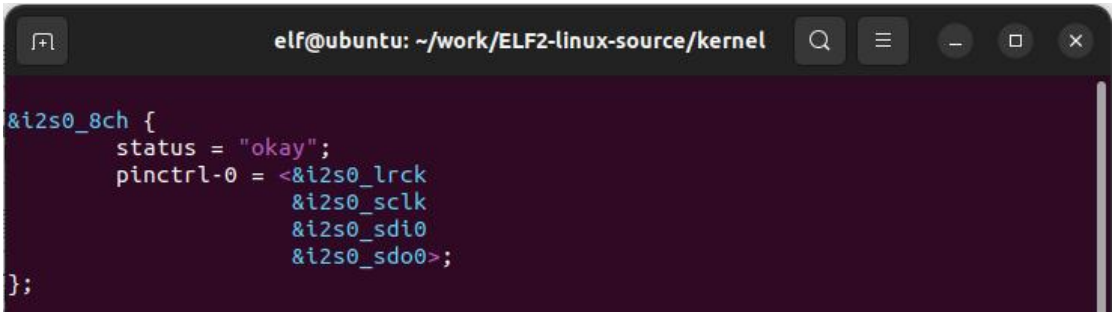


i2s0_mclk:

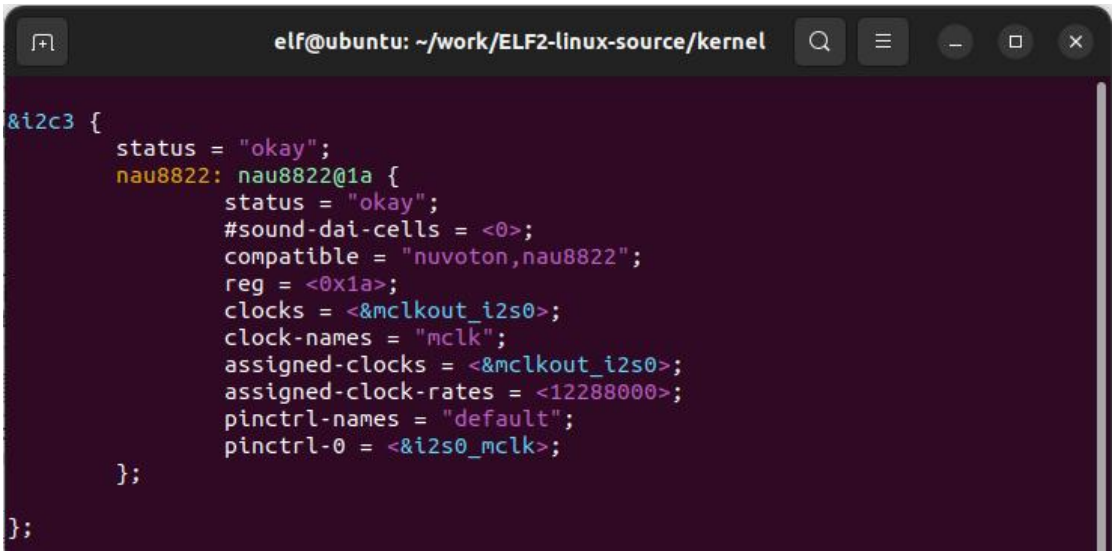


1.10.6.3 设备节点配置

DAI 部分需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中引用 i2s0_8ch，将查询到的引脚描述添加上，并且将 status 状态设置为”okay”。



codec 部分需要在 i2c3 节点下创建子节点，用来描述 codec 芯片。



compatible = "nuvoton,nau8822";：标识了设备的兼容性，用来匹配驱动。

reg = <0x1a>;：即 I2C 地址为 0x1a。

clocks = <&mclkout_i2s0>;：指定了设备使用的时钟，这里连接到了名为 mclkout_i2s0 的时钟控制器。

clock-names = "mclk";：指定了时钟的名称为 mclk。

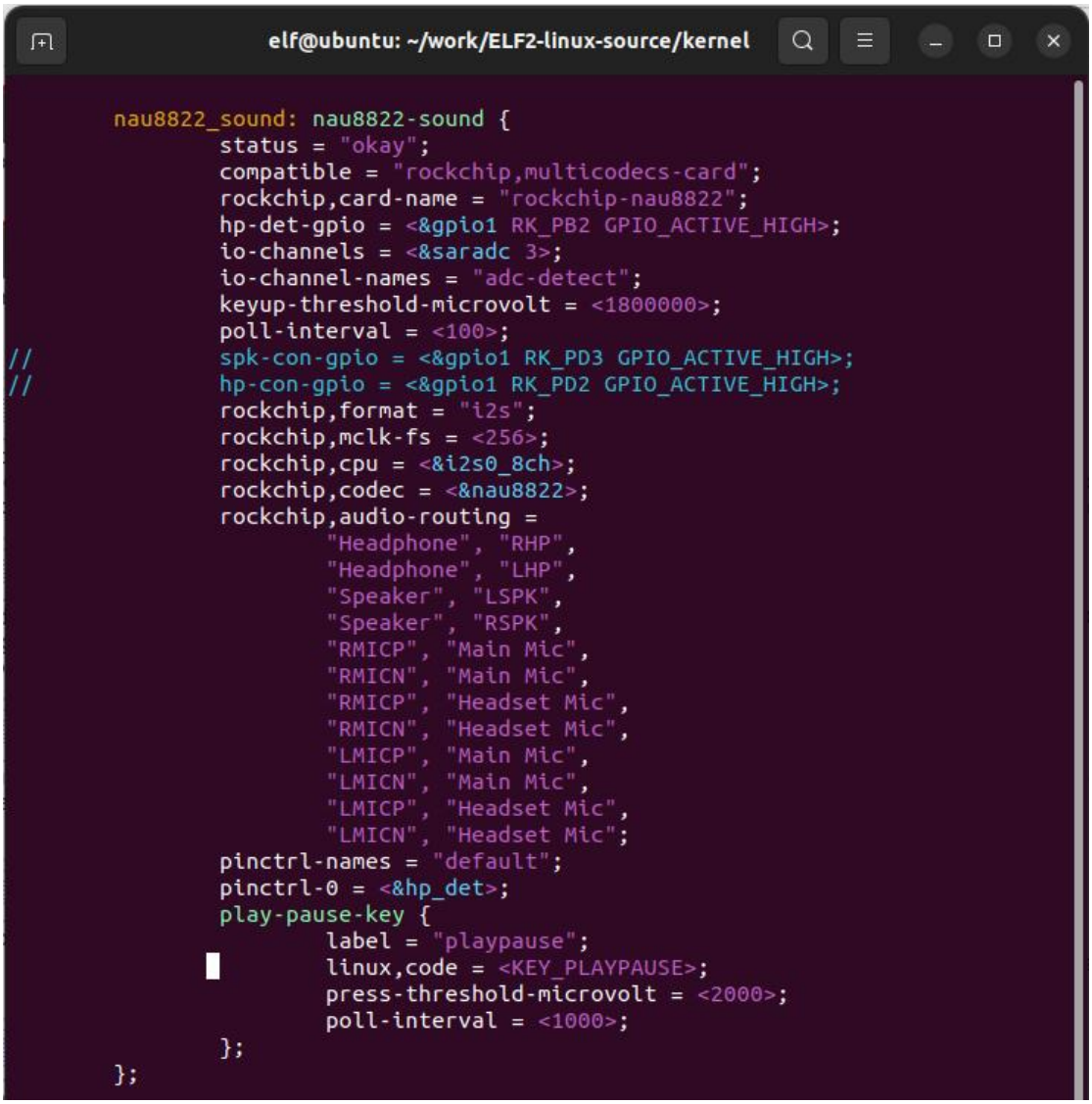
assigned-clocks = <&mclkout_i2s0>;：指定了设备使用的时钟为 mclkout_i2s0。

assigned-clock-rates = <12288000>;：指定了时钟速率为 12288000。

pinctrl-names = "default";：设备的引脚控制配置中定义了一个名称为 default 的配置。

pinctrl-0 = <&i2s0_mclk>;：引脚控制配置中指定了使用 i2s0_mclk 用来做 i2s 的 mclk 引脚。

Machine 部分是在设备树 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中的 nau8822_sound 节点中描述的。



compatible = "rockchip,multicodecs-card";：标识设备的兼容性，用来匹配驱动。

rockchip,card-name = "rockchip-nau8822";：指定音频卡的名称为 "rockchip-nau8822"。

hp-det-gpio = <&gpio1 RK_PB2 GPIO_ACTIVE_HIGH>;：指定了耳机检测引脚的 GPIO 控制器和配置，通过 gpio1 的引脚 RK_PB2 检测耳机状态为高电平表示插入。

io-channels = <&saradc 3>;：指定了输入/输出通道，连接到 saradc 控制器的第三个通道，同样可以通过 adc 的第三通道来检测耳机的插拔。

keyup-threshold-microvolt = <1800000>;：指定输入电压的阈值，当超过 1800000 微伏时被视为按键释放。

poll-interval = <100>;：指定轮询输入设备的间隔，单位为毫秒。

rockchip,format = "i2s";：指定音频数据的格式为 I2S。

rockchip,mclk-fs = <256>;：指定主时钟频率（Master Clock Frequency）的除数，这里设置为 256。

rockchip,cpu = <&i2s0_8ch>;：指定处理器使用的 I2S 接口，这里使用名为 i2s0_8ch 的 I2S 接口。

rockchip,codec = <&nau8822>;：指定了该音频卡使用的 Codec，连接到 nau8822 设备。

rockchip,audio-routing = ... ;：定义了音频流的路由路径，描述了不同输入和输出之间的连接关系。

1.10.6.4 编译测试

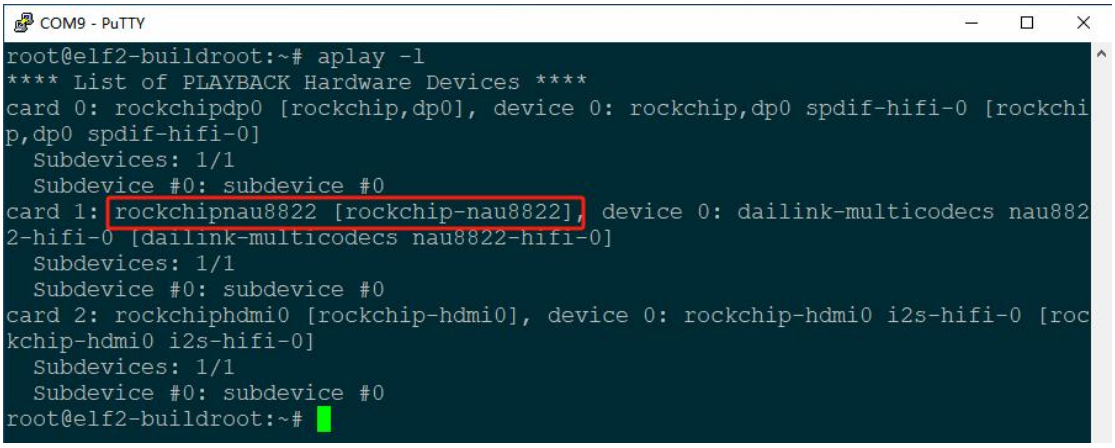
编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

输入 aplay -l 查询声卡设备：

```
root@elf2-buildroot:~# aplay -l
```



可以看到设备为 card1，device0。

设置音频 PCM 参数：

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "PCM" 255
Simple mixer control 'PCM',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 255
  Front Left: 255 [100%] [0.00dB]
  Front Right: 255 [100%] [0.00dB]
```

打开 Headphone：

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Headphone" on
Simple mixer control 'Headphone',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
```

设置音量：

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Headphone" 63
Simple mixer control 'Headphone',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
```

播放音频文件：

```
root@elf2-buildroot:~#  gst-play-1.0  /data/piano2-CoolEdit.mp3  --audiosink="alsasink
```

```
device=plughw:1,0"
Press 'k' to see a list of keyboard shortcuts.
Now playing /data/piano2-CoolEdit.mp3
Redistribute latency...
Redistribute latency...
0:00:04.6 / 0:00:06.3
```

播放 mp3 文件后能在耳机中听到声音输出。

1.10.7 MIPI-CSI

MIPI-CSI（Camera Serial Interface） 是由 MIPI（Mobile Industry Processor Interface）联盟推出的串行接口标准，主要用于连接摄像头模块，提供高速的数据传输。

CSI（Camera Serial Interface）和 DSI（Display Serial Interface）都是由 MIPI（Mobile Industry Processor Interface）联盟制定的串行接口标准，但它们有着不同的用途和功能，主要区别如下：

1.用途

CSI（Camera Serial Interface）：主要用于连接摄像头模块，将摄像头采集到的图像或视频数据传输到处理器。常见于手机、安防监控、无人机等需要图像采集的设备。

DSI（Display Serial Interface）：主要用于连接显示屏模块，将处理器生成的图像或视频内容发送到显示设备。常见于智能手机、平板电脑、车载显示屏等设备。

2.数据流方向

CSI：数据流是从摄像头到处理器，即输入设备到处理器的数据流。

DSI：数据流是从处理器到显示屏，即从处理器输出图像数据到显示设备。

3.协议标准

CSI：采用 MIPI-CSI 协议，用于处理视频/图像数据的输入。支持高速传输图像数据，特别是在高分辨率和高帧率摄像头上。

DSI：采用 MIPI-DSI 协议，用于处理视频/图像数据的输出。支持高分辨率显示屏，能够高效传输显示内容，支持触摸屏等附加功能。

RK3588 的 camera 通路：

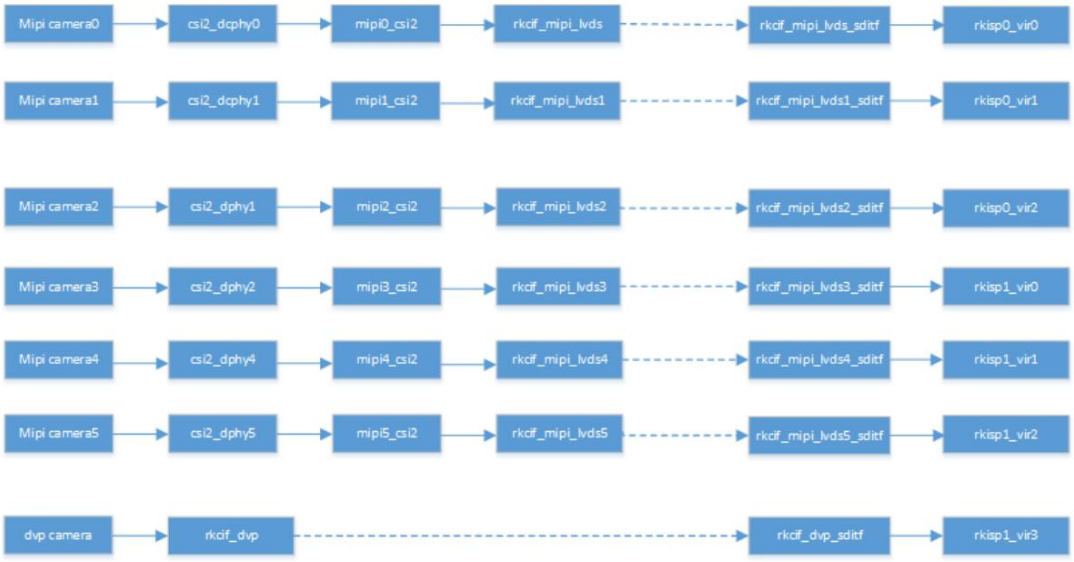
多 sensor 支持

单路硬件 isp 最多支持 4 路复用，isp 复用情况支持分辨率如下：

2 路复用：最大分辨率 3840x2160，dts 对应配置 2 路 rkisp_vir 设备。

3 路或 4 路复用：最大分辨率 2560x1536，dts 对应配置 3 或 4 路 rkisp_vir 设备。

硬件支持最多采集 7 路 sensor：6mipi + 1dvp，多 sensor 软件通路如下：



rk3588 支持两个 dcphy，节点名称分别为 csi2_dcphy0/csi2_dcphy1。每个 dcphy 硬件支持 RX/TX 同时使用，对于 camera 输入使用的是 RX。支持 DPHY/CPHY 协议复用；需要注意的是同一个 dcphy 的 TX/RX 只能同时使用 DPHY 或同时使用 CPHY。其他 dcphy 参数请查阅 rk3588 数据手册。

rk3588 支持 2 个 dphy 硬件，这里我们称之为 dphy0_hw/dphy1_hw，两个 dphy 硬件都可以工作在 fullmode 和 split mode 两种模式下。

1.dphy0_hw

full mode：节点名称使用 csi2_dphy0，最多支持 4 lane。

split mode： 拆分成 2 个 phy 使用，分别为 csi2_dphy1（使用 0/1 lane）、csi2_dphy2(使用 2/3 lane)，每个 phy 最多支持 2 lane。

当 dphy0_hw 使用 full mode 时，链路需要按照 csi2_dphy1 这条链路来配置，但是节点名称 csi2_dphy1 需要修改为 csi2_dphy0，软件上是通过 phy 的序号来区分 phy 使用的模式。

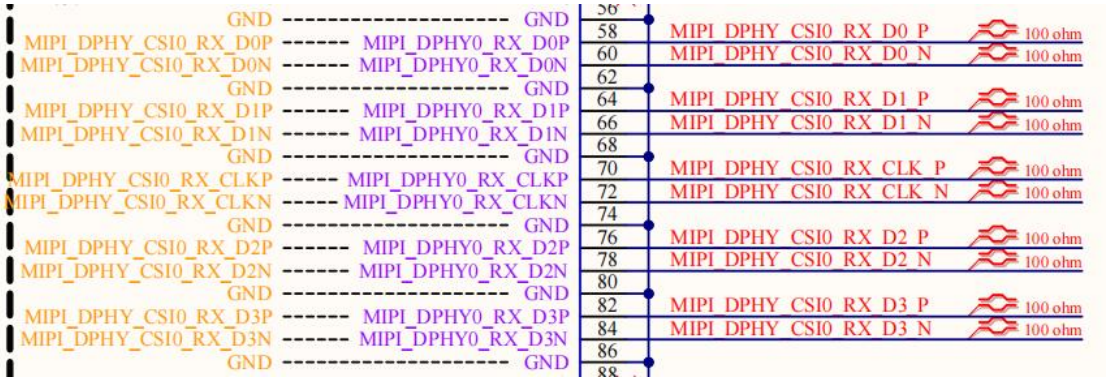
2.dphy1_hw

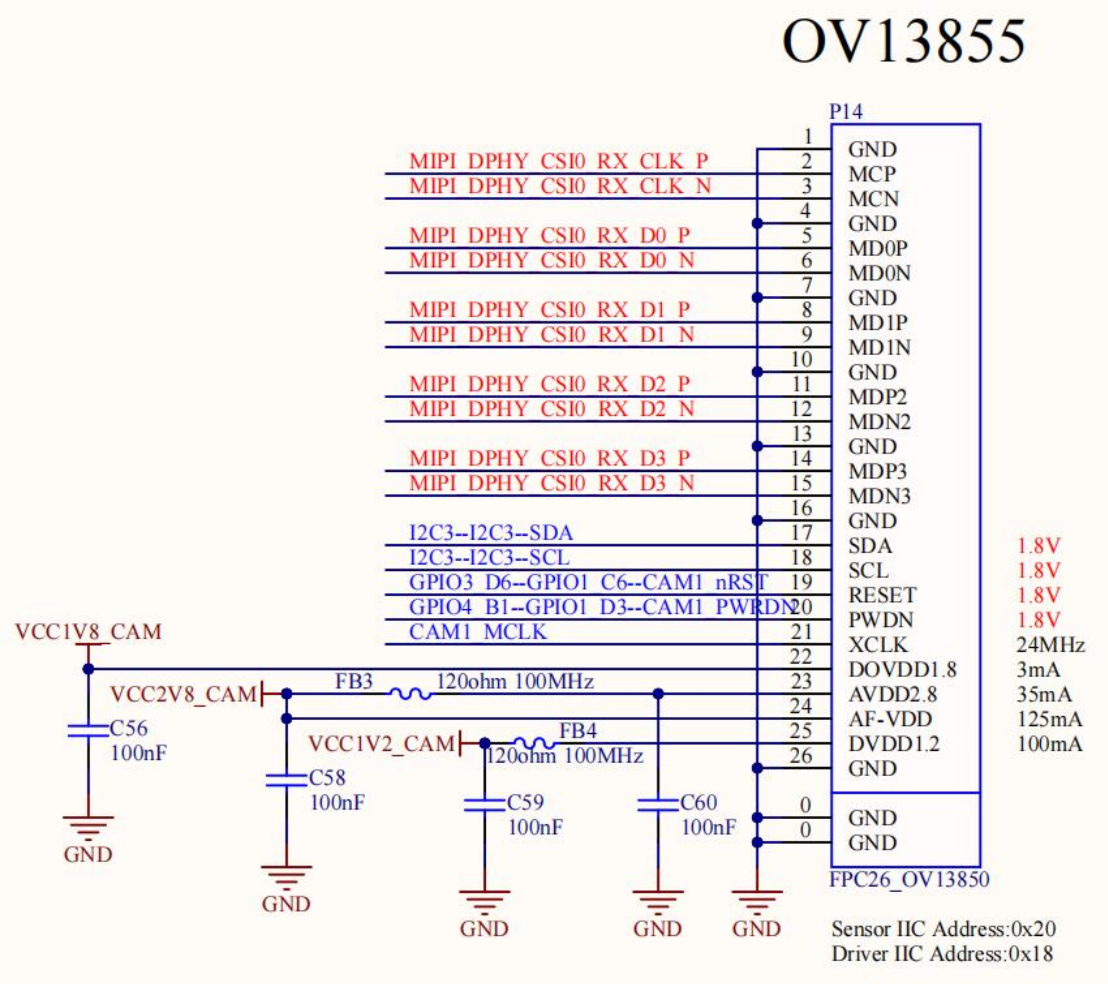
full mode：节点名称使用 csi2_dphy3，最多支持 4 lane。

split mode： 拆分成 2 个 phy 使用，分别为 csi2_dphy4（使用 0/1 lane）、csi2_dphy5(使用 2/3 lane)，每个 phy 最多支持 2 lane。

当 dphy1_hw 使用 full mode 时，链路需要按照 csi2_dphy4 这条链路来配置，但是节点名称 csi2_dphy4 需要修改为 csi2_dphy3，软件上是通过 phy 的序号来区分 phy 使用的模式。

1.10.7.1 硬件原理





MIPI_DPHY_CSI0_RX_CLK 是数据传输参考时钟。

MIPI_DPHY_CSI0_RX_D[3:0]数据信号引脚。

I2C3_SDA 和 I2C3_SCL 用来配置摄像头设备，地址为 0x36。

CAM1_nRST 是摄像头的复位引脚。

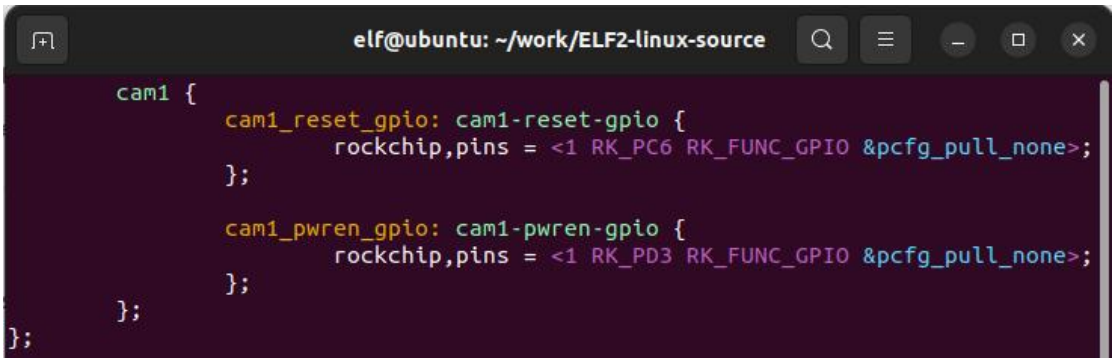
CAM1_PWRDN 是摄像头的电源使能引脚。

CAM1_MCLK 是摄像头的工作时钟，由外部晶振提供。

1.10.7.2 IOMUX 配置

由于 MIPI-CSI 接口使用的是专属引脚，所以不需要 IOMUX 配置。需要添加的为复位引脚和电源使能引脚。从原理图中可以看到复位引脚使用的是 GPIO1_C6，电源使能引脚为 GPIO1_D3。

打开 kernel/arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 设备树在&pinctrl 节点下添加 cam1：

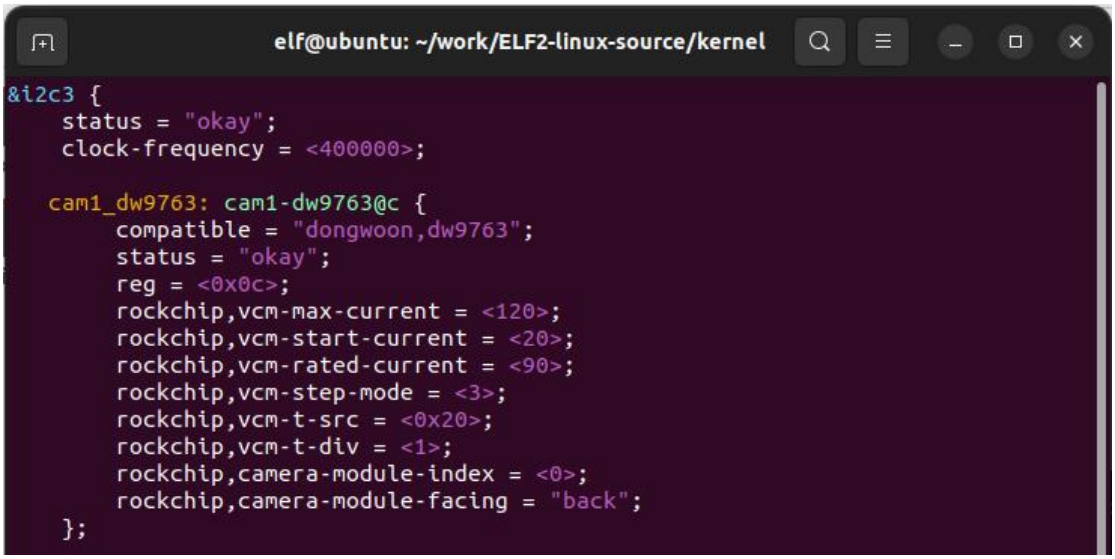


1.10.7.3 设备节点配置

下面以讲解适配的 ov13855 为例进行讲解。

路由链路：mipicamera0 --> csi2_dcphy0 --> mipi0_csi2 -->rkcif_mipi_lvds --> rkcif_mipi_lvds_sditf --> rkisp0_vir0

ov13855 的相关配置在设备树 arch/arm64/boot/dts/rockchip/elf2-3588-Camera.dtsi 中。摄像头是连接在 i2c3 下的，在 i2c3 下添加相关节点：



cam1_dw9763 定义的是与摄像头相关的 VCM（Voice Coil Motor，音圈马达）模块，用于摄像头的自动对焦。定义了设备的兼容性信息，表明该节点描述的硬件设备是由 Dongwoon 生产的 DW9763 VCM 驱动器。驱动程序通过此属性来匹配和加载对应的设备驱动。

reg = <0x0c>;定义该设备的 I2C 地址。

rockchip,vcm-max-current = <120>;定义了 VCM 驱动器的最大工作电流，单位通常为 mA。

rockchip,vcm-start-current = <20>;定义了 VCM 启动时的初始电流（单位：mA）。

rockchip,vcm-rated-current = <90>;定义了 VCM 的额定工作电流（单位：mA）。

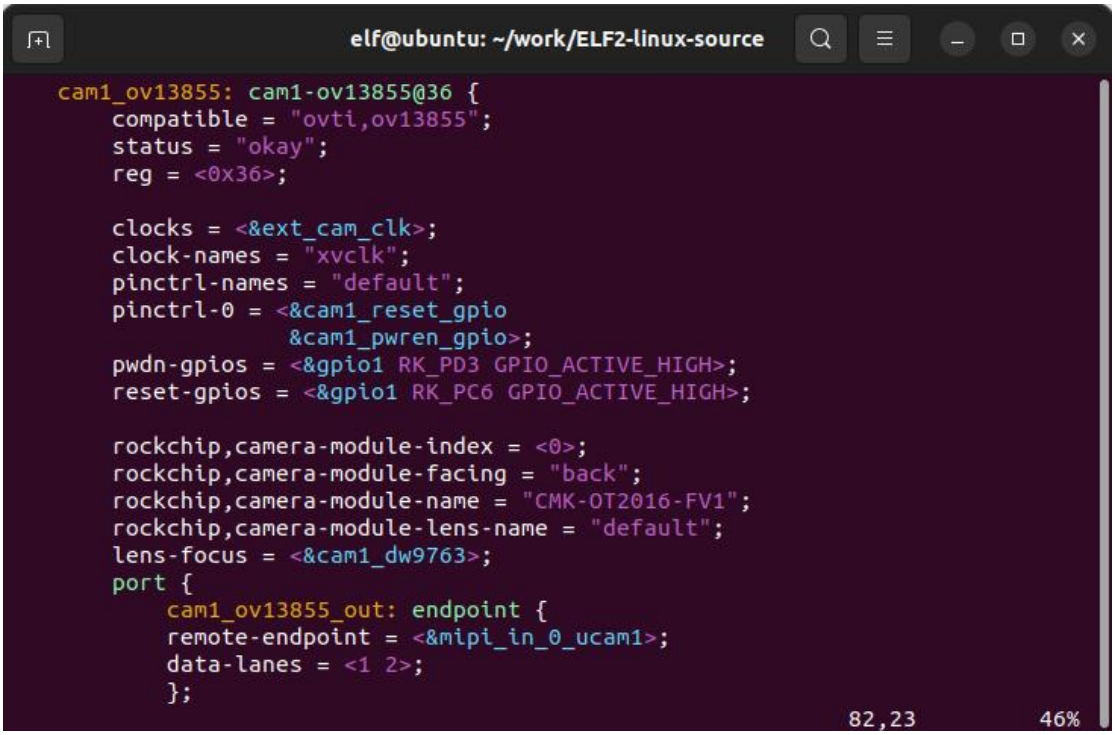
rockchip,vcm-step-mode = <3>;定义 VCM 的步进模式，通常与控制马达移动的精细程度相关。

rockchip,vcm-t-src = <0x20>;这个参数定义了 VCM 驱动中的时间源设置（单位：一般为时钟周期）。该值 0x20 是特定时间源的配置值，用于决定驱动器如何进行时间控制，例如控制对焦动作的时长。

rockchip,vcm-t-div = <1>;时间分频器，用于进一步分配或减少时间源的频率。值为 1 表示时间源不进行分频，直接使用原始时间源。

rockchip,camera-module-index = <0>;定义了摄像头模块的索引，通常用于多摄像头系统中标识不同的摄像头模块。0 表示这是系统中的第一个摄像头。

rockchip,camera-module-facing = "back";定义摄像头模块的朝向。"back" 表示这是后置摄像头，可能还会有 "front" 用于前置摄像头的配置。



cam1_ov13855 描述的是 OV13855 传感器的相机模块。

pinctrl-0；引用 IOMUX 配置中定义的复位引脚和电源使能引脚。

pwdn-gpios = <&gpio1 RK_PD3 GPIO_ACTIVE_HIGH>;电源关闭 GPIO 引脚，高电平有效，由 gpio1_d3 控制。

reset-gpios = <&gpio1 RK_PC6 GPIO_ACTIVE_HIGH>;重置 GPIO 引脚，高电平有效，由 gpio1_c6 控制。

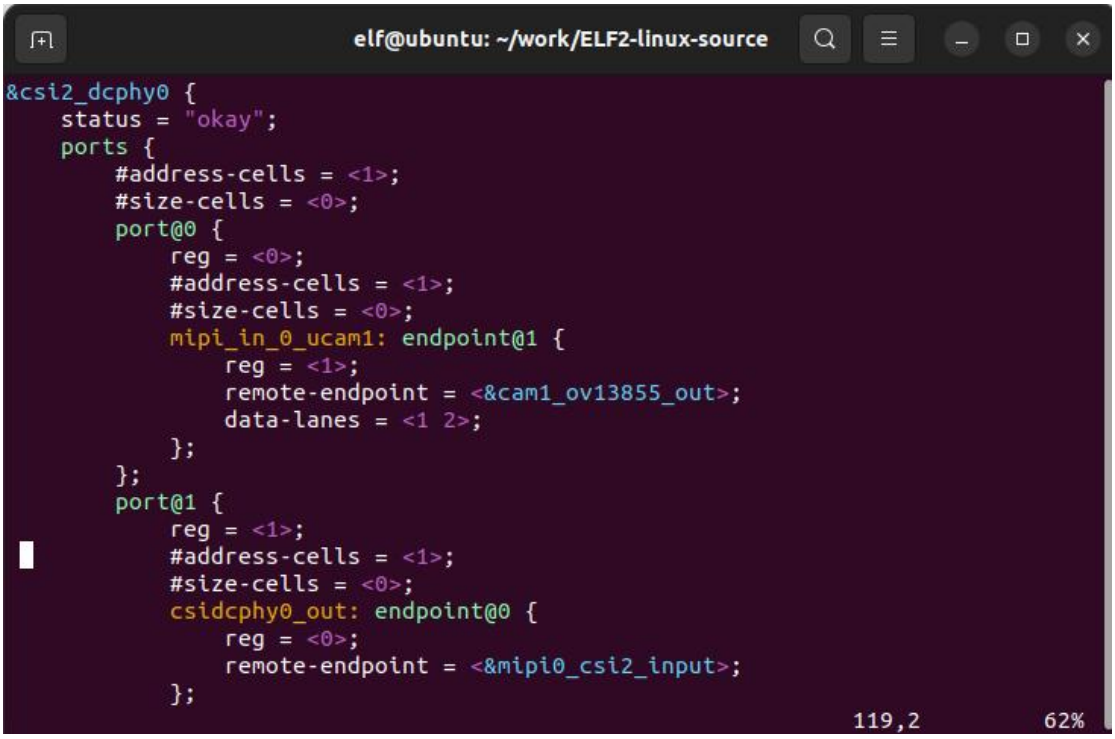
lens-focus = <&cam1_dw9763>;与前边的对焦节点关联。

port 部分定义了连接：

cam1_ov13855_out: endpoint 描述了相机输出的一个端点。

remote-endpoint = <&mipi_in_0_ucam1>; 将其连接到相机接口的 MIPI 输入。

data-lanes = <1 2>; 指定传感器使用的 MIPI 数据通道。

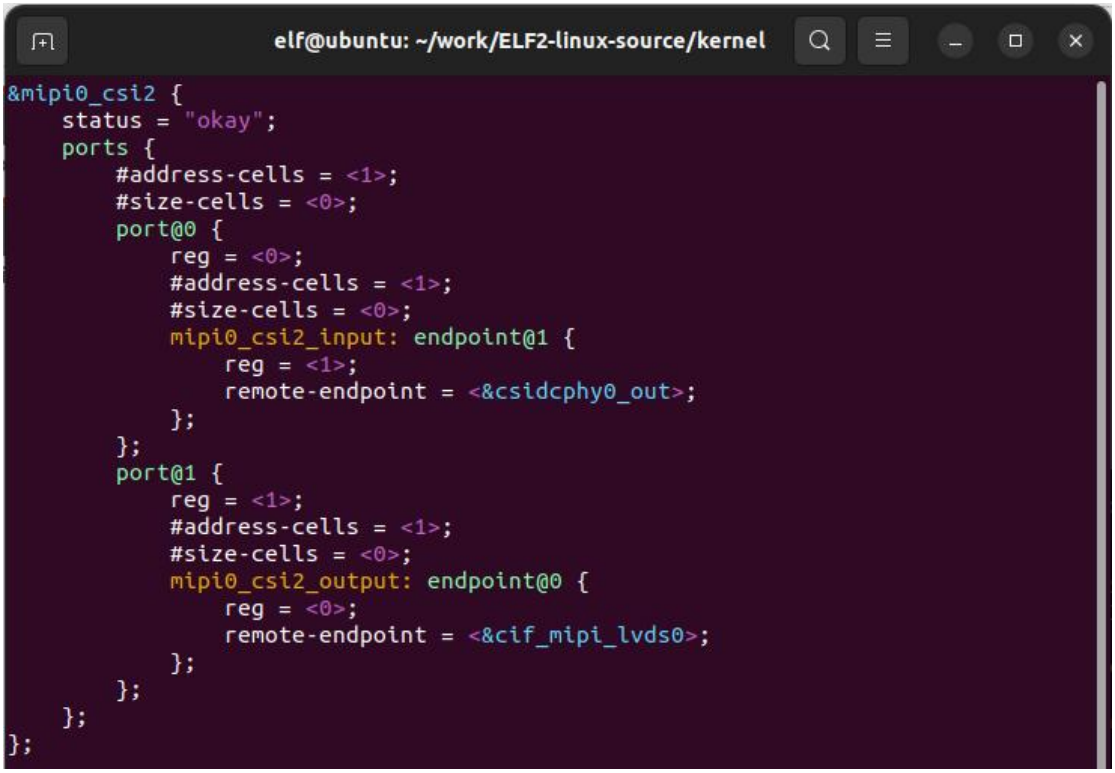


csi2_dcphy0 节点为 CSI2 数据和时钟物理层 (DC-PHY) 设备节点，用于处理从摄像头传感器接收的 MIPI CSI2 数据。

remote-endpoint = <&cam1_ov13855_out>;：将此端点连接到来自 cam1_ov13855 的输出端点，即摄像头传感器的数据输出。

data-lanes = <1 2 >;：使用 MIPI CSI2 数据通道 1、2 进行数据传输。

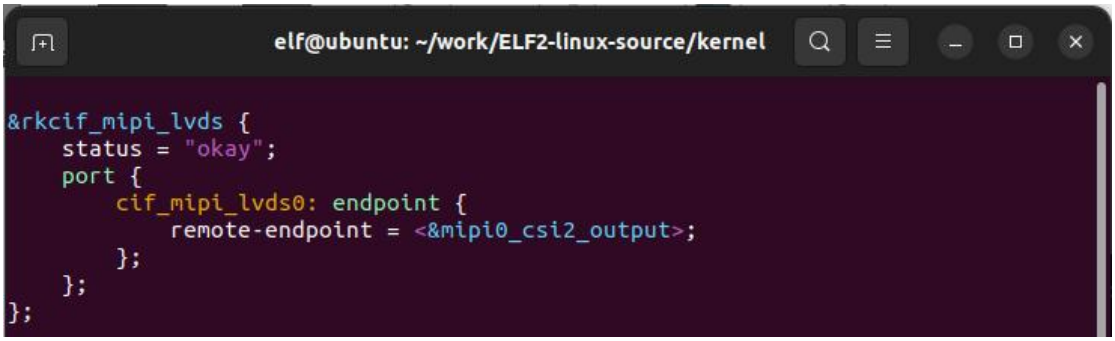
remote-endpoint = <&mipi0_csi2_input>;：将此端点连接到 mipi0_csi2_input，通常这是摄像头接口控制器的输入端。



mipi0_csi2：表示 MIPI CSI2 接口的设备节点，处理来自 MIPI 摄像头传感器的数据。

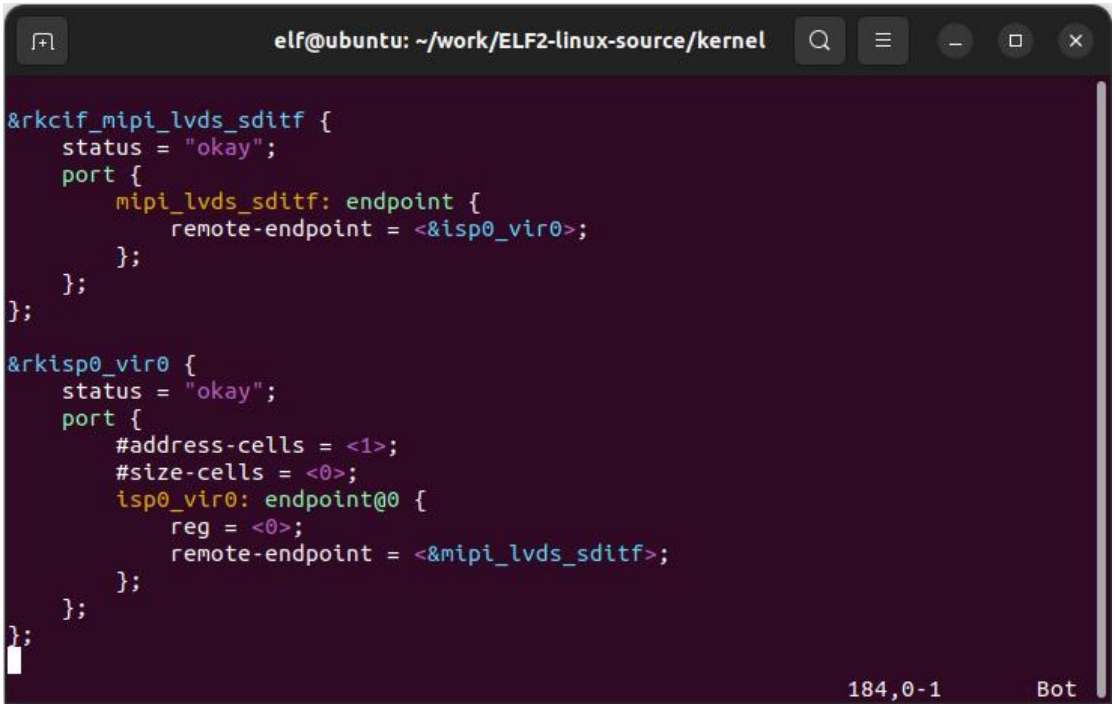
remote-endpoint = <&csidcphy0_out>;：将此端点与 csidcphy0_out 连接，即从 CSI2 PHY 接收数据。这是来自物理层的数据输入端。

remote-endpoint = <&cif_mipi_lvds0>;：将该端点与 cif_mipi_lvds0 连接，通常这是将数据传递给下游处理器或图像接口模块的输出端。



rkcif_mipi_lvds：表示 Rockchip 摄像头接口（CIF）的 MIPI/LVDS 设备节点，用于接收和处理来自 MIPI 或 LVDS 摄像头接口的数据。

remote-endpoint = <&mipi0_csi2_output>;将此端点连接到 mipi0_csi2_output，即从 mipi0_csi2 接收的摄像头数据。这意味着该节点接收经过 MIPI CSI2 接口传递的数据，并将其传递给 Rockchip 的摄像头接口框架进行进一步处理。



rkcif_mipi_lvds_sditf：这是 Rockchip 摄像头接口框架 (CIF) 的 MIPI/LVDS 数据接口节点，表示此接口处理 MIPI/LVDS 的摄像头数据。

remote-endpoint = <&isp0_vir0>;将该端点连接到图像信号处理器 (ISP) 的虚拟节点 isp0_vir0，表示数据将传递给 ISP 进行进一步处理。

rkisp0_vir0：这是 Rockchip 的 ISP（图像信号处理器）的虚拟端口节点，负责处理从摄像头传输来的数据。

remote-endpoint = <&mipi_lvds_sditf>;将该端点连接到 mipi_lvds_sditf，即接收来自 MIPI/LVDS 接口的摄像头数据。

1.10.7.4 编译测试

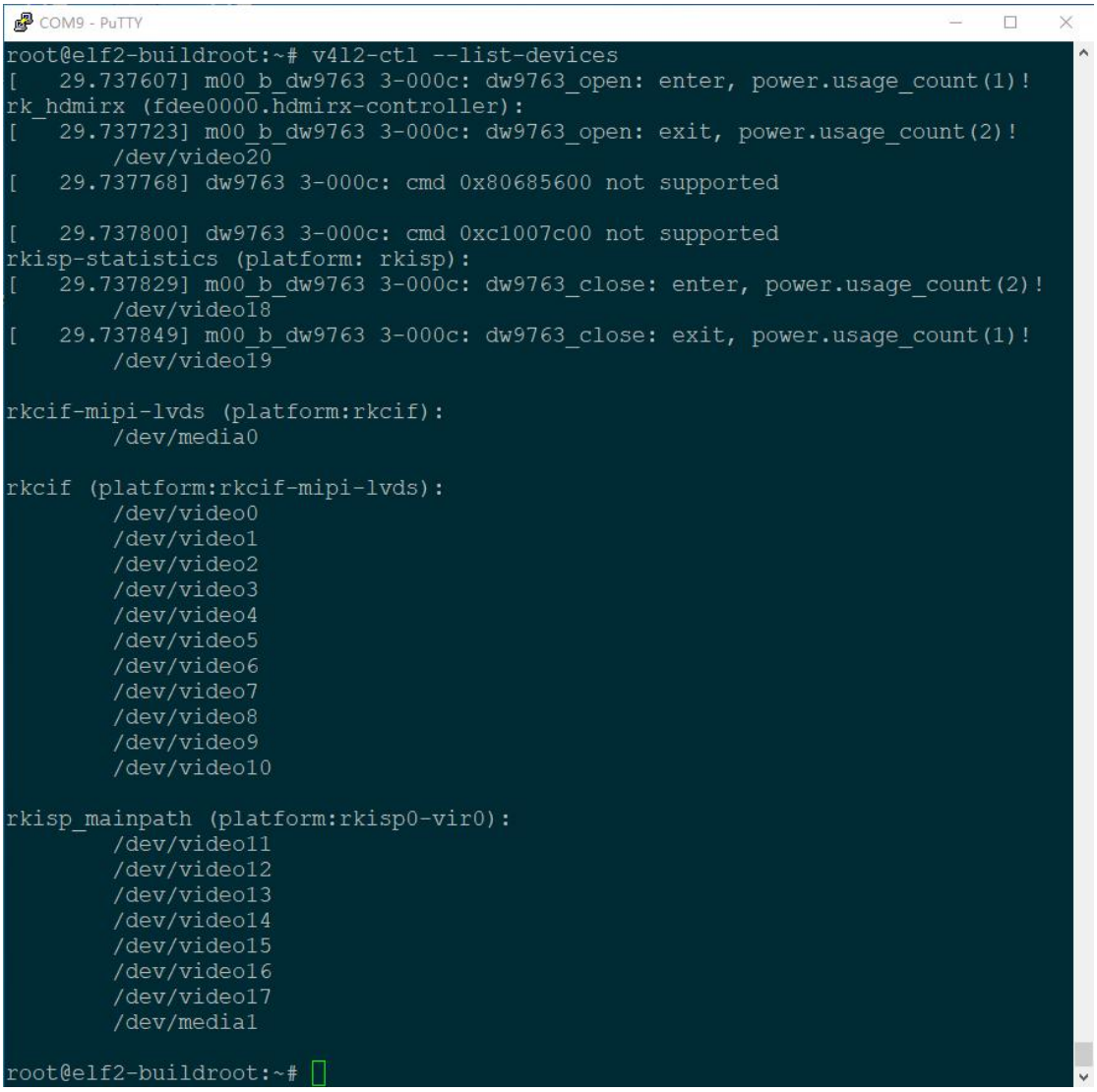
编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

接上 ov13855 摄像头后，重新上电，在命令行终端输入查询指令：

```
root@elf2-buildroot:~# v4l2-ctl --list-devices
```



rkcif-mipi-lvds (platform: rkcif)：这是一个 MIPI/LVDS 摄像头接口节点，通常用来捕获来自 MIPI 或 LVDS 摄像头的数据流。负责接收视频数据并将其传递给图像信号处理器 (ISP) 进行进一步处理。

rkcif (platform: rkcif-mipi-lvds)：这个节点代表 Rockchip 摄像头接口 (CIF)，它用于管理和控制从 MIPI/LVDS 接口接收到的数据。负责图像数据的流控、格式转换及其它处理任务，以确保数据能够顺利流向图像信号处理器 (ISP)。

rkisp_mainpath (platform: rkisp0-vir0)：这是 ISP 的主要数据路径，用于处理来自摄像头的数据。负责执行图像处理任务（例如去噪、色彩校正等），并将处理后的数据输出给显示设备或存储。

摄像头预览：

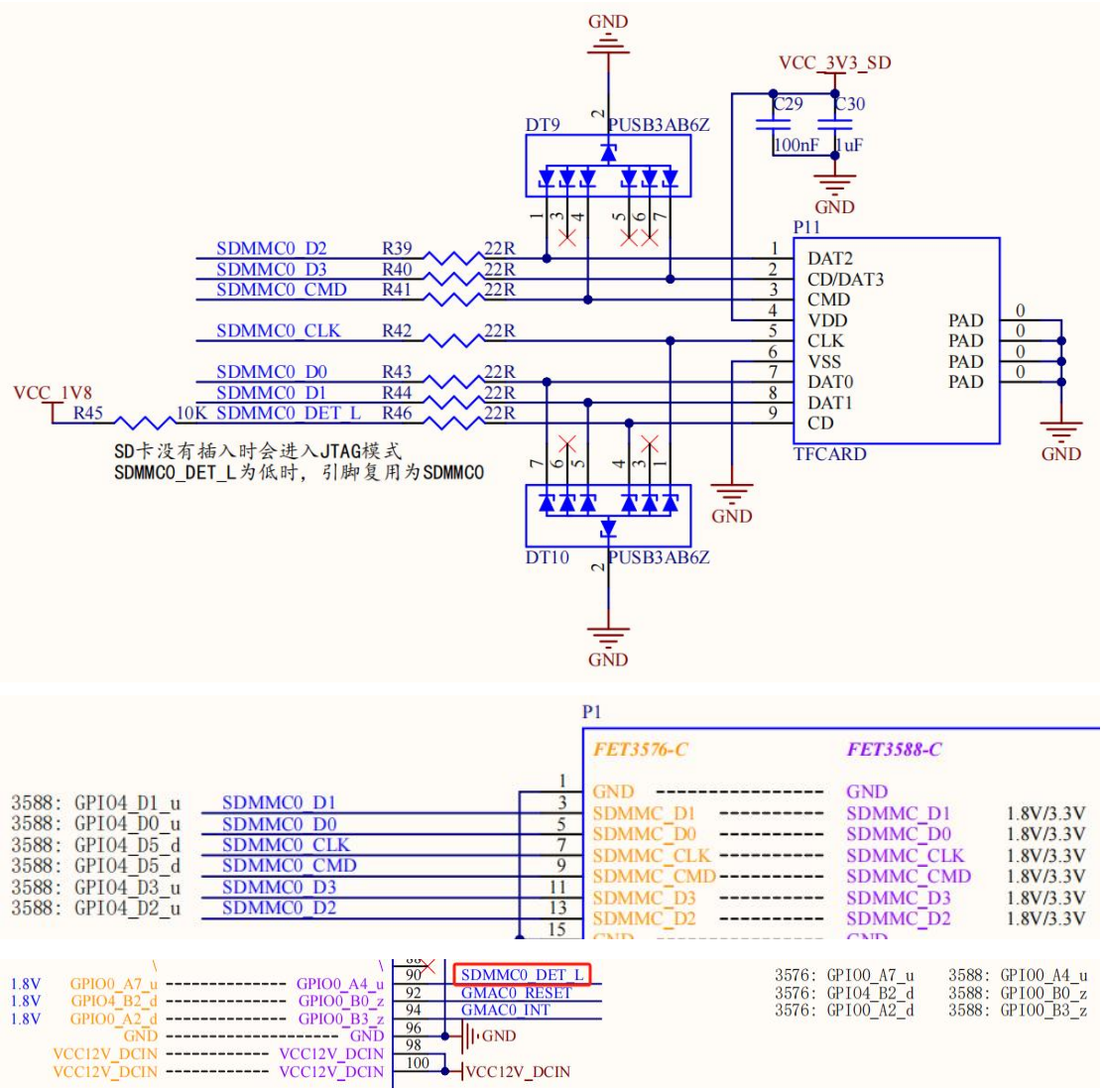
```
root@elf2-buildroot:~# gst-launch-1.0 v4l2src device=/dev/video11 ! video/x-raw, format=NV12, width=640, height=480, framerate=30/1 ! waylandsink
```

输入预览指令后，在屏幕上可以看到摄像头 ov13855 的预览画面。

1.10.8 SDMMC

SDMMC（Secure Digital MultiMediaCard）接口是一种支持 Secure Digital (SD) 和 MultiMediaCard (MMC) 的通用接口。它主要用于嵌入式系统中，支持可移动的存储卡设备。

1.10.8.1 硬件原理



TF 卡使用的是 SDMMC 接口，对应的引脚：SDMMC0_D0、SDMMC0_D1、SDMMC0_D2、SDMMC0_D3、SDMMC0_CMD、SDMMC0_CLK，以及卡检测引脚 SDMMC0_DET_L（GPIO0_A4），通过卡检测引脚检测 TF 卡是否插入或者拔出。

1.10.8.2 IOMUX 配置

在设备树中 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 中 pinctrl-0 描述了默认使用的引脚：

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel

        sdmmc: mmc@fe2c0000 {
                compatible = "rockchip,rk3588-dw-mshc", "rockchip,rk3288-dw-mshc
";
                reg = <0x0 0xfe2c0000 0x0 0x4000>;
                interrupts = <GIC_SPI 203 IRQ_TYPE_LEVEL_HIGH>;
                clocks = <&scmi_clk SCMI_HCLK_SD>, <&scmi_clk SCMI_CCLK_SD>,
                         <&cru SCLK_SDMMC_DRV>, <&cru SCLK_SDMMC_SAMPLE>;
                clock-names = "biu", "ciu", "ciu-drive", "ciu-sample";
                fifo-depth = <0x100>;
                max-frequency = <200000000>;
                pinctrl-names = "default";
                pinctrl-0 = <&sdmmc_clk &sdmmc_cmd &sdmmc_det &sdmmc_bus4>;
                power-domains = <&power RK3588_PD_SDMMC>;
                status = "disabled";
        };
```

打开 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 可以看到这些描述的定义：

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel

        sdmmc {
                /omit-if-no-ref/
                sdmmc_bus4: sdmmc-bus4 {
                        rockchip,pins =
                                /* sdmmc_d0 */
                                <4 RK_PD0 1 &pcfg_pull_up_drv_level_2>,
                                /* sdmmc_d1 */
                                <4 RK_PD1 1 &pcfg_pull_up_drv_level_2>,
                                /* sdmmc_d2 */
                                <4 RK_PD2 1 &pcfg_pull_up_drv_level_2>,
                                /* sdmmc_d3 */
                                <4 RK_PD3 1 &pcfg_pull_up_drv_level_2>;
                };

                /omit-if-no-ref/
                sdmmc_clk: sdmmc-clk {
                        rockchip,pins =
                                /* sdmmc_clk */
                                <4 RK_PD5 1 &pcfg_pull_up_drv_level_2>;
                };

                /omit-if-no-ref/
                sdmmc_cmd: sdmmc-cmd {
                        rockchip,pins =
                                /* sdmmc_cmd */
                                <4 RK_PD4 1 &pcfg_pull_up_drv_level_2>;
                };

                /omit-if-no-ref/
                sdmmc_det: sdmmc-det {
                        rockchip,pins =
                                /* sdmmc_det */
                                <0 RK_PA4 1 &pcfg_pull_up>;
                };
```

我们看到定义中使用的引脚与原理图中是一致的，所以不需要修改。

1.10.8.3 设备节点配置

在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 中已经定义 sdmmc 节点了，所以只需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 引用就可以了：



max-frequency = <150000000>;此配置设置 SD 卡的运行频率，虽然设置为 150M，但是还要根据 SD 卡的不同模式进行调整。这部分不需要关心，实际运行频率和模块的关系软件会关联。最大不超过 150MHz。

no-sdio;禁用 SDIO 功能。

no-mmc;禁用 MMC 功能。

bus-width = <4>;此配置标识需要使用 SD 卡的线宽。SD 卡最大支持 4 线模式，如果不配置就模式使用 1 线模式。另外，这个位只支持的数值为 1，4，配置其他数值会认为是非法数值，强制按照 1 线模式进行使用。

disable-wp;禁用写保护。

sd-uhs-sdr104;支持 SD 卡的 UHS-I（Ultra High Speed）模式中的 SDR104 模式。

vqmmc-supply = <&vccio_sd_s0>;设置 SD 卡的 I/O 电压（VQMMC）电源供应。

cd-gpios = <&gpio0 RK_PA4 GPIO_ACTIVE_LOW>;定义卡检测（Card Detect，CD）引脚的 GPIO 配置。

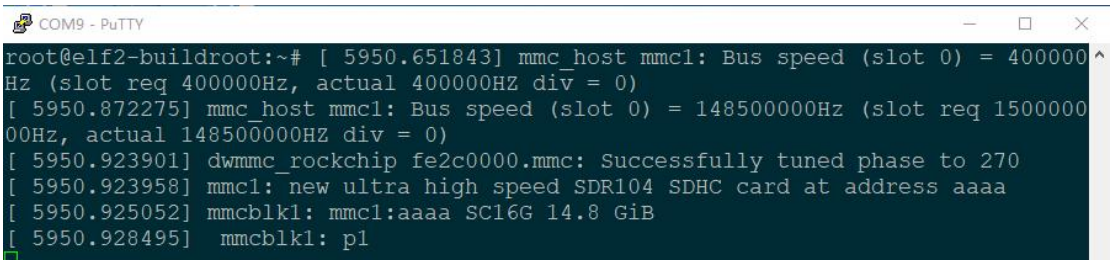
1.10.8.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

启动之后，插上 tf 卡可以看到打印识别 tf 的信息：



输入 df 指令，可以看到 mmcblk1p1 挂载的路径：

```
COM9 - PuTTY
root@elf2-buildroot:~# df
Filesystem     1K-blocks     Used Available Use% Mounted on
/dev/root       14424824  1147892  12662580   9% /
devtmpfs         7998936        8   7998928   1% /dev
tmpfs            8133496      248   8133248   1% /tmp
tmpfs            8133496      404   8133092   1% /run
tmpfs            8133496      148   8133348   1% /var/log
tmpfs            8133496    80480   8053016   1% /dev/shm
/dev/mmcblk0p7    125804    11952    110316  10% /oem
/dev/mmcblk0p8 100155600   290784  95616632   1% /userdata
/dev/mmcblk1p1  15340028 10596416   4743612  70% /mnt/sdcard
root@elf2-buildroot:~#
```

系统会自动将 mmcblk1p1 挂载到/mnt/sdcard 目录下。查看 TF 卡中的文件，命令如下：

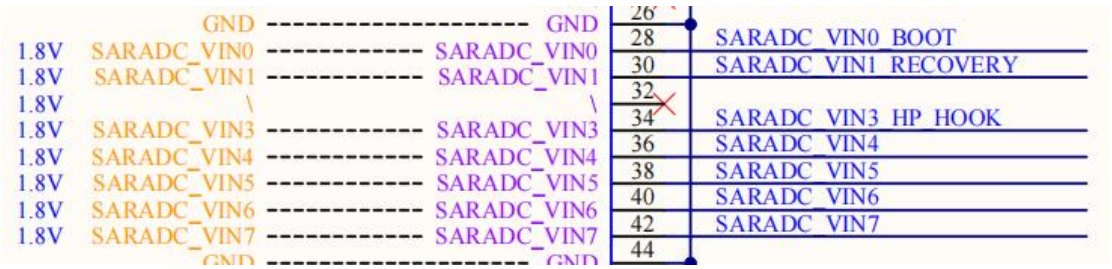
```
root@elf2-buildroot:~# ls /mnt/sdcard/
```

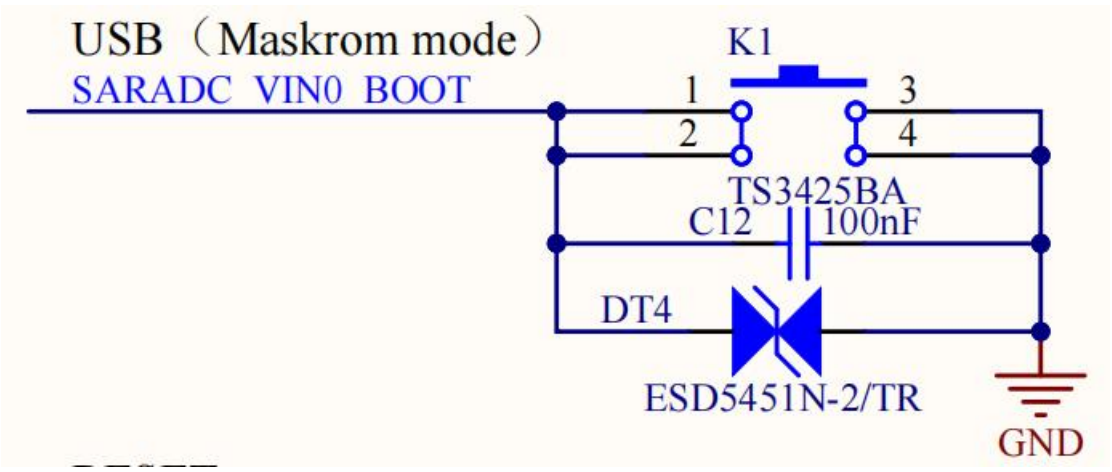
```
COM9 - PuTTY
root@elf2-buildroot:~# ls /mnt/sdcard/
 ELF2                        rksdfw.tag              update.img
'System Volume Information'  sd_boot_config.config  update_MD5.txt
 boot.img                    sdupdate.img
 config.cfg                  source_MD5.txt
root@elf2-buildroot:~#
```

1.10.9 SARADC

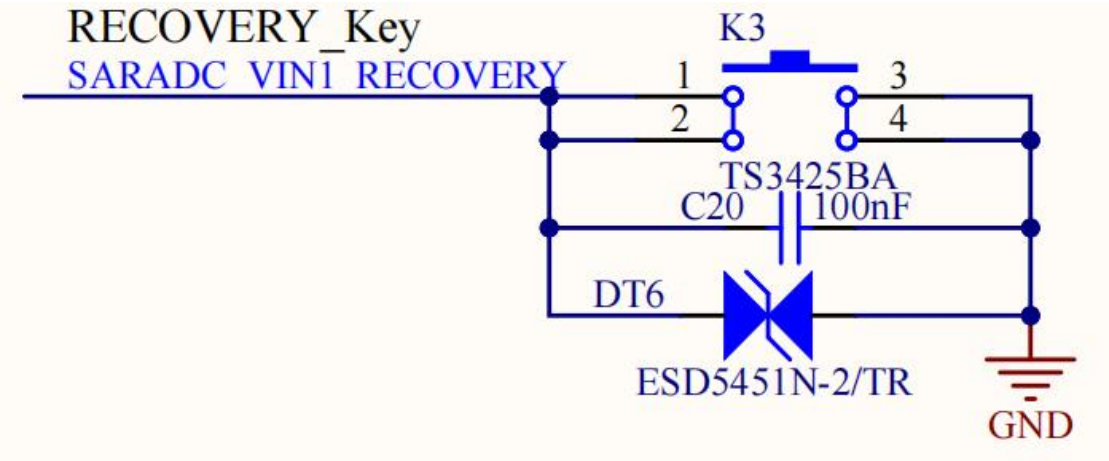
SARADC 是一个 8 通道 12bit 有效位的数模转化器，转换速度为 1MSPS。

1.10.9.1 硬件原理

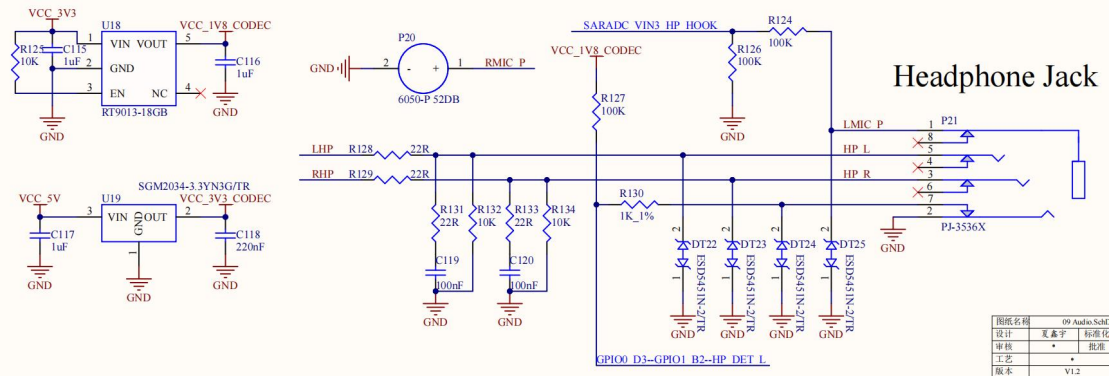




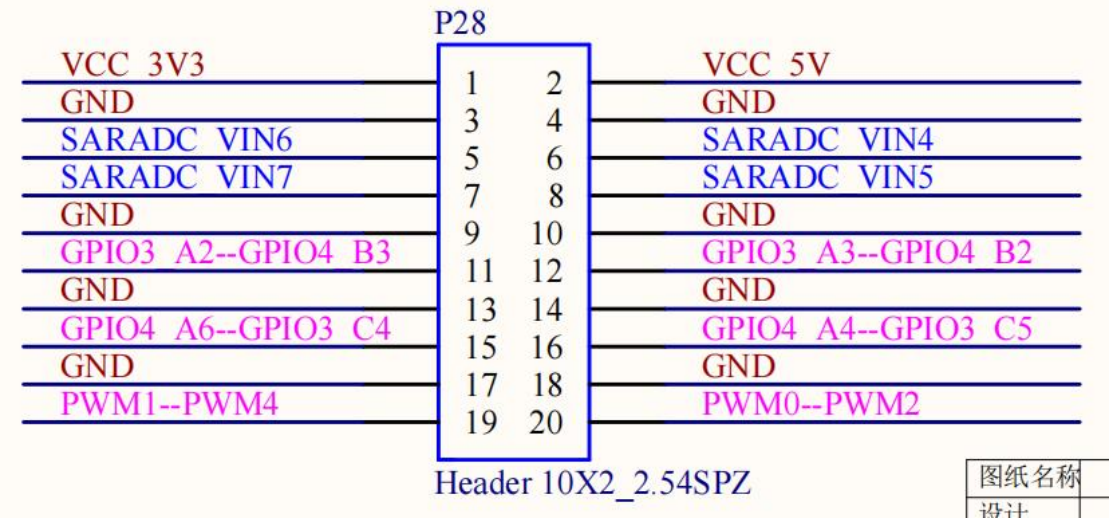
其中 VIN0 接到了 K1 也就是 BOOT 按键上，上电时按住此按键可以进入到 Maskrom 模式。



VIN1 接到了 K3 也就是 RECOVERY 按键上，上电时按住此按键可以进入到 loader 模式。



VIN3 接到了耳机插座上，用作了耳机插拔检测引脚。



剩余的 VIN4-VIN7 接到了 P28 的端子座上。

1.10.9.2 IOMUX 配置

在 RK3588 中只有固定引脚可以用来当作 SARADC 功能，所以不需要额外进行 IOMUX 的配置。

1.10.9.3 设备节点配置

在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 文件中已经存在 saradc 节点：



在 compatible 属性中描述的是驱动的匹配项，"rockchip,rk3588-saradc"。

reg 是 ADC1 对应的一些寄存器的起始地址和这些寄存器所占用的长度；

interrupts = <GIC_SPI 398 IRQ_TYPE_LEVEL_HIGH>;指定使用的中断配置。GIC_SPI 表示该中断使用了通用中断控制器的 SPI（共享中断），398 是中断的编号，IRQ_TYPE_LEVEL_HIGH 表示中断类型为高电平触发。

clocks = <&cru CLK_SARADC>, <&cru PCLK_SARADC>;开启时钟

clock-names = "saradc", "apb_pclk";时钟名称。

resets = <&cru SRST_P_SARADC>;定义设备使用的复位控制信号。

reset-names = "saradc-apb";定义复位信号的名称，"saradc-apb" 指向 APB 总线上与 SARADC 相关的复位信号。

status = disable：默认是关闭状态。

我们需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中，增加和修改 saradc 节点配置：

```
&saradc {
        status = "okay";
        vref-supply = <&vcc_1v8_s0>;
};
```



status = "okay";开启 saradc 功能。

vref-supply = <&vcc_1v8_s0>;引脚的最大输入电压为 1.8v。

1.10.9.4 编译测试

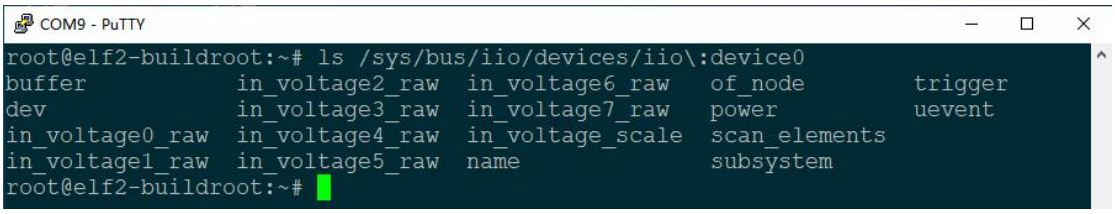
编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板。

重新启动开发板，在 /sys/bus/iio/devices/iio\:device0 下发现 8 个节点分别是：in_voltage0_raw 到 in_voltage7_raw：

```
root@elf2-buildroot:~# ls /sys/bus/iio/devices/iio\:device0
```



查看 VIN0 的值，使用 cat in_voltage0_raw：

```
root@elf2-buildroot:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage0_raw
4095
```

按下 BOOT 按键，再次查询：

```
root@elf2-buildroot:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage0_raw
7
```

按下按键后，引脚电平变为低电平，查询出来的值对应变成低电平。

1.10.10 USB 控制器

RK3588 支持 5 个独立的 USB 控制器，包括：2 个 USB 2.0 HOST 控制器，2 个 USB 3.1 OTG 控制器，1 个 USB 3.1 HOST 控制器。

在 ELF2 开发板上，默认使用了 1 个 USB2.0 HOST，1 个 USB3.1 OTG，1 个 USB3.1 HOST 控制器。

RK3588 USB 接口和 USB DTS 节点的对应关系：（在 ELF 2 开发板资料包\06-RK 原厂资料\cn\Common\USB 中的 Rockchip_RK3588_Developer_Guide_USB_CN.pdf 文档中可以找到此部分描述）

USB 接口名称	USB 控制器 DTS 节点	USB PHY DTS 节点
TYPEC0	usbdrd3_0 usbdrd_dwc3_0	u2phy0 u2phy0_otg usbdp_phy0 usbdp_phy0_u3
TYPEC1	usbdrd3_1 usbdrd_dwc3_1	u2phy1 u2phy1_otg usbdp_phy1 usbdp_phy1_u3
USB20_HOST0	usb_host0_ehci usb_host0_ohci	u2phy2 u2phy2_host
USB20_HOST1	usb_host1_ehci usb_host1_ohci	u2phy3 u2phy3_host
USB30_2	usbhost3_0 usbhost_dwc3_0	combphy2_psu

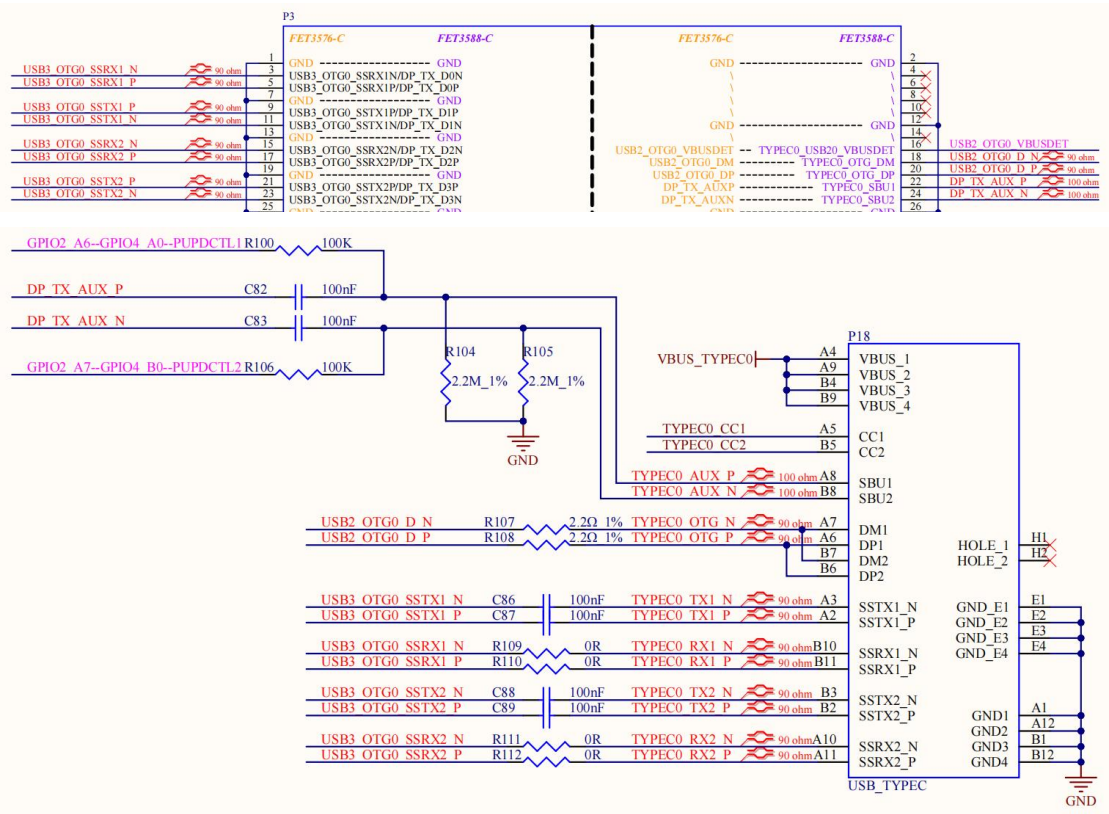
ELF2 开发板使用的 USB 接口和 USB DTS 节点对应关系：

USB 接口名称	USB 控制器 DTS 节点	USB PHY DTS 节点
TYPEC0 (底板丝印为 USB-DP)	usbdrd3_0 usbdrd_dwc3_0	u2phy0 u2phy0_otg usbdp_phy0 usbdp_phy0_u3
USB20_HOST1	usb_host1_ehci usb_host1_ohci	u2phy3 u2phy3_host
USB30_2	usbhost3_0	combphy2_psu

	usbhost_dwc3_0	

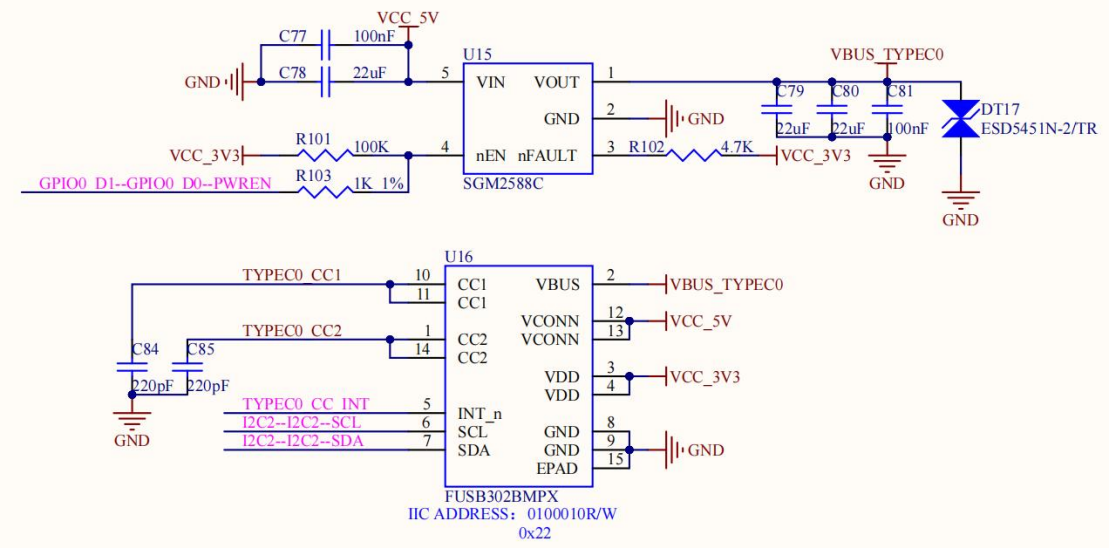
1.10.10.1 硬件原理

1.TYPEC0（底板丝印为 USB-DP)：



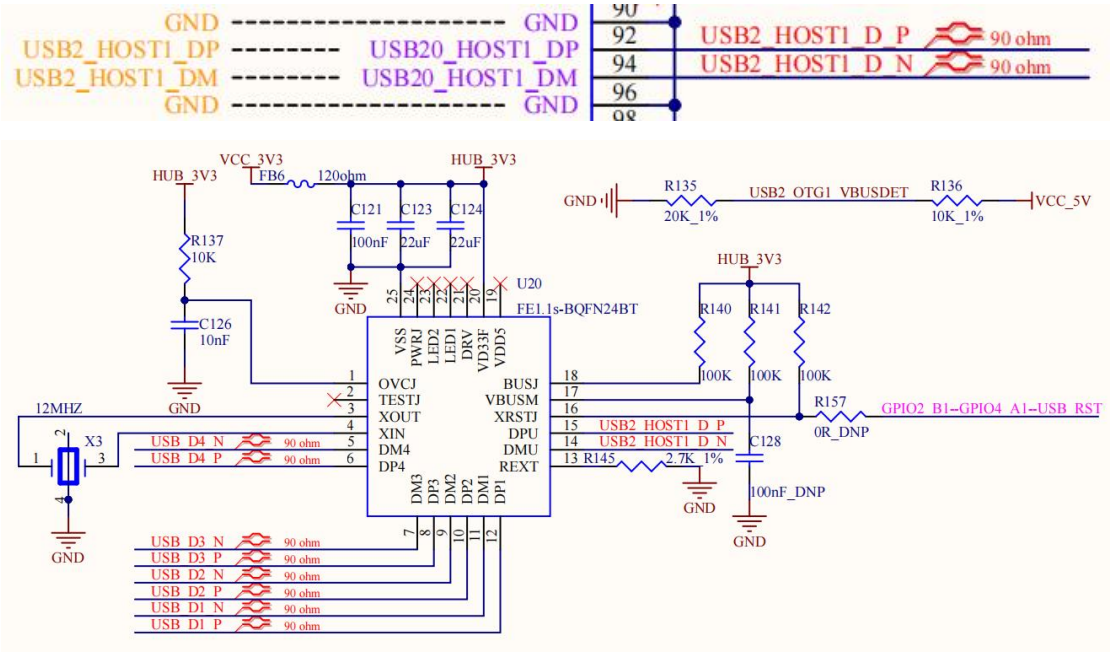
从原理图中可以看出，typec 相关的引脚直接从核心板连接到了底板的 P18 上，这是一个全功能 Type-C 接口，不仅支持 USB2.0、USB3.0 和 USB3.1 的数据传出，还支持 DP（DisplayPort Alt Mode）功能，可以将 Type-C 接口用于视频输出，连接外部显示器或投影仪，实现高分辨率的视频传输。

除此之外，硬件上还需要搭配一个 Type-C 控制芯片（FUSB302)。

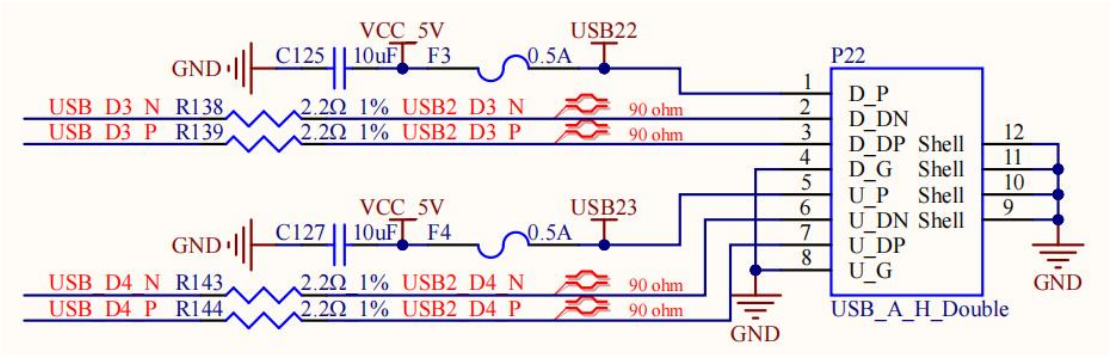


FUSB302 能够识别和管理 USB Type-C 连接，包括正确的插头方向检测。支持 USB 数据传输，能够处理 Type-C 接口下的数据交换。通过监测电流和电压，提供过载保护，确保连接的设备安全。

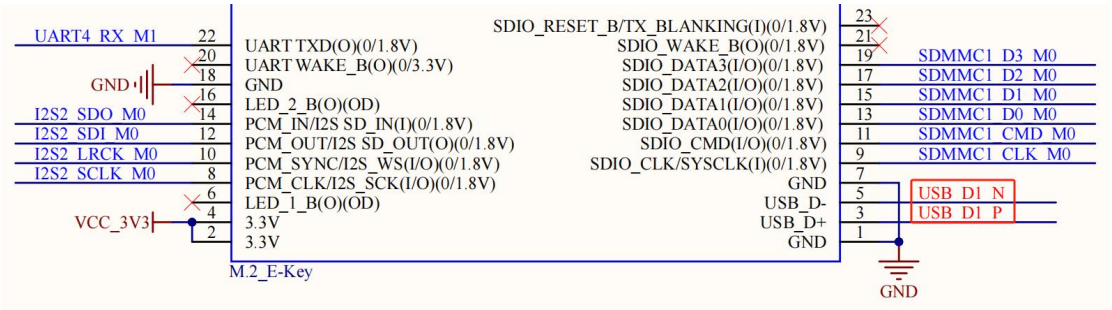
2.USB2.0



USB2.0 的两根线从核心板连接到了一个一转四的 HUB 芯片上。

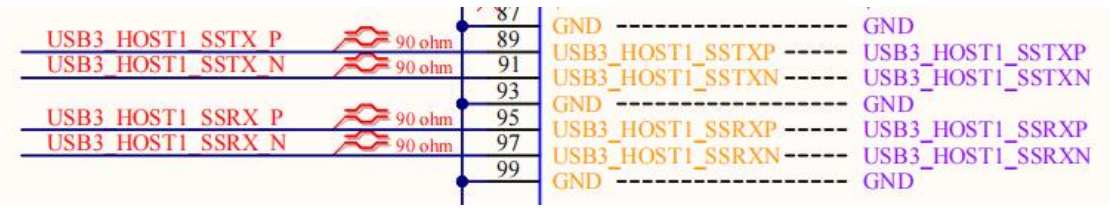


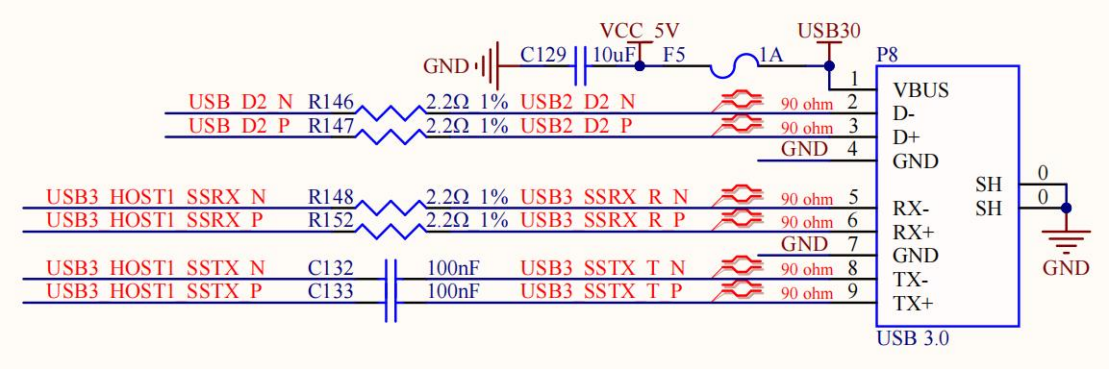
其中有两个 USB2.0 分别是 USB_D3 和 USB_D4 接到了开发板的 P22 上，也就是两个 usb2.0 的接口。



USB_D1 接到了 M.2 的接口。USB_D2 被 USB3.0 的接口使用了，从下边的原理图中能够看出。

3.USB3.0





1.10.10.2 IOMUX 配置

由于 USB 接口使用的是专属引脚，所以不需要 IOMUX 配置。

1.10.10.3 设备节点配置

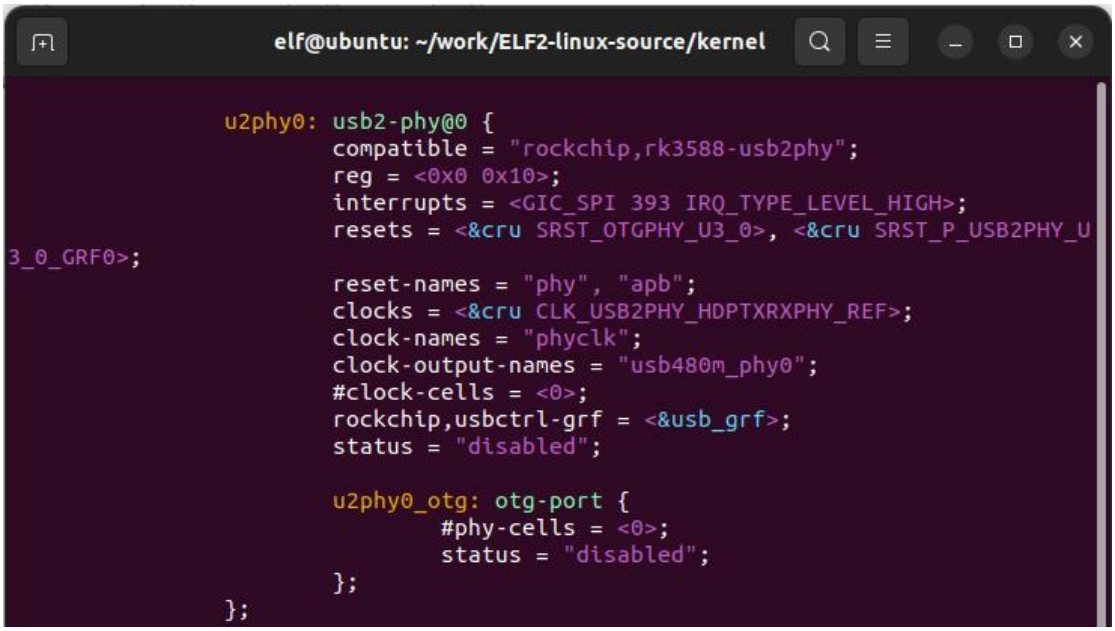
在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 文件中默认已经添加了 typec0 相关的节点，首先是控制器的 DTS 节点：

elf@ubuntu: ~/work/ELF2-linux-source/kernel

```
usbdrd3_0: usbdrd3_0 {
        compatible = "rockchip,rk3588-dwc3", "rockchip,rk3399-dwc3";
        clocks = <&cru REF_CLK_USB3OTG0>, <&cru SUSPEND_CLK_USB3OTG0>,
                 <&cru ACLK_USB3OTG0>;
        clock-names = "ref", "suspend", "bus";
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;
        status = "disabled";

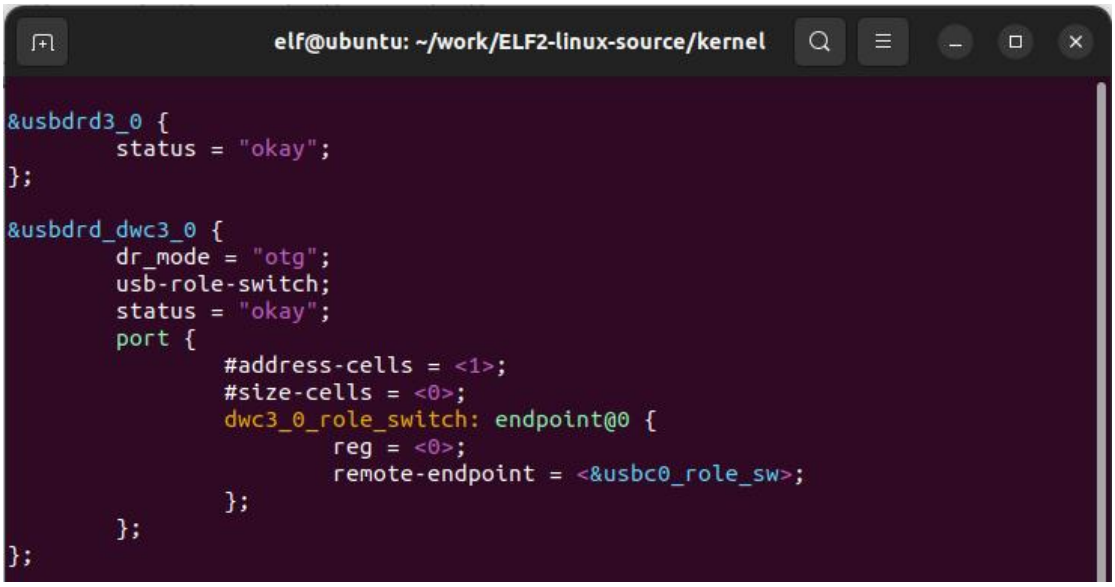
        usbdrd_dwc3_0: usb@fc000000 {
                compatible = "snps,dwc3";
                reg = <0x0 0xfc000000 0x0 0x400000>;
                interrupts = <GIC_SPI 220 IRQ_TYPE_LEVEL_HIGH>;
                power-domains = <&power RK3588_PD_USB>;
                resets = <&cru SRST_A_USB3OTG0>;
                reset-names = "usb3-otg";
                dr_mode = "otg";
                phys = <&u2phy0_otg>, <&usbdp_phy0_u3>;
                phy-names = "usb2-phy", "usb3-phy";
                phy_type = "utmi_wide";
                snps,dis_enblslpm_quirk;
                snps,dis-u1-entry-quirk;
                snps,dis-u2-entry-quirk;
                snps,dis-u2-freeclk-exists-quirk;
                snps,dis-del-phy-power-chg-quirk;
                snps,dis-tx-ipgap-linecheck-quirk;
                snps,parkmode-disable-hs-quirk;
                snps,parkmode-disable-ss-quirk;
                quirk-skip-phy-init;
                status = "disabled";
        };
};
```

USB PHY DTS 节点，在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 中：

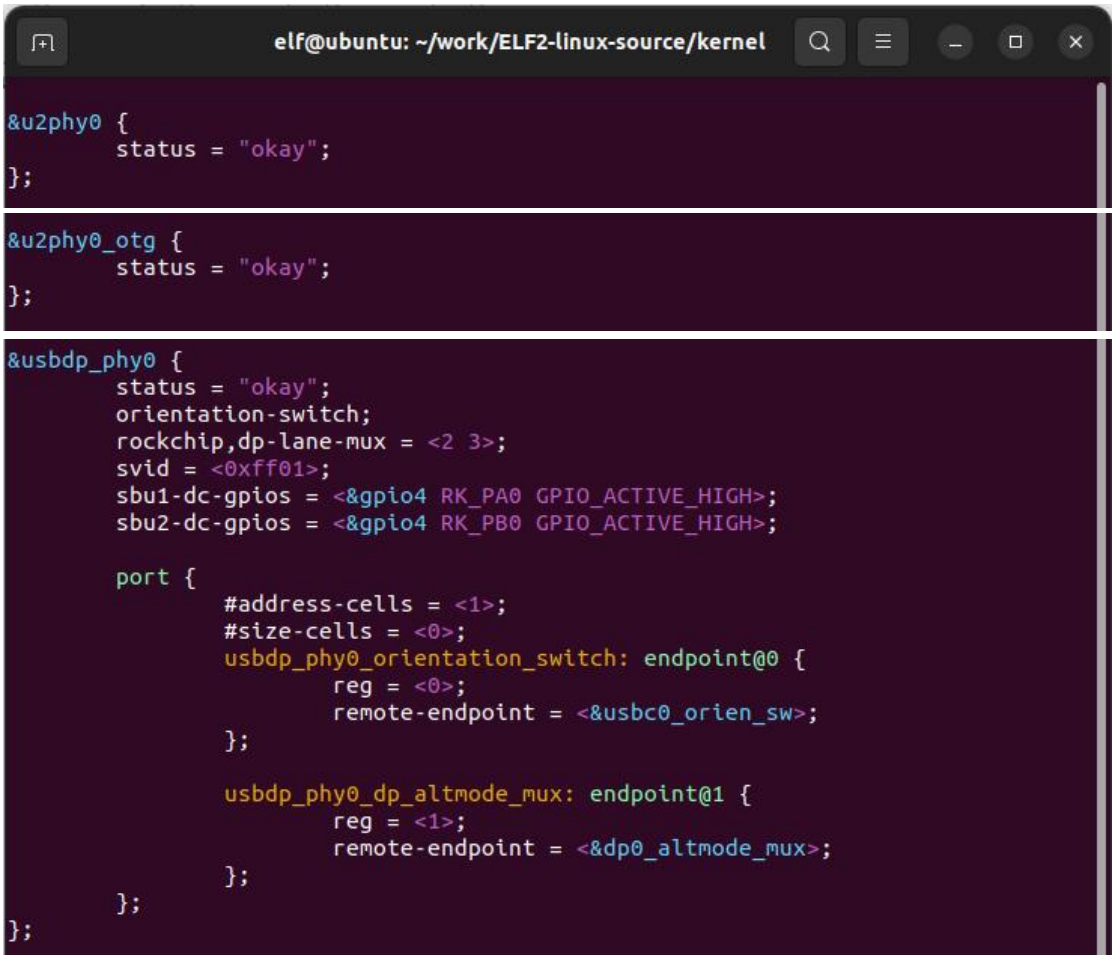




接下来需要打开 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 进行配置，把需要用到的节点打开。



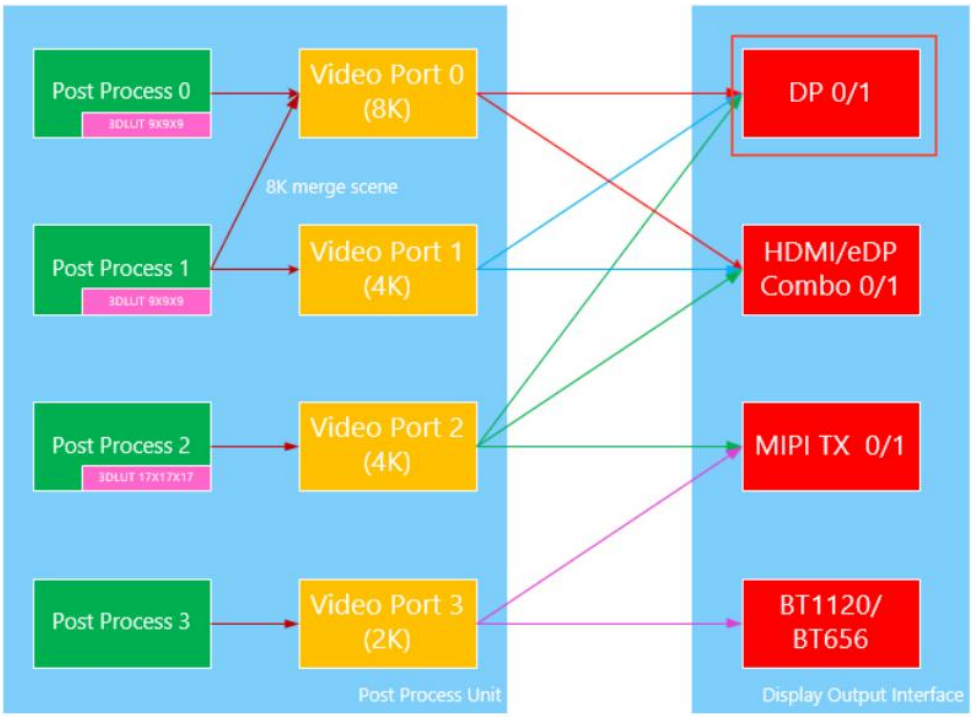
dr_mode = "otg";设置设备工作模式为 OTG（On-The-Go），允许设备在主机和从设备之间切换。
usb-role-switch;表示启用 USB 角色切换功能，使得设备能够在不同的 USB 角色间进行切换。
子节点 dwc3_0_role_switch：代表一个 USB 角色切换的端点。
remote-endpoint = <&usbc0_role_sw>; - 表示该端点的远程端点，通常用于连接到其他设备或功能。
PHY DTS 节点：



```
&usbdp_phy0_u3 {
        status = "okay";
};
```

rockchip,dp-lane-mux = <2 3>;配置 DP 使用 lane2/3，驱动会自动分配 lane0/1 给 USB3.1 Rx/Tx。从显示接口的连接关系中可以看出 DP 接口是可以连接到 VP0、VP1 或者 VP2 上的。默认 ELF2 开发板是连接到了 VP2 上。所以还需要添加 dp 和 vp 相关的节点。

4.3.2 RK3588 VP 和各显示接口的连接关系




```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
&dp0 {
        status = "okay";
};

&dp0_in_vp2 {
        status = "okay";
};
```

除了 USB 控制器和 PHY 节点的配置外，还需要对 FUSB302 进行配置，这个芯片属于 i2c 设备，所以需要在 i2c 节点下创建子节点，可以在 i2c2 的子节点中找到：

elf@ubuntu: ~/work/ELF2-linux-source/kernel

```
usbc0: fusb302@22 {
        compatible = "fcs,fusb302";
        reg = <0x22>;
        interrupt-parent = <&gpio1>;
        interrupts = <RK_PB0 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&usbc0_int>;
        vbus-supply = <&vbus5v0_typec0>;
        status = "okay";

        ports {
                #address-cells = <1>;
                #size-cells = <0>;

                port@0 {
                        reg = <0>;
                        usbc0_role_sw: endpoint@0 {
                                remote-endpoint = <&dwc3_0_role_switch>;
                        };
                };
        };

        usb_con0: connector {
                compatible = "usb-c-connector";
                label = "USB-C";
                data-role = "dual";
                power-role = "dual";
                try-power-role = "sink";
                op-sink-microwatt = <1000000>;
                sink-pdos =
                        <PDO_FIXED(5000, 1000, PDO_FIXED_USB_COMM)>;
                source-pdos =
                        <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)>;

                altmodes {
                        #address-cells = <1>;
                        #size-cells = <0>;

                        altmode@0 {
                                reg = <0>;
                                svid = <0xff01>;
                                vdo = <0xffffffff>;
                        };
                };
```

```
                        ports {
                                #address-cells = <1>;
                                #size-cells = <0>;

                                port@0 {
                                        reg = <0>;
                                        usbc0_orien_sw: endpoint {
                                                remote-endpoint = <&usbdp_phy0_o
rientation_switch>;
                                        };
                                };

                                port@1 {
                                        reg = <1>;
                                        dp0_altmode_mux: endpoint {
                                                remote-endpoint = <&usbdp_phy0_d
p_altmode_mux>;
                                        };
                                };
                        };
                };
        };
```

这些就是 Type-C 相关的所有节点的配置了。

USB3.0 的相关节点：

打开 arch/arm64/boot/dts/rockchip/rk3588s.dtsi

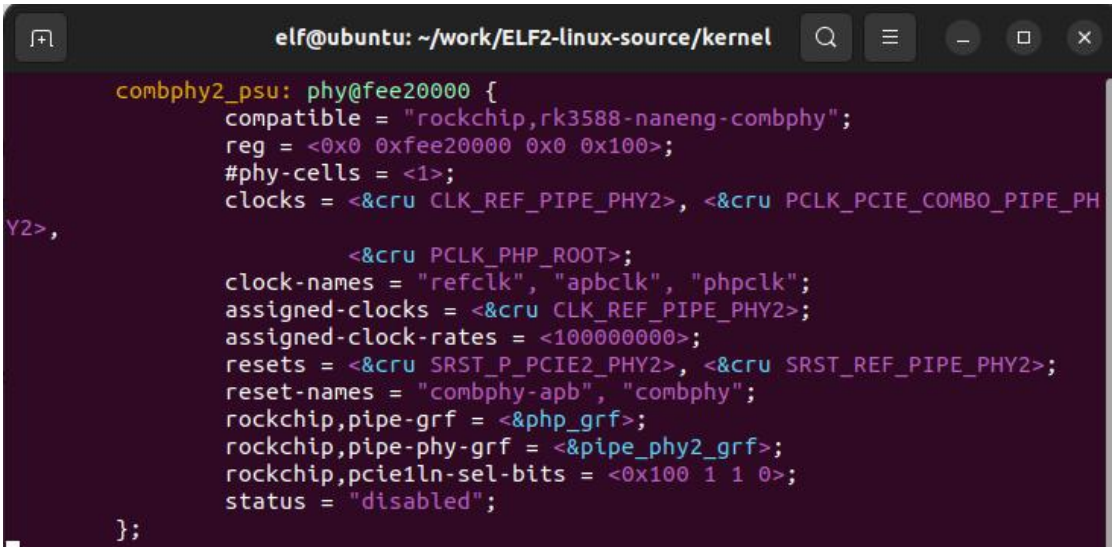
```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
        usbhost3_0: usbhost3_0 {
                compatible = "rockchip,rk3588-dwc3", "rockchip,rk3399-dwc3";
                clocks = <&cru REF_CLK_USB3OTG2>, <&cru SUSPEND_CLK_USB3OTG2>,
                         <&cru ACLK_USB3OTG2>, <&cru CLK_UTMI_OTG2>,
                         <&cru PCLK_PHP_ROOT>, <&cru CLK_PIPEPHY2_PIPE_U3_G>;
                clock-names = "ref", "suspend", "bus", "utmi", "php", "pipe";
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;
                status = "disabled";

                usbhost_dwc3_0: usb@fcd00000 {
                        compatible = "snps,dwc3";
                        reg = <0x0 0xfcd00000 0x0 0x400000>;
                        interrupts = <GIC_SPI 222 IRQ_TYPE_LEVEL_HIGH>;
                        resets = <&cru SRST_A_USB3OTG2>;
                        reset-names = "usb3-host";
                        dr_mode = "host";
                        phys = <&combphy2_psu PHY_TYPE_USB3>;
                        phy-names = "usb3-phy";
                        phy_type = "utmi_wide";
                        snps,dis_enblslpm_quirk;
                        snps,dis-u2-freeclk-exists-quirk;
```

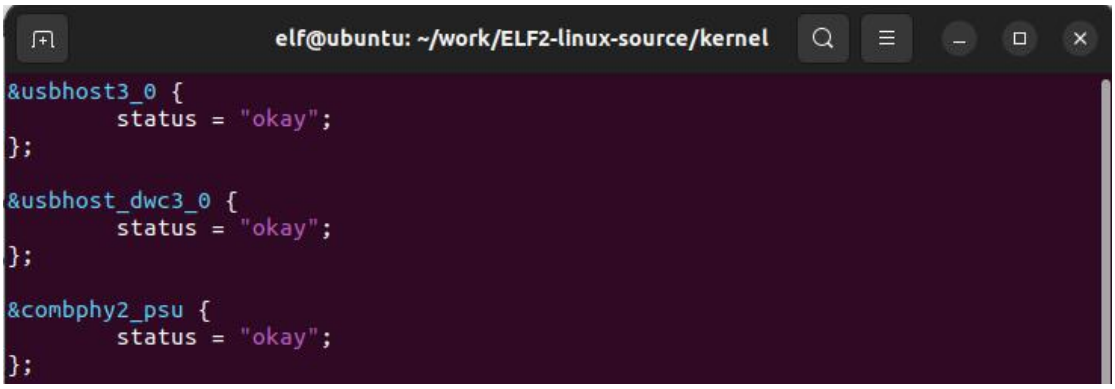
```
                        snps,dis-del-phy-power-chg-quirk;
                        snps,dis-tx-ipgap-linecheck-quirk;
                        snps,dis_rxdet_inp3_quirk;
                        snps,parkmode-disable-hs-quirk;
                        snps,parkmode-disable-ss-quirk;
                        status = "disabled";
                };
        };
```

控制器部分的节点已经配置好了，而且使用的都是专用引脚，不需要进行 IOMUX 配置，并且之只能当作 host 模式使用。

从上图 phys = <&combphy2_psu PHY_TYPE_USB3>; 可以看出控制器使用的 usbphy 为 combphy2_psu。相关节点如下也是已经配置好了。

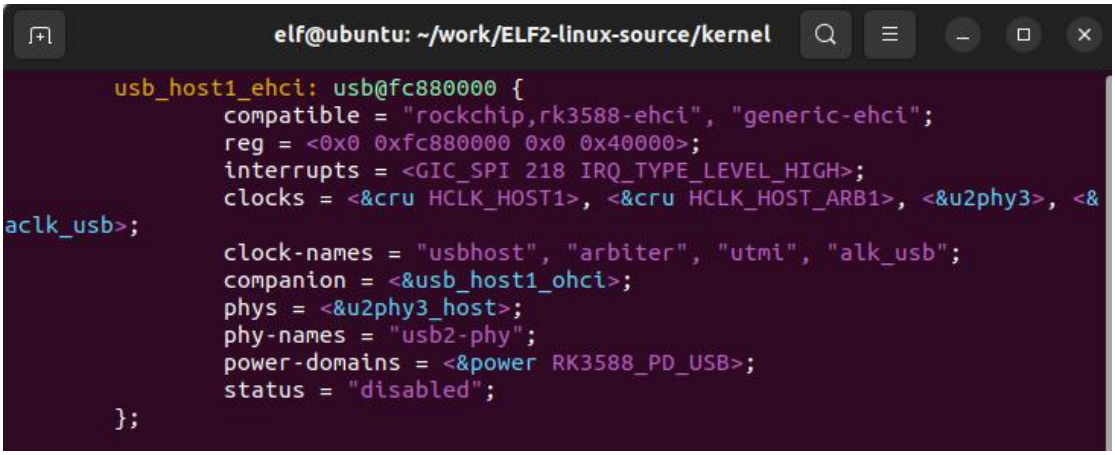


接下来需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中打开这些节点。



USB2.0 相关节点：

打开设备树 arch/arm64/boot/dts/rockchip/rk3588s.dtsi



```
        usb_host1_ohci: usb@fc8c0000 {
                compatible = "rockchip,rk3588-ohci", "generic-ohci";
                reg = <0x0 0xfc8c0000 0x0 0x40000>;
                interrupts = <GIC_SPI 219 IRQ_TYPE_LEVEL_HIGH>;
                clocks = <&cru HCLK_HOST1>, <&cru HCLK_HOST_ARB1>, <&u2phy3>, <&
aclk_usb>;
                clock-names = "usbhost", "arbiter", "utmi", "alk_usb";
                phys = <&u2phy3_host>;
                phy-names = "usb2-phy";
                power-domains = <&power RK3588_PD_USB>;
                status = "disabled";
        };
```

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
                u2phy3: usb2-phy@c000 {
                        compatible = "rockchip,rk3588-usb2phy";
                        reg = <0xc000 0x10>;
                        interrupts = <GIC_SPI 392 IRQ_TYPE_LEVEL_HIGH>;
                        resets = <&cru SRST_OTGPHY_U2_1>, <&cru SRST_P_USB2PHY_U
2_1_GRF0>;
                        reset-names = "phy", "apb";
                        clocks = <&cru CLK_USB2PHY_HDPTXRXPHY_REF>;
                        clock-names = "phyclk";
                        clock-output-names = "usb480m_phy3";
                        #clock-cells = <0>;
                        status = "disabled";

                        u2phy3_host: host-port {
                                #phy-cells = <0>;
                                status = "disabled";
                        };
                };
        };
```

控制器和 phy 部分同样已经配置好了，不需要做任何修改，只需要在 ELF2 使用的设备树 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中将其状态打开就可以了。

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
&u2phy3_host {
        status = "okay";
};

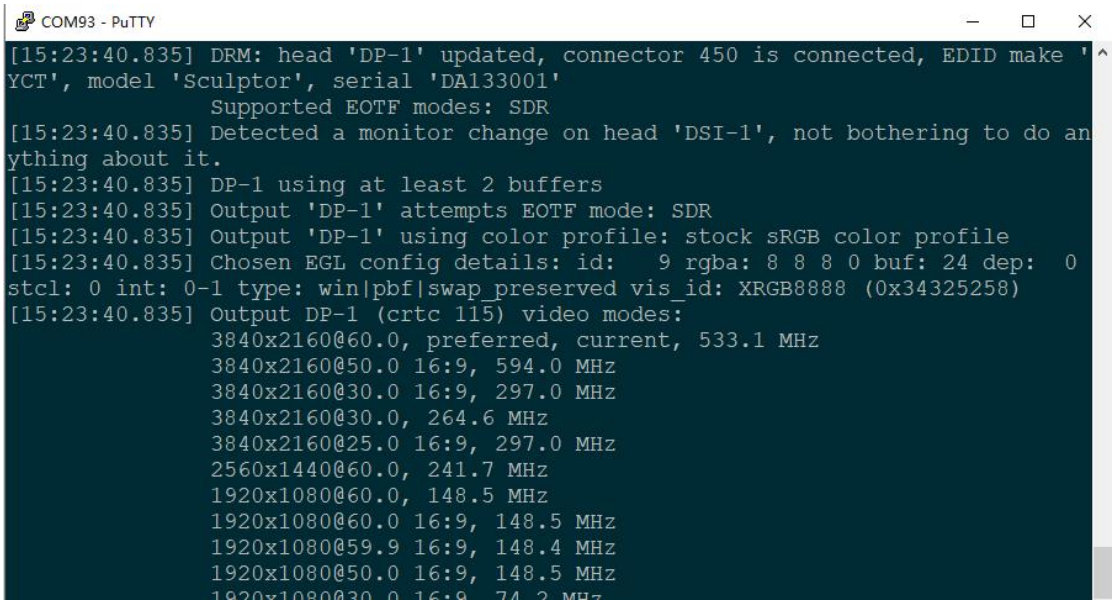
&usb_host1_ehci {
        status = "okay";
};

&usb_host1_ohci {
        status = "okay";
};
```

```
&u2phy3 {
        status = "okay";
};
```

1.10.10.4 编译测试

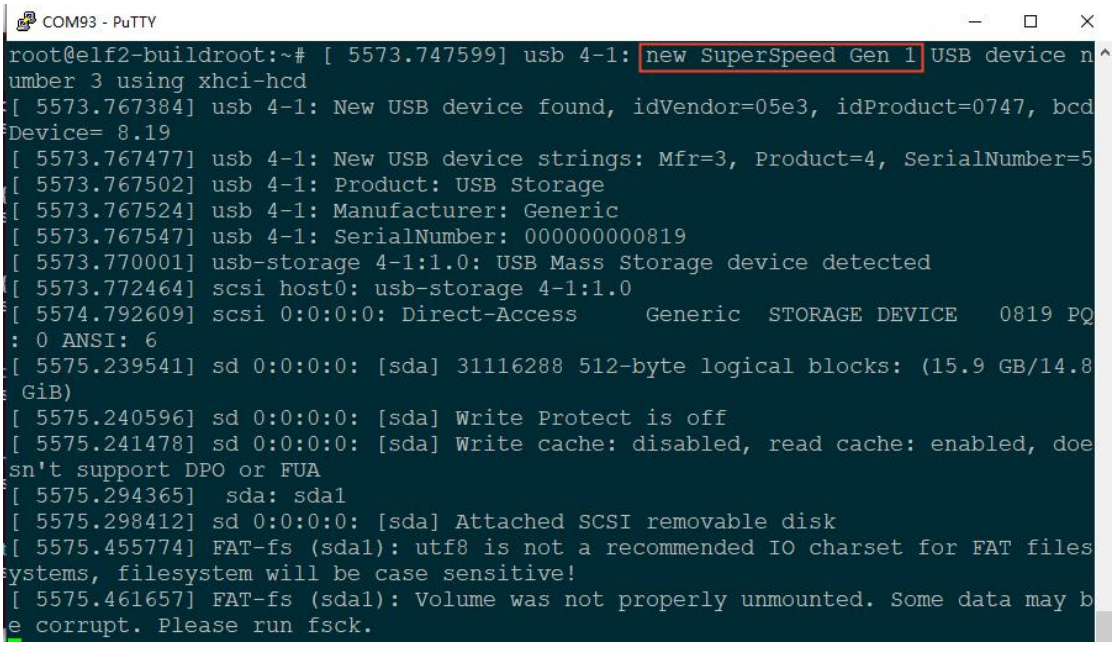
Type-C 接口插上 DP 屏幕，能够正常显示界面，并且 DEBUG 终端会打印 DP 屏幕所支持的分辨率。



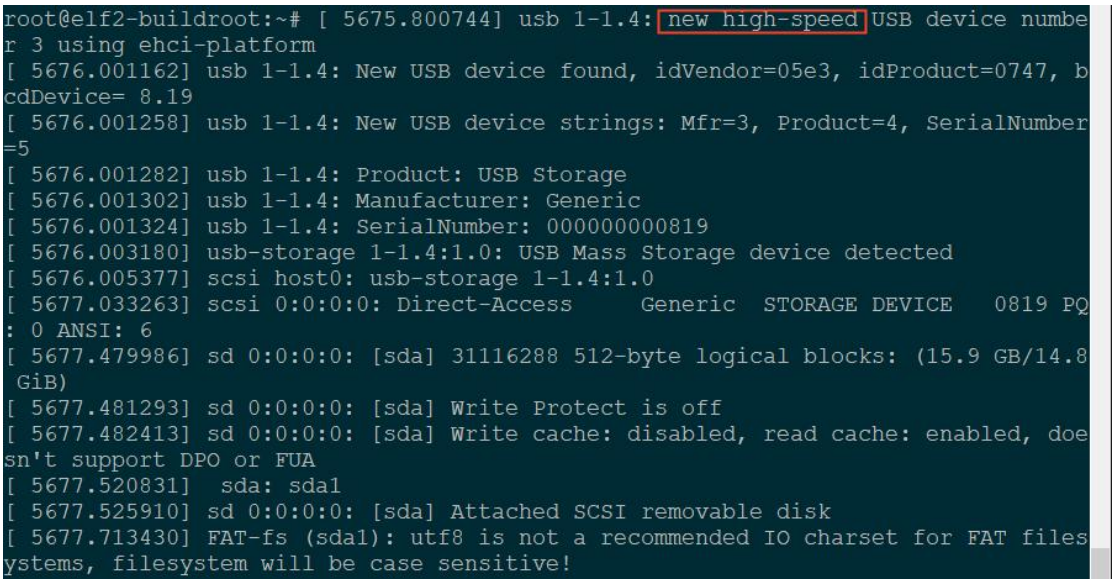
Type-C 接口插上 usb3.0 的 U 盘，可以通过 log 信息看到能够正常识别并且挂载：

```
[ 1090.579235] usb usb8: We don't know the algorithms for LPM for this host, disabling LPM.
[ 1090.579304] usb usb8: New USB device found, idVendor=1d6b, idProduct=0003, bcdDevice= 5.10
[ 1090.579315] usb usb8: New USB device strings: Mfr=3, Product=2, SerialNumber=1
[ 1090.579325] usb usb8: Product: xHCI Host Controller
[ 1090.579334] usb usb8: Manufacturer: Linux 5.10.209 xhci-hcd
[ 1090.579344] usb usb8: SerialNumber: xhci-hcd.9.auto
[ 1090.579913] hub 8-0:1.0: USB hub found
[ 1090.579936] hub 8-0:1.0: 1 port detected
[ 1091.583695] usb 8-1: new SuperSpeed Gen 1 USB device number 2 using xhci-hcd
[ 1091.603624] usb 8-1: New USB device found, idVendor=05e3, idProduct=0747, bcdDevice= 8.19
[ 1091.603723] usb 8-1: New USB device strings: Mfr=3, Product=4, SerialNumber=5
[ 1091.603747] usb 8-1: Product: USB Storage
[ 1091.603768] usb 8-1: Manufacturer: Generic
[ 1091.603788] usb 8-1: SerialNumber: 000000000819
[ 1091.605210] usb-storage 8-1:1.0: USB Mass Storage device detected
[ 1091.605690] scsi host0: usb-storage 8-1:1.0
[ 1092.612141] scsi 0:0:0:0: Direct-Access     Generic  STORAGE DEVICE   0819 PQ: 0 ANSI: 6
[ 1093.024873] sd 0:0:0:0: [sda] 31116288 512-byte logical blocks: (15.9 GB/14.8 GiB)
[ 1093.025490] sd 0:0:0:0: [sda] Write Protect is off
[ 1093.026033] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
```

USB3.0 接口测试，可以将 USB3.0 的 U 盘插上查看 log 信息：



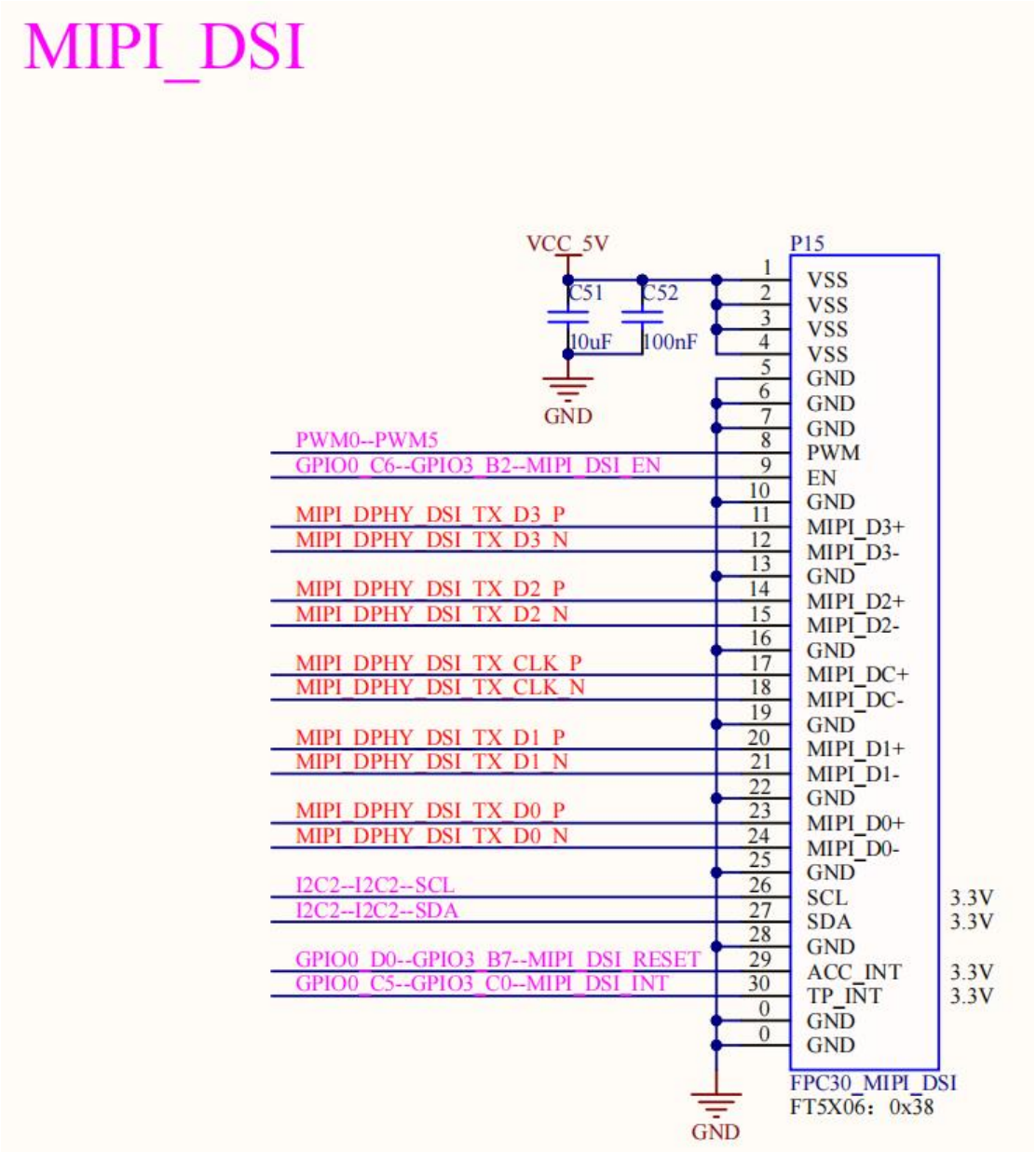
USB2.0 接口测试，ELF2 开发板上有两个 USB2.0 接口，都是通过 HUB 芯片转接出来的，同样可以通过 log 信息判断：



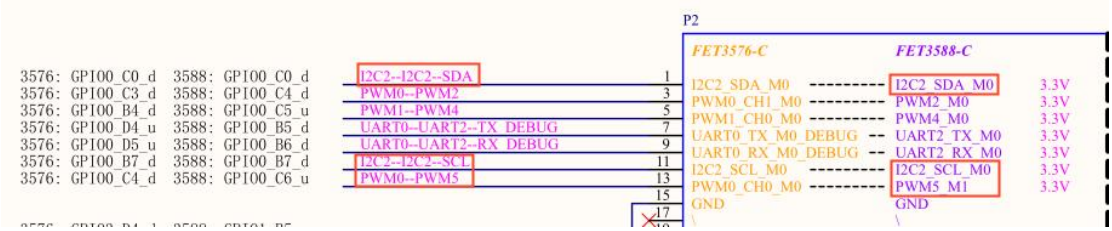
1.10.11 MIPI-DSI

DSI-2 是 MIPI 联盟定义的一组通信协议的一部分， DWC-MIPI-DSI2 是一个实现 MIPI-DSI2 规范中定义的所有协议功能的数字核控制器，可以兼容 D-PHY 和 C-PHY 的物理接口，支持两路的 Display StreamCompression (DSC) 数据传输，RK3588 有两个 DSI-2 控制器和两个独立的物理的 D/C-PHY, 可以同时最多支持两路 MIPI 输出。其中 ELF2 开发板设计了一路 MIPI 输出。

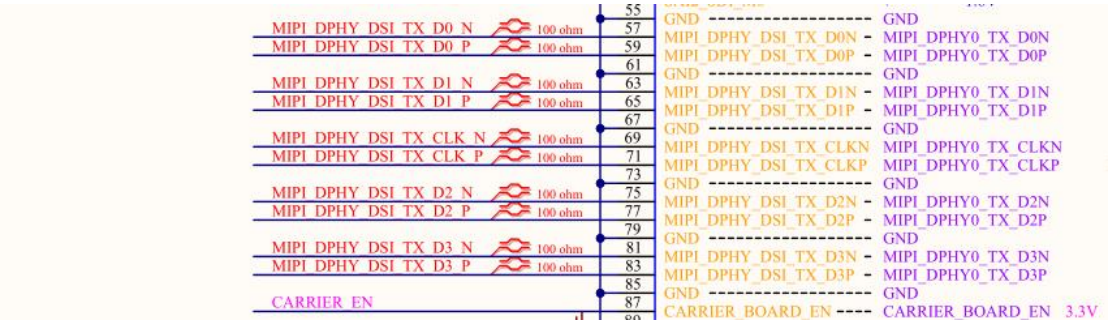
1.10.11.1 硬件原理



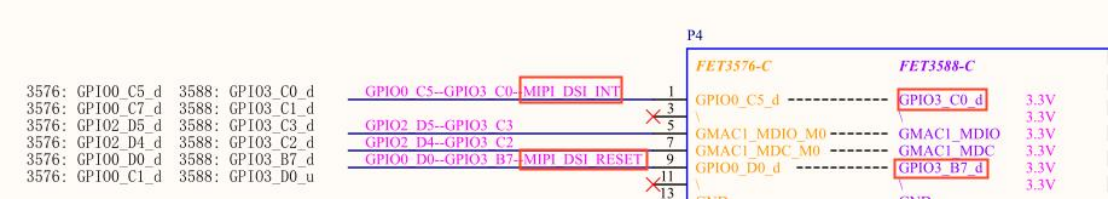
PWM 连接到了 3588 核心板的 PWM5_M1 引脚上，负责控制屏幕的背光。I2C2 连接到了 3588 核心板的 I2C2_SDA_M0 核 I2C2_SCL_M0 引脚上。i2c2 上接的是电容触摸芯片 ft5x06。



MIPI_DPHY_DSI_TX_D0 到 MIPI_DPHY_DSI_TX_D3 为 4 组数据差分线，MIPI_DPHY_DSI_TX_CLK 为 MIPI 的差分时钟线，连接到了 3588 核心板的 MIPI_DPHY0_TX 上。



MIPI 的中断引脚 MIPI_DSI_INT 连接到了核心板的 GPIO3_C0，屏幕的复位引脚 MIPI_DSI_RESET 连接到了核心板的 GPIO3_B7。

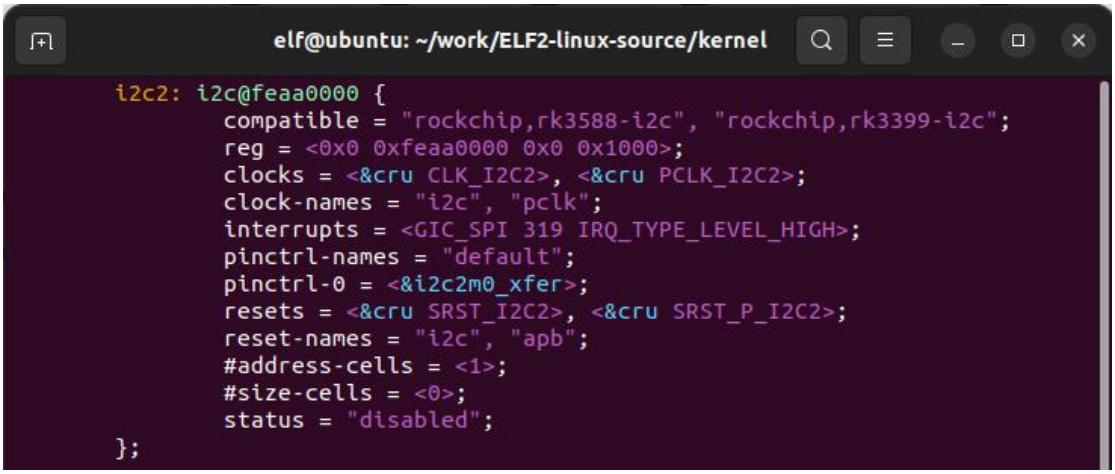


1.10.11.2 IOMUX 配置

MIPI_DPHY0_TX 引脚为专用引脚，不需要单独进行配置。

PWM5 引脚在前面讲解 pwm 章节时已经讲解，所以也不在重复讲解。

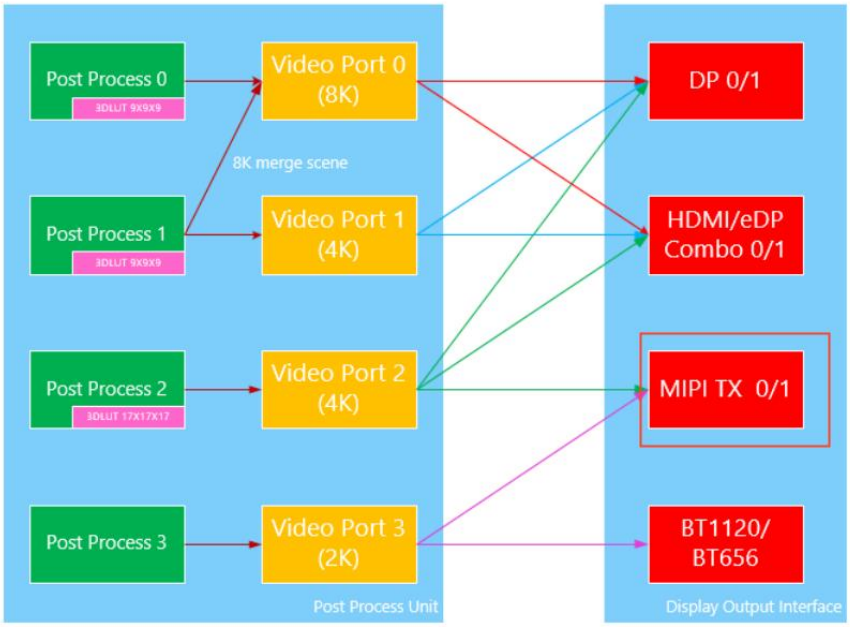
I2C2 引脚使用的是 I2C2 的 M0 引脚，打开 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 查看 I2C2 默认使用的就是 M0 这一组引脚，所以也是不需要修改的。



1.10.11.3 设备节点配置

RK3588 最多支持两路 MIPI 输出，ELF2 开发板支持一路，使用到了 DSI0 接到了 VP3 上，需要注意的是 DSI0 只能选择接到 VP2 或者 VP3 上。从 VP 和各显示接口的连接关系中可以看出。下图来自于：ELF 2 开发板资料包\06-RK 原厂资料\cn\Common\DISPLAY 中的
<<Rockchip_Developer_Guide_DRM_Display_Driver_CN.pdf>>

4.3.2 RK3588 VP 和各显示接口的连接关系



ELF2 开发板默认是将 dsi0 接到 vp3 上的，所以需要打开 dsi0_in_vp3，关闭 dsi0_in_vp2。

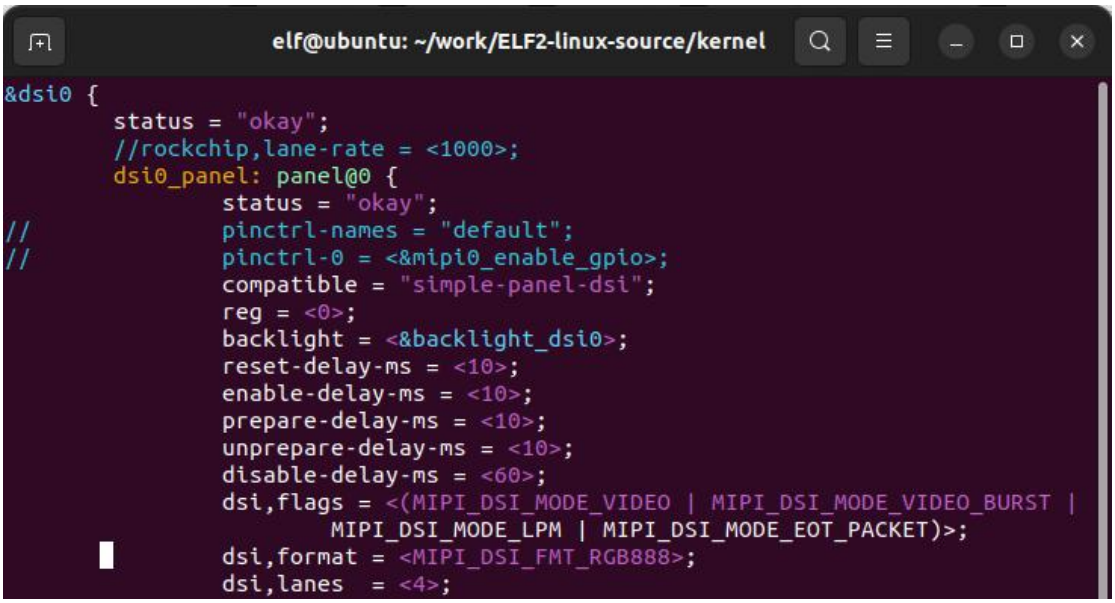
打开：arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
&dsi0_in_vp2 {
        status = "disabled";
};

&dsi0_in_vp3 {
        status = "okay";
};
```

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel
&route_dsi0 {
        status = "okay";
        connect = <&vp3_out_dsi0>;
};
```

然后需要对 dsi0 节点进行配置。



compatible = "simple-panel-dsi"; 驱动兼容属性。

backlight = <&backlight_dsi0>;屏幕背光，引用外部的 backlight_dsi0 节点。

reset-delay-ms = <10>;屏幕的复位延时时间。

enable-delay-ms = <10>;屏幕启动延时时间。

prepare-delay-ms = <10>;屏幕准备延时时间。

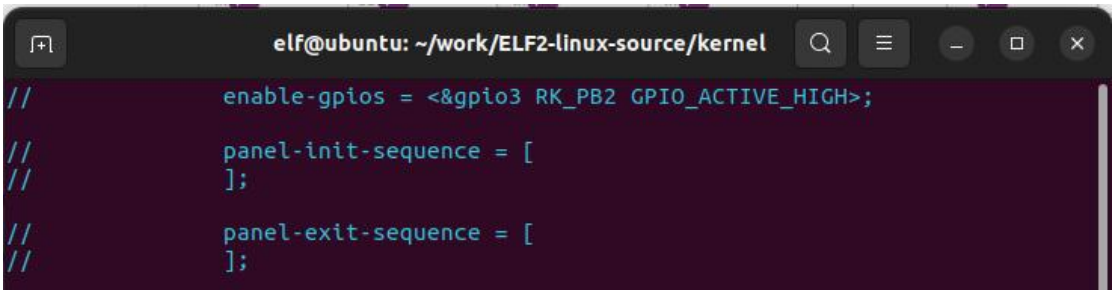
unprepare-delay-ms = <10>;屏幕取消准备延时时间。

disable-delay-ms = <60>;关闭屏幕时的延时时间。

dsi,flags；DSI2 的工作模式，这里设置为启用视频模式、视频突发模式、低功耗模式和 EOT 包模式。

dsi,format = <MIPI_DSI_FMT_RGB888>;数据格式为 RGB888。

dsi,lanes = <4>; // DSI 通道数为 4，从原理图中也可以看出使用了 4 组差分数据线。

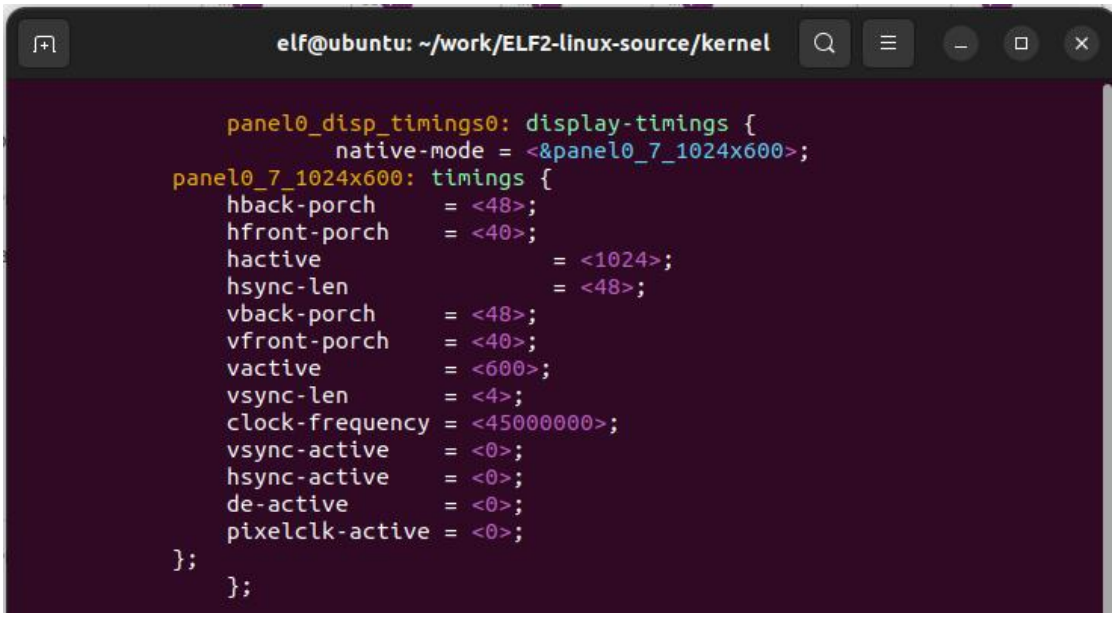


enable-gpios；是使能屏幕的 GPIO，因为没有用到所以屏蔽了。

panel-init-sequence；这个属性是设置 mipi 屏幕初始化序列的，因为默认适配的 MIPI 屏幕不需要初始化序列，所以此处也是屏蔽了。

panel-exit-sequence；为屏幕关闭序列，同样没有涉及到。

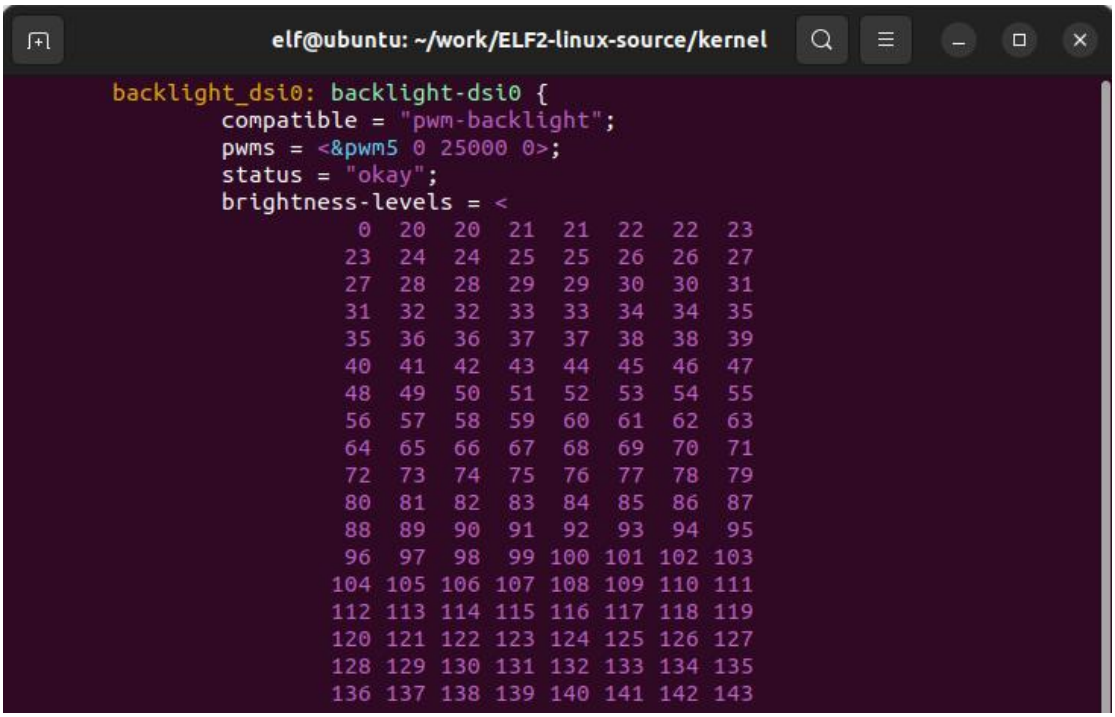
接下来就是需要根据屏幕规格手册设置的一些屏幕参数了，默认 ELF2 开发板适配的为 7 寸，1024x600 分辨率的屏幕。



再接下来就是 dsi0 的输入输出端口，这部分不需要修改，是固定不变的。

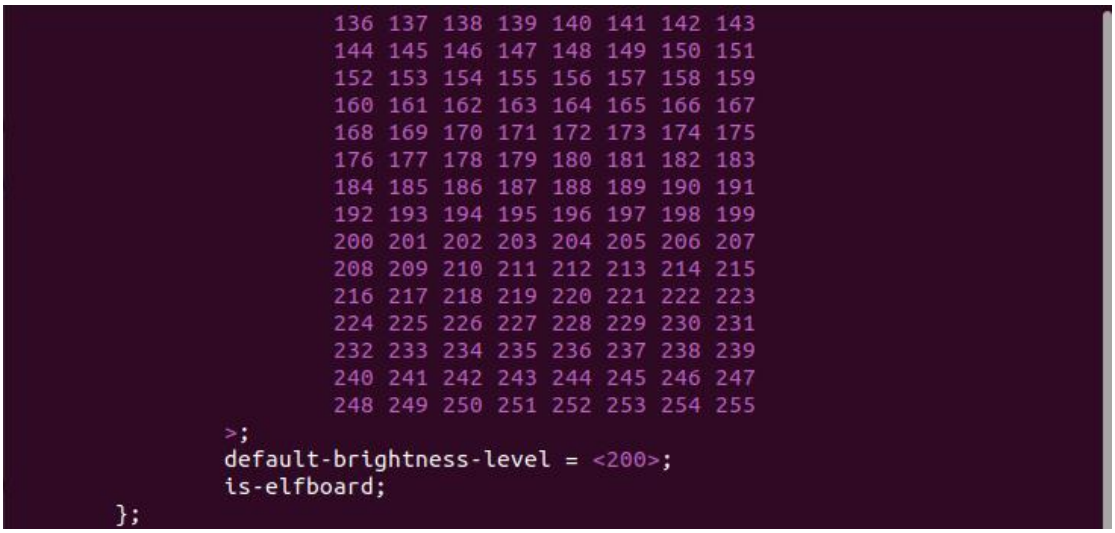


背光的节点如下：



compatible = "pwm-backlight";驱动的兼容属性，用来匹配驱动。

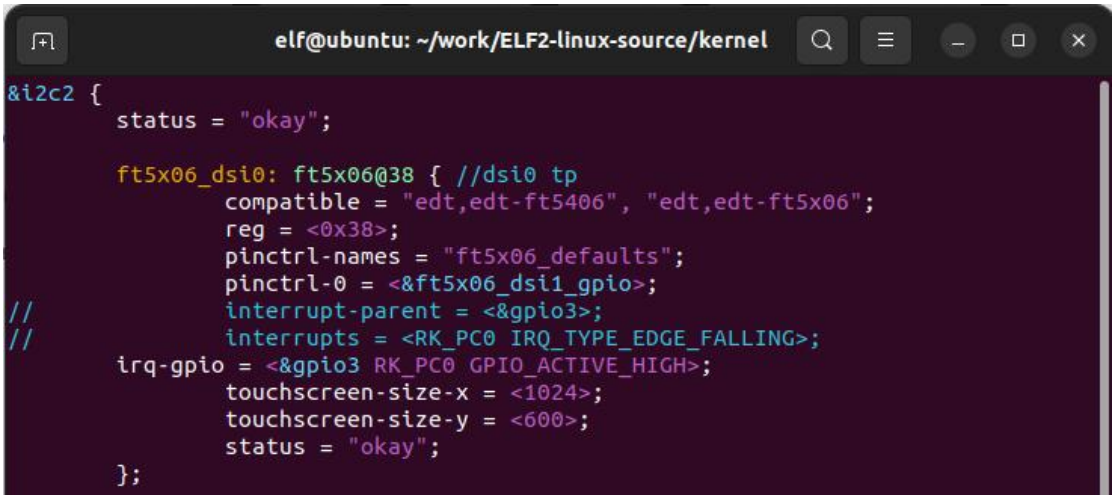
pwms = <&pwm5 0 25000 0>;引用了 pwm5 节点，表示使用 pwm5 来控制背光。0 表示 pwm 的通道编号，25000 表示 pwm 的频率，单位是 HZ，这里表示 25kHz。最后边的 0 表示初始状态的占空比。



brightness-levels；用来控制亮度等级，这里分成了 0-255 份。

default-brightness-level = <200>;表示默认的亮度为 200。

触摸芯片是挂载在 i2c2 上面的。



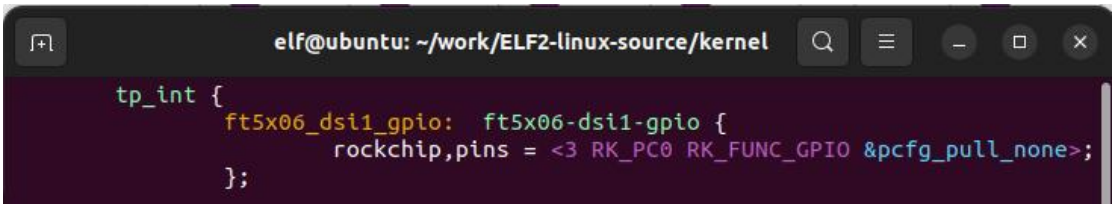
7 寸 MIPI 屏幕默认使用的触摸芯片为 ft5x06。

compatible = "edt,edt-ft5406", "edt,edt-ft5x06";驱动兼容属性，用来匹配驱动。

reg = <0x38>;这表示该设备在 I2C 总线上的设备地址为 0x38。

pinctrl-names = "ft5x06_defaults";该字段指定了使用的 pinctrl 配置名称。

pinctrl-0 = <&ft5x06_dsi1_gpio>;这表示使用 ft5x06_dsi1_gpio 配置来控制触摸屏相关的 GPIO 引脚。



irq-gpio = <&gpio3 RK_PC0 GPIO_ACTIVE_HIGH>;用于指定触摸屏的中断引脚。

touchscreen-size-x = <1024>;这是触摸屏的 X 轴分辨率，意味着触摸屏的宽度支持 1024 个触摸点。

touchscreen-size-y = <600>;这是触摸屏的 Y 轴分辨率，意味着触摸屏的高度支持 600 个触摸点。

以上这些就是涉及到 7 寸 MIPI 显示和触摸的相关节点。

1.10.11.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板，插上 7 寸 MIPI 屏幕上电，能看到显示界面。

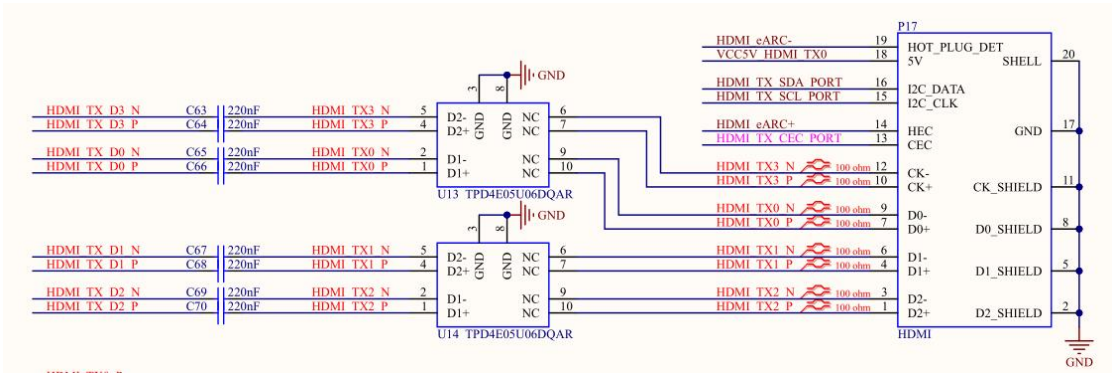


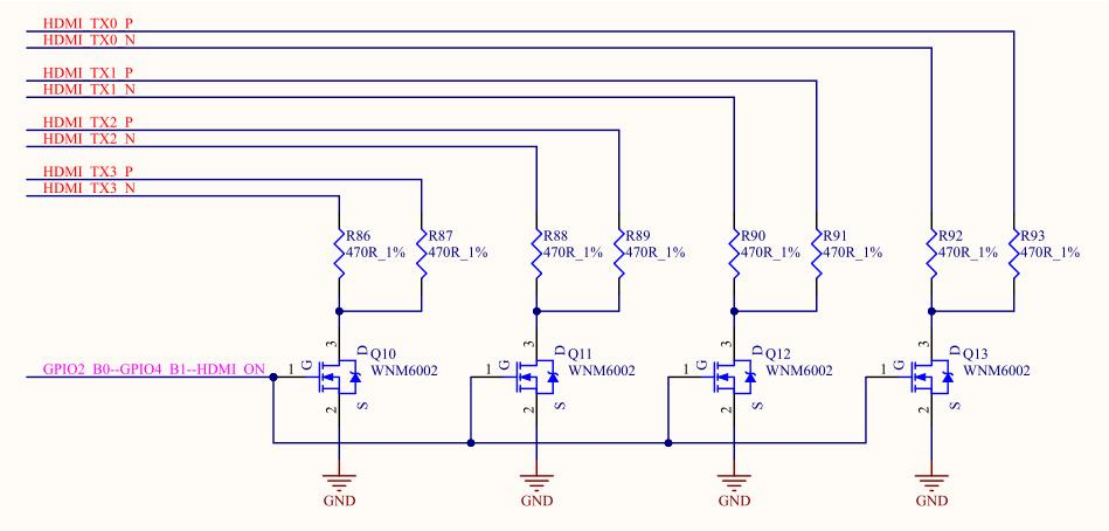
1.10.12 HDMI

HDMI（High-Definition Multimedia Interface） 是一种用于传输高清视频和音频的数字接口标准。它广泛应用于电视、显示器、投影仪等设备之间的数据传输。HDMI 能够提供高质量的视频和音频信号传输，同时减少了传统模拟信号传输中的质量损失和电磁干扰。ELF2 开发板设计了一路 HDMI 输出接口。

1.10.12.1 硬件原理

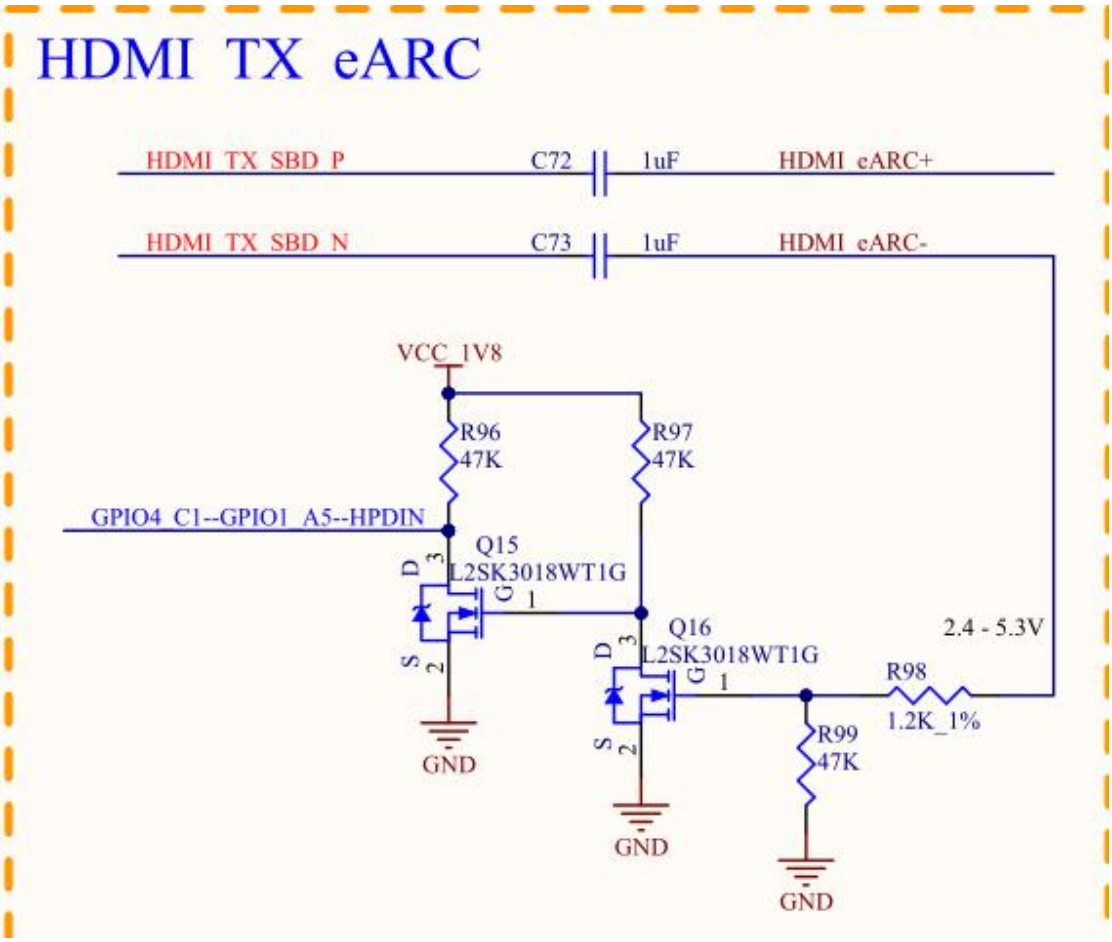
HDMI 接口部分的原理图：





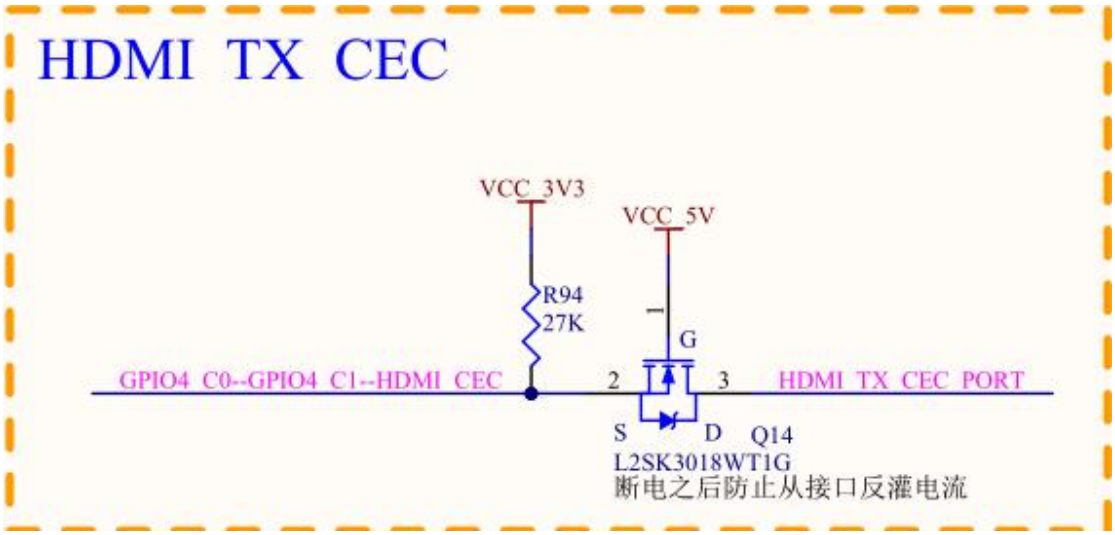
FRL 模式：在传统的 TMDS 架构下，是利用一个独立的通道来传送 Clock，但在 FRL 的架构中，将 Clock 嵌入在 Data 的通道中，在 Sink 端透过 Clock Recovery 解析出 Clock。

支持 ARC/eARC 通过 HDMI0_TX_SBDP/HDMI0_TX_SBDN 信号到 RK3588 内部解析出音频数据。

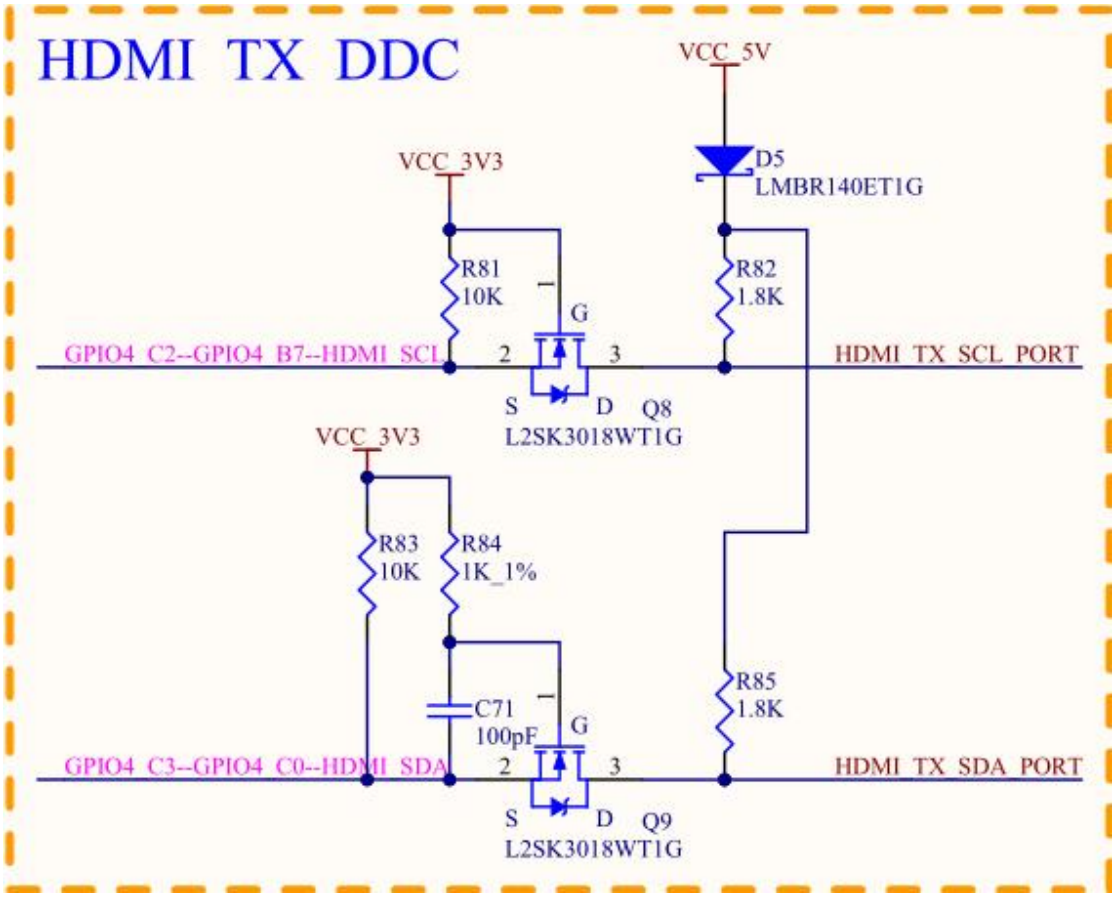


HDMI_TX0_HPD 是 HDMI TX 控制器复用到普通 GPIO 上，电平随所在电源域电压，电源域供电电压有更改，外围电路的上拉电阻电源也必须同步调整。

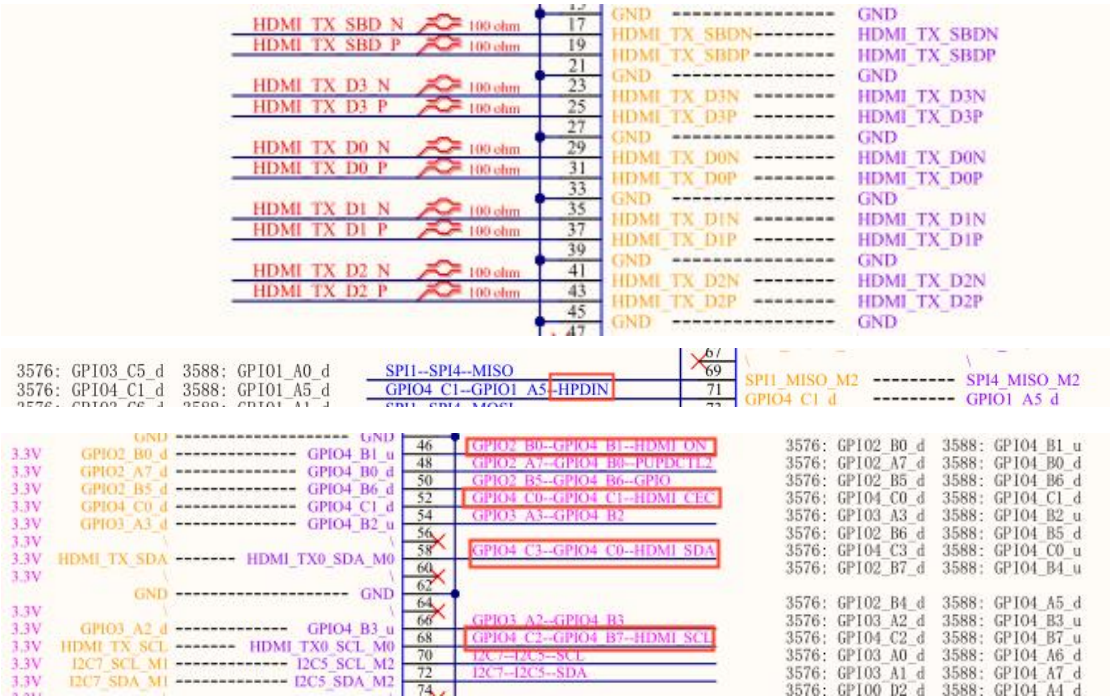
HDMI_TX0_CEC 是 HDMI 控制器 CEC 功能复用到普通 GPIO 上功能，电平随所在电源域电压，电源域供电电压有更改，外围电路的上拉电阻电源也必须同步调整。



HDMI_TX0 DDC_SCL/DDC_SDA 是 HDMI TX0 控制器的 I2C/DDC 总线，电平随所在电源域电压，电源域供电电压有更改，外围电路的上拉电阻电源也必须同步调整。DDC_SCL/DDC_SDA 协议规定是 5V 电平，RK3588 IO 不支持 5V 电平，必须增加电平转换电路，不得删减，默认使用 MOS 管电平转换，MOS 型号默认选择 2SK3018，如果要换其它型号，结电容必须相当，如果用结电容过大，不仅影响工作，认证也会过不了。



核心板端的原理图如下：



1.10.12.2 IOMUX 配置

HDMI 的数据引脚为专用引脚不需要配置。

HDMI 的使能引脚 HDMI_ON 使用的为 GPIO4_B1。此功能为 GPIO 功能，也不需要进行 IOMUX 配置。

HDMI_CEC 使用的为 GPIO4_C1，HDMI_HPD 使用的 GPIO1_A5，HDMI_SCL 使用的 GPIO4_B7，HDMI_SDA 使用的 GPIO4_B4。

打开 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 搜索 hdmi0 节点，能检索到相关的引脚描述：

HDMI_CEC 使用的是 hdmim0_tx0_cec

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel

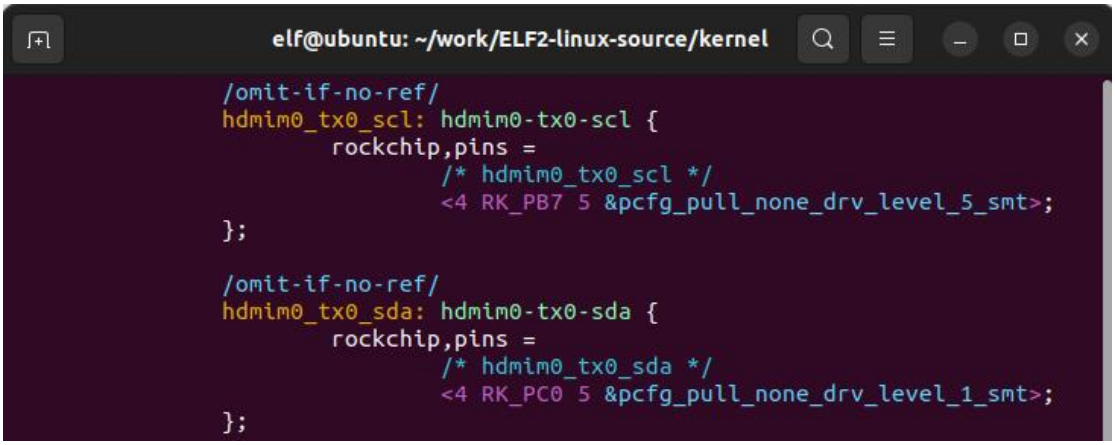
/omit-if-no-ref/
hdmim0_tx0_cec: hdmim0-tx0-cec {
        rockchip,pins =
                /* hdmim0_tx0_cec */
                <4 RK_PC1 5 &pcfg_pull_none>;
};
```

HDMI_HPD 使用的是 hdmim0_tx0_hpd

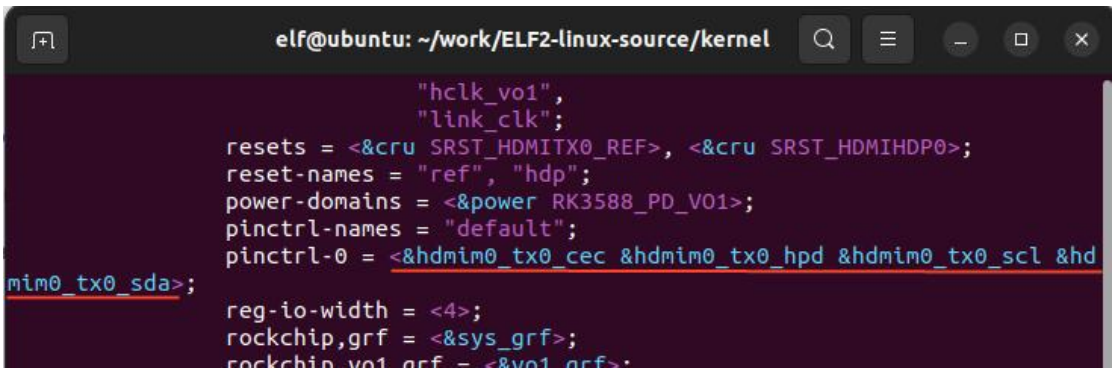
```
elf@ubuntu: ~/work/ELF2-linux-source/kernel

/omit-if-no-ref/
hdmim0_tx0_hpd: hdmim0-tx0-hpd {
        rockchip,pins =
                /* hdmim0_tx0_hpd */
                <1 RK_PA5 5 &pcfg_pull_none>;
};
```

HDMI_SCL 使用的是 hdmi0_rx0_scl，HDMI_SDA 使用的是 hdmi0_tx_sda



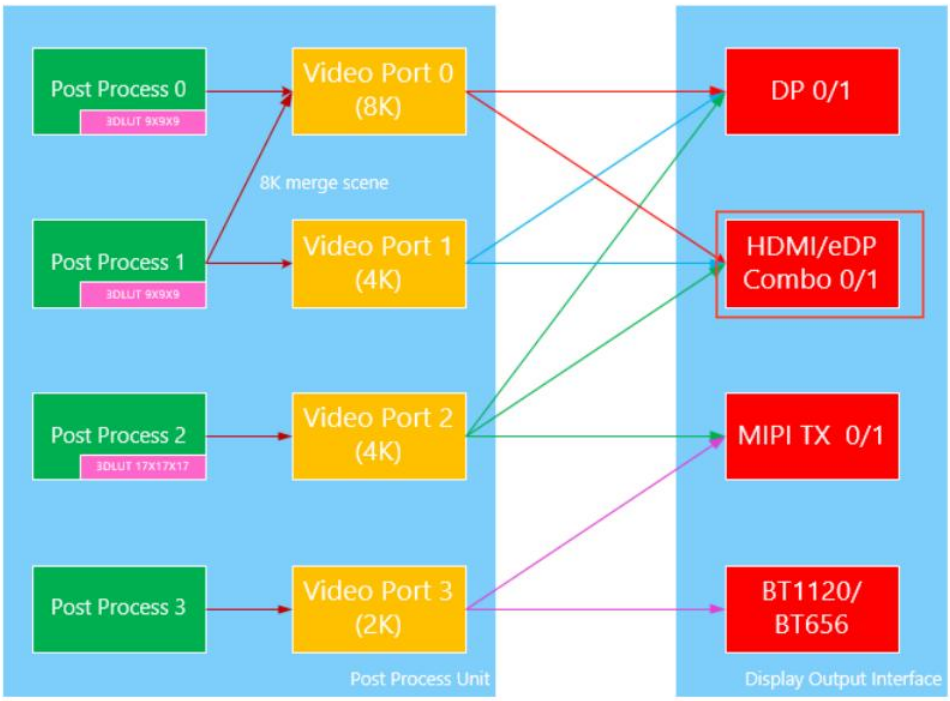
打开 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 设备树文件，查找 hdmi0 的默认引脚配置，可以看到默认使用的就是这些引脚，所以不需要单独配置。



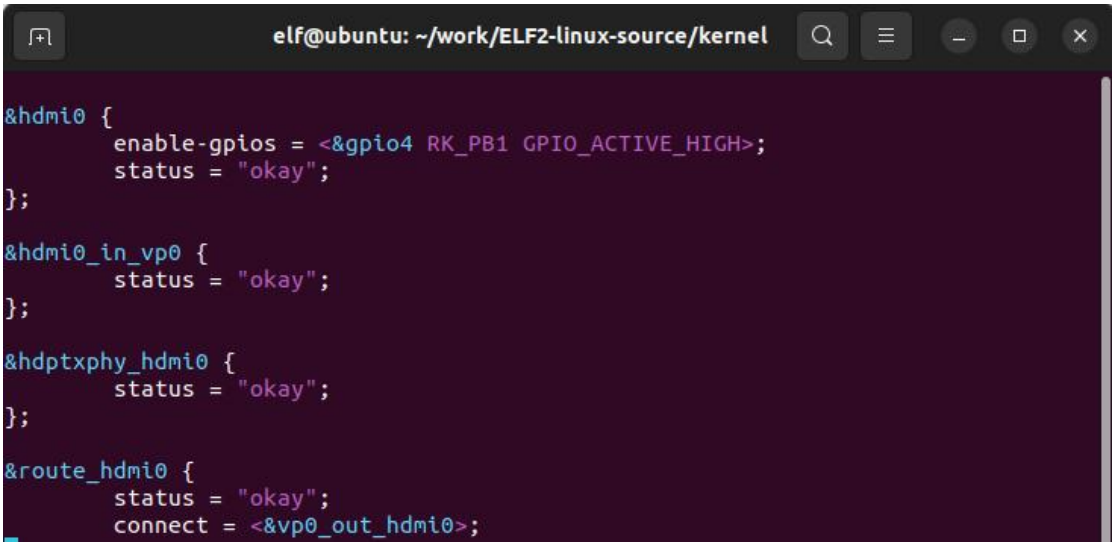
1.10.12.3 设备节点配置

同样从接口连接关系可以看出，HDMI 可以选择接到 VP0、VP1 或者 VP2 上，但是需要注意的是，RK3588 的 HDMI 和 DP 支持 8K 输出，在 8K 的输出模式下，一个显示接口需要同时占用 VP0 和 VP1。ELF2 默认是将 HDMI 连接到了 VP0 上。

4.3.2 RK3588 VP 和各显示接口的连接关系



在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 设备树中将 hmdi0 节点打开，并且将其连接到 VP0 上。

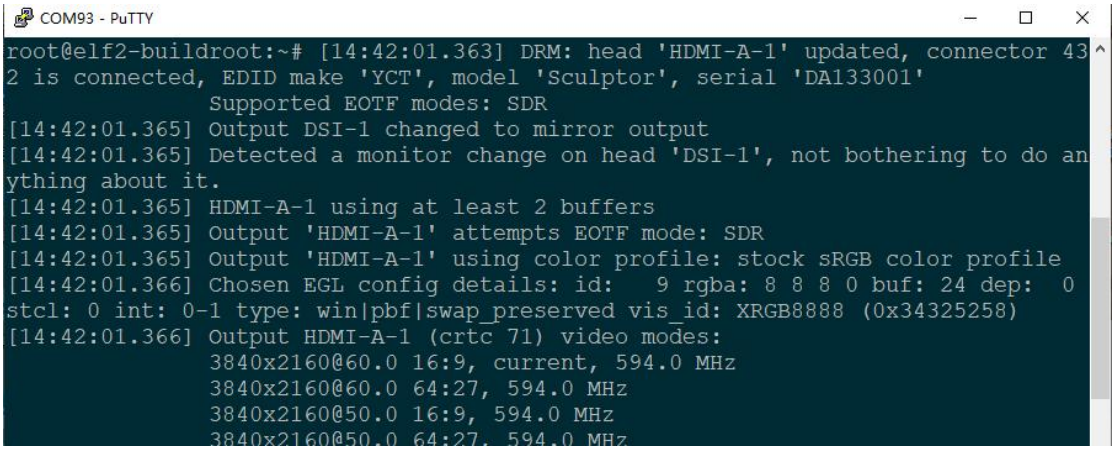


1.10.12.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板，HDMI 支持热插拔，启动完成后插上 HDMI 屏幕，可以看到显示正常，并且能够在调试终端中显示屏幕的信息。从打印信息中可以看出我插入的是一个 3840x2160 分辨率的 HDMI 屏幕。



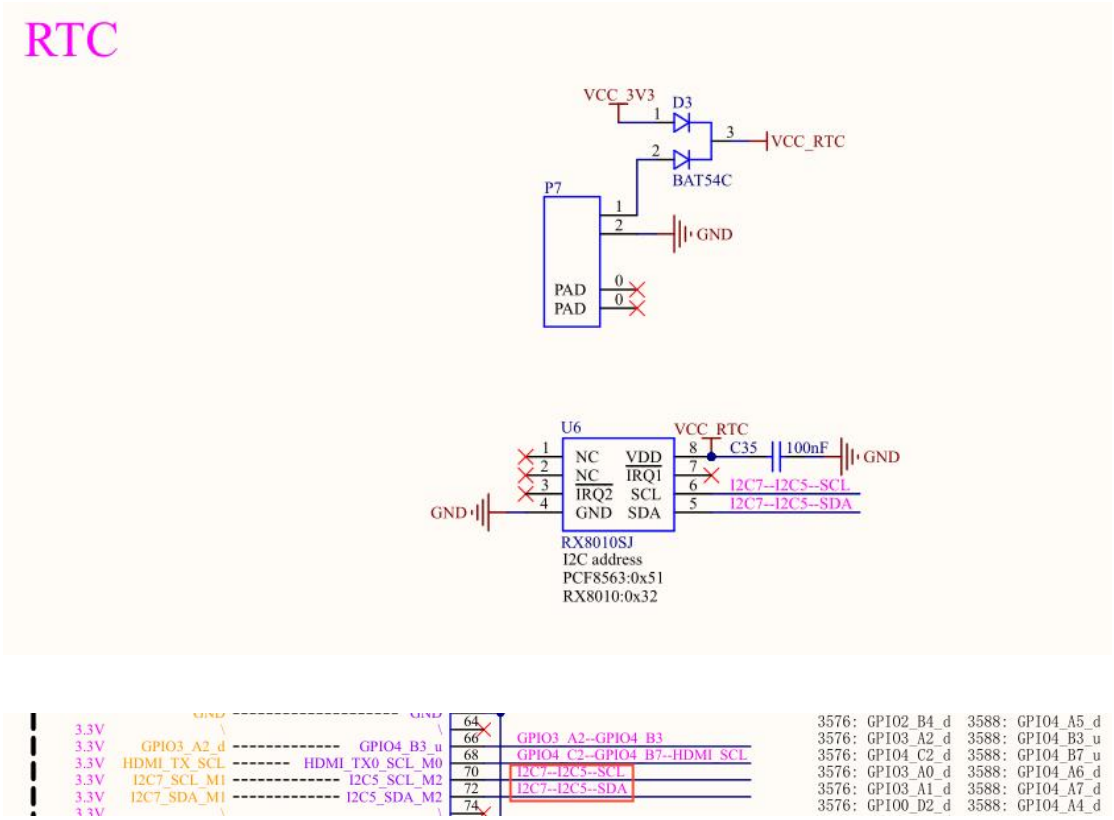
1.10.13 DP

DisplayPort（DP） 是一种数字显示接口标准，用于将视频和音频信号从计算机等设备传输到显示设备（如显示器、电视或投影仪）。ELF2 默认支持一路 DP 输出，可以将屏幕接到开发板的 TYPE-C 接口。前面“1.10.10 USB 控制器”章节的 TYPEC 部分已经对相关配置进行了讲解。此部分不在赘述。

1.10.14 RTC

RTC（Real-Time Clock，实时时钟），能够持续记录当前时间的硬件时钟模块。RTC 模块非常节能，在纽扣电池的支持下，RTC 可以保持长时间计时（通常数年），非常适合用在嵌入式系统中。RTC 的精度通常较高，可以在断电情况下保持时间同步。ELF2 开发板默认使用的是 RX8010RTC 芯片。

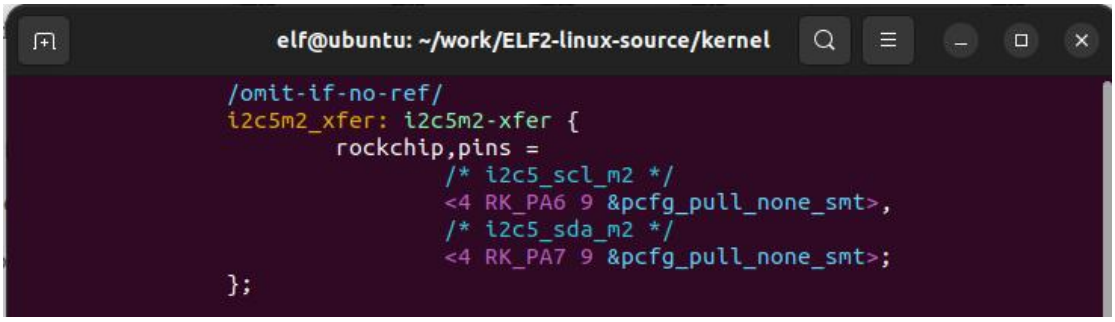
1.10.14.1 硬件原理



1.10.14.2 IOMUX 配置

RTC 芯片是 I2C 接口的，从原理图可以看出连接到的是 I2C5_M2 引脚，使用的是 GPIO4_A6 和

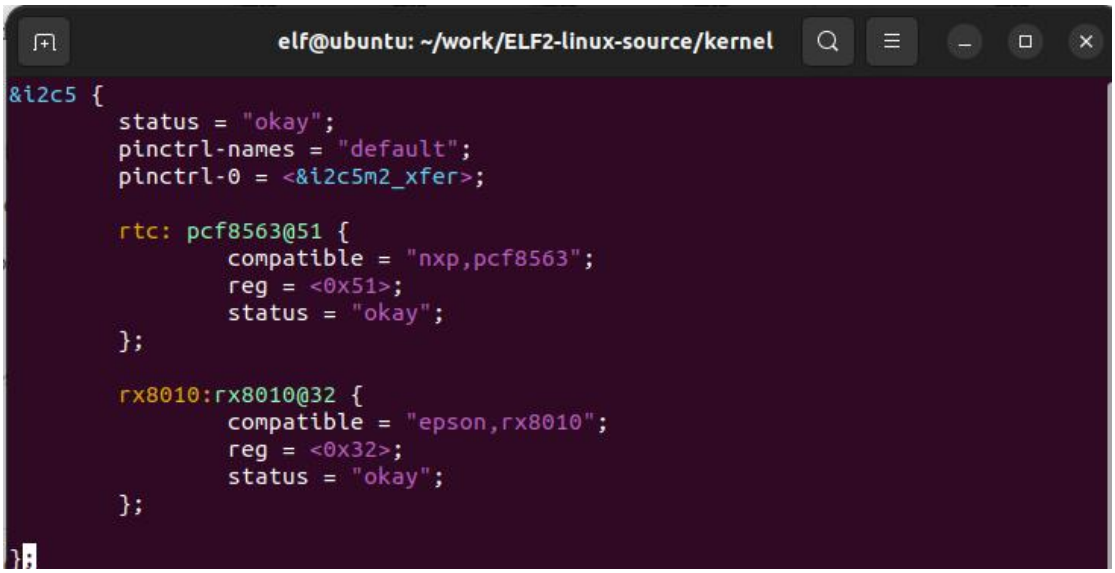
GPIO4_A7 两个引脚，打开 arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi 找到引脚描述。



检索 i2c5 可以找到对应的引脚描述为：i2c5m2_xfer。

1.10.14.3 设备节点配置

因为 RTC 芯片为 I2C 设备，且是连接到 I2C5 上的，所以需要在 I2C5 下创建子节点。打开 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 文件修改。



i2c5 的功能引脚修改为使用 i2c5m2_xfer，并且在下边创建 rx8010 的子节点，i2c 设备地址为 0x32。所匹配的驱动为”epson,rx8010"。

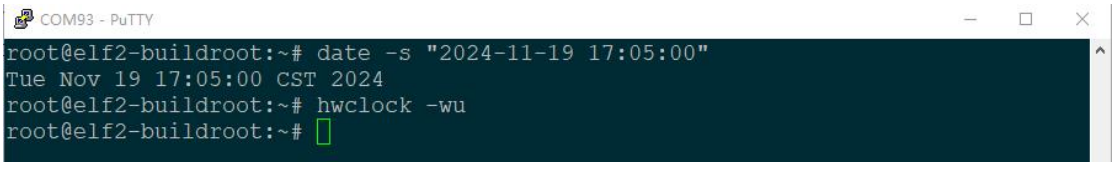
1.10.14.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板，启动后设置时间，然后给开发板断电，再上电查看时间依然能够同步，需要确保正常接上了纽扣电池，因为在断电状态下是由纽扣电池给 RTC 芯片供电的。

```
root@elf2-buildroot:~# date -s "2024-11-19 17:05:00"
Tue Nov 19 17:05:00 CST 2024
root@elf2-buildroot:~# hwclock -wu
```



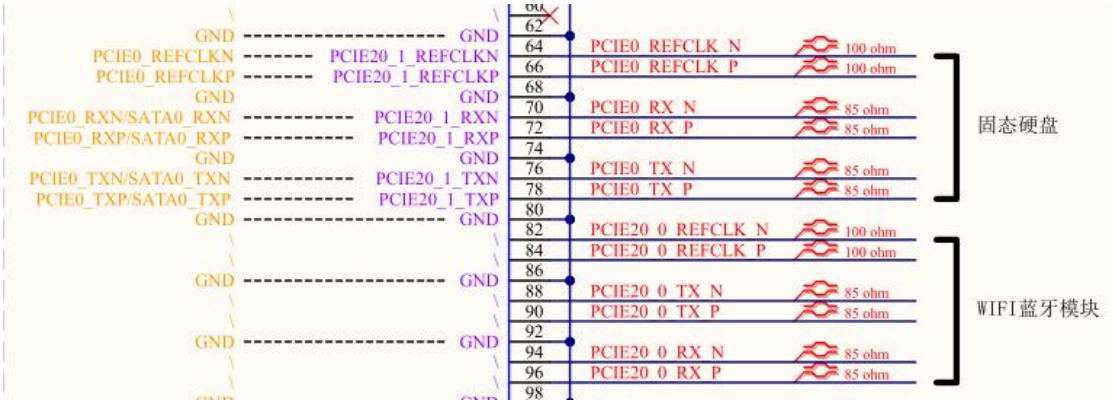
断电后重新使用 date 命令查询，可以看到时间是同步更新的。

```
COM93 - PuTTY
root@elf2-buildroot:~# date
Tue Nov 19 17:09:22 CST 2024
root@elf2-buildroot:~#
```

1.10.15 PCIe

PCIe（Peripheral Component Interconnect Express）是一种高速串行计算机扩展总线标准，由 Intel 在 2001 年提出，旨在替代旧的 PCI、PCI-X 和 AGP 总线标准。ELF2 开发板设计了两路 PCIe2.0 lLane。

1.10.15.1 硬件原理



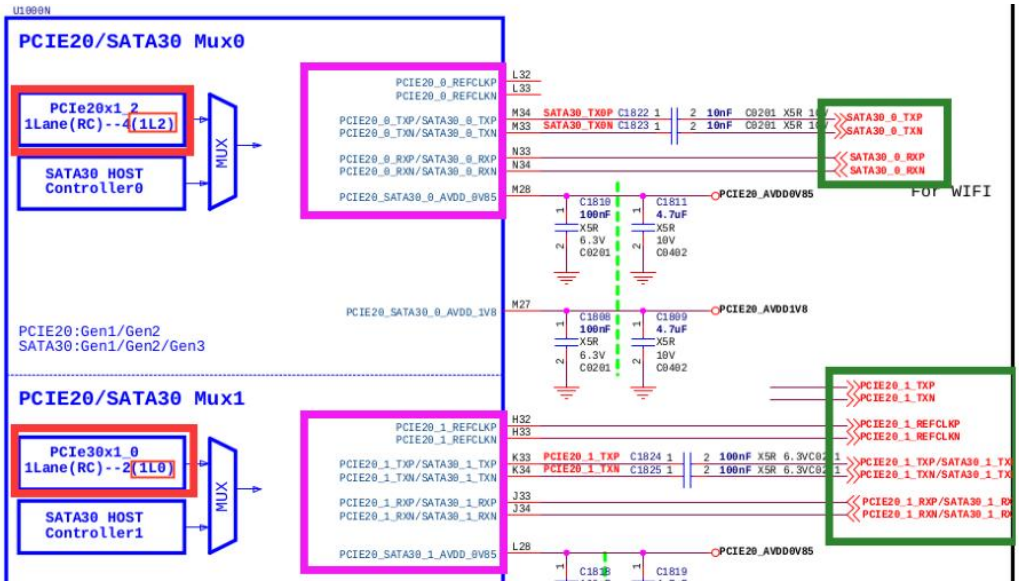
从原理图中可以看出，一路用于接固态硬盘，一路用于接 WIFI 蓝牙模块。

1.10.15.2 IOMUX 配置

PCIe 使用的都为专用引脚不需要单独配置。

1.10.15.3 设备节点配置

从原理图中可以看出使用的 PCIE20_0 和 PCIE20_1，对应的控制器为 1L2 和 1L0。这个可以从 ELF 2 开发板资料包\06-RK 原厂资料\cn\Common\PCIe 文件夹中的 Rockchip_Developer_Guide_PCIe_CN.pdf 文档中查看。

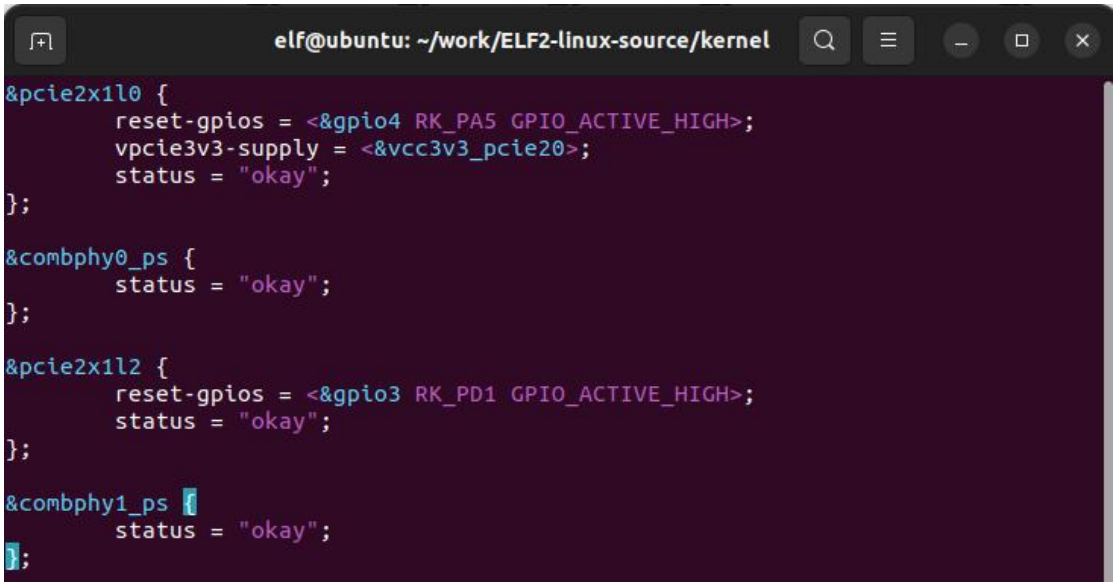


dts 节点同样从文档中能够找到，分别为 pcie2x1l0 和 pcie2x1l2：

资源	模式	dts 节点	可用phy	内部DMA	支持ASPM	支持MMU
PCIe Gen3 x 4 lane	RC/EP	pcie3x4: pcie@fe150000	pcie30phy	2个读Channel + 2个写 Channel	ALL	是
PCIe Gen3 x 2 lane	RC only	pcie3x2: pcie@fe160000	pcie30phy	否	ALL	是
PCIe Gen3 x 1 lane	RC only	pcie2x1l0: pcie@fe170000	pcie30phy, combphy1_ps	否	ALL	是
PCIe Gen3 x 1 lane	RC only	pcie2x1l1: pcie@fe180000	pcie30phy, combphy2_psu	否	ALL	是
PCIe Gen3 x 1 lane	RC only	pcie2x1l2: pcie@fe190000	combphy0_ps	否	ALL	是

pcie2x1l0 可用的 phy 为 pcie30phy 和 combphy1_ps。pcie2x1l2 可用的 phy 为 combphy0_ps。

这些节点已经在 arch/arm64/boot/dts/rockchip/rk3588s.dtsi 设备树中定义好了，不需要修改，只需要在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 设备树中引用，把状态改为 okay，并添加对应的复位引脚就可以了。

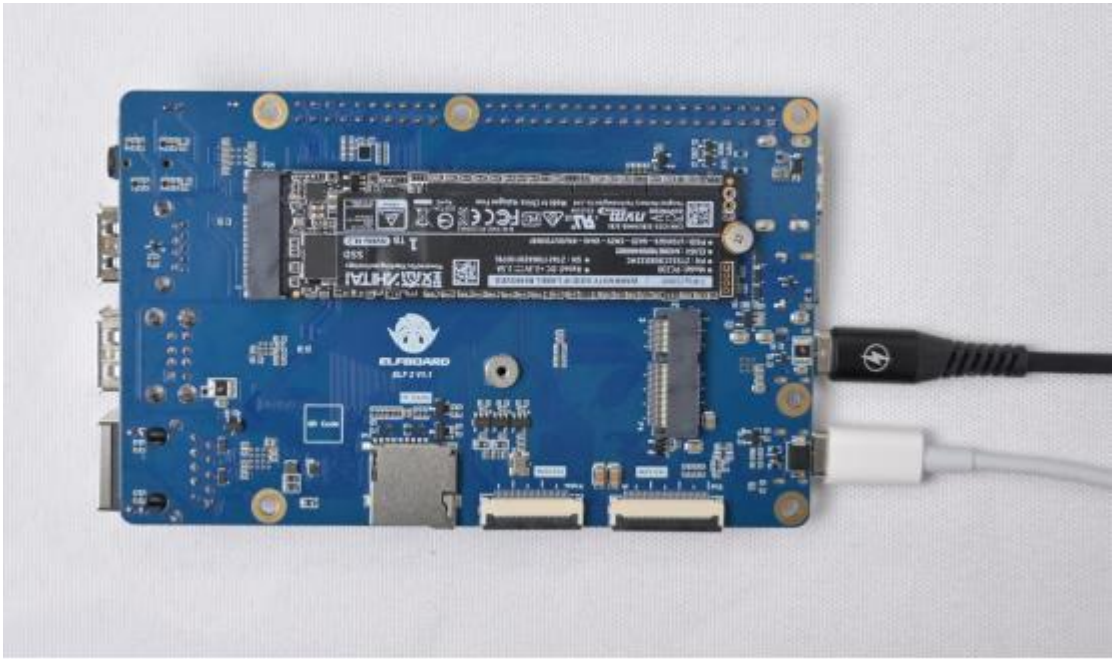


1.10.15.4 编译测试

编译内核和设备树：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板，插上固态硬盘，启机后 lscpi 可以查询到固态硬盘的信息。



```
root@elf2-buildroot:~# lspci
```

```
COM93 - PuTTY
root@elf2-buildroot:/# lspci
21:00.0 Class 0108: 144d:a808
20:00.0 Class 0604: 1d87:3588
root@elf2-buildroot:/#
```

内核中默认已经配置了固态硬盘的驱动，在/dev 目录下会生成 nvme 节点。

```
root@elf2-buildroot:~# ls /dev/nvme0*
/dev/nvme0   /dev/nvme0n1   /dev/nvme0n1p1
```

```
COM93 - PuTTY
root@elf2-buildroot:/# ls /dev/nvme0*
/dev/nvme0   /dev/nvme0n1   /dev/nvme0n1p1
root@elf2-buildroot:/#
```

可以手动将/dev/nvme0n1p1 挂载到文件系统中。比方说挂载到/mnt/sdcard 路径下：

```
root@elf2-buildroot:~#   mount /dev/nvme0n1p1 /mnt/sdcard/
[ 2009.199684] EXT4-fs (nvme0n1p1): mounted filesystem with ordered data mode. Opts: (null)
```

```
COM93 - PuTTY
root@elf2-buildroot:/# mount /dev/nvme0n1p1 /mnt/sdcard/
[ 2009.199684] EXT4-fs (nvme0n1p1): mounted filesystem with ordered data mode. O
pts: (null)
root@elf2-buildroot:/#
```

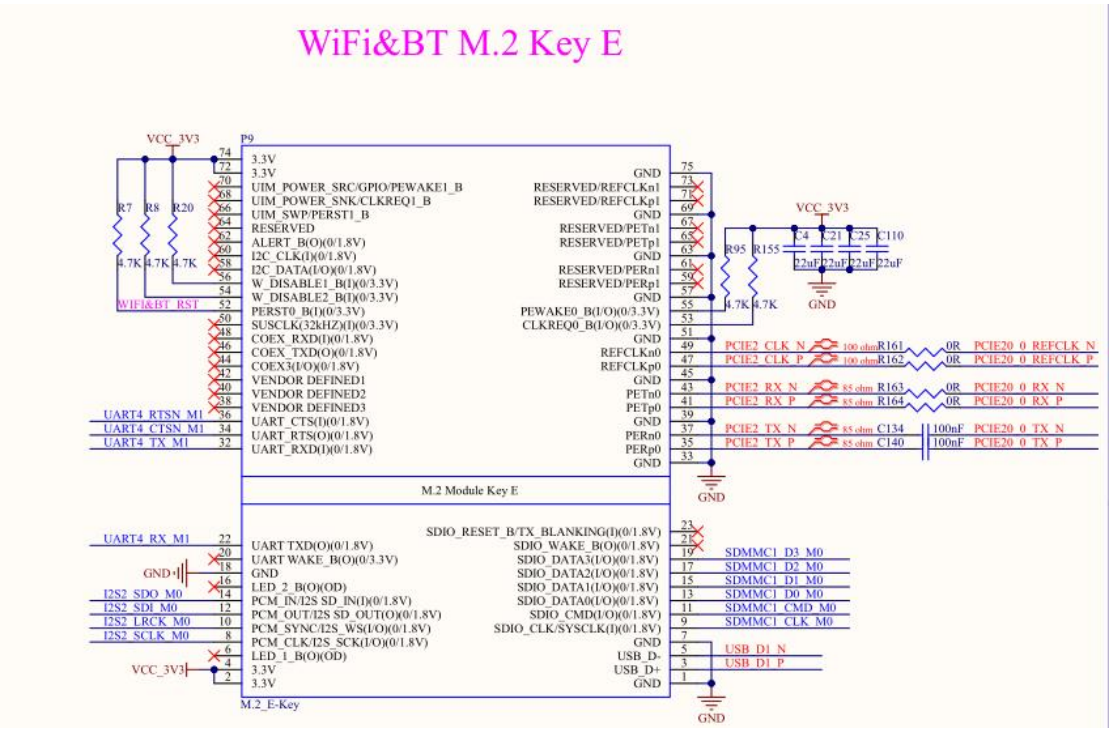
df -h 指令可以查看硬盘的大小及剩余空间。

```
COM93 - PuTTY
root@elf2-buildroot:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        14G  1.2G   13G   9% /
devtmpfs        1.8G  8.0K  1.8G   1% /dev
tmpfs           1.9G  248K  1.9G   1% /tmp
tmpfs           1.9G  408K  1.9G   1% /run
tmpfs           1.9G  148K  1.9G   1% /var/log
tmpfs           1.9G   79M  1.8G   5% /dev/shm
/dev/mmcblk0p7  123M   12M  108M  10% /oem
/dev/mmcblk0p8   14G  285M   13G   3% /userdata
/dev/nvme0n1p1  230G  1.3G  219G   1% /mnt/sdcard
root@elf2-buildroot:/#
```

1.10.16 Wifi&蓝牙

ELF2 开发板默认适配的是 CF-AX200 的 Wifi&蓝牙二合一模块。基于 Intel AX200 芯片，使用 M.2 接口。

1.10.16.1 硬件原理图



Wifi 使用的是 PCIe 接口，蓝牙使用的是 usb 接口。

1.10.16.2 IOMUX 配置

PCIe 和 usb 接口都是使用的专用接口，不涉及到 IOMUX 的配置。

1.10.16.3 设备节点配置

PCIe 的节点配置和 USB 接口的节点配置在前面章节已经讲解，Wifi 和蓝牙不涉及到设节点的配置。

1.10.16.4 内核驱动配置

在 SDK 目录下执行./build.sh kconfig 指令进入到图形界面进行配置：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kconfig
```

将 Wifi 和蓝牙相关的驱动都配置上。

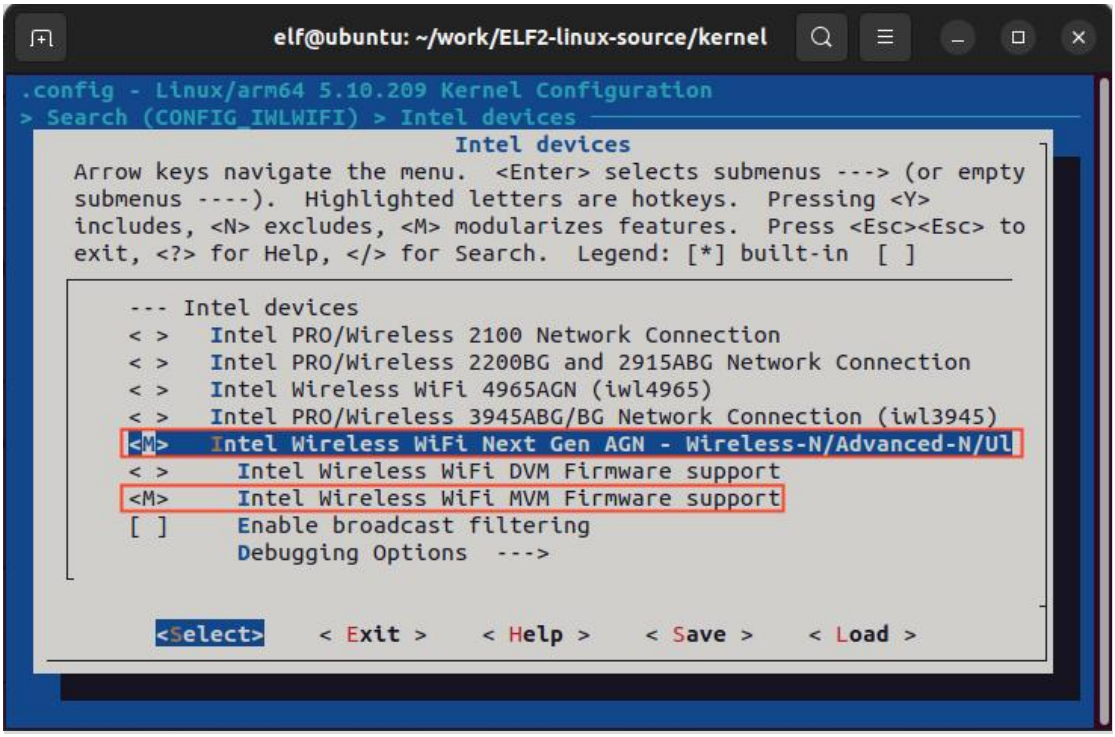
进入到如下路径：

```
-> Device Drivers
    -> Network device support (NETDEVICES [=y])
```

```
-> Wireless LAN (WLAN [=y])
    -> Intel devices (WLAN_VENDOR_INTEL [=y])
```

将 CONFIG_IWLWIFI 和 CONFIG_IWLMVM 都配置上，编译成模块。

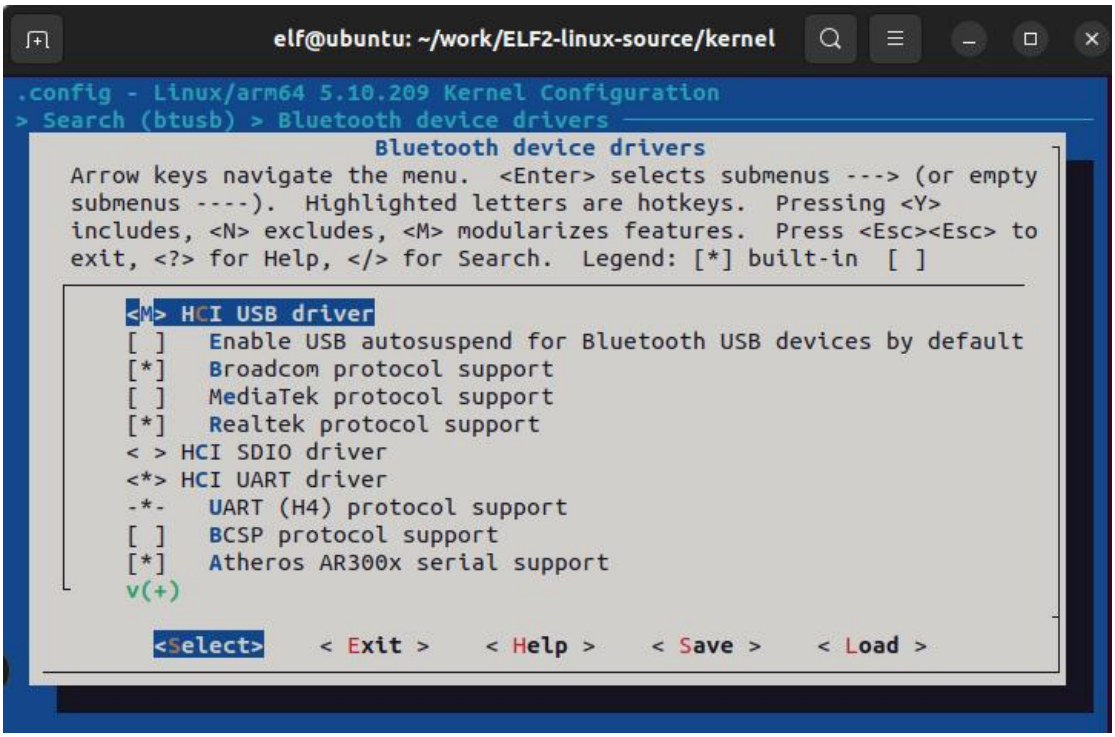
其中 CONFIG_IWLWIFI 表示将 Intel 的 Wifi 驱动编译上，CONFIG_IWLMVM 表示支持 MVM 固件。



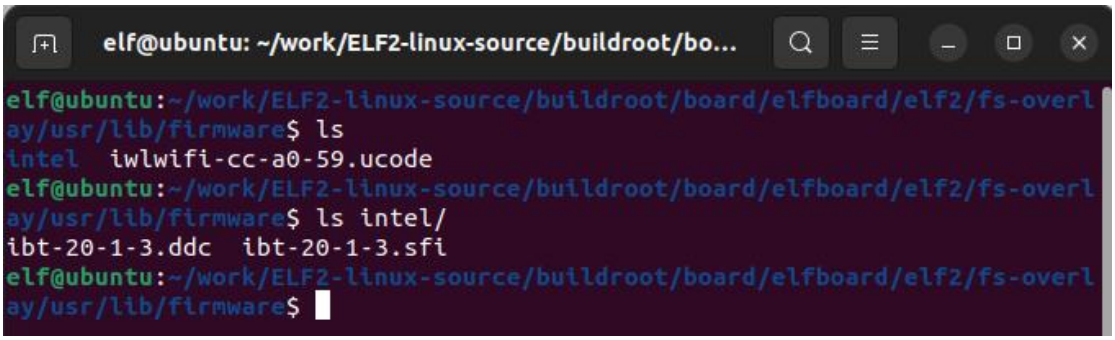
蓝牙驱动配置：

```
-> Networking support (NET [=y])
    -> Bluetooth subsystem support (BT [=y])
        -> Bluetooth device drivers
```

将 CONFIG_BT_HCIBTUSB 编译成模块。



驱动配置完成之后还需要添加 Wifi 和蓝牙的固件，默认已经添加到了 buildroot 中，在 buildroot 中的路径为：buildroot/board/elfboard/elf2/fs-overlay/usr/lib/firmware，iwlwifi-cc-a0-59.ucode 为 Wifi 的固件，intel 目录下的 ibt-20-1-3.ddc 和 ibt-20-1-3.sfi 为蓝牙的固件。



1.10.16.5 编译测试

编译内核和设备树：

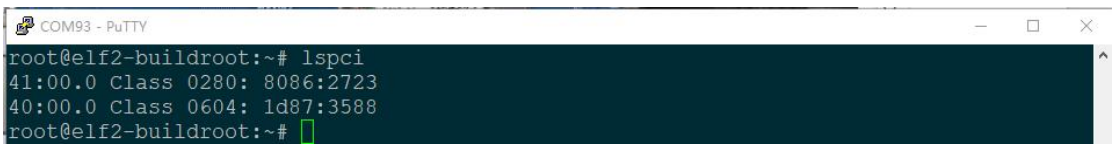
```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

编译完成后，将 kernel 目录下生成的 boot.img 烧录到开发板，插上 Wifi 模组。

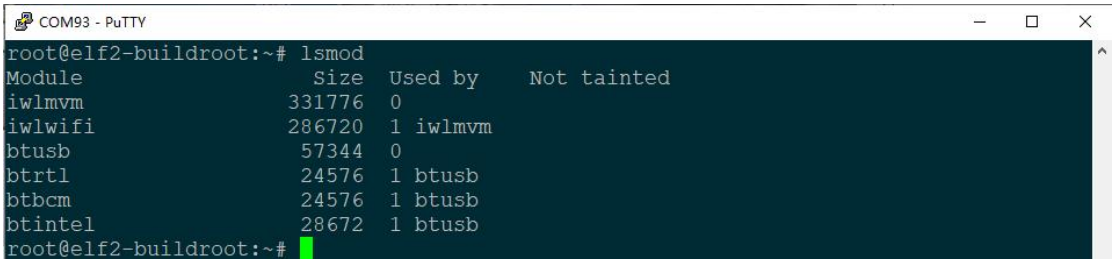


输入 lspci 能够查询到 Wifi 设备：

```
root@elf2-buildroot:~# lspci
41:00.0 Class 0280: 8086:2723
40:00.0 Class 0604: 1d87:3588
```



输入 lsmod 能够查询到 Wifi 和蓝牙的驱动模块。



Wifi 和蓝牙的功能测试，参考《ELF2 开发板快速启动手册》的 3.1.4 的 WIFI 测试和 3.1.5 的蓝牙测试。

第二章 驱动开发

2.1 驱动基本介绍

在 Linux 系统中，驱动（Driver）用于与硬件设备进行交互，以便操作系统可以控制和管理设备的功能。驱动程序充当操作系统和硬件设备之间的桥梁，使它们能够相互通信和协调工作。

驱动程序负责向操作系统提供设备的抽象接口，隐藏硬件的细节，并为操作系统和应用程序提供一致的接口。它通过操作硬件设备的寄存器、访问设备的内存和执行设备特定的指令，来实现对设备的控制和通信。

在 Linux 系统中，有两种主要的方式可以将驱动程序与内核进行集成：驱动模块和编译到内核。

1.驱动模块（Loadable Kernel Module，LKM）

驱动模块是一种动态加载到内核的可执行代码，它可以在运行时加载和卸载。驱动模块通常作为独立的文件存在，拥有自己的源代码和编译对象。它们可以通过内核模块加载器（如 insmod、modprobe）在运行时加载到内核中，从而与内核进行交互。

驱动模块的优势在于可以在系统运行时动态加载和卸载，从而实现灵活性和可扩展性。此外，驱动模块允许开发人员通过模块参数来配置和调整驱动程序的行为，而无需重新编译整个内核。这样可以减少开发时间和系统维护的复杂性。

2.编译到内核（Built-in）

编译驱动程序到内核意味着将驱动代码与内核代码合并，并通过重新编译整个内核将其包含在内核镜像中。这种方式将驱动程序与内核完全集成，使其成为内核的一部分，因此在系统启动时就会加载和初始化。

编译驱动程序到内核的优势在于能够获得更高的性能和更低的系统开销，因为驱动程序与内核紧密耦合，可以直接访问内核数据结构和函数。然而，这也意味着每次更新驱动程序时都需要重新编译整个内核，这可能会增加开发和维护的复杂性。

2.1.1 Linux 设备驱动的分类

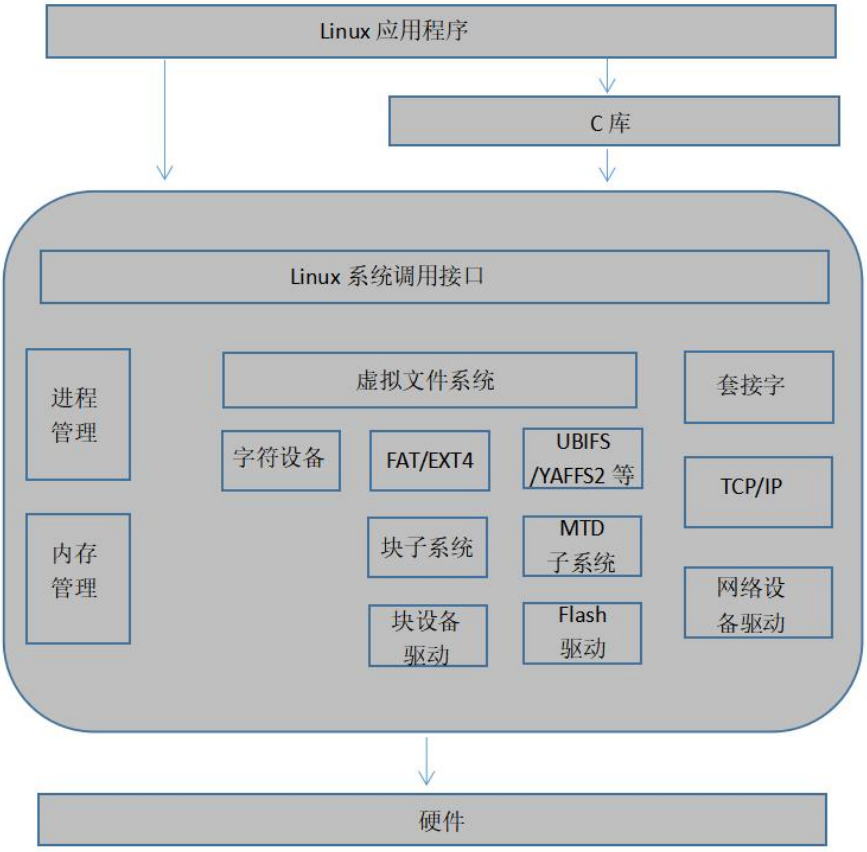
Linux 设备驱动可以分为以下三类:

字符设备驱动（Character Device Drivers）：这种驱动用于管理字符设备，例如终端设备、串口设备等。它们通过提供读取和写入字符流的功能来与设备进行交互。

块设备驱动（Block Device Drivers）：块设备驱动用于管理块设备，例如硬盘驱动器、固态硬盘等。它们支持以块为单位进行数据读写操作。

网络设备驱动（Network Device Drivers）：这类驱动程序用于支持网络接口卡（NIC）和其他网络设备。它们负责管理数据包的传输、网络连接和协议处理等功能。

2.1.2 Linux 设备驱动与整个软硬件系统的关系



从上图可以看出，除网络设备外，字符设备和块设备都映射到了虚拟文件系统目录下。应用程序可以通过系统调用接口 open、close、write、read 等相关 API 进行访问字符设备和块设备，继而实现对硬件的操作。

Linux 设备驱动程序作为内核模块嵌入到 Linux 内核中，位于内核空间。它们直接与内核进行交互，通过内核提供的接口与硬件设备进行通信和控制。

用户空间的应用程序通过系统调用和设备文件接口与设备驱动进行交互。设备驱动负责接收来自用户空间的请求，并将其转化为相应的硬件操作。

2.2 Linux 驱动模块

2.2.1 helloworld 驱动

本章节先写一个 helloworld 驱动，不涉及对硬件的操作，它的目的是展示驱动程序的基本结构和加载过程。

1.源码编写

1）首先包含头文件

```
#include <linux/module.h>     // 包含模块相关函数的头文件
#include <linux/kernel.h>     // 包含内核相关函数的头文件
#include <linux/init.h>       // 包含初始化和清理函数的头文件
```

2）驱动模块入口和出口函数

```
module_init(helloworld_init);   // 指定驱动程序的初始化函数
module_exit(helloworld_exit);   // 指定驱动程序的清理函数
```

module_init()的定义：

```
#define module_init(init_fn)                \
    static inline initcall_t __initcall_##init_fn##_wrapper(void) \
    {                                                   \
        return init_fn;                                 \
    }                                                   \
    int init_module(void) __attribute__((alias(#init_fn)));
```

module_init()宏接受一个初始化函数作为参数，并将其定义为内联函数__initcall_##init_fn##_wrapper()。然后，它使用__attribute__((alias(#init_fn)))将 init_module()函数与初始化函数进行关联。

在 Linux 内核加载模块时，会调用 init_module()函数，它实际上是一个入口点函数。通过将 module_init()定义为 init_module()的别名，使得初始化函数成为模块加载时的入口点函数。

使用 module_init()宏的目的是在驱动程序中指定初始化函数，以便在模块加载时调用该函数执行必要的初始化操作。

当使用 insmod 将模块加载进内核的时候，初始化函数的代码将会被执行。模块初始化代码只与内核模块管理子系统打交道，并不与应用程序直接交互。

module_exit()的定义：

```
#define module_exit(exit_fn)                \
    static inline exitcall_t __exitcall_##exit_fn##_wrapper(void) \
    {                                                   \
        return exit_fn;                                 \
    }                                                   \
    void cleanup_module(void) __attribute__((alias(#exit_fn)));
```

module_exit()宏接受一个退出函数作为参数，并将其定义为内联函数__exitcall_##exit_fn##_wrapper()。然后，它使用__attribute__((alias(#exit_fn)))将 cleanup_module()函数与退出函数进行关联。

在 Linux 内核卸载模块时，会调用 cleanup_module()函数，它实际上是一个出口点函数。通过将 module_exit()定义为 cleanup_module()的别名，使得退出函数成为模块卸载时的出口点函数。

使用 module_exit()宏的目的是在驱动程序中指定退出函数，以便在模块卸载时调用该函数执行必要的清理操作。

几点说明：

l 模块退出没有返回值；

l __exit 标记这段代码仅用于模块卸载；

l module_exit 不是必须的。但是，没有 module_exit 定义的模块无法被卸载，如果需要支持模块卸载则必须有 module_exit。

当使用当使用 rmmod 卸载模块时，退出函数的代码将被执行。模块退出代码只与内核模块管理子系统打交道，并不直接与应用程序交互。

3）入口函数和出口函数实现

```
static int __init helloworld_init(void)
{
    printk(KERN_INFO "Hello, World!\n");   // 打印消息到内核日志
    return 0;
```

```
}

static void __exit helloworld_exit(void)
{
    printk(KERN_INFO "Goodbye, World!\n");  // 打印消息到内核日志
}
```

内核加载时输出 Hello, World!，内核卸载时输出 Goodbye, World!

4）声明模块信息

```
MODULE_LICENSE("GPL");  // 指定模块的许可证信息
MODULE_AUTHOR("Your Name");  // 指定模块的作者信息
MODULE_DESCRIPTION("A simple Hello World driver");  // 指定模块的描述信息
```

2.hello.c 示例源码

```
#include <linux/module.h>     // 包含模块相关函数的头文件
#include <linux/kernel.h>     // 包含内核相关函数的头文件
#include <linux/init.h>       // 包含初始化和清理函数的头文件

static int __init helloworld_init(void)
{
    printk(KERN_INFO "Hello, World!\n"); // 打印消息到内核日志
    return 0;
}

static void __exit helloworld_exit(void)
{
    printk(KERN_INFO "Goodbye, World!\n"); // 打印消息到内核日志
}

module_init(helloworld_init); // 指定驱动程序的初始化函数
module_exit(helloworld_exit); // 指定驱动程序的清理函数

MODULE_LICENSE("GPL"); // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple Hello World driver"); // 指定模块的描述信息
```

3.编译

编译驱动分为两种形式，一种是编译现成模块，生成.ko 文件然后通过手动加载；另一种是编译到内核，在内核启动时自动加载。一般在驱动调试阶段会编译成模块，这样便于调试，不用每次都重新编译整个内核，本章节以编译成模块进行讲解。

在 hello.c 目录下编写 Makefile：

```
LINUX_KERNEL_PATH = /home/elf/work/ELF2-linux-source/kernel
CURRENT_PATH := $(shell pwd)
obj-m :=hello.o
```

```
kernel_modules:
	$(MAKE) -C $(LINUX_KERNEL_PATH) M=$(CURRENT_PATH) modules \
		ARCH=arm64 \

CROSS_COMPILE=/home/elf/work/ELF2-linux-source/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
clean:
	$(MAKE) -C $(LINUX_KERNEL_PATH) M=$(CURRENT_PATH) clean
```

-C：参数后面加 config 文件所在的文件夹；

M：参数后面是要编译的模块；

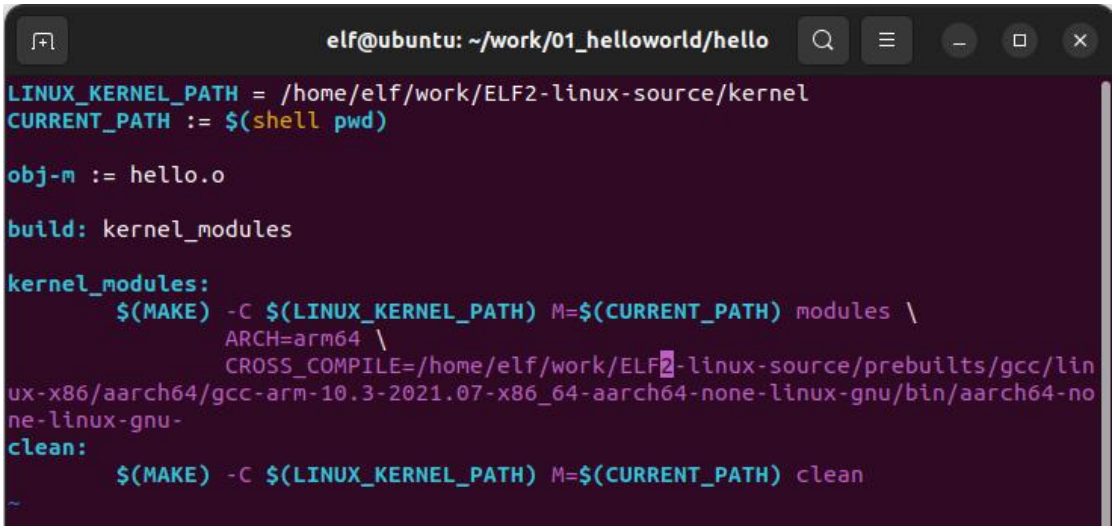
LINUX_KERNEL_PATH：为要依赖内核的目录；

CURRENT_PATH：目前模块的目录；

CROSS_COMPILE：编译器的前缀；

⚠**注意**：LINUX_KERNEL_PATH 的路径要以实际源码存放的路径为准。

添加后效果如下：



执行 make 编译，将生成的 hello.ko 文件拷贝到开发板中测试。在开发板中执行 insmod 加载驱动，rmmod 卸载驱动：

```
root@elf2-buildroot:/# insmod hello.ko
[    64.758236] Hello, World!
root@elf2-buildroot:/# rmmod hello.ko
[    67.517591] Goodbye, World!
```

可以看出加载时进入到了 helloworld_init 函数，并打印了 Hello, World!，卸载函数时进入了 helloworld_exit 函数并打印了 Goodbye, World!

2.2.2 带参数的驱动模块

Linux 驱动模块可以接受参数，这些参数可以在加载模块时通过命令行或配置文件进行配置。驱动模块参数提供了一种动态配置和调整驱动行为的方式。

在 Linux 内核中，驱动模块参数通常使用 module_param 宏进行定义和注册。下面是一些常用的驱动模块参数相关的宏和函数。

module_param：用于定义和注册一个驱动模块参数。

```
module_param(name, type, permissions);
```

name：参数名称，通常使用驼峰命名法。
type：参数的数据类型，可以是整型、字符型或布尔型等。
permissions：指定参数的访问权限。

MODULE_PARM_DESC：用于定义驱动模块参数的描述信息。

```
MODULE_PARM_DESC(name, description);
```

name：参数名称，与之前使用 module_param 或 module_param_array 定义的参数名称相同。
description：参数的描述信息。

通过使用这些宏和函数，可以定义和注册驱动模块参数。在加载驱动模块时，可以通过命令行或配置文件设置这些参数的值。

1.hello_param.c 示例源码

```
#include <linux/module.h>     // 包含模块相关函数的头文件
#include <linux/kernel.h>     // 包含内核相关函数的头文件
#include <linux/init.h>       // 包含初始化和清理函数的头文件
#include <linux/moduleparam.h> //包含模块参数相关函数的头文件

static int my_param = 0;
static char *who = "helloworld";

module_param(my_param, int, S_IRUSR | S_IWUSR);
MODULE_PARM_DESC(my_param, "An integer parameter");

module_param(who, charp, S_IRUSR | S_IWUSR);
MODULE_PARM_DESC(who, "An integer parameter");

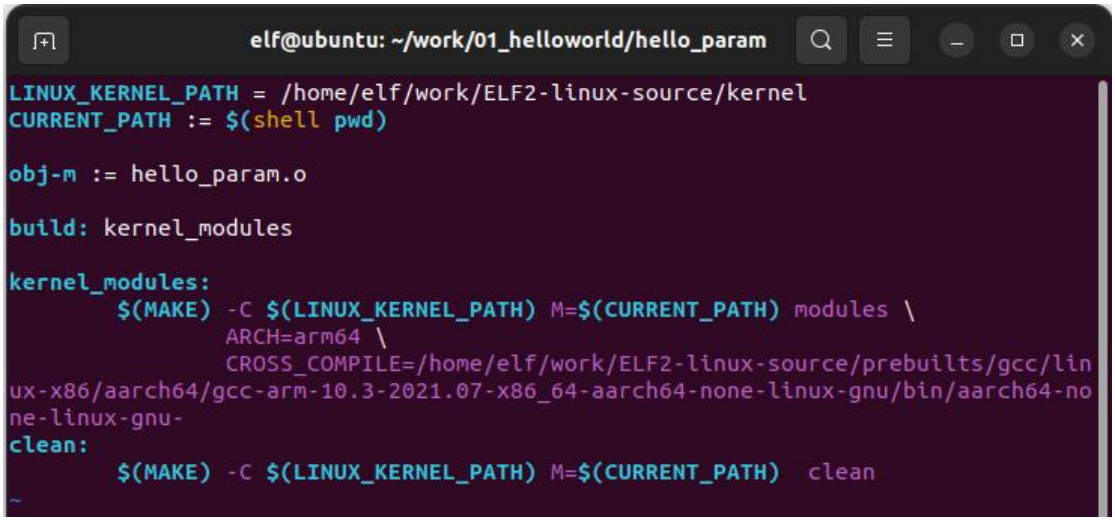

static int __init helloworld_init(void)
{
    printk(KERN_INFO "Hello, World! my_param is %d\n",my_param); // 打印消息到内核日志
    printk(KERN_INFO "Hello, World! this is %s\n",who);
 return 0;
}
static void __exit helloworld_exit(void)
{
    printk(KERN_INFO "Goodbye, World!\n"); // 打印消息到内核日志
}

module_init(helloworld_init); // 指定驱动程序的初始化函数
module_exit(helloworld_exit); // 指定驱动程序的清理函数

MODULE_LICENSE("GPL"); // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple Hello World driver"); // 指定模块的描述信息
```

2.编译

复制 helloworld 驱动中的 Makefile 文件，将其中的 hello.o 修改为 hello_param.o，效果如下：



执行 make 编译，将生成的 hello_param.ko 文件拷贝到开发板中测试。

在开发板中执行 insmod 加载驱动，rmmod 卸载驱动：

```
root@elf2-buildroot:~# insmod  hello_param.ko
[  627.884478] Hello, World! my_param is 0
[  627.884514] Hello, World! this is helloworld
root@elf2-buildroot:~# rmmod hello_param.ko
[  666.760635] Goodbye, World!
```

可以看出不输入参数，加载时打印出的是初始化参数。下面我们传入参数看一下效果：

```
root@elf2-buildroot:~# insmod hello_param.ko my_param=72 who=elfboard
[ 1725.489043] Hello, World! my_param is 72
[ 1725.489086] Hello, World! this is elfboard
root@elf2-buildroot:~# rmmod hello_param.ko
[ 1763.150887] Goodbye, World!
```

可以看到我们写的参数传到了驱动中，并打印了出来。

2.3 字符驱动

2.3.1 字符驱动描述

字符驱动是一种在 Linux 内核中实现的设备驱动程序，用于管理和操作字符设备。字符设备是以字符为单位进行输入和输出的设备，如终端、串口、打印机等。

字符驱动的主要任务是提供对字符设备的访问接口，使应用程序能够通过文件操作的方式读取和写入字符设备的数据。它将字符设备抽象为文件，应用程序可以像读写文件一样操作字符设备。这种通过文件操作接口进行访问的方式简化了应用程序与设备之间的交互。

字符驱动通常包含以下基本组件：

设备文件：字符驱动通过在文件系统中创建设备文件来表示字符设备。应用程序可以通过打开设备文件来访问字符设备。

初始化和清理函数：字符驱动在加载时执行初始化函数，用于初始化驱动程序的状态和资源。在卸载时执行清理函数，用于释放驱动程序占用的资源。

文件操作函数：字符驱动提供一组文件操作函数，用于处理设备文件的打开、关闭、读取和写入等操

作。这些函数与应用程序通过设备文件进行交互，实现数据的读取和写入。

设备号和设备节点：字符驱动使用设备号来唯一标识字符设备，并通过设备节点与设备文件进行关联。设备节点是在/dev 目录下的特殊文件，它与设备文件一一对应。

锁和同步机制：字符驱动可能需要使用锁和同步机制来保护共享资源，以确保多个进程或线程之间的正确操作。

IOCTL 和其他功能：字符驱动还可以支持 IOCTL 命令和其他功能，以提供更丰富的设备控制和操作选项。

2.3.2 字符驱动框架描述

字符驱动框架的核心组件包括以下部分：

文件操作函数 (file_operations)：字符驱动框架通过 file_operations 结构体定义了一组函数指针，用于处理设备文件的各种操作，如打开(open)、关闭(release)、读取(read)、写入(write)、定位(llseek)等。驱动程序需要实现这些函数，以响应应用程序对设备文件的操作。

主设备号(Major Number)：主设备号用于标识字符设备驱动程序所控制的设备类型。驱动程序在注册时可以选择由内核分配主设备号，或者指定自己的主设备号。

设备号分配：在字符驱动加载时，可以使用 register_chrdev_region 或 alloc_chrdev_region 函数来动态分配设备号，或者通过 cdev_add 函数将设备号与字符设备关联。

字符设备结构(cdev)：字符设备结构 cdev 是在字符驱动中用于表示和管理字符设备的数据结构。它包含了设备号、文件操作函数等信息，并通过 cdev_init 和 cdev_add 函数与字符设备关联。

设备文件的创建与删除：通过 class_create 和 device_create 函数可以在/dev 目录下创建设备文件，并通过 device_destroy 和 class_destroy 函数进行删除。

内核缓冲区和用户空间数据交换：驱动程序可以使用 copy_to_user 和 copy_from_user 函数在内核缓冲区和用户空间之间传输数据。

同步与互斥机制：驱动程序可能需要使用信号量、互斥锁或自旋锁等同步机制，以确保多个进程或线程之间的数据安全性和一致性。

错误处理：驱动程序需要正确处理各种错误情况，包括参数校验、资源分配失败、设备访问错误等，并向用户空间返回适当的错误码。

2.3.3 测试示例

1.源码编写，新建一个 mydevice.c

1）首先包含头文件

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
```

2）声明变量

```
#define DEVICE_NAME "mydevice"    // 设备名称
static dev_t dev_num;                  //分配的设备号
struct    cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
```

3）驱动模块入口和出口函数

```
module_init(mydevice_init); // 指定驱动程序的初始化函数
module_exit(mydevice_init); // 指定驱动程序的清理函数
```

4）入口函数和出口函数实现

```
static int __init mydevice_init(void)
{
    int ret;

    // 在这里执行驱动程序的初始化操作

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
    }
        major=MAJOR(dev_num);          //获取主设备号
        minor=MINOR(dev_num);          //获取次设备号
            printk(KERN_INFO "major number: %d\n",major);
            printk(KERN_INFO "minor number: %d\n",minor);

            my_cdev.owner = THIS_MODULE;
            cdev_init(&my_cdev,&fops);      //初始化字符设备结构体
            cdev_add(&my_cdev,dev_num,1);   //将字符设备添加到内核中
                printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}

static void __exit mydevice_exit(void)
{
    // 在这里执行驱动程序的清理操作
    // 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);
        printk(KERN_INFO "Device unregistered.\n");
}
```

入口函数 mydevice_init(void)中主要有以下几个函数的调用：

调用 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, const char *name);函数进行设备号分配。

dev：指向 dev_t 类型的指针，用于接收分配的设备号范围的起始设备号。

firstminor：指定次设备号的起始值。次设备号用于区分同一主设备号下的不同设备实例。

count：指定要分配的设备号数量。

name：指定字符设备的名称。这是一个用于标识设备的字符串，可以在/proc/devices 中找到。

调用 MAJOR(dev)获取主设备号：

MAJOR(dev)宏接受一个 dev_t 类型的设备号作为参数，并返回该设备号的主设备号。

调用 MINOR(dev)获取次设备号：

MINOR(dev)宏接受一个 dev_t 类型的设备号作为参数，并返回该设备号的次设备号。

调用 void cdev_init(struct cdev *cdev, const struct file_operations *fops);函数初始化字符设备结构体

cdev：指向要初始化的 struct cdev 结构体的指针。

fops：指向包含设备操作函数的 struct file_operations 结构体的指针。

调用 int cdev_add(struct cdev *p, dev_t dev, unsigned int count);函数将字符设备添加到内核中。

p：指向要添加的 struct cdev 结构体的指针。

dev：指定要添加的设备号。通常是由 alloc_chrdev_region 或 register_chrdev_region 函数分配得到的设备号。

count：指定要添加的设备号数量。

cdev_add 函数将字符设备添加到内核中，并与指定的设备号相关联。当成功添加字符设备后，它将被分配一个主设备号和次设备号，并成为系统中的一个可用设备。

⚠**注意：**在调用 cdev_add 之前，应先使用 cdev_init 函数对 struct cdev 进行初始化，并设置正确的文件操作函数。

出口函数 mydevice_exit(void)中主要有以下两个函数的调用：

调用 void unregister_chrdev(unsigned int major, const char *name);函数注销已注册的字符设备。

major：要注销的字符设备的主设备号。

name：字符设备的名称。

unregister_chrdev 函数将指定的字符设备从内核中注销，并释放相关的资源。注销后，该字符设备将不再可用。

调用 void cdev_del(struct cdev *p);函数删除字符设备。

p：指向要删除的 struct cdev 结构体的指针。

cdev_del 函数从内核中删除指定的字符设备，并释放相关的资源。删除后，该字符设备将不再可用。

5）定义 fops 结构体

```
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .read = device_read,
    .write = device_write,
};
```

file_operations 结构体定义如下：

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    // 其他成员函数...
};
```

file_oprations 结构中定义了非常多的成员，但是对于大多数字符驱动而言，只需要实现，其中很少的几个方法即可。下面将忽略不常用的成员，对常用成员进行介绍：

owner：指向持有该结构体的模块的指针。
llseek：用于实现文件的定位（seek）操作。
read：用于实现从文件读取数据的操作。
write：用于实现向文件写入数据的操作。
open：用于打开文件时的处理操作。
release：用于关闭文件时的处理操作。

6）设备操作函数的实现

```
static int device_open(struct inode *inode, struct file *file)
{
    // 在这里处理设备打开的操作
    printk(KERN_INFO "This is device_open.\n");
    return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
    // 在这里处理设备关闭的操作
    printk(KERN_INFO "This is device_release.\n");

    return 0;
}

static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t *offset)
{
    // 在这里处理设备读取的操作
    printk(KERN_INFO "This is device_read.\n");
    return 0;
}

static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t *offset)
{
    // 在这里处理设备写入的操作
    printk(KERN_INFO "This is device_write.\n");
    return 0;
}
```

7）声明模块信息

```
MODULE_LICENSE("GPL"); // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple Hello World driver"); // 指定模块的描述信息
```

2.完整驱动示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件

#define DEVICE_NAME "mydevice"    // 设备名称
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;             //字符设备指针
int major;    //主设备号
int minor;    //次设备号

static int device_open(struct inode *inode, struct file *file)
{
    // 在这里处理设备打开的操作
    printk(KERN_INFO "This is device_open.\n");
    return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
    // 在这里处理设备关闭的操作
    printk(KERN_INFO "This is device_release.\n");

    return 0;
}

static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t *offset)
{
    // 在这里处理设备读取的操作
    printk(KERN_INFO "This is device_read.\n");
    return 0;
}

static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t
*offset)
{
    // 在这里处理设备写入的操作
    printk(KERN_INFO "This is device_write.\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
```

```
    .open = device_open,
    .release = device_release,
    .read = device_read,
    .write = device_write,
};

static int __init mydevice_init(void)
{
    int ret;

    // 在这里执行驱动程序的初始化操作

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
    }
        major=MAJOR(dev_num);     //获取主设备号
        minor=MINOR(dev_num);     //获取次设备号
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

        my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);     //初始化字符设备结构体
        cdev_add(&my_cdev,dev_num,1);     //将字符设备添加到内核中
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}

static void __exit mydevice_exit(void)
{
    // 在这里执行驱动程序的清理操作
    // 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

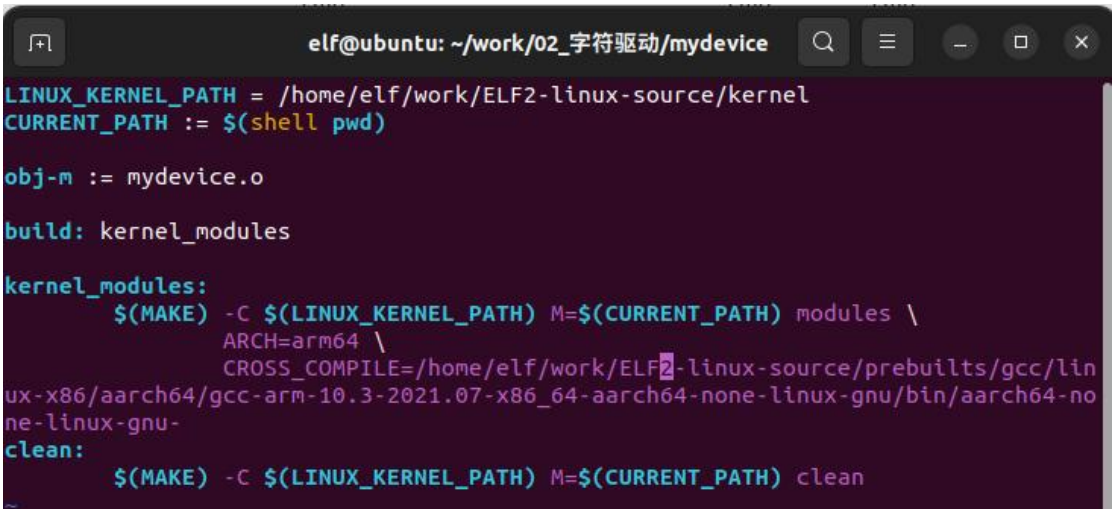
    printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);
```

```
MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3.编译

在 mydevice.c 目录下创建 Makefile 文件：



执行 make 编译，将生成的 mydevice.ko 文件拷贝到开发板中。

4.编写应用测试源码 device_app.c

驱动中已经给应用层提供了 open、read、write 的接口，接下来应用层就可以进行相应的系统调用。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <fcntl.h>

#define DEV_NAME "/dev/my_device"

int main(int argc, char *argv[])
{
   int reg;
   int fd = 0;
   int dat = 0;

  fd = open (DEV_NAME, O_RDWR);
  if (fd < 0) {
  perror("Open "DEV_NAME" Failed!\n");
  exit(1);
  }

  reg = read(fd, &dat, 1);
```

```
  if (reg<0) {
  perror("read "DEV_NAME" Failed!\n");
  exit(1);
}

  dat = 0;
  reg = write(fd, &dat, 1);
  if (reg<0) {
  perror("write "DEV_NAME" Failed!\n");
  exit(1);
  }
  close(fd);
  return 0;
  }
```

设置环境变量（需要根据实际交叉编译链的路径来确认）：

```
elf@ubuntu:~/work/testapp$export
PATH=/home/elf/work/ELF2-linux-source/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2
021.07-x86_64-aarch64-none-linux-gnu/bin/:$PATH
```

编译，将生成的 device_app 文件拷贝到开发板中：

```
elf@ubuntu:~/work/testapp$ aarch64-none-linux-gnu-gcc device_app.c -o device_app
```

5.测试

1）加载驱动

```
root@elf2-buildroot:~# insmod mydevice.ko
[12532.109794] major number: 234
[12532.109818] minor number: 0
[12532.109827] Device registered successfully.
```

2）可以看到生成的主设备号为 234，次设备号为 0。我们需要使用主设备号和次设备号手动创建/dev/下的设备节点。

```
root@elf2-buildroot:~# mknod /dev/my_device c 234 0
```

mknod 的基本语法如下

```
mknod <device_file> <file_type> <major> <minor>
```

<device_file>：指定要创建的设备文件节点的路径和名称。

<file_type>：指定设备文件的类型，可以是 b（块设备）或 c（字符设备）。

<major>：指定设备文件的主设备号。

<minor>：指定设备文件的次设备号。

3）运行测试应用程序

```
root@elf2-buildroot:~# ./device_app
[12696.061876] This is device_open.
[12696.061905] This is device_read.
[12696.061913] This is device_write.
[12696.061922] This is device_release.
```

4）卸载驱动

```
root@elf2-buildroot:~# rmmod mydevice.ko
[12766.952863] Device unregistered.
```

2.3.4 自动创建节点

测试示例章节写的驱动加载后，需要手动使用 mknod 命令在/dev 下创建设备节点，本节讲解如何让驱动自动在/dev 目录下生成设备节点。

想要实现自动创建节点，首先需要创建设备类，然后创建设备节点并关联到设备类。

1.在测试示例源码的基础上进行添加

1）添加头文件

```
#include <linux/device.h>
```

2）声明变量

```
static struct class *my_class;
static struct device *my_device;
```

3）在 mydevice_init(void)函数中创建类和设备节点

```
static int __init mydevice_init(void)
{
    int ret;

    // 在这里执行驱动程序的初始化操作

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
    }
     major=MAJOR(dev_num);     //获取主设备号
     minor=MINOR(dev_num);     //获取次设备号
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

        my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);     //初始化字符设备结构体
        cdev_add(&my_cdev,dev_num,1);     //将字符设备添加到内核中

      // 创建设备类
    my_class = class_create(THIS_MODULE, "my_class");
    if (IS_ERR(my_class)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(my_class);
   }
     // 创建设备节点并关联到设备类
```

```
        my_device = device_create(my_class, NULL, MKDEV(major, minor), NULL,
"my_device");
        if (IS_ERR(my_device)) {
            pr_err("Failed to create device\n");
            class_destroy(my_class);
            return PTR_ERR(my_device);
        }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
```

class_create()函数的原型：

```
struct class *class_create(struct module *owner, const char *name);
```

owner：指向拥有该设备类的内核模块的指针。通常使用 THIS_MODULE 来指定当前驱动模块作为拥有者。

name：指定设备类的名称。该名称将用于在 sysfs 文件系统中创建对应的类目录。

class_create()函数返回一个指向新创建的设备类的 struct class 指针。如果创建失败，它将返回一个指向 NULL 的指针

设备类的创建是设备驱动程序中重要的一步，它为设备提供了一个统一的命名空间和属性集合，并将相关设备节点组织在一起。创建设备类后，驱动程序可以使用 device_create()函数创建设备节点，并将其关联到该设备类下。

device_create()函数的原型：

```
struct device *device_create(struct class *class, struct device *parent,
                             dev_t dev, void *drvdata, const char *fmt, ...);
```

class：指向设备节点所属的设备类的 struct class 指针。

parent：指向设备节点的父设备的 struct device 指针，如果没有父设备，则可以设置为 NULL。

dev：指定设备节点的设备号，使用 MKDEV(major, minor)来创建。

drvdata：指向与设备节点相关的私有数据的指针，可以将驱动程序的特定数据与设备节点关联起来。

fmt：设备节点名称的格式字符串，用于在 sysfs 文件系统中创建设备节点。可以使用类似"mydevice%d"的格式，并使用可变参数来指定设备节点名称的后缀。

device_create()函数返回一个指向新创建的设备节点的 struct device 指针。如果创建失败，它将返回一个指向 NULL 的指针。

设备节点的创建需要先创建一个设备类（使用 class_create()函数），然后使用 device_create()函数创建设备节点并将其关联到设备类下。

4）mydevice_exit(void)函数中添加销毁设备节点和设备类

```
static void __exit mydevice_exit(void)
{
    // 在这里执行驱动程序的清理操作
    // 销毁设备节点
      device_destroy(my_class, MKDEV(major, minor));
    // 销毁设备类
     class_destroy(my_class);
     // 删除字符设备
     cdev_del(&my_cdev);
```

```
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

    printk(KERN_INFO "Device unregistered.\n");
}
```

2.完整驱动示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>
#define DEVICE_NAME "mydevice"   // 设备名称
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;            //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *my_class;
static struct device *my_device;

static int device_open(struct inode *inode, struct file *file)
{
    // 在这里处理设备打开的操作
    printk(KERN_INFO "This is device_open.\n");
    return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
    // 在这里处理设备关闭的操作
    printk(KERN_INFO "This is device_release.\n");

    return 0;
}

static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t
*offset)
{
    // 在这里处理设备读取的操作
    printk(KERN_INFO "This is device_read.\n");
    return 0;
}

static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t
```

```
*offset)
{
    // 在这里处理设备写入的操作
    printk(KERN_INFO "This is device_write.\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .read = device_read,
    .write = device_write,
};

static int __init mydevice_init(void)
{
    int ret;

    // 在这里执行驱动程序的初始化操作

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
    }
        major=MAJOR(dev_num);    //获取主设备号
        minor=MINOR(dev_num);    //获取次设备号
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

        my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);    //初始化字符设备结构体
        cdev_add(&my_cdev,dev_num,1);    //将字符设备添加到内核中

       // 创建设备类
    my_class = class_create(THIS_MODULE, "my_class");
    if (IS_ERR(my_class)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(my_class);
    }
      // 创建设备节点并关联到设备类
        my_device = device_create(my_class, NULL, MKDEV(major, minor), NULL,
```

```
"my_device");
        if (IS_ERR(my_device)) {
            pr_err("Failed to create device\n");
            class_destroy(my_class);
            return PTR_ERR(my_device);
        }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}

static void __exit mydevice_exit(void)
{
    // 在这里执行驱动程序的清理操作
    // 销毁设备节点
      device_destroy(my_class, MKDEV(major, minor));
    // 销毁设备类
     class_destroy(my_class);
    // 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

    printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);

MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3.编译测试

在 mydevice-auto.c 目录下创建 Makefile 文件：



执行 make 编译，将生成的 mydevice-auto.ko 文件拷贝到开发板中。

```
root@elf2-buildroot:/# insmod mydevice-auto.ko
[   366.138374] major number: 511
[   366.138395] minor number: 0
[   366.138638] Device registered successfully.
root@elf2-buildroot:#   ls /dev/my_device
/dev/my_device
root@elf2-buildroot:# ./device_app
[   427.877346] This is device_open.
[   427.877376] This is device_read.
[   427.877385] This is device_write.
[   427.877393] This is device_release.
```

可以看出在使用 insmod 加载驱动后，在/dev 下就生成了 my_device 节点，使用 device_app 测试正常。

2.4 内核空间与用户空间的数据拷贝

2.4.1 数据拷贝介绍

在 Linux 系统中，内核空间和用户空间是两个独立的地址空间，它们有不同的访问权限和内存保护机制。在内核空间和用户空间之间进行数据传输时，需要进行数据拷贝操作。

Linux 内核提供了几种方法来实现内核空间与用户空间之间的数据拷贝：

1.copy_to_user()和 copy_from_user()

这两个函数用于在内核空间和用户空间之间进行数据拷贝。

copy_to_user()函数用于将数据从内核空间复制到用户空间。

copy_from_user()函数用于将数据从用户空间复制到内核空间。

这两个函数的原型如下：

```
unsigned long copy_to_user(void __user *to, const void *from, unsigned long n);
unsigned long copy_from_user(void *to, const void __user *from, unsigned long n);
```

参数说明：

to：目标缓冲区的指针，用于存储数据。

from：源缓冲区的指针，数据的来源。

n：要拷贝的字节数。

2.get_user()和 put_user()

这两个函数用于在内核空间和用户空间之间进行单个数据的拷贝。

get_user()函数用于从用户空间获取单个数据到内核空间。

put_user()函数用于将单个数据从内核空间写入到用户空间。

这两个函数的原型如下：

```
int get_user(x, p);
int put_user(x, p);
```

参数说明：

x：数据的值。

p：指向用户空间的指针。

需要注意的是，在进行数据拷贝时，应确保源地址和目标地址的有效性，并进行适当的错误处理。此外，数据拷贝涉及到用户空间和内核空间的切换，可能会引发性能开销，因此在性能敏感的场景下，应谨慎使用数据拷贝操作。

2.4.2 获取内核空间数据

1.在自动创建节点章节源码的基础上进行添加

1）添加头文件

```
#include <linux/uaccess.h>
```

2）定义变量

```
#define DEVICE_NAME "copy_to_user"  // 设备名称
#define BUFFER_SIZE 1024
static char kernel_buffer[BUFFER_SIZE] = "Hello, User! This is kernel_buffer!";
```

3）在 device_read()函数添加拷贝操作，用户空间使用 read 函数时，会进入此函数。

```
static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t *offset)
{
        ssize_t bytes_read = 0;
// 在这里处理设备读取的操作
        printk(KERN_INFO "This is device_read.\n");

        if (*offset >= BUFFER_SIZE)
                return 0;
        if (*offset + length > BUFFER_SIZE)
                length = BUFFER_SIZE - *offset;
        if (copy_to_user(buffer, kernel_buffer + *offset, length))
                return -EFAULT;
        *offset += length;
        bytes_read = length;
        return bytes_read;
```

```
}
```

offset 参数表示当前文件位置的偏移量。它用于跟踪读取或写入操作在文件中的位置。

对于 device_read 函数来说，offset 指向一个 loff_t 类型的变量，表示当前读取操作的文件偏移量。在读取数据之前，可以通过检查 offset 来确定从文件的哪个位置开始读取数据。读取完成后，可以通过更新 offset 来指示下一次读取操作应该从哪个位置开始。

在使用 copy_to_user()函数将数据复制到用户空间之前，通常会更新 offset 的值，以确保下一次读取操作从正确的位置开始。这样可以实现逐步读取大文件或在多次读取中定位到不同位置的功能。

2.完整的示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>
#include <linux/uaccess.h>
#define DEVICE_NAME "copy_to_user"   // 设备名称
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *my_class;
static struct device *my_device;

#define BUFFER_SIZE 1024
static char kernel_buffer[BUFFER_SIZE] = "Hello, User! This is kernel_buffer!";


static int device_open(struct inode *inode, struct file *file)
{
// 在这里处理设备打开的操作
printk(KERN_INFO "This is device_open.\n");
      return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
// 在这里处理设备关闭的操作
printk(KERN_INFO "This is device_release.\n");
      return 0;
}
static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t
*offset)
{
            ssize_t bytes_read = 0;
// 在这里处理设备读取的操作
```

```
        printk(KERN_INFO "This is device_read.\n");

        if (*offset >= BUFFER_SIZE)
                return 0;
        if (*offset + length > BUFFER_SIZE)
                length = BUFFER_SIZE - *offset;
        if (copy_to_user(buffer, kernel_buffer + *offset, length))
                return -EFAULT;
        *offset += length;
        bytes_read = length;
        return bytes_read;


}

static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t
*offset)
{
// 在这里处理设备写入的操作
printk(KERN_INFO "This is device_write.\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .read = device_read,
    .write = device_write,
};
static int __init mydevice_init(void)
{
    int ret;

    // 在这里执行驱动程序的初始化操作

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
    }
   major=MAJOR(dev_num);
   minor=MINOR(dev_num);
```

```
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

    my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);
        cdev_add(&my_cdev,dev_num,1);

       // 创建设备类
    my_class = class_create(THIS_MODULE, "my_class");
    if (IS_ERR(my_class)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(my_class);
}
  // 创建设备节点并关联到设备类
    my_device = device_create(my_class, NULL, MKDEV(major, minor), NULL,
DEVICE_NAME);
    if (IS_ERR(my_device)) {
        pr_err("Failed to create device\n");
        class_destroy(my_class);
        return PTR_ERR(my_device);
    }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
static void __exit mydevice_exit(void)
{
// 在这里执行驱动程序的清理操作
// 销毁设备节点
 device_destroy(my_class, MKDEV(major, minor));
// 销毁设备类
 class_destroy(my_class);
// 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

    printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);

MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
```

```
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3. 将驱动模块编译成.ko 并传到开发板中

在 copy_to_user.c 目录下创建 Makefile 文件：



执行 make 编译，将生成的 copy_to_user.ko 拷贝到开发板中。

4.编写测试应用程序

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <fcntl.h>
#include <string.h>

#define DEV_NAME "/dev/copy_to_user"
#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
int reg;
int fd = 0;
char read_buffer[BUFFER_SIZE];

 fd = open (DEV_NAME, O_RDWR);
 if (fd < 0) {
 perror("Open "DEV_NAME" Failed!\n");
 exit(1);
 }

 if (read(fd, read_buffer, BUFFER_SIZE) < 0) {
            printf("Failed to read from device file: %s\n", DEV_NAME);
            close(fd);
```

```
        exit(1);
    }
    printf("Read from device file: %s\n", read_buffer);
    close(fd);
    return 0;
    }
```

将编译好的应用程序发送到开发板中。

5.测试

```
root@elf2-buildroot:~# insmod copy_to_user.ko
[    20.081508] major number: 234
[    20.081519] minor number: 0
[    20.081759] Device registered successfully.
root@elf2-buildroot:~# ./copy_to_user
[    79.725313] This is device_open.
[    79.725363] This is device_read.
Read from device file: Hello, User! This is kernel_buffer!
[    79.725457] This is device_release.
```

可以看到运行测试程序后，把内核空间中 kernel_buffer[]中的数据打印了出来。

2.4.3 获取用户空间数据

1.在获取内核空间数据章节源码的基础上添加 device_write 函数的实现即可

1）定义变量

```
#define DEVICE_NAME "copy_form_user"    // 设备名称
static char user_buffer[BUFFER_SIZE] = "";
```

2）在 device_write()函数添加拷贝操作，用户空间使用 write 函数时，会进入此函数。

```
static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t
*offset)
{
        ssize_t bytes_written = 0;
// 在这里处理设备写入的操作
        printk(KERN_INFO "This is device_write.\n");

        if (*offset >= BUFFER_SIZE)
                return -ENOSPC;
         if (*offset + length > BUFFER_SIZE)
                 length = BUFFER_SIZE - *offset;
         if (copy_from_user(user_buffer + *offset, buffer, length))
                 return -EFAULT;
         *offset += length;
         bytes_written = length;
         printk(KERN_INFO "Received from user: %s", user_buffer);
```

```
        return bytes_written;


}
```

offset 参数表示当前文件位置的偏移量。它用于跟踪读取或写入操作在文件中的位置。

对于 device_write()函数来说，offset 参数用于指示应写入数据的起始位置。根据需要，可以在函数内部修改该偏移量，以便在下一次写操作时从适当的位置开始写入。

在上述示例中，首先执行写操作，并使用 offset 参数作为起始位置。然后，通过将 length 添加到 offset 上来更新偏移量，以便在下一次写操作时从正确的位置开始。

2.完整的示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>
#include <linux/uaccess.h>
#define DEVICE_NAME "copy_form_user"   // 设备名称
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *my_class;
static struct device *my_device;

#define BUFFER_SIZE 1024
static char kernel_buffer[BUFFER_SIZE] = "Hello, User! This is kernel_buffer!";
static char user_buffer[BUFFER_SIZE] = "";

static int device_open(struct inode *inode, struct file *file)
{
// 在这里处理设备打开的操作
printk(KERN_INFO "This is device_open.\n");
      return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
// 在这里处理设备关闭的操作
printk(KERN_INFO "This is device_release.\n");
      return 0;
}
static ssize_t device_read(struct file *file, char __user *buffer, size_t length, loff_t
*offset)
{
            ssize_t bytes_read = 0;
```

```
// 在这里处理设备读取的操作
        printk(KERN_INFO "This is device_read.\n");

        if (*offset >= BUFFER_SIZE)
                return 0;
        if (*offset + length > BUFFER_SIZE)
                length = BUFFER_SIZE - *offset;
        if (copy_to_user(buffer, kernel_buffer + *offset, length))
                return -EFAULT;
        *offset += length;
        bytes_read = length;
        return bytes_read;


}

static ssize_t device_write(struct file *file, const char __user *buffer, size_t length, loff_t
*offset)
{
    ssize_t bytes_written = 0;

// 在这里处理设备写入的操作
        printk(KERN_INFO "This is device_write.\n");

        if (*offset >= BUFFER_SIZE)
                return -ENOSPC;
         if (*offset + length > BUFFER_SIZE)
                 length = BUFFER_SIZE - *offset;
         if (copy_from_user(user_buffer + *offset, buffer, length))
                 return -EFAULT;
         *offset += length;
         bytes_written = length;
         printk(KERN_INFO "Received from user: %s", user_buffer);

         return bytes_written;
}

static struct file_operations fops = {
      .owner = THIS_MODULE,
      .open = device_open,
      .release = device_release,
      .read = device_read,
      .write = device_write,
};
```

```
static int __init mydevice_init(void)
{
	int ret;

	// 在这里执行驱动程序的初始化操作

	// 注册字符设备驱动程序
	ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
	if (ret < 0) {
		printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
		return ret;
}
    major=MAJOR(dev_num);
    minor=MINOR(dev_num);
printk(KERN_INFO "major number: %d\n",major);
printk(KERN_INFO "minor number: %d\n",minor);

my_cdev.owner = THIS_MODULE;
		cdev_init(&my_cdev,&fops);
		cdev_add(&my_cdev,dev_num,1);

	    // 创建设备类
	my_class = class_create(THIS_MODULE, "my_class");
	if (IS_ERR(my_class)) {
		pr_err("Failed to create class\n");
		return PTR_ERR(my_class);
}
  // 创建设备节点并关联到设备类
	my_device = device_create(my_class, NULL, MKDEV(major, minor), NULL,
DEVICE_NAME);
	if (IS_ERR(my_device)) {
		pr_err("Failed to create device\n");
		class_destroy(my_class);
		return PTR_ERR(my_device);
	}
			printk(KERN_INFO "Device registered successfully.\n");
	return 0;
}
static void __exit mydevice_exit(void)
{
// 在这里执行驱动程序的清理操作
// 销毁设备节点
 device_destroy(my_class, MKDEV(major, minor));
// 销毁设备类
```

```
 class_destroy(my_class);
// 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

    printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);

MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3.将驱动模块编译成.ko 并传到开发板中。

在 copy_form_user.c 文件下创建 Makefile 文件：



执行 make 编译，将生成的 copy_form_user.ko 拷贝到开发板。

4.编写测试应用程序

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <fcntl.h>
#include <string.h>

#define DEV_NAME "/dev/copy_form_user"
#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
```

```
int reg;
int fd = 0;
char *write_buffer="Hello,kernel!This is user_buffer!";

 fd = open (DEV_NAME, O_RDWR);
 if (fd < 0) {
 perror("Open "DEV_NAME" Failed!\n");
 exit(1);
 }

if (write(fd, write_buffer, strlen(write_buffer)) < 0) {
            printf("Failed to write to device file: %s\n", DEV_NAME);
            close(fd);
            exit(1);
 }

 printf("Wrote to device file: %s\n", DEV_NAME);
 close(fd);
 return 0;
 }
```

5.将编译好的应用程序发送到开发板中。

6.测试

```
root@elf2-buildroot:~# insmod copy_form_user.ko
[   226.767110]major number: 248
[   226.767150]minor number: 0
[   226.767379]Device registered successfully.
root@elf2-buildroot:~# ./copy_form_user
[   256.555274]This is device_open.
[   256.555329]This is device_write.
Received from user: Hello,kernel!This is user_buffer!Wrote to device file: /dev/copy_f
orm_user
[   256.555562]This is device_release.
```

可以看到运行测试程序后，可以看到驱动里把用户空间传进去的字符串打印了出来。

2.5 Pinctrl 和 GPIO 子系统

2.5.1 Pinctrl 子系统

pinctrl（Pin Control）子系统是 Linux 内核中的一个模块化子系统，用于管理和控制硬件引脚（Pin）的配置和功能。它提供了一种统一的接口，使驱动程序可以对硬件引脚进行灵活的配置和控制。

pinctrl 子系统的主要目的是将引脚的配置和控制与设备驱动程序的实现分离开来，从而提高驱动程序的可移植性和可重用性。通过使用 pinctrl 子系统，设备驱动程序可以独立于特定的硬件平台，并在不同的平台上共享或重新使用。

pinctrl 子系统的核心是 pinctrl 核心层和 pinctrl 驱动程序两个部分：

pinctrl 核心层：提供了 pinctrl 子系统的核心功能，包括设备引脚的注册、配置和控制等。它负责管理

所有的 pinctrl 驱动程序和设备引脚配置数据，并为设备驱动程序提供统一的接口。

pinctrl 驱动程序：每个特定的硬件平台都需要相应的 pinctrl 驱动程序来管理和控制硬件引脚。这些驱动程序负责与硬件平台的引脚控制器交互，执行引脚的配置和控制操作。每个 pinctrl 驱动程序通常与特定的硬件平台或芯片系列相关联。

RK3588 的 pinctrl 子系统具有以下特点和功能：

1.引脚配置：pinctrl 子系统允许驱动程序对每个引脚进行灵活的配置。它提供了一组接口，允许驱动程序选择引脚的功能、复用模式和电气属性。通过配置 pinctrl，驱动程序可以将引脚用于不同的外设接口，例如 UART、SPI、I2C、GPIO 等。

2.引脚复用：RK3588 支持引脚的多功能复用，即一个引脚可以用于多种不同的功能。pinctrl 子系统允许驱动程序将引脚设置为特定功能的输入或输出，以适应不同的外设连接需求。

3.中断控制：对于支持中断的引脚，pinctrl 子系统提供了接口用于配置中断模式和触发方式。这使得 RK3588 能够与外部设备进行有效的中断信号传递和处理。

4.状态保存和恢复：pinctrl 子系统支持状态的保存和恢复，以便在系统休眠和唤醒过程中保持引脚配置的一致性。

RK3588 的引脚描述使用设备树来定义。设备树中定义了引脚的功能选项、复用模式、电气属性等信息。通过设备树，可以将引脚与相应的功能关联起来。也就是前面讲的 IOMUX。

以 i2c2 为例，在设备树中的体现形式如下：

（arch/arm64/boot/dts/rockchip/rk3588s-pinctrl.dtsi）

```
elf@ubuntu: ~/work/ELF2-linux-source/kernel

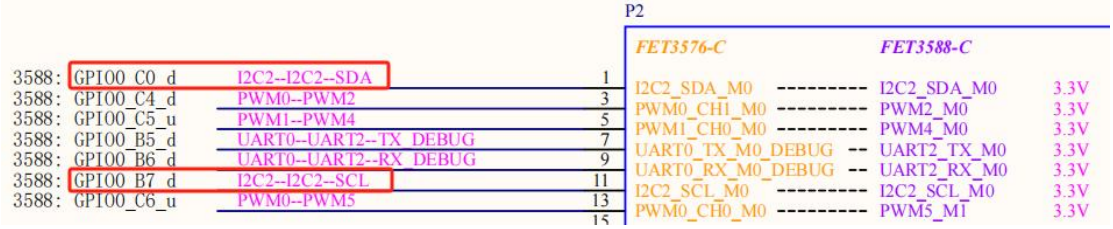
i2c2 {
        /omit-if-no-ref/
        i2c2m0_xfer: i2c2m0-xfer {
                rockchip,pins =
                        /* i2c2_scl_m0 */
                        <0 RK_PB7 9 &pcfg_pull_none_smt>,
                        /* i2c2_sda_m0 */
                        <0 RK_PC0 9 &pcfg_pull_none_smt>;
        };

        /omit-if-no-ref/
        i2c2m2_xfer: i2c2m2-xfer {
                rockchip,pins =
                        /* i2c2_scl_m2 */
                        <2 RK_PA3 9 &pcfg_pull_none_smt>,
                        /* i2c2_sda_m2 */
                        <2 RK_PA2 9 &pcfg_pull_none_smt>;
        };

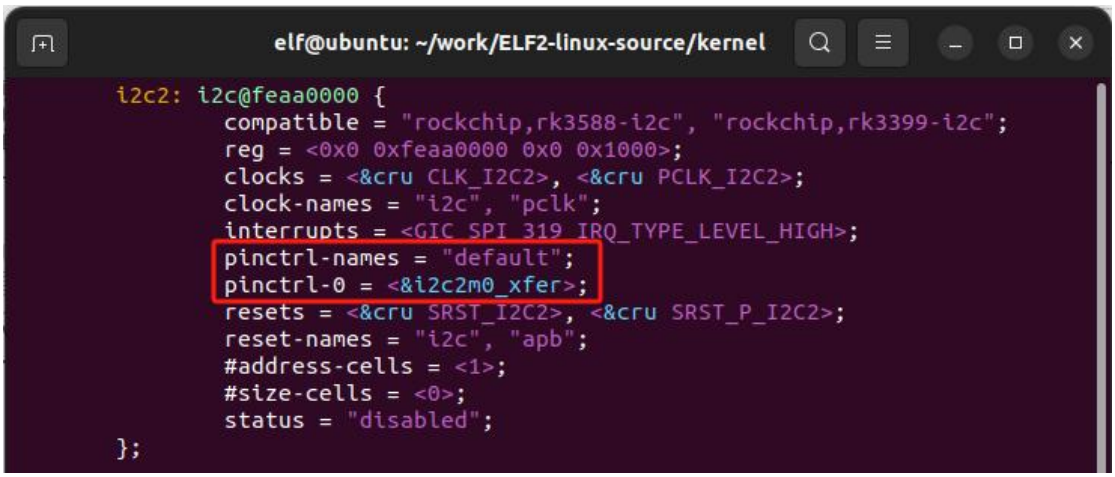
        /omit-if-no-ref/
        i2c2m3_xfer: i2c2m3-xfer {
                rockchip,pins =
                        /* i2c2_scl_m3 */
                        <1 RK_PC5 9 &pcfg_pull_none_smt>,
                        /* i2c2_sda_m3 */
                        <1 RK_PC4 9 &pcfg_pull_none_smt>;
        };

        /omit-if-no-ref/
        i2c2m4_xfer: i2c2m4-xfer {
                rockchip,pins =
                        /* i2c2_scl_m4 */
                        <1 RK_PA1 9 &pcfg_pull_none_smt>,
                        /* i2c2_sda_m4 */
                        <1 RK_PA0 9 &pcfg_pull_none_smt>;
        };
};
```

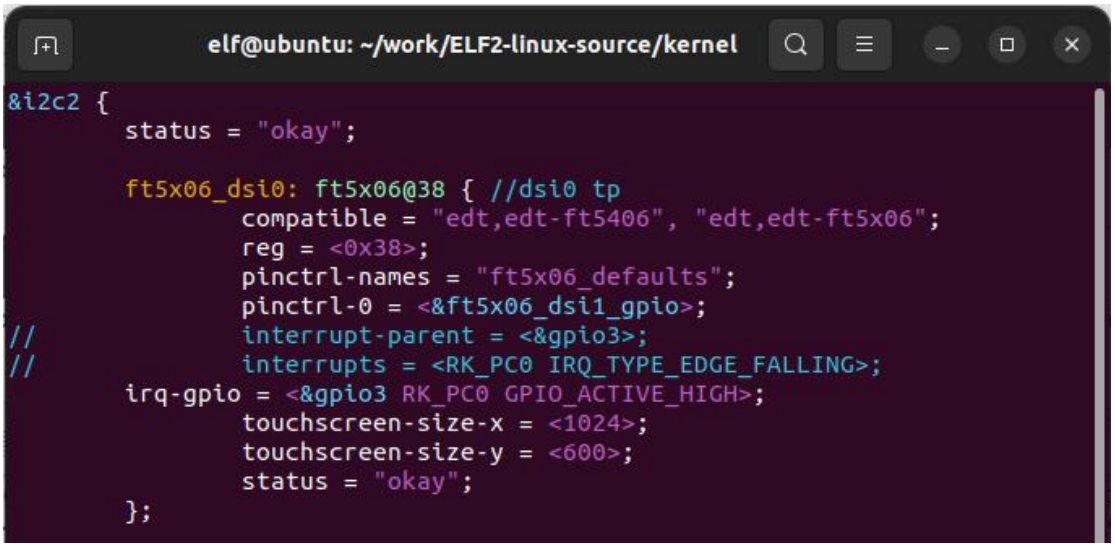
从上边的引脚定义可以看出，一共有 4 组引脚是可以复用成 i2c2 的功能的。我们需要根据硬件上实际使用的引脚来确定使用哪组。从 ELF2 的原理图中可以看出，使用的是 GPIO0_C0 和 GPIO0_B7，也就是 i2c2m0_xfer。



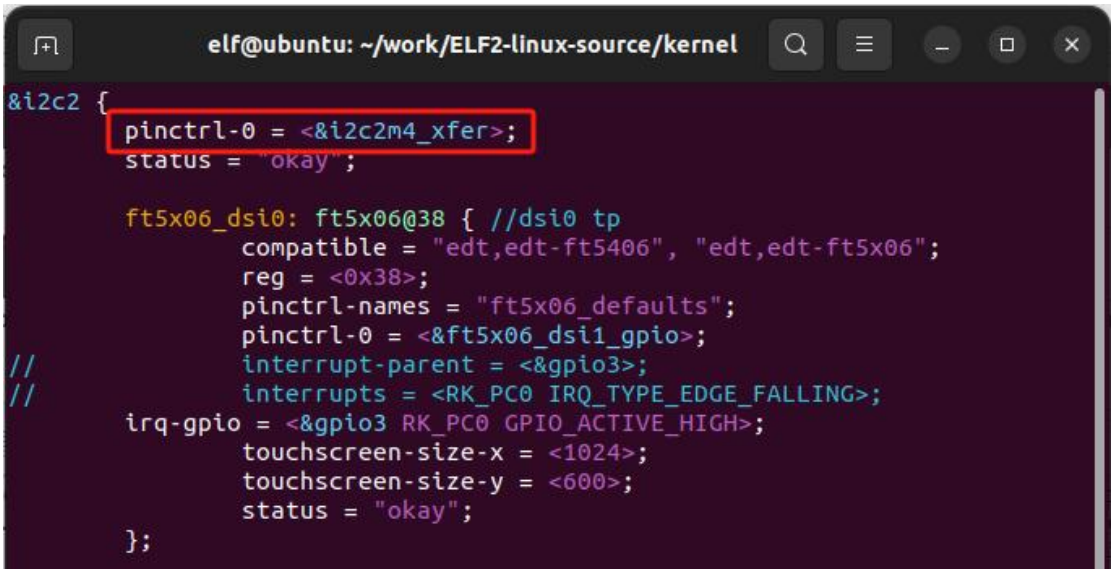
从 arch/arm64/boot/dts/rockchip/rk3588s.dtsi，能够看到默认使用的引脚就是 i2c2m0_xfer。



在 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 设备树文件中，有引用&i2c2 的节点，其中没有 pinctrl-0 的属性，说明使用的就是默认的属性。



如果硬件设计时使用的不是这一组引脚，比方说使用的是 GPIO1_A0 和 GPIO1_A1 引脚，那么在引用时就可以重新定义 pinctrl，来使用 i2c2m4_xfer，如下：



可以看出在 iomux 节点中定义了各个引脚的复用功能，需要注意的是一个引脚只能复用成一种功能，

所以在添加引脚复用时一定要检查，其它地方有没有用到选用的引脚，如果选用了要将其屏蔽掉，避免复用冲突。

2.5.2 GPIO 子系统

GPIO（General Purpose Input/Output）子系统是用于控制和管理通用输入/输出引脚的一部分。它提供了一种标准化的方式来访问和操作硬件平台上的 GPIO 引脚，使开发人员能够通过软件控制设备上的 GPIO 信号。

在 Linux 的 GPIO 子系统中，提供了一组 API 函数来管理和操作 GPIO 引脚。以下是一些常用的 GPIO 子系统 API 函数：

gpio_request()：请求一个 GPIO 引脚并进行相关的配置，使其可用于驱动程序的操作。

gpio_free()：释放之前请求的 GPIO 引脚，将其解除分配并释放资源。

gpio_direction_input()：将指定的 GPIO 引脚配置为输入模式。

gpio_direction_output()：将指定的 GPIO 引脚配置为输出模式。

gpio_get_value()：获取指定 GPIO 引脚的当前输入值。

gpio_set_value()：设置指定 GPIO 引脚的输出值。

gpio_cansleep()：检查指定的 GPIO 引脚是否支持睡眠（sleep）操作。

gpio_to_desc()：根据 GPIO 引脚的编号获取对应的 GPIO 描述符。

这些函数通常定义在<linux/gpio.h>头文件中，使用时需要包含该头文件。

2.5.3 ioctl 介绍

在 Linux 中，GPIO 引脚通常可以通过 ioctl 系统调用与相应的 GPIO 设备进行交互和控制。通过 ioctl 命令，可以向 GPIO 设备发送不同的命令，并传递相应的参数来实现对 GPIO 引脚的配置和操作。所以在操作 GPIO 之前有必要先介绍一下 ioctl 的使用方法。

具体而言，GPIO 设备驱动程序通常会实现字符设备接口，并在驱动程序中注册相应的 file_operations 结构体。其中，包含一个成员函数.unlocked_ioctl，用于处理 ioctl 系统调用。

在用户空间，通过打开 GPIO 设备文件并调用 ioctl 系统调用，可以向驱动程序发送不同的 ioctl 命令来控制 GPIO 引脚的配置和操作。驱动程序则根据接收到的 ioctl 命令和参数进行相应的处理，并执行相应的 GPIO 操作。

ioctl 命令的构成

ioctl 命令由三个部分组成：幻数（Magic number）、命令序号（Command number）和参数方向（Direction）。

1.幻数（Magic number）

幻数是一个 32 位的标识符，用于将 ioctl 命令与特定的设备或驱动程序关联起来。

幻数可以通过预定义的宏定义或自定义的方式来定义，以确保唯一性。

在定义幻数时，通常使用_IO、_IOW、_IOR 或_IOWR 宏来设置幻数的方向和大小。

2.命令序号（Command number）

命令序号是一个 8 位的整数，用于区分不同的 ioctl 命令。

每个命令序号通常与特定的设备功能或操作相关联。

3.参数方向（Direction）

参数方向指示了 ioctl 命令的数据传输方向，可以是输入、输出或输入输出。

参数方向使用_IO、_IOW、_IOR 或_IOWR 宏来表示。

常用的宏定义如下：

_IO(type, nr)：表示无参数的 ioctl 命令。

_IOW(type, nr, dtype)：表示带有写入数据的 ioctl 命令，参数为 dtype 类型。
_IOR(type, nr, dtype)：表示带有读取数据的 ioctl 命令，参数为 dtype 类型。
_IOWR(type, nr, dtype)：表示带有读写数据的 ioctl 命令，参数为 dtype 类型。
其中，type 为幻数，nr 为命令序号，dtype 为数据类型。
示例：

```
//选定幻数后，可以这样来进行定义：#define LED_IOC_MAGIC 'Z'
#define LED_IOC_MAGIC 'Z'

//控制 LED 无数据传输时
#define SET_LED_ON          _IO(LED_IOC_MAGIC, 1)
#define SET_LED_OFF     _IO(LED_IOC_MAGIC, 0)

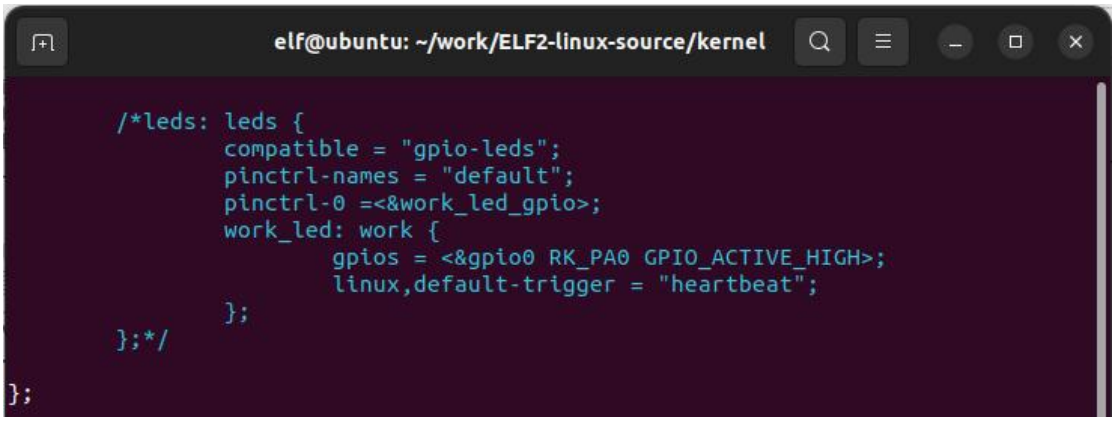
//驱动写入一个 int 型的数据
#define CHAR_WRITE_DATA   _IOW(CHAR_IOC_MAGIC, 2, int)

//驱动读取一个 int 型的数据
#define CHAR_READ_DATA    _IOR(CHAR_IOC_MAGIC, 3, int)
```

2.5.4 LED 驱动

前面章节的驱动，没有对实际的硬件进行操作，主要通过一些打印信了解了驱动的加载流程和调用流程。本章节开始，以实际操作硬件为例进行讲解，首先使用 GPIO 子系统 API 函数编写一个用户层能够控制核心板 LED 灯亮灭的驱动。

核心板上的 LED 灯使用的是 GPIO0_A0，默认已经配置成了呼吸灯，编写驱动前需要先将呼吸灯的功能取消掉。打开 arch/arm64/boot/dts/rockchip/elf2-3588.dtsi，将 leds 部分屏蔽掉。



保存退出后，重新单独编译内核，然后单独烧录，烧录完成后核心板上的蓝灯为长灭的状态。

1.驱动源码编写，新建一个 myled.c

1）头文件引用

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>          //包含设备节点相关的头文件
#include <linux/gpio.h>            //包含 gpio 操作函数的相关头文件
```

2）创建相关宏定义

```
#define DEVICE_NAME "mydevice"   // 设备名称
#define LED_IOC_MAGIC 'k'            //定义 ioctl 幻数
#define SET_LED_ON _IO(LED_IOC_MAGIC, 1)   //定义 LED 开命令
#define SET_LED_OFF _IO(LED_IOC_MAGIC, 0)   //定义 LED 关命令
#define GPIO_LED_PIN_NUM 0       //定义操作的 GPIO 编号
```

ioctl 幻数：

ioctl 幻数是一个用于区分 ioctl 命令的标识符。它是一个唯一的整数值，并且通常使用 ASCII 字符来表示。幻数的目的是确保 ioctl 命令的唯一性，以免不同设备的命令发生冲突。

_IO 宏定义：

_IO 宏定义用于创建无参数的 ioctl 命令代码。

_IO(LED_IOC_MAGIC, 0)，LED_IOC_MAGIC 表示定义的幻数，0 或者 1 表示具体的命令代码。

GPIO 编号：A 对应 1，B 对应 2，C 对应 3，D 对应 4

在 rk3588 上确定 GPIO 编号的公式为：GPIOn_xy=n*32+(x-1)*8+y

因为选择的引脚为 GPIO0_A0，所以通过 n*32+(x-1)*8+y 计算得到的编号为 0。

3）相关变量定义

```
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;                 //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *my_led;
static struct device *my_device;
```

4）mydevice_init(void)函数的实现

```
static int __init mydevice_init(void)
{
      int ret;
       //释放之前申请的 GPIO,避免申请失败
       gpio_free(GPIO_LED_PIN_NUM);
      // 申请 GPIO，并判断是否申请成功
            if (gpio_request(GPIO_LED_PIN_NUM, "led_run")) {
                        printk("request %s gpio faile \n", "led_run");
                         return -1;
             }
      // 注册字符设备驱动程序
      ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
      if (ret < 0) {
            printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
            return ret;
}
    major=MAJOR(dev_num);
    minor=MINOR(dev_num);
        printk(KERN_INFO "major number: %d\n",major);
```

```
        printk(KERN_INFO "minor number: %d\n",minor);

            my_cdev.owner = THIS_MODULE;
            cdev_init(&my_cdev,&fops);
            cdev_add(&my_cdev,dev_num,1);

           // 创建设备类
    my_led = class_create(THIS_MODULE, "my_led");
    if (IS_ERR(my_led)) {
            pr_err("Failed to create class\n");
            return PTR_ERR(my_led);
}
  // 创建设备节点并关联到设备类
    my_device = device_create(my_led, NULL, MKDEV(major, minor), NULL,
"my_device");
    if (IS_ERR(my_device)) {
            pr_err("Failed to create device\n");
            class_destroy(my_led);
            return PTR_ERR(my_device);
    }
                printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
```

主要添加了 gpio_free()和 gpio_request()函数的调用，gpio_free()函数原型如下：

```
void gpio_free(unsigned int gpio);
```

参数说明：

gpio：要释放的 GPIO 引脚的编号。

返回值：

无返回值。

函数功能：

gpio_free()函数释放先前请求的 GPIO 引脚，并将其解除分配。释放后的 GPIO 引脚可以被其他驱动程序或设备重新请求使用。

gpio_request()函数原型如下：

```
int gpio_request(unsigned int gpio, const char *label);
```

参数说明：

gpio：要请求的 GPIO 引脚的编号。

label：用于标识该 GPIO 引脚的描述字符串。

返回值：

成功时，返回 0 表示请求成功。

失败时，返回负数错误代码，表示请求失败。

函数功能：

gpio_request()函数请求一个 GPIO 引脚并进行相关的配置，以便将其用于后续的操作。

它会检查 GPIO 引脚是否已经被其他驱动程序或内核使用，如果被占用则请求失败。
请求成功后，该 GPIO 引脚将被锁定，使其他驱动程序无法再次请求该引脚。

5）mydevice_exit(void)函数的实现

```
static void __exit mydevice_exit(void)
{
// 在这里执行驱动程序的清理操作
//释放申请的 GPIO 资源
gpio_free(GPIO_LED_PIN_NUM);
// 销毁设备节点
 device_destroy(my_led, MKDEV(major, minor));
// 销毁设备类
 class_destroy(my_led);
// 删除字符设备
 cdev_del(&my_cdev);
// 注销字符设备驱动程序
 unregister_chrdev(0, DEVICE_NAME);

    printk(KERN_INFO "Device unregistered.\n");
}
```

6）定义 fops 结构体

```
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .unlocked_ioctl = myled_ioctl,
};
```

此处用到了 unlocked_ioctl 函数，unlocked_ioctl 函数是 Linux 内核中用于设备控制和通信的重要函数之一。它允许用户空间程序与设备驱动程序进行交互，发送特定的命令和参数来执行设备相关的操作。

unlocked_ioctl（）函数原型如下：

```
long unlocked_ioctl(struct file *filp, unsigned int cmd, unsigned long arg);
```

参数说明：

filp：指向 struct file 结构的指针，表示要执行 ioctl 操作的文件。

cmd：请求代码，用于指定要执行的具体操作。请求代码通常是一个整数，可以是预定义的常量或自定义的命令。

arg：参数，根据具体的请求代码而定。

返回值：

unlocked_ioctl 函数返回一个 long 类型的值，通常用于表示操作的成功与否，返回值为负数表示出现错误。

7）device_open（）函数的实现：

```
static int device_open(struct inode *inode, struct file *file)
{
    // 在这里处理设备打开的操作
```

```
    //设置 GPIO 引脚为输出模式，并将其初始化为高电平
    gpio_direction_output(GPIO_LED_PIN_NUM, 1);
    printk(KERN_INFO "This is device_open.\n");
    return 0;
}
```

gpio_direction_output()函数原型如下：

```
int gpio_direction_output(unsigned int gpio, int value);
```

参数说明：

gpio：要配置的 GPIO 引脚的编号。

value：设置 GPIO 引脚的初始输出值，0 表示低电平，非零表示高电平。

返回值：

成功时，返回 0 表示设置成功。

失败时，返回负数错误代码，表示设置失败。

函数功能：

gpio_direction_output()函数将指定的 GPIO 引脚配置为输出模式，并设置其初始输出值。

在配置为输出模式后，可以使用其他函数（如 gpio_set_value()）来更改 GPIO 引脚的输出值。

8）myled_ioctl（）函数的实现

```
static long myled_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{

    switch (cmd) {
            case SET_LED_ON:
            // 设置 GPIO 引脚为高电平
            gpio_set_value(GPIO_LED_PIN_NUM, 1);
            break;

        case SET_LED_OFF:
          //设置 GPIO 引脚为低电平
          gpio_set_value(GPIO_LED_PIN_NUM, 0);
            break;

        default:
            return -ENOTTY;
    }

    return 0;
}
```

gpio_set_value()函数原型如下：

```
void gpio_set_value(unsigned int gpio, int value);
```

参数说明：

gpio：要设置的 GPIO 引脚的编号。

value：要设置的输出值，0 表示低电平，非零表示高电平。

返回值：

无返回值。

函数功能：

gpio_set_value()函数用于设置指定 GPIO 引脚的输出值。该函数会将 GPIO 引脚配置为输出模式（如果尚未配置），并设置其输出值为指定的值。

9）device_release（）函数的实现：

```
static int device_release(struct inode *inode, struct file *file)
{
// 在这里处理设备关闭的操作
//设置 GPIO 引脚为输出模式，并将置为高电平
gpio_direction_output(GPIO_LED_PIN_NUM, 1);
printk(KERN_INFO "This is device_release.\n");

    return 0;
}
```

2.完整驱动示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>
#include <linux/gpio.h>

#define DEVICE_NAME "mydevice"   // 设备名称
#define LED_IOC_MAGIC 'k'
#define SET_LED_ON _IO(LED_IOC_MAGIC, 1)
#define SET_LED_OFF _IO(LED_IOC_MAGIC, 0)
#define GPIO_LED_PIN_NUM 0
static dev_t dev_num;     //分配的设备号
struct    cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *my_led;
static struct device *my_device;

static int device_open(struct inode *inode, struct file *file)
{
// 在这里处理设备打开的操作
//设置 GPIO 引脚为输出模式，并将其初始化为低电平
gpio_direction_output(GPIO_LED_PIN_NUM, 0);
printk(KERN_INFO "This is device_open.\n");
    return 0;
}
```

```
static int device_release(struct inode *inode, struct file *file)
{
// 在这里处理设备关闭的操作
gpio_direction_output(GPIO_LED_PIN_NUM, 0);
printk(KERN_INFO "This is device_release.\n");

    return 0;
}

static long myled_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{

    switch (cmd) {
            case SET_LED_ON:
            // 设置 GPIO 引脚为高电平
            gpio_set_value(GPIO_LED_PIN_NUM, 1);
            break;

        case SET_LED_OFF:
          //设置 GPIO 引脚为低电平
          gpio_set_value(GPIO_LED_PIN_NUM, 0);
            break;

        default:
            return -ENOTTY;
    }

    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .unlocked_ioctl = myled_ioctl,
};
static int __init mydevice_init(void)
{
    int ret;

   // 在这里执行驱动程序的初始化操作
   //释放之前申请的 GPIO,避免申请失败
        gpio_free(GPIO_LED_PIN_NUM);
```

```
        if (gpio_request(GPIO_LED_PIN_NUM, "led_run")) {
                printk("request %s gpio faile \n", "led_run");
                 return -1;
         }
    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
}
   major=MAJOR(dev_num);
   minor=MINOR(dev_num);
      printk(KERN_INFO "major number: %d\n",major);
      printk(KERN_INFO "minor number: %d\n",minor);

        my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);
        cdev_add(&my_cdev,dev_num,1);

       // 创建设备类
    my_led = class_create(THIS_MODULE, "my_led");
    if (IS_ERR(my_led)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(my_led);
}
  // 创建设备节点并关联到设备类
    my_device = device_create(my_led, NULL, MKDEV(major, minor), NULL,
"my_device");
    if (IS_ERR(my_device)) {
        pr_err("Failed to create device\n");
        class_destroy(my_led);
        return PTR_ERR(my_device);
    }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
static void __exit mydevice_exit(void)
{
// 在这里执行驱动程序的清理操作
//释放申请的 GPIO 资源
gpio_free(GPIO_LED_PIN_NUM);
// 销毁设备节点
 device_destroy(my_led, MKDEV(major, minor));
// 销毁设备类
```

```
 class_destroy(my_led);
// 删除字符设备
    cdev_del(&my_cdev);
    // 注销字符设备驱动程序
    unregister_chrdev(0, DEVICE_NAME);

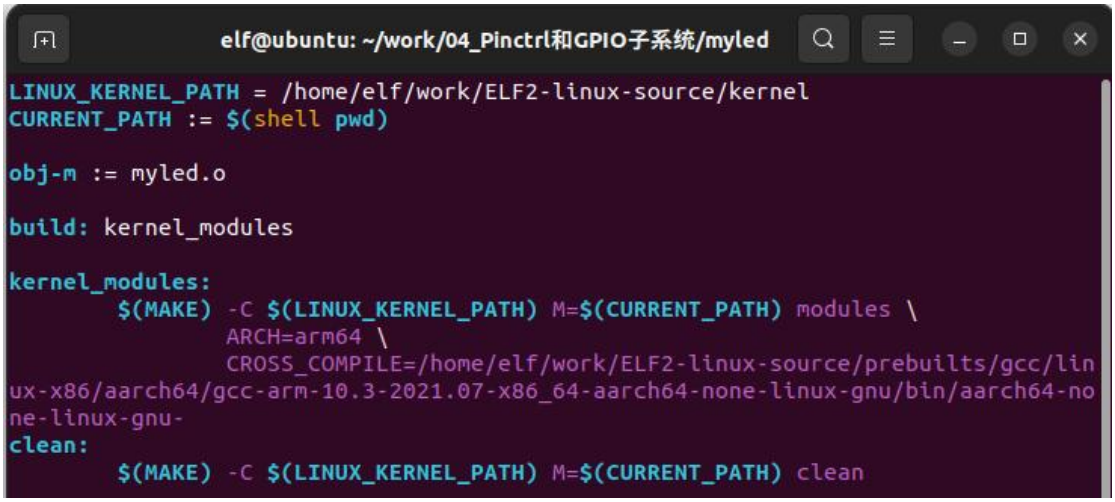
    printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);

MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3.编译成 ko 模块并发送到开发板中。

在 myled.c 目录下创建 Makefile 文件：



执行 make 编译，将生成的 myled.ko 拷贝到开发板中。

4.编写应用测试源码 led_app.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <fcntl.h>

#define DEV_NAME "/dev/my_device"
#define LED_IOC_MAGIC 'k'
#define SET_LED_ON _IO(LED_IOC_MAGIC, 1)
#define SET_LED_OFF _IO(LED_IOC_MAGIC, 0)

int main(int argc, char *argv[])
```

```
{
        int i;
        int fd = 0;
        fd = open (DEV_NAME, O_RDONLY);
        if (fd < 0) {
                perror("Open "DEV_NAME" Failed!\n");
         exit(1);
        }

        for (i=0; i<5; i++)
        {
                ioctl(fd, SET_LED_ON);
                sleep(1);
                ioctl(fd, SET_LED_OFF);
                sleep(1);
        }

        close(fd);
        return 0;
}
```

6.编译测试源码并拷贝到开发板中。

7.测试。

```
root@elf2-buildroot:~# insmod myled.ko
[ 3304.975488] major number: 234
[ 3304.975497] minor number: 0
[ 3304.975738] Device registered successfully.
root@elf2-buildroot:~# ./led_app
[ 3310.389439] This is device_open.
[ 3320.391119] This is device_release.
```

执行 ledapp 后，核心板 LED 灯循环闪烁 5 次。

2.6 Linux 系统中的中断

在 Linux 系统中，中断用于处理硬件设备的异步事件，如外部设备的输入、定时器的触发等。Linux 内核提供了中断处理程序的机制，允许驱动程序捕获和处理中断事件。

2.6.1 Linux 中断介绍

以下是在 Linux 系统中使用中断的一般步骤：

1.中断请求（IRQ Request）

驱动程序使用 request_irq()函数向内核请求分配中断资源，并注册中断处理程序。该函数的原型如下：

```
int request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
                const char *name, void *dev);
```

参数说明：

irq：中断请求号（IRQ），对应于硬件设备的中断信号。
handler：中断处理程序的函数指针，用于处理中断事件。
flags：中断处理程序的属性和配置标志。
name：中断处理程序的名称，用于标识中断请求。
dev：传递给中断处理程序的设备数据指针。

2.中断处理程序（Interrupt Handler）

中断处理程序是一个函数，用于处理中断事件。
中断处理程序需要满足特定的格式和要求，以便能够正确地响应中断事件。
在中断处理程序中，可以执行相应的操作，如读取设备数据、处理中断状态等。

3.中断释放（IRQ Free）

在不再需要中断资源时，驱动程序应调用 free_irq()函数释放中断资源。该函数的原型如下：

```
void free_irq(unsigned int irq, void *dev);
```

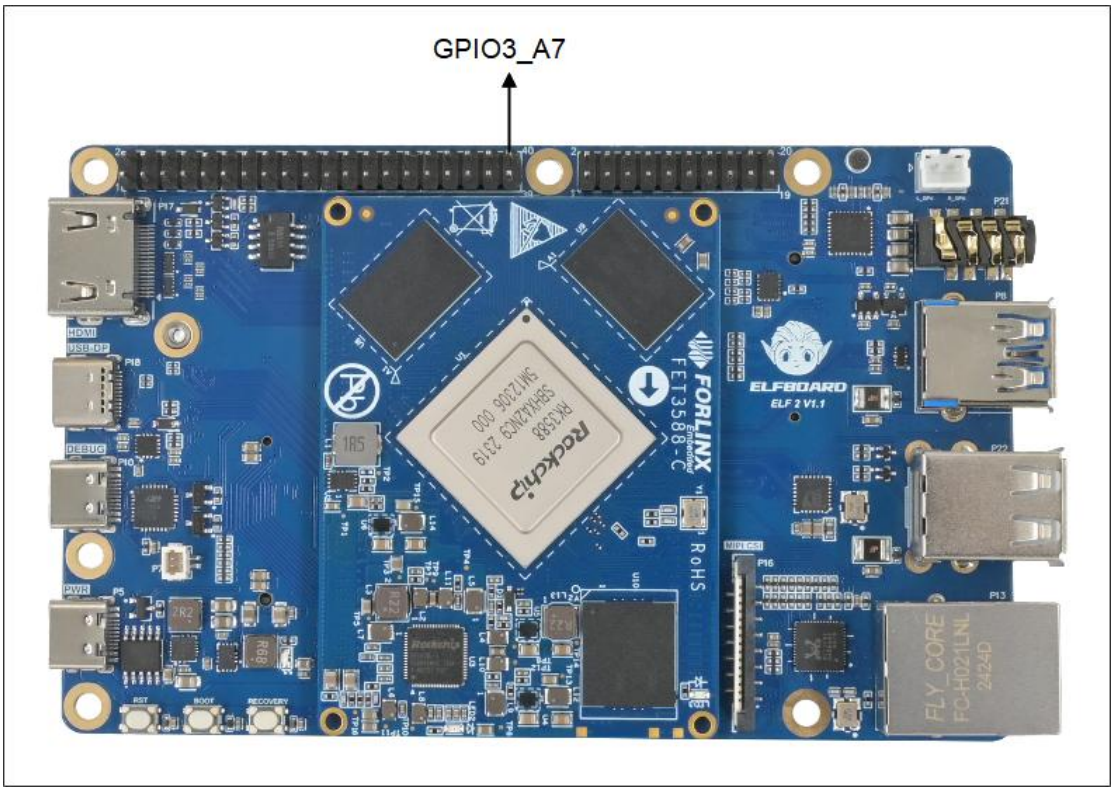
参数说明：

irq：要释放的中断请求号（IRQ）。
dev：与中断处理程序相关的设备数据指针。

2.6.2 按键中断驱动

上一节 LED 驱动中，使用了 GPIO 子系统的 API 函数将引脚配置为输出来控制 LED 的亮灭，本节讲解将引脚配置为输入，来获取按键状态。并且还使用到了中断的概念。

接下来编写一个按键的驱动，因为开发板上没有 GPIO 控制的物理按键，所以在 P26 的排针上选择一个 GPIO 来做测试。咱们这里选择 P26 的 40 脚，也就是 GPIO3_A7。



1.驱动源码编写，新建一个 myirq_key.c

1）头文件引用

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>          //包含设备节点相关的头文件
#include <linux/gpio.h>            //包含 gpio 操作函数的相关头文件
#include <linux/interrupt.h>       //包含中断函数相关头文件
```

2）创建相关宏定义和变量

```
#define DEVICE_NAME "button_irq"   // 设备名称
#define GPIO_KEY_PIN_NUM 103       //定义操作的 GPIO 编号
#define BUTTON_IRQ   gpio_to_irq(GPIO_KEY_PIN_NUM)//GPIO 引脚中断号

static dev_t dev_num;     //分配的设备号
struct   cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *button_irq;
static struct device *my_device;
```

GPIO 编号：A 对应 1，B 对应 2，C 对应 3，D 对应 4

在 rk3588 上确定 GPIO 编号的公式为：GPIOn_xy=n*32+(x-1)*8+y

因为选择的引脚为 GPIO3_A7，所以通过 n*32+(x-1)*8+y 计算得到的编号为 103。

gpio_to_irq()函数用于将 GPIO 引脚编号转换为对应的中断号。函数原型如下：

```
int gpio_to_irq(unsigned int gpio);
```

参数 gpio 是要转换的 GPIO 引脚号。该函数返回与给定 GPIO 引脚相关联的中断号。如果转换失败或引脚没有关联的中断，函数将返回一个负值。

3）mydevice_init(void)函数的实现

```
static int __init mydevice_init(void)
{
    int ret;

    gpio_free(GPIO_KEY_PIN_NUM);
    // 在这里执行驱动程序的初始化操作
    if (gpio_request(GPIO_KEY_PIN_NUM, "button_irq")) {
        printk("request %s gpio faile \n", "button_irq");
        return -1;
    }
    //将引脚设置为输入模式
    gpio_direction_input(GPIO_KEY_PIN_NUM);

    // 注册中断处理函数
    ret = request_irq(BUTTON_IRQ, button_interrupt_handler,
IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING, "button_irq", NULL);
```

```
    if (ret < 0) {
        printk(KERN_ALERT "Failed to register interrupt handler\n");
        gpio_free(GPIO_KEY_PIN_NUM);
        return ret;
    }

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
}
   major=MAJOR(dev_num);
   minor=MINOR(dev_num);
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

    my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);
        cdev_add(&my_cdev,dev_num,1);
       // 创建设备类
    button_irq = class_create(THIS_MODULE, "button_irq");
    if (IS_ERR(button_irq)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(button_irq);
}
  // 创建设备节点并关联到设备类
    my_device = device_create(button_irq, NULL, MKDEV(major, minor), NULL,
DEVICE_NAME);
    if (IS_ERR(my_device)) {
        pr_err("Failed to create device\n");
        class_destroy(button_irq);
        return PTR_ERR(my_device);
    }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
```

与前面 LED 驱动的区别主要是使用 gpio_direction_input 函数将引脚配置为了输入模式，使用 request_irq 函数申请了中断，触发方式为：IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING。表示上升沿和下降沿都会触发中断。

4）中断服务函数 irqreturn_t button_interrupt_handler()实现

```
//中断服务函数
static irqreturn_t button_interrupt_handler(int irq, void *dev_id)
{
```

```
    // 在此处执行按键中断处理代码

    // 检查按键状态
    int button_state = gpio_get_value(GPIO_KEY_PIN_NUM);
    if (button_state) {
        printk(KERN_INFO "Button released\n");
    } else {
        printk(KERN_INFO "Button pressed\n");
    }

    return IRQ_HANDLED;
}
```

使用 gpio_get_value()函数获取 gpio 引脚的电平状态。函数原型如下：

```
int gpio_get_value(unsigned int gpio);
```

参数 gpio 是要获取值的 GPIO 引脚号。该函数返回 GPIO 引脚的当前值，如果引脚处于高电平状态，则返回 1；如果引脚处于低电平状态，则返回 0。如果读取 GPIO 值失败，函数将返回一个负值。通过判断电平引脚的电平状态，来输出对应的打印信息。

2.完整驱动示例源码

```
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/uaccess.h>         // 包含用户空间数据访问函数的头文件
#include <linux/cdev.h>            //包含字符设备头文件
#include <linux/device.h>          //包含设备节点相关的头文件
#include <linux/gpio.h>            //包含 gpio 子系统相关函数头文件
#include <linux/interrupt.h>       //包含中断函数相关头文件

#define DEVICE_NAME "button_irq"   // 设备名称
#define GPIO_KEY_PIN_NUM 103
#define BUTTON_IRQ   gpio_to_irq(GPIO_KEY_PIN_NUM)//GPIO 引脚中断号

static dev_t dev_num;     //分配的设备号
struct   cdev my_cdev;              //字符设备指针
int major;   //主设备号
int minor;   //次设备号
static struct class *button_irq;
static struct device *my_device;


//中断服务函数
static irqreturn_t button_interrupt_handler(int irq, void *dev_id)
{
    // 在此处执行按键中断处理代码

    // 检查按键状态
```

```
    int button_state = gpio_get_value(GPIO_KEY_PIN_NUM);
    if (button_state) {
        printk(KERN_INFO "Button released\n");
    } else {
        printk(KERN_INFO "Button pressed\n");
    }

    return IRQ_HANDLED;
}

static int device_open(struct inode *inode, struct file *file)
{
// 在这里处理设备打开的操作
printk(KERN_INFO "This is device_open.\n");
    return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
// 在这里处理设备关闭的操作
printk(KERN_INFO "This is device_release.\n");

    return 0;
}



static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
};

static int __init mydevice_init(void)
{
    int ret;
    //申请 GPIO 前先释放资源
    gpio_free(GPIO_KEY_PIN_NUM);
    // 在这里执行驱动程序的初始化操作
    if (gpio_request(GPIO_KEY_PIN_NUM, "button_irq")) {
        printk("request %s gpio faile \n", "button_irq");
          return -1;
     }
     //将引脚设置为输入模式
```

```
        gpio_direction_input(GPIO_KEY_PIN_NUM);


        // 注册中断处理函数
    ret          =          request_irq(BUTTON_IRQ,          button_interrupt_handler,
IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING, "button_irq", NULL);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to register interrupt handler\n");
        gpio_free(GPIO_KEY_PIN_NUM);
        return ret;
    }

    // 注册字符设备驱动程序
    ret = alloc_chrdev_region(&dev_num,0,1,DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ALERT "Failed to allocate device number: %d\n", ret);
        return ret;
}
   major=MAJOR(dev_num);
   minor=MINOR(dev_num);
    printk(KERN_INFO "major number: %d\n",major);
    printk(KERN_INFO "minor number: %d\n",minor);

    my_cdev.owner = THIS_MODULE;
        cdev_init(&my_cdev,&fops);
        cdev_add(&my_cdev,dev_num,1);
       // 创建设备类
    button_irq = class_create(THIS_MODULE, "button_irq");
    if (IS_ERR(button_irq)) {
        pr_err("Failed to create class\n");
        return PTR_ERR(button_irq);
}
  // 创建设备节点并关联到设备类
    my_device  =  device_create(button_irq,  NULL,  MKDEV(major,  minor),  NULL,
DEVICE_NAME);
    if (IS_ERR(my_device)) {
        pr_err("Failed to create device\n");
        class_destroy(button_irq);
        return PTR_ERR(my_device);
    }
            printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}
```

```
static void __exit mydevice_exit(void)
{
// 释放中断
   free_irq(BUTTON_IRQ, NULL);
// 在这里执行驱动程序的清理操作
 gpio_free(GPIO_KEY_PIN_NUM);
// 销毁设备节点
 device_destroy(button_irq, MKDEV(major, minor));
// 销毁设备类
 class_destroy(button_irq);
// 删除字符设备
 cdev_del(&my_cdev);
     // 注销字符设备驱动程序
     unregister_chrdev(0, DEVICE_NAME);
     printk(KERN_INFO "Device unregistered.\n");
}

module_init(mydevice_init);
module_exit(mydevice_exit);

MODULE_LICENSE("GPL");          // 指定模块的许可证信息
MODULE_AUTHOR("Your Name"); // 指定模块的作者信息
MODULE_DESCRIPTION("A simple character device driver"); // 指定模块的描述信息
```

3.编译模块驱动并发送到开发板中。

在 myirq_key.c 目录下创建 Makefile 文件：



执行 make 编译，将生成的 myirq_key.ko 文件拷贝到开发板中。

4.测试。

```
root@elf2-buildroot:~# insmod myirq_key.ko
[   494.231695] major number: 511
[   494.231741] minor number: 0
[   494.232097] Device registered successfully.
```

```
root@elf2-buildroot:/# Button pressed
[   565.714322] Button pressed
[   565.729380] Button released
[   565.729546] Button released
[   565.729580] Button released
[   565.729660] Button pressed
[   565.730933] Button released
[   565.731243] Button pressed
[   565.736169] Button released
[   565.736197] Button pressed
```

加载驱动后，将 P26 的 40 脚使用杜邦线接地，然后抬起，就能看到 Button pressed 和 Button released 的打印信息。由于没有做消抖处理，终端可能一直会打印 Button released 和 Button pressed，这里只是验证驱动中申请和使用中断的情况，所以不做优化。

2.7 Platform 总线

2.7.1 platform 总线简介

平台总线（Platform Bus）是 Linux 内核中的一个基础架构，用于支持硬件平台上的设备驱动程序的开发和管理。它提供了一种统一的方式来表示和操作与特定平台相关的设备。

平台总线的设计目标是将特定平台的设备抽象为一个个的平台设备（Platform Device），并提供与设备相关的配置和控制接口。这些平台设备通过设备树（Device Tree）描述在内核中进行注册和管理。

平台总线的特点包括：

设备树描述：平台总线使用设备树来描述和配置平台上的设备。设备树是一种硬件描述语言，用于描述硬件平台的设备和资源分配情况。通过设备树，可以定义平台设备的特性、功能和资源，以及它们与其他设备之间的连接关系。

平台设备注册：通过设备树描述，平台设备可以在内核启动期间进行自动注册。内核会根据设备树中的描述信息，创建对应的平台设备对象，并将其注册到平台总线中。

驱动程序匹配：平台总线会根据设备树中的描述信息，自动匹配合适的设备驱动程序来与平台设备进行绑定。内核会遍历已加载的驱动程序，查找与平台设备兼容的驱动程序，并将其与设备进行绑定。

设备配置和控制：平台总线提供了一组统一的接口和方法，使驱动程序能够配置和控制平台设备。通过这些接口，驱动程序可以访问设备寄存器、设置设备的属性和参数，以及响应设备的中断和事件。

2.7.2 platform 总线驱动简单示例

1.示例源码

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/of.h>

static int my_platform_probe(struct platform_device *pdev)
{
    printk(KERN_INFO "my_platform_probe: Platform device probed\n");
    return 0;
}
```

```
static int my_platform_remove(struct platform_device *pdev)
{
	printk(KERN_INFO "my_platform_remove: Platform device removed\n");
	return 0;
}

static const struct of_device_id of_platform_match[] = {
		{ .compatible = "platform", },
		{},
};
MODULE_DEVICE_TABLE(of, of_platform_match);

static struct platform_driver my_platform_driver = {
	.driver = {
		.name = "my_platform_driver",
		.owner = THIS_MODULE,
		.of_match_table = of_platform_match,
	},
	.probe = my_platform_probe,
	.remove = my_platform_remove,
};

module_platform_driver(my_platform_driver);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("Platform Driver Example");
```

上述示例定义了一个名为 "my_platform_driver" 的平台驱动程序。驱动程序中的 my_platform_probe() 函数是探测函数，my_platform_remove() 函数是移除函数。还定义了一个 platform_driver 类型的结构体，结构体定义如下：

```
struct platform_driver {
	int (*probe)(struct platform_device *);
	int (*remove)(struct platform_device *);
	void (*shutdown)(struct platform_device *);
	int (*suspend)(struct platform_device *, pm_message_t);
	int (*resume)(struct platform_device *);
	struct device_driver driver;
	const struct platform_device_id *id_table;
	bool prevent_deferred_probe;
};
```

定义说明：

probe：驱动探测函数指针，当设备与驱动匹配时调用。它接收一个指向 structplatform_device 的指针作为参数，返回一个整数类型的值。

remove：驱动移除函数指针，当设备被移除时调用。它接收一个指向 structplatform_device 的指针作为参数，返回一个整数类型的值。

shutdown：关机回调函数指针，当系统关机时调用。它接收一个指向 struct platform_device 的指针作为参数，无返回值。

suspend：挂起回调函数指针，当系统进入挂起状态时调用。它接收一个指向 struct platform_device 的指针和 pm_message_t 类型的参数作为输入，返回一个整数类型的值。

resume：恢复回调函数指针，当系统从挂起状态恢复时调用。它接收一个指向 struct platform_device 的指针作为参数，返回一个整数类型的值。

driver：包含了驱动的名称、拥有者等信息的 struct device_driver 结构体。

id_table：一个指向平台设备 ID 表的指针，用于与设备匹配。它可以为空，或者指向一个以 NULL 结尾的数组。

prevent_deferred_probe：一个布尔值，用于指示是否禁止推迟的探测。如果设置为 true，则在设备匹配时不会推迟探测过程。

通过使用 module_platform_driver 宏，我们可以将驱动程序的注册和注销过程简化为一行代码。宏会自动为驱动程序注册和注销函数，并处理必要的初始化和清理工作。

需要注意的是，驱动程序中的 probe 和 remove 回调函数必须与平台驱动程序的结构体字段对应，以正确地处理设备的探测和移除。

of_platform_match[]数组中定义了设备兼容性匹配项，当内核在设备初始化过程中加载驱动时，会遍历设备树中的设备节点，并将每个设备节点的 compatible 属性与驱动中的兼容属性进行匹配。只有当匹配成功时，才会执行相应的 probe 函数。

2.在设备树根节点下添加设备。

打开 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 添加如下内容：

```
my_device {
                compatible = "platform";
        };
```

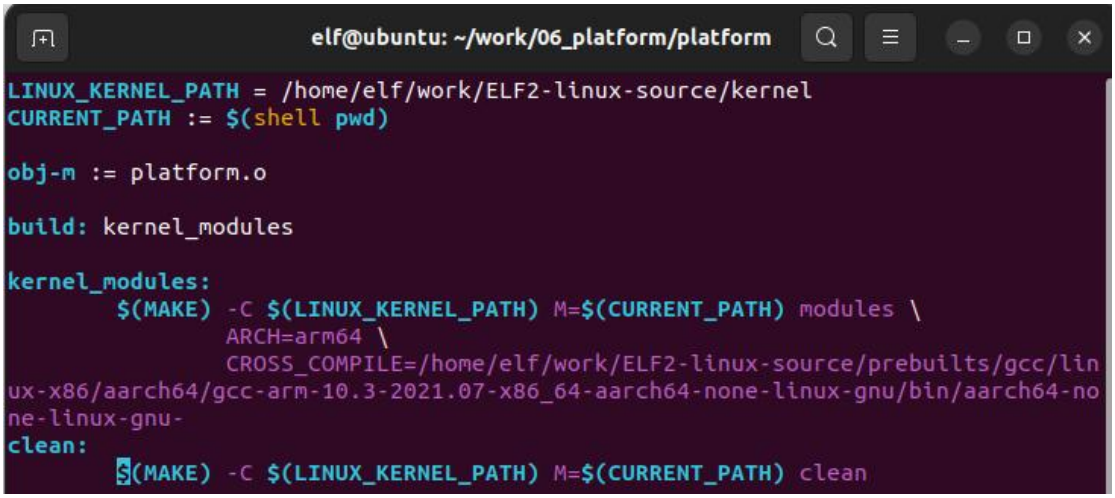
添加后效果如下：

```
my_device {
        compatible = "platform";
};

fan: pwm-fan {
        compatible = "pwm-fan";
        #cooling-cells = <2>;
        pwms = <&pwm2 0 50000 0>;
};
```

3.编译驱动。

在 platform.c 目录下创建 Makefile 文件：



执行 make 指令，编译生成 platform.ko 文件。

4.编译内核。

因为修改了设备树文件，所以需要重新编译内核：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

将编译生成的 boot.img 镜像烧录到开发板中。

5.测试。

将上文编译生成的 platform.ko 文件拷贝到开发板中，然后执行如下指令：

```
root@elf2-buildroot:~# insmod platform.ko
[    37.305039] my_platform_probe: Platform device probed
root@elf2-buildroot:~# rmmod platform.ko
[    79.122608] my_platform_remove: Platform device removed
```

可以看到在驱动加载时，进入到了 probe 函数，在驱动卸载时进入到了 remove 函数。从上面示例中可以了解到使用 module_platform_driver 宏可以大大简化平台驱动程序的编写过程，并提高代码的可读性和可维护性。

2.7.3 platform 驱动控制 LED

同样下面以控制核心板上的 LED 灯为例进行讲解：

核心板上的 LED 灯使用的是 GPIO0_A0。

在设备树 arch/arm64/boot/dts/rockchip/elf2-3588-common.dtsi 中添加 leds 节点和引脚复用，并检查设备树中是否有其它的地方也用到了此引脚，如果用到了就需要将其屏蔽掉，避免复用冲突。

添加节点：

```
platform-leds {
                compatible = "platform_led";
                pinctrl-names = "default";
                pinctrl-0 = <&pinctrl_led>;
                gpios = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>;
        };
```

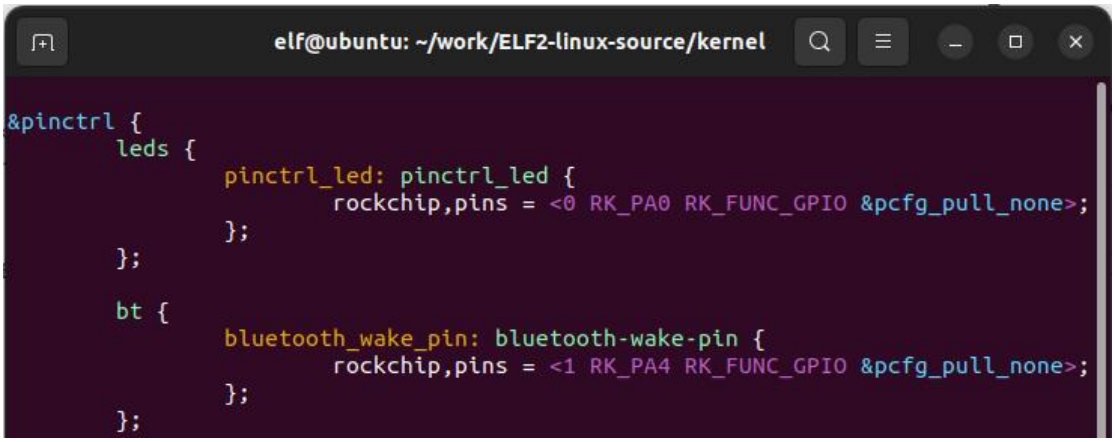
添加后效果如下：



添加复用，在 iomux 中新建 pinctrl_led：

```
&pinctrl {
        leds {
                pinctrl_led: pinctrl_led {
                        rockchip,pins = <0 RK_PA0 RK_FUNC_GPIO
&pcfg_pull_none>;
                };
        };
...
};
```

添加后效果如下:



1.驱动源码编写，新建一个 platform_led.c

1）头文件引用

```
#include <linux/init.h>
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/device.h>          //包含设备节点相关的头文件
#include <linux/gpio.h>            //包含 gpio 操作函数的相关头文件
#include <linux/of_gpio.h>
```

```
#include <linux/of.h>
```

2）创建相关宏定义

```
#define DEVICE_NAME "my_device"
#define LED_IOC_MAGIC 'k'
#define SET_LED_ON _IO(LED_IOC_MAGIC, 0)
#define SET_LED_OFF _IO(LED_IOC_MAGIC, 1)

struct device_node *node;    //设备树节点
int gpio;    //gpio 编号
```

这里同样使用 ioctl 来控制，ioctl 相关的定义本节不在重复讲解。

3）定义 platform_driver 类型结构体

```
static struct platform_driver my_platform_driver = {
    .driver = {
        .name = "my_platform_driver",
        .owner = THIS_MODULE,
        .of_match_table = of_platform_match,
    },
    .probe = my_platform_probe,
    .remove = my_platform_remove,
};
```

4）定义 of_platform_match，用来与设备树中的 compatible 匹配，匹配成功后才会进入到 probe 函数中。

```
static const struct of_device_id of_platform_match[] = {
        { .compatible = "platform_led", },
        {},
};
```

5）probe 函数的实现

```
static int my_platform_probe(struct platform_device *pdev)
{
        int ret;
        //注册杂项设备
        ret = misc_register(&my_misc_device);
        if (ret) {
        pr_err("Failed to register misc device\n");
        return ret;
        }
        // 获取设备节点
        node = of_find_node_by_name(NULL,"platform-leds");
        // 获取 gpios 属性中的引脚编号
        gpio = of_get_named_gpio(node, "gpios", 0);
        //判断是否获取成功
          if (!gpio_is_valid(gpio)) {
          pr_err("Failed to get GPIO\n");
```

```
            return -1;
        }

        gpio_free(gpio);
        if (gpio_request(gpio, "led_run")) {
                printk("request %s gpio faile \n", "led_run");
                return -1;
        }
        printk(KERN_INFO "my_platform_probe: Platform device probed\n");
    return 0;
}
```

misc_register()用户注册杂项设备，原型如下：

```
int misc_register(struct miscdevice *misc);
```

参数说明：

misc：指向 struct miscdevice 结构体的指针，表示要注册的杂项设备。

返回值是一个整数，表示注册结果。如果注册成功，函数会返回 0；如果注册失败，函数会返回一个负数，表示错误代码。

struct miscdevice 结构体用于描述杂项设备的属性，包括设备的次设备号（minor number）、设备名称、设备文件操作函数等。在注册杂项设备之前，需要先初始化 struct miscdevice 结构体的字段，然后将其作为参数传递给 misc_register()函数。

of_find_node_by_name()用于查找设备树中的节点，原型如下：

```
struct device_node *of_find_node_by_name(struct device_node *prev, const char *name);
```

参数说明：

prev：指向前一个设备树节点的指针。如果要从整个设备树中开始查找节点，则可以传递 NULL。

name：字符串，表示要查找的设备树节点的名称。

返回值是一个指向找到的设备树节点的指针。如果找到匹配的节点，则返回该节点的指针；如果未找到匹配的节点，则返回 NULL。

of_get_named_gpio()用于从设备树中获取 GPIO 引脚编号，原型如下：

```
int of_get_named_gpio(const struct device_node *np, const char *propname, int index);
```

参数说明：

np：指向设备树节点的指针，表示要从该节点获取 GPIO 引脚。

propname：字符串，表示要获取的 GPIO 引脚属性的名称。

index：整数，表示在属性中指定的 GPIO 引脚的索引。如果属性中有多个 GPIO 引脚定义，可以通过指定索引来获取其中的一个引脚。

返回值是一个整数，表示获取到的 GPIO 引脚编号。如果获取失败，函数会返回一个负数。

6）miscdevice 类型结构体定义

```
static struct miscdevice my_misc_device = {
    .minor = MISC_DYNAMIC_MINOR,     // 动态分配次设备号
    .name = DEVICE_NAME,             // 设备名称
    .fops = &my_device_fops,         // 设备文件操作函数
};
```

7）文件操作函数，file_operations 类型结构体定义

```
static struct file_operations my_device_fops = {
    .owner = THIS_MODULE,
    .open = my_device_open,
    .release = my_device_release,
    .unlocked_ioctl = myled_ioctl,
};
```

8）文件操作函数实现：

```
static int my_device_open(struct inode *inode, struct file *file)
{
    // 打开设备的操作
    //将引脚配置为输出
    gpio_direction_output(gpio, 1);
    printk(KERN_INFO "This is device_open.\n");
        return 0;
}

static long myled_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{

    switch (cmd) {
            case SET_LED_ON:
                printk(KERN_INFO "This is set_led_on.\n");
            // 设置 GPIO 引脚为高电平
            gpio_set_value(gpio, 1);
            break;

        case SET_LED_OFF:
          //设置 GPIO 引脚为低电平
          printk(KERN_INFO "This is set_led_off.\n");
          gpio_set_value(gpio, 0);
            break;

        default:
            return -ENOTTY;
    }

    return 0;
}

static int my_device_release(struct inode *inode, struct file *file)
{
    // 关闭设备的操作
    return 0;
}
```

文件操作函数同样使用 ioctl 来实现，与之前的 LED 操作一样，接收到 SET_LED_ON 命令后将引脚拉低，接收到 SET_LED_OFF 命令后，将引脚电平拉高。

2.完整驱动示例源码

```
#include <linux/init.h>
#include <linux/module.h>          // 包含模块相关函数的头文件
#include <linux/fs.h>              // 包含文件系统相关函数的头文件
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/device.h>          //包含设备节点相关的头文件
#include <linux/gpio.h>            //包含 gpio 操作函数的相关头文件
#include <linux/of_gpio.h>
#include <linux/of.h>

#define DEVICE_NAME "my_device"
#define LED_IOC_MAGIC 'k'
#define SET_LED_ON _IO(LED_IOC_MAGIC, 0)
#define SET_LED_OFF _IO(LED_IOC_MAGIC, 1)

struct device_node *node;    //设备树节点
int gpio;    //gpio 编号

static int my_device_open(struct inode *inode, struct file *file)
{
    // 打开设备的操作
    //将引脚配置为输出
     gpio_direction_output(gpio, 1);
     printk(KERN_INFO "This is device_open.\n");
          return 0;
}

static long myled_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{

     switch (cmd) {
               case SET_LED_ON:
                    printk(KERN_INFO "This is set_led_on.\n");
               // 设置 GPIO 引脚为高电平
               gpio_set_value(gpio, 1);
               break;

          case SET_LED_OFF:
            //设置 GPIO 引脚为低电平
            printk(KERN_INFO "This is set_led_off.\n");
            gpio_set_value(gpio, 0);
```

```
                break;

        default:
                return -ENOTTY;
    }

    return 0;
}

static int my_device_release(struct inode *inode, struct file *file)
{
    // 关闭设备的操作
    return 0;
}

static struct file_operations my_device_fops = {
    .owner = THIS_MODULE,
    .open = my_device_open,
    .release = my_device_release,
    .unlocked_ioctl = myled_ioctl,
};

static struct miscdevice my_misc_device = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = DEVICE_NAME,
    .fops = &my_device_fops,
};

static int my_platform_probe(struct platform_device *pdev)
{
        int ret;
        //注册杂项设备
        ret = misc_register(&my_misc_device);
        if (ret) {
        pr_err("Failed to register misc device\n");
        return ret;
        }
        // 获取设备节点
        node = of_find_node_by_name(NULL,"platform-leds");
        // 获取 gpios 属性中的引脚编号
        gpio = of_get_named_gpio(node, "gpios", 0);
        //判断是否获取成功
          if (!gpio_is_valid(gpio)) {
          pr_err("Failed to get GPIO\n");
```

```
                return -1;
            }

            gpio_free(gpio);
            if (gpio_request(gpio, "led_run")) {
                    printk("request %s gpio faile \n", "led_run");
                    return -1;
             }
            printk(KERN_INFO "my_platform_probe: Platform device probed\n");
      return 0;
}
static int my_platform_remove(struct platform_device *pdev)
{
            misc_deregister(&my_misc_device);

            printk(KERN_INFO "my_platform_remove: Platform device removed\n");
      return 0;
}

static const struct of_device_id of_platform_match[] = {
            { .compatible = "platform_led", },
            {},
};

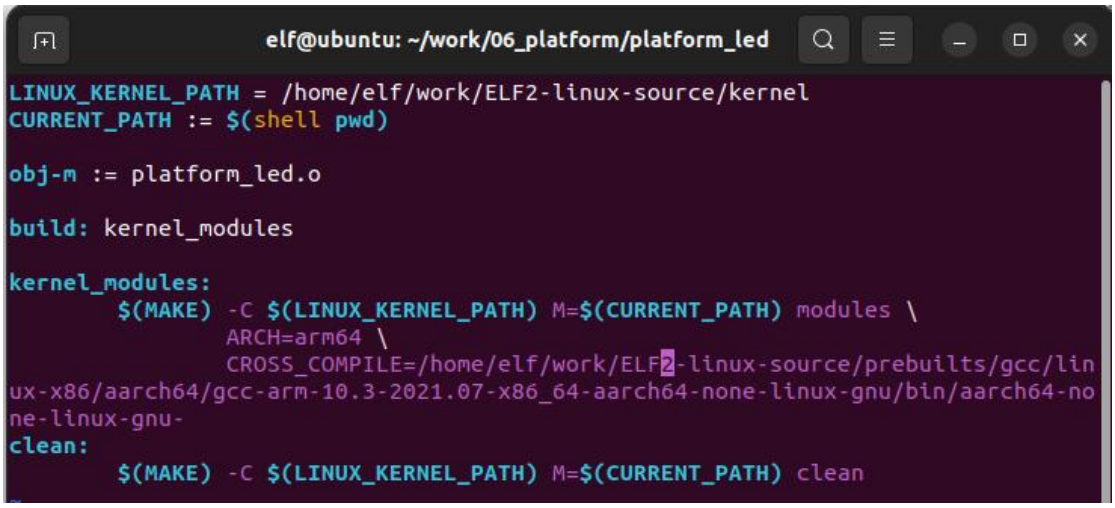
static struct platform_driver my_platform_driver = {
      .driver = {
            .name = "my_platform_driver",
            .owner = THIS_MODULE,
            .of_match_table = of_platform_match,
      },
      .probe = my_platform_probe,
      .remove = my_platform_remove,
};

module_platform_driver(my_platform_driver);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("Platform Driver Example");
```

3.编译驱动。

在 platform_led.c 目录下创建 Makefile 文件：



执行 make 指令，编译生成 platform_led.ko 文件。

4.编译内核。

因为修改了设备树文件，所以需要重新编译内核：

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh kernel
```

将编译生成的 boot.img 镜像烧录到开发板中。

5.测试，测试 app 使用之前章节的 app 即可。

将前面编译生成的 platform_led.ko 文件拷贝到开发板，并执行如下指令：

```
root@elf2-buildroot:~# insmod platform_led.ko
 [    20.927322] my_platform_probe: Platform device probed
root@elf2-buildroot:~# ./led_app
[    25.360878] This is device_open.
[    25.360913] This is set_led_on.
[    26.361118] This is set_led_off.
[    27.361396] This is set_led_on.
[    28.361656] This is set_led_off.
[    29.361828] This is set_led_on.
[    30.361924] This is set_led_off.
[    31.362014] This is set_led_on.
[    32.362105] This is set_led_off.
[    33.362197] This is set_led_on.
[    34.362294] This is set_led_off.
```

此时可以看到核心板上的蓝色 LED，循环闪烁 5 次。

第三章 Uboot 介绍

3.1 Uboot 简介

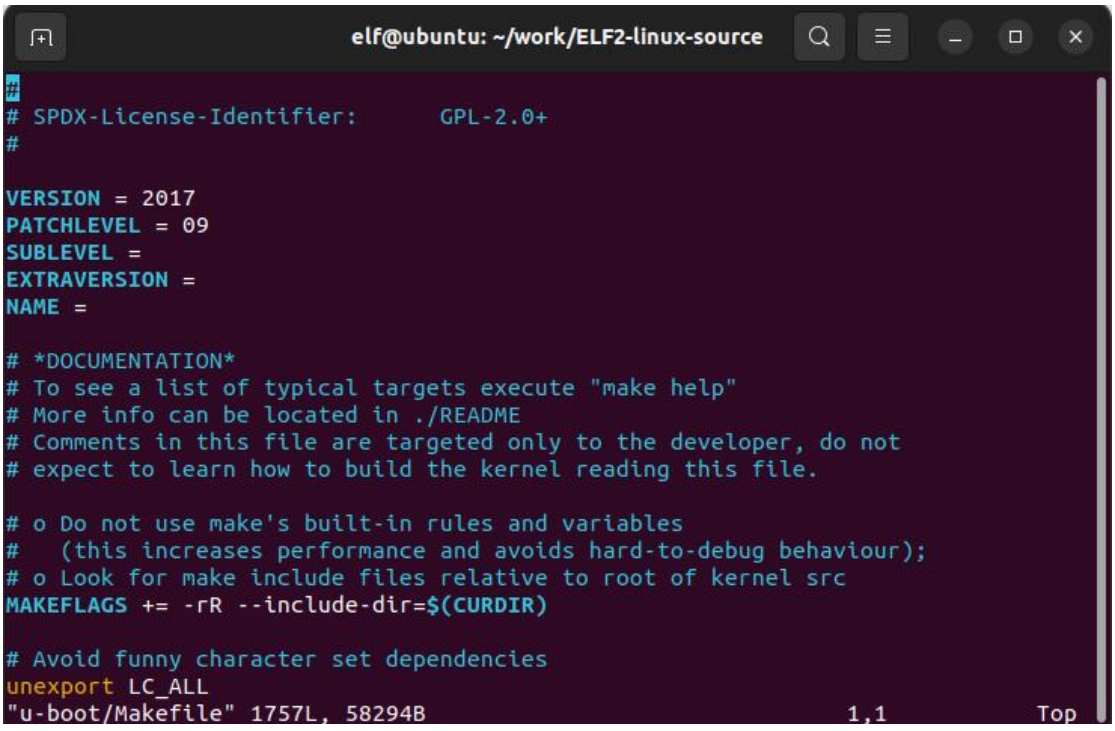
3.1.1 BootLoder

BootLoder 是什么，从名字上来看，BootLoder 由 boot 和 loader 两部分组成，这也是它的主要的两个功能。boot 主要是指初始化一些系统的硬件并建立内存映射图，为内核的运行创建一个合适的环境，loader 就是指将内核从存储介质(启动介质)中加载到内存 RAM 中，然后跳转到内核所在内存地址运行内核。总之，BootLoder 是内核启动之前为内核搭建软硬件环境的一段程序。BootLoder 是不同平台或项目的启动加载程序的统称，不单指某一个程序，比如，常见的 BootLoder 有，Redboot，Armboot，U-Boot 等。

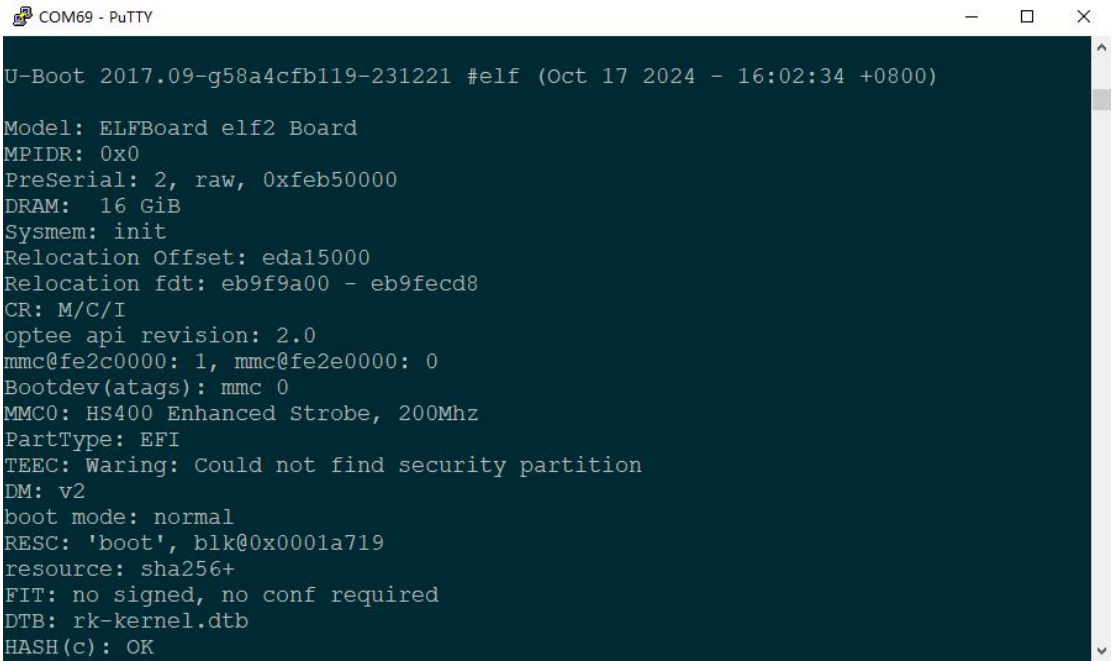
3.1.2 U-Boot 版本

U-Boot 是遵循 GPL 条款的开放源码项目，是在 PPCBoot 以及 ARMBoot 的基础上逐步发展和演化而来的，最新版本目前已经更新到 u-boot-2020.10 版本，我们可以从 uboot 官方网站：http://www.denx.de/wiki/U-Boot/SourceCode 下载。不过我们目前使用的 uboot 源码不是在这个网站下载的，而是由瑞芯微公司提供的。ELF2 开发板使用的 uboot 是 2017 版本的，并且瑞芯微公司已经将 uboot 源码开源到了 Github 上，并会定期更新：https://github.com/rockchip-linux/u-boot

可以在 u-boot 目录下的 Makefile 文件中查看版本号。

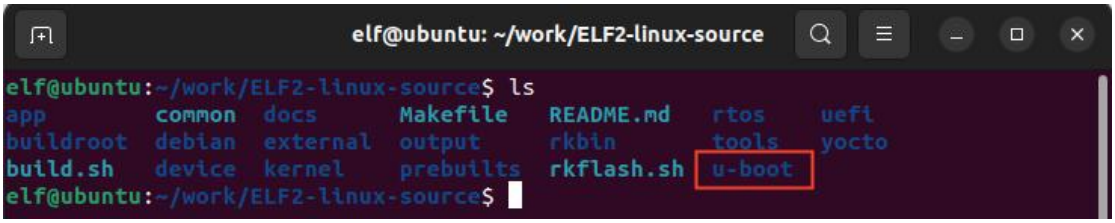


同时也可以在开发板启动时，通过 debug 口的信息查看。



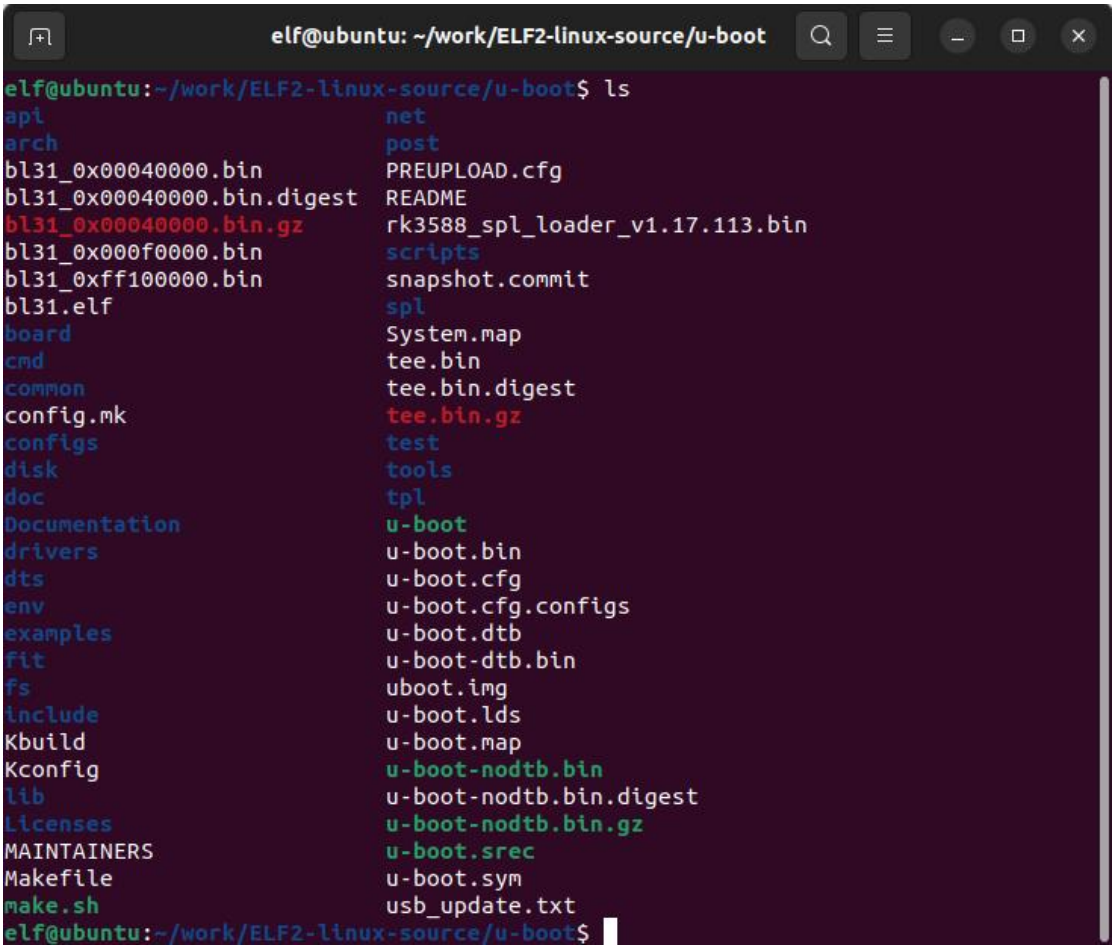
3.2 Uboot 目录结构介绍

SDK 源码解压后，uboot 源码在 u-boot 目录下：



进入 u-boot 目录，使用 ls 命令查看 uboot 源码目录：

```
elf@ubuntu:~/work/ELF2-linux-source/u-boot$ ls
```



1.目录说明

api：存放 uboot 提供的功能函数接口。

arch：系统架构相关的代码，这里每一个文件夹名都代表着一种处理器类型，我们关心的是 arm，在 arch/arm/cpu/armv8 里面存在着重要的与 uboot 启动相关的.S 文件。

board：存放着根据不同开发板定制的板级代码

common： 一些与具体硬件无关的通用代码

cmd：以 uboot 命令行处理代码为主

disk ：磁盘分区相关代码

doc：里面存放了很多 uboot 相关文档，这些文档可以帮助我们理解和使用 uboot

drivers ： 驱动代码

examples：示例程序

fs：文件系统，支持嵌入式开发板常见的文件系统

include：头文件，以通用的头文件为主

lib：通用库文件

net：网络相关代码

post： 加电自检程序

tools：辅助程序，用于编译和检查 uboot 目标文件

2.文件说明

Makefile：一个致力于工程编译的文件,将来整个 uboot 编译的时候就是用这个 Makefile 来编译管理的

Kconfig:用于生成内核配置界面的的文件

Kbuild：跟 Makefile 文件配合编译 uboot，它对 Makefile 进行了功能上的扩充，使其在编译 uboot 文

件时更加高效简介

config.mk：为一个 Makefile 文件,被主 Makefile 调用。

MAINTAINERS：维护者，就是当前在参与维护 uboot 源码的社区工作者。

README：源码说明文件

3.3 U-Boot 编译

U-Boot 编译可以分为两种形式，第一种方式是在 SDK 目录下使用 build.sh 单独编译，另一种方式是在 u-boot 目录下使用 make.sh 进行编译。

3.3.1 SDK 目录下编译

在 sdk 目录下直接执行./build.sh uboot，即可单独编译 U-boot。

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh uboot
Log saved at /home/elf/work/ELF2-linux-source/output/sessions/2024-09-26_18-55-54
Using last kernel version(5.10)

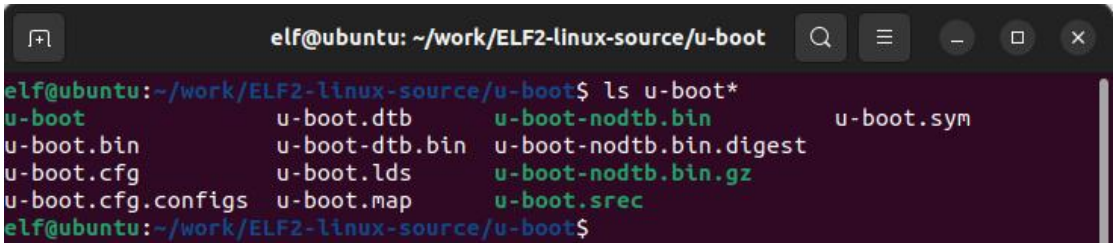
============================================
              Final configs
============================================
RK_BOOT_FIT_ITS=/home/elf/work/ELF2-linux-source/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=elf2_fs
RK_CHIP=elf2
RK_CHIP_FAMILY=elf2
......
home/elf/work/ELF2-linux-source/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
Thu Sep 26 18:56:09 CST 2024
+ cd ..
Running 70-loader.sh - build_uboot succeeded.
```

编译完成后显示：build_uboot succeeded。

3.3.2 编译后生成的文件

U-Boot 编译成功后会在根目录下生成一些重要文件

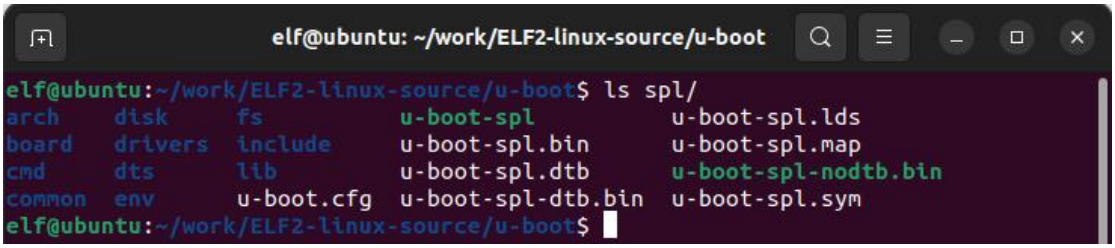
U-Boot 阶段




```
u-boot.map          // MAP 表文件
u-boot.sym          // SYMBOL 表文件
```

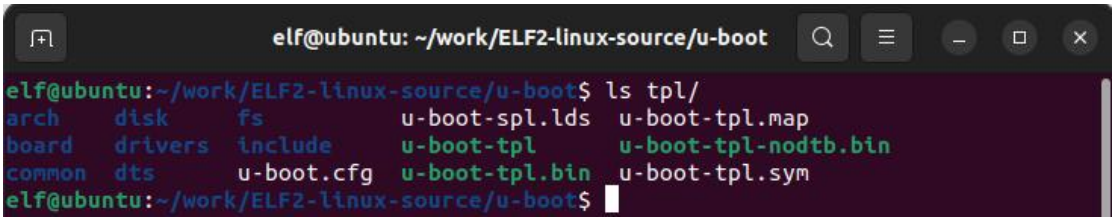
```
u-boot              // ELF文件，类同内核的 vmlinux（重要！）
u-boot.dtb          // u-boot 自己的 dtb 文件
u-boot.bin          //  可执行二进制文件，会被打包成 uboot.img 用于烧写
```

SPL 阶段




```
spl/u-boot-spl.map     // MAP 表文件
spl/u-boot-spl.sym     // SYMBOL 表文件
spl/u-boot-spl         // ELF 文件，类同内核的 vmlinux（重要！）
spl/u-boot-spl.dtb     // spl 自己的 dtb 文件
spl/u-boot-spl.bin     //  可执行二进制文件，会被打包成 loader 用于烧写
```

TPL 阶段




```
tpl/u-boot-tpl.map     // MAP 表文件
tpl/u-boot-tpl.sym     // SYMBOL 表文件
tpl/u-boot-tpl         // ELF 文件，类同内核的 vmlinux（重要！）
tpl/u-boot-tpl.dtb     // tpl 自己的 dtb 文件
tpl/u-boot-tpl.bin     //  可执行二进制文件，会被打包成 loader 用于烧写
```

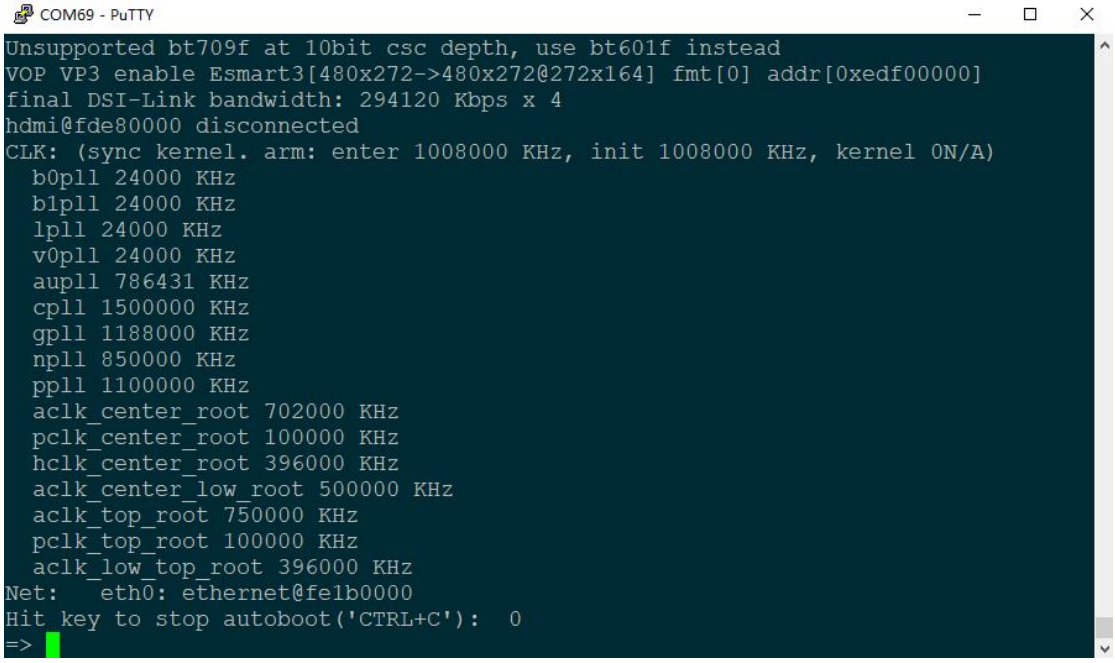
3.4 uboot 常用命令

3.4.1 查看命令

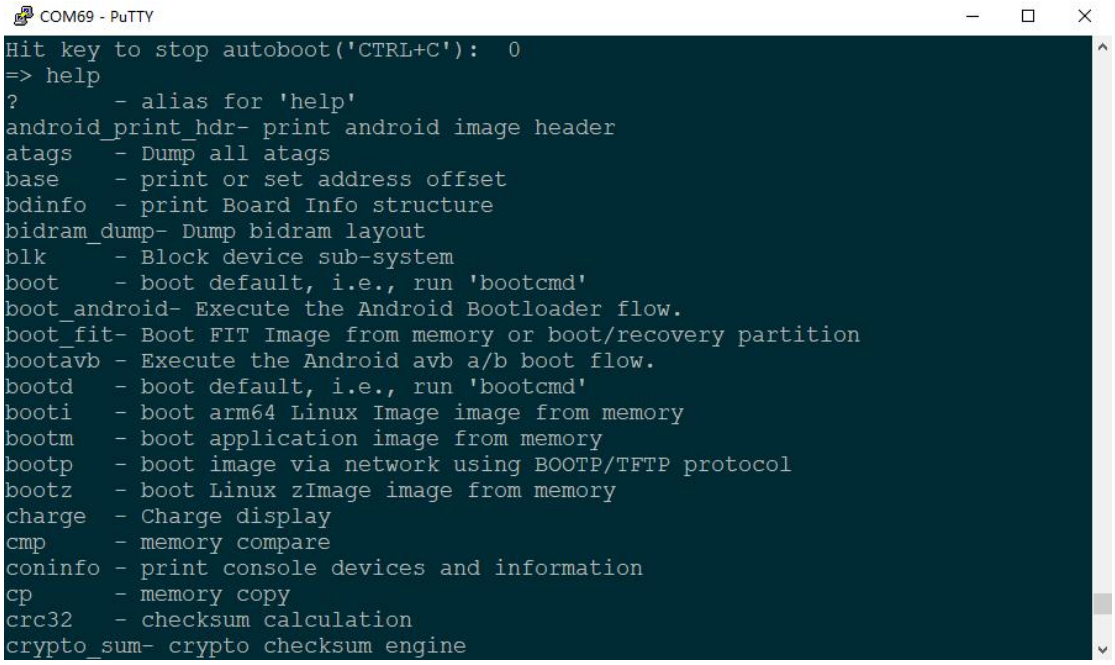
RK 平台提供串口组合键触发一些事件用于调试、烧写（如果无法触发，请多尝试几次；启用 secure-boot 时无效）。开机时长按：

参数	说明
ctrl+c	进入 U-Boot 命令行模式
ctrl+d	进入 loader 烧写模式
ctrl+b	进入 maskrom 烧写模式
ctrl+f	进入 fastboot 模式
ctrl+m	打印 bidram/system 信息
ctrl+i	使能内核 initcall_debug
ctrl+p	打印 cmdline 信息
ctrl+s	"Starting kernel..."之后进入 U-Boot 命令行

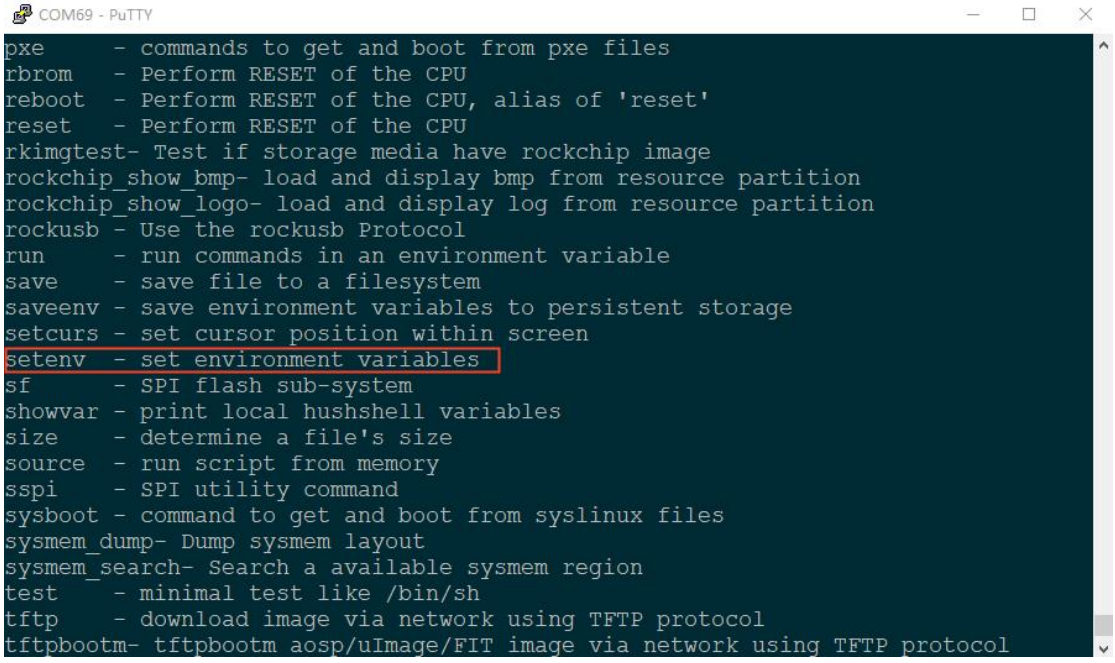
上电后，按“Ctrl+C”进入 boot 命令行：



进入 uboot 命令行之后，可以通过输入“？”或者 help 可以查看 uboot 全部命令。如图（图未截全）：



每一个命令后面有一个功能说明，使用 help 命令，可以查看该命令的使用说明，例如我们比较常用的 setenv：



setenv - set environment variables 该命令是设置环境变量功能，使用 help setenv 查看使用说明：

```
=>help setenv
```

```
=> help setenv
setenv - set environment variables

Usage:
setenv [-f] name value ...
    - [forcibly] set environment variable 'name' to 'value ...'
setenv [-f] name
    - [forcibly] delete environment variable 'name'
=>
```

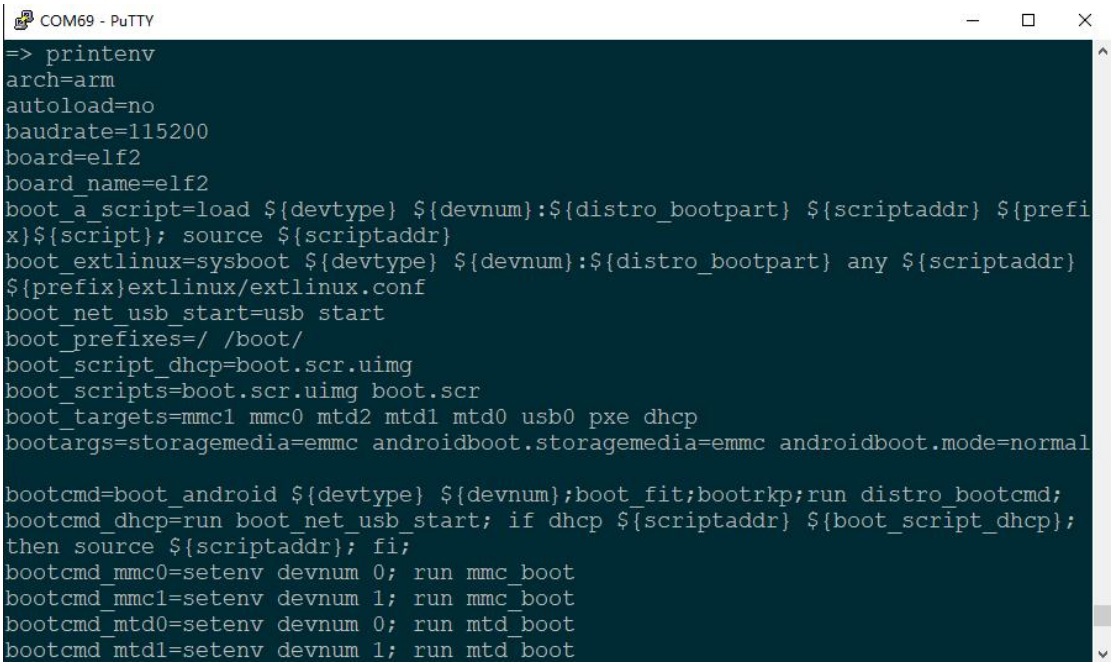
这里可以看到 setenv 命令有两种用法，一种是设置环境变量值，一种是删除环境变量，具体使用方法在下一节中详细说明。

3.4.2 环境变量命令

uboot 中环境变量的作用就是在不改变源码、不用重新编译的情况下，可以使我们通过设置环境变量的值来改变 uboot 的一些设置。uboot 开机时会一次性从存储介质(启动介质)中读取全部的环境变量到内存中作为环境变量的初始值，uboot 运行过程中用到的环境变量，都是内存中的环境变量，用户可以使用 saveenv 指令将内存中的环境变量重新写入存储介质(启动介质)中，下次重启系统时会从存储介质(启动介质)中读取更新过的环境变量。

1.查看环境变量

输入 printenv 命令，即可查看所有环境变量（图片只截取了一部分）：



图中我们看到环境变量的基本模式是：变量名=变量值，这个变量值就是一串字符串，程序通过解析字符串，得到所需参数（如 baudrate=115200）或者所要执行的操作（如 mmc_boot）。

2.增改删环境变量

1）我们可以添加一个自己的环境变量，比如添加一个 firstenv 变量，并设置值为 elf2：

```
=>setenv firstenv elf2
```

保存:

```
=>saveenv
```

saveenv 命令是保存环境变量到 eMMC Flash，如果不使用此命令，那么只是将环境变量存储到了内存中，断电之后，此变量将丢失。

然后使用 printenv 命令查看环境变量添加成功：

```
=>printenv
fdtcontroladdr=0x8300000
firstenv=elf2
kernel_addr_c=0x05480000
```

或者使用“printenv 变量名”查看：

```
=> printenv firstenv
firstenv=elf2
```

2）更改环境变量的值，这次我们不保存 saveenv：

```
=> setenv firstenv rk3588
```

然后使用 printenv 命令查看结果，变量值更改成功：

```
=> printenv firstenv
firstenv=rk3588
```

3）删除一个环境变量，删除 firstenv 环境变量,使用 setenv 变量名，不赋任何值：

```
=> setenv firstenv
=> saveenv
```

使用 printenv 查看结果，发现该变量不存在，表示删除成功：

```
=> printenv firstenv
```

```
# # Error: “firstenv” not defined
```

3.bootdelay

用于倒计时，等待用户进入命令行。经过以上章节讲解，通过 sentenv 可以设置 bootdelay 时间为 2s。

```
=> printenv bootdelay
bootdelay=2
```

3.4.3 常用操作命令

3.4.3.1 网络命令

使用网络命令之前，首先需要设置开发板的 IP 地址：

```
=> setenv ipaddr 192.168.0.232
=> saveenv
```

我们使用 PC 机和开发板进行通信测试，所以需要设置 PC 机的 IP 地址为同一网段的：192.168.0.102。

ping 命令主要用于测试网络通信状态，设置好开发板 IP 和 PC 机 IP 之后，使用网线连接好开发板和 PC 机，使用 ping 命令：

```
=> ping 192.168.0.102
ethernet@fe1b0000 Waiting for PHY auto negotiation to complete. Done
Using ethernet@fe1b0000 device
host 192.168.0.102 is alive
=>
```

host is alive，表示网络通信畅通。

3.4.3.2 内存命令

1.md

md 是 memory display 的缩写，即显示内存数据。使用 help md 命令查看 md 使用方法：

```
=> help md
md - memory display

Usage:
md [.b, .w, .l, .q] address [# of objects]
=>
```

其中：

参数	说明
.b	显示 8 位数据
.w	显示 16 位数据
.l	显示 32 位数据
.q	退出内存显示或终止当前的操作

address 表示显示的数据地址，of objects 表示显示的数据个数（注意是十六进制）。例如：显示内存地址 0X83000000 地址中的 2 个 8 位数据：

```
=> md.b 83000000 2
83000000: ff ff
=>
```

显示内存地址 0X83000000 地址中的 1 个 16 位数据：

```
=> md.w 83000000 1
83000000: ffff
=>
```

如果不加.b，.w，.l，也不设置显示数据的个数的话，默认是显示 1 个 32 位数据：

```
=> md 83000000
83000000: ffffffff
=>
```

2.mw

mw 是 memory write 的缩写，即向内存中写入数据。使用 help mw 命令显示 mw 的使用方法：

```
=> help mw
mw - memory write (fill)

Usage:
mw [.b, .w, .l, .q] address value [count]
=>
```

显然跟 md 命令使用方式类似，只是多了一个 value，即要写入的数据，而 count 是指要在起始地址为 address 的内存中写入 value 的个数。

直接举例说明，向内存 0x80000000 的地址写入 2 个 8 位数据 a8，然后使用 md 命令查看是否写入成功：

```
=> mw.b 80000000 a8 2
=> md.b 80000000 2
80000000: a8 a8
=>
```

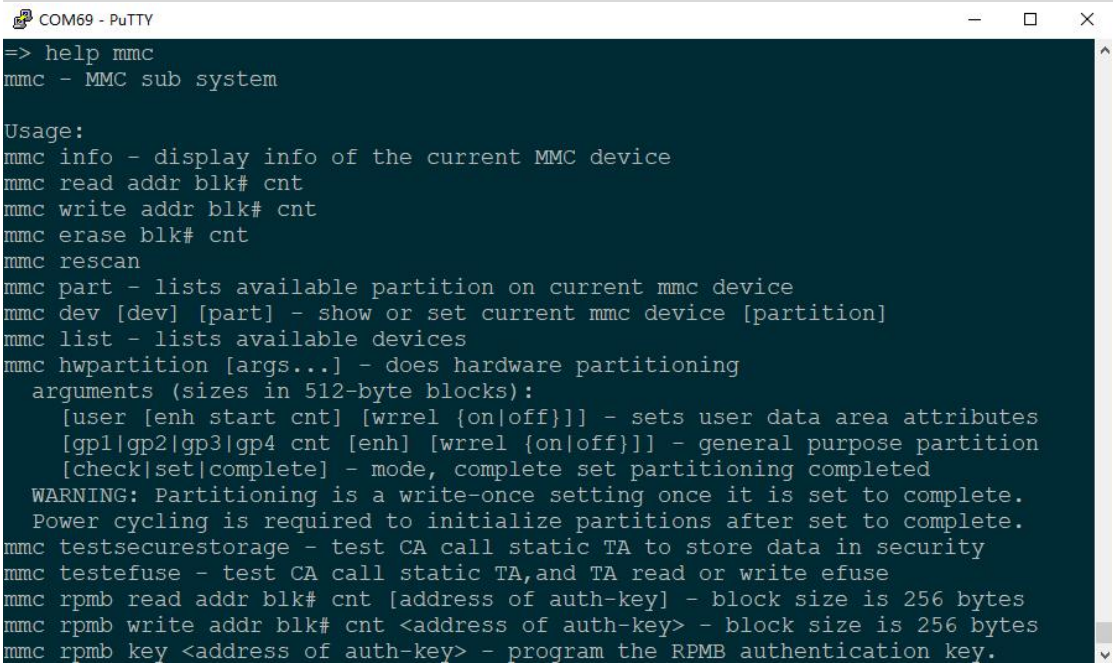
显然写入数据成功。

3.4.3.3 eMMC/SD 卡命令

eMMC 和 SD 卡(TF 卡)是同一类设备，以下命令二者是通用，本章节主要以 eMMC 举例说明命令的使用。

使用 help mmc 可以看到 mmc 相关命令列表以及其对应命令用法：

```
=> help mmc
```



1.mmc dev

使用 mmc list 可以看到当前系统挂载的所有 mmc 设备：

```
=> mmc list
mmc@fe2c0000: 1
mmc@fe2e0000: 0 (eMMC)Device 'mmc@fe2c0000': seq 1 is in use by 'mmc@fe
2c0000'
mmc@fe2c0000: 1Device 'mmc@fe2e0000': seq 0 is in use by 'mmc@fe2e0000'
mmc@fe2e0000: 0
=>
```

本系统的 mmc 设备有设备 0 和设备 1,设备 0 是 eMMC 设备，设备 1 就是 SD 卡(TF 卡)设备。

使用 mmc dev 命令可以查看当前选中正在使用的 mmc 设备。

mmc dev 使用方法为：mmc dev [dev] [part]

参数	说明
[dev]	要选择的设备
[part]	所选设备的分区

如果不加[dev]和[part]两个参数，则显示当前选中设备，如果不加[part]参数，则默认使用所选设备的 0 分区。

```
=> mmc dev
switch to partitions #0, OK
mmc0(part 0) is current device
=>
```

当前选中正在使用的是 mmc0 设备的 0 分区，即 eMMC Flash 的 0 分区，此处的 0 分区其实是指 eMMC Flash 的用户分区(用户分区的概念在以下 mmc write 章节会有说明)。

使用 mmc dev 切换到使用 SD 设备：mmc dev 1：

```
=> mmc dev 1
switch to partitions #0, OK
mmc1 is current device
=>
```

切换成功之后，现在使用的就是 SD 卡(TF 卡)设备，此时使用的 mmc 命令都是对 SD 卡进行的操作。为了使用 eMMC 进行命令使用举例演示，我们再将 mmc 设备更换为 eMMC：

mmc dev 0：

```
=> mmc dev 0
switch to partitions #0, OK
mmc0(part 0) is current device
=>
```

2.mmc info

该命令是查看当前 mmc 设备相关信息：mmc info 或者 mmcinfo：

```
=> mmc info
Device: mmc@fe2e0000
Manufacturer ID: d6
OEM: 103
Name: A3A55
Timing Interface: HS400 Enhanced Strobe
Tran Speed: 200000000
Rd Block Len: 512
MMC version 5.1
```

```
High Capacity: Yes
Capacity: 28.9 GiB
Bus Width: 8-bit DDR
Erase Group Size: 512 KiB
HC WP Group Size: 8 MiB
User Capacity: 28.9 GiB WRREL
Boot Capacity: 4 MiB ENH
RPMB Capacity: 16 MiB ENH
=>
```

3.mmc read

mmc read 命令是读 mmc 中的数据到内存地址。

使用方法为 mmc read addr blk# cnt

参数	说明
addr	将要存放数据的内存地址
blk#	要读的 mmc 的起始块号
cnt	块数目

举例，从 mmc 第 5 块开始读取 2 块的数据到内存 0x8300000：mmc read 8300000 5 2：

```
=> mmc read 8300000 5 2

MMC read: dev # 0, block # 5, count 2 ... 2 blocks read: OK
=>
```

每一块是 512 个字节，读取 2 块，就是读取 2*512=1024 个字节。

4.mmc write

顾名思义，mmc write 是向 mmc 中写入数据，mmc write addr blk# cnt

参数	说明
addr	将要存放数据的内存地址
blk#	要读的 mmc 的起始块号
cnt	块数目

5.mmc erase

mmc erase 是擦除 mmc 中数据。使用方法：

mmc erase blk# cnt

参数	说明
blk#	要擦除的 mmc 起始地址
cnt	要擦除的块数量

一般情况很少使用 mmc erase 命令进行 mmc 擦除操作。此处不进行举例，可能会导致无法正常启动。

3.5 uboot 启动流程分析

RK 平台的 U-Boot 启动流程如下，仅列出一些重要步骤：

```
start.s
    // 汇编环境
    => IRQ/FIQ/lowlevel/vbar/errata/cp15/gic    // ARM 架构相关的 lowlevel 初始化
    => _main
      => stack                                  // 准备好 C 环境需要的栈
```

```
// 【第一阶段】C 环境初始化，发起一系列的函数调用
=> board_init_f: init_sequence_f[]
    initf_malloc
    arch_cpu_init                   // 【SoC 的 lowlevel 初始化】
    serial_init                     // 串口初始化
    dram_init                       // 【获取 ddr 容量信息】
    reserve_mmu                     // 从 ddr 末尾开始往低地址 reserve 内存
    reserve_video
    reserve_uboot
    reserve_malloc
    reserve_global_data
    reserve_fdt
    reserve_stacks
    dram_init_banksize
    sysmem_init
    setup_reloc                     // 确定 U-Boot自身要 reloc 的地址
// 汇编环境
=> relocate_code                    // 汇编实现 U-Boot 代码的 relocation
// 【第二阶段】C 环境初始化，发起一系列的函数调用
=> board_init_r: init_sequence_r[]
    initr_caches                    // 使能 MMU 和 I/Dcache
    initr_malloc
    bidram_initr
    sysmem_initr
    initr_of_live                   // 初始化 of_live
    initr_dm                        // 初始化 dm 框架
    board_init                      // 【平台初始化，最核心部分】
      board_debug_uart_init         // 串口 iomux、clk 配置
      init_kernel_dtb               // 【切到 kernel dtb】！
      clks_probe                    // 初始化系统频率
      regulators_enable_boot_on     // 初始化系统电源
      io_domain_init                // io-domain 初始化
      set_armclk_rate               // __weak，ARM 提频(平台有需求才实现)
      dvfs_init                     // 宽温芯片的调频调压
      rk_board_init                 // __weak，由各个具体平台进行实现
    console_init_r
    board_late_init                 // 【平台 late 初始化】
      rockchip_set_ethaddr          // 设置 mac 地址
      rockchip_set_serialno         // 设置 serialno
      setup_boot_mode               // 解析"reboot xxx"命令、
                                    // 识别按键和 loader 烧写模式、recovery
      charge_display                // U-Boot 充电
      rockchip_show_logo            // 显示开机 logo
      soc_clk_dump                  // 打印 clk tree
```

```
    rk_board_late_init          // __weak，由各个具体平台进行实现
run_main_loop                   // 【进入命令行模式，或执行启动命令】
```

第四章 Buildroot

4.1 Buildroot 简介

Rockchip Buildroot Linux SDK 是基于 Buildroot-2018.02 的版本的软件开发包，其包含了基于 Linux 系统开发用到的各种系统源码，驱动，工具，应用软件包。Buildroot 是 Linux 平台上一个开源的嵌入式 Linux 系统自动构建框架。整个 Buildroot 是由 Makefile 脚本和 Kconfig 配置文件构成的。你可以通过 Buildroot 配置，编译出一个完整的可以直接烧写到机器上运行的 Linux 系统软件。

Buildroot 有以下几点优势：

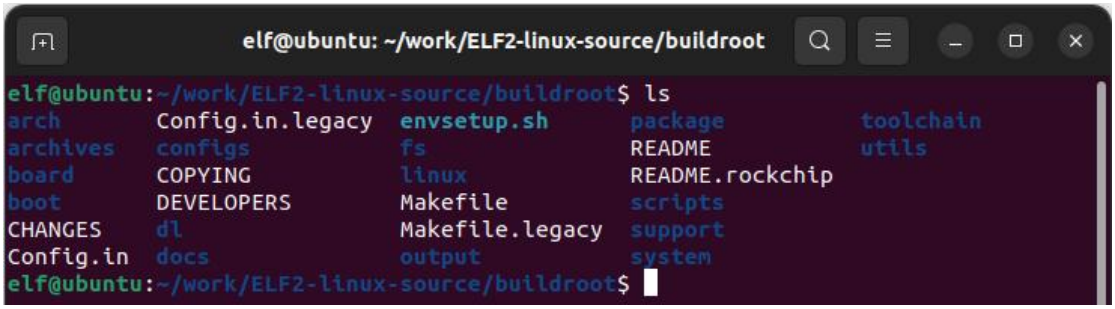
1.通过源码构建，有很大的灵活性。

2.方便的交叉编译环境，可以进行快速构建。

3.方便各系统组件配置及定制开发。

4.2 目录结构介绍

在 SDK 源码下的 buildroot 目录下输入 ls，可以查看到 buildroot 包含的目录。



目录	说明
arch	主要存放不同体系架构的相关文件，用于支持 Buildroot 针对多种处理器架构（如 ARM、x86、MIPS 等）进行定制和配置。
board	包含了不同开发板的配置文件（如 U-Boot 配置、内核配置等）。开发者可以在此特定开发板的启动和配置文件，并进行定制。
configs	包含预设的 Buildroot 构建配置文件，开发者可以通过（make <config_name>）的方式快速加载预定义配置。这些配置文件以不同硬件平台为基础。
docs	Buildroot 的文档，涵盖了安装指南、使用手册、如何添加新包、开发指导等详细内容。开发者可以在此查阅开发过程中可能遇到的常见问题。
fs	文件系统相关的支持内容。这里包括不同文件系统的生成工具（如 ext4、squashfs 等）和构建选项。
linux	包含与 Linux 内核相关的代码及其构建脚本。开发者可以在此进行内核配置和定制。
output	构建过程中生成的所有文件的存放位置。这个目录非常重要，因为它保存了构建过程中的临时文件、最终的镜像文件以及编译日志。
package	Buildroot 中包的定义目录。每个软件包都在这里定义，包含下载 URL、配置脚本、补丁、构建选项等。可以通过添加新目录来引入新的软件包。
support	包含支持脚本和工具，如用于编译过程的各种帮助工具。这些脚本

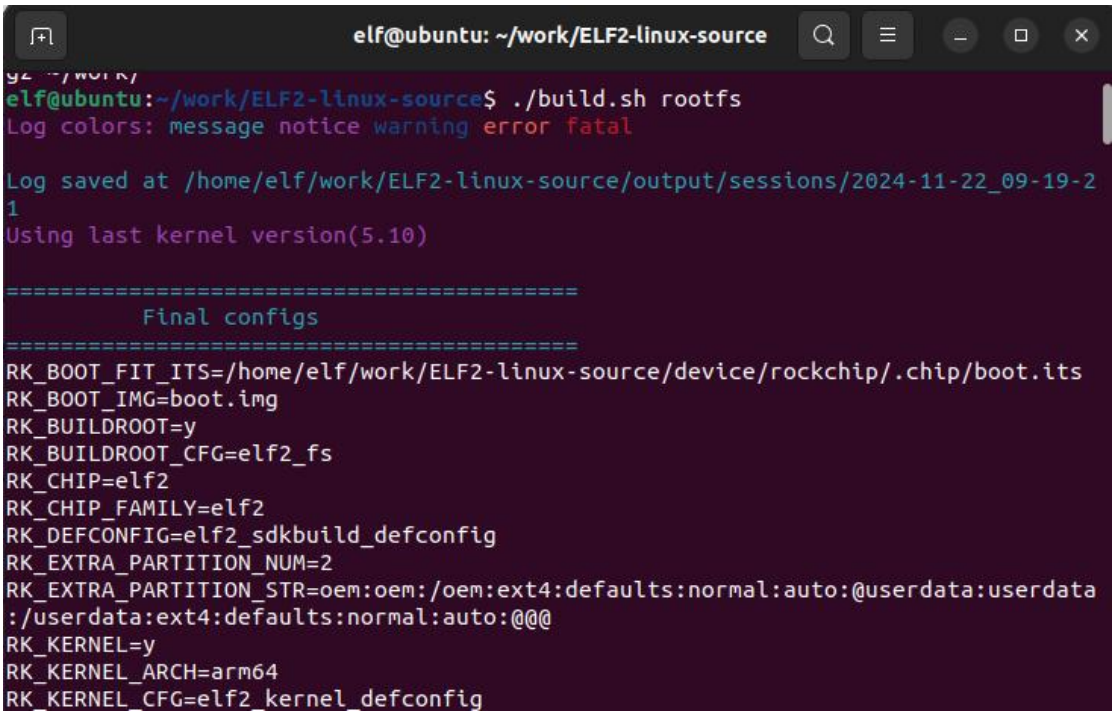
	简化了构建过程和环境设置。
system	用于构建系统镜像的文件和配置。包括初始系统配置、系统启动脚本等内容。
toolchain	处理交叉编译工具链的生成和配置。此目录包括工具链下载、构建和配置的相关文件。
utils	各种实用工具和脚本，帮助用户调试、构建和处理其他开发任务。
Makefile	Buildroot 的顶层 Makefile，构建系统的核心入口，负责协调整个构建过程。
Config.in	Buildroot 配置界面的顶层文件。它定义了系统的基本构建选项，并通过配置菜单（如 make menuconfig）供用户选择。 通过这些目录结构，Buildroot 能够将不同的开发任务模块化，并让用户根据自己的需求快速定制一个嵌入式系统。

4.3 Buildroot 编译

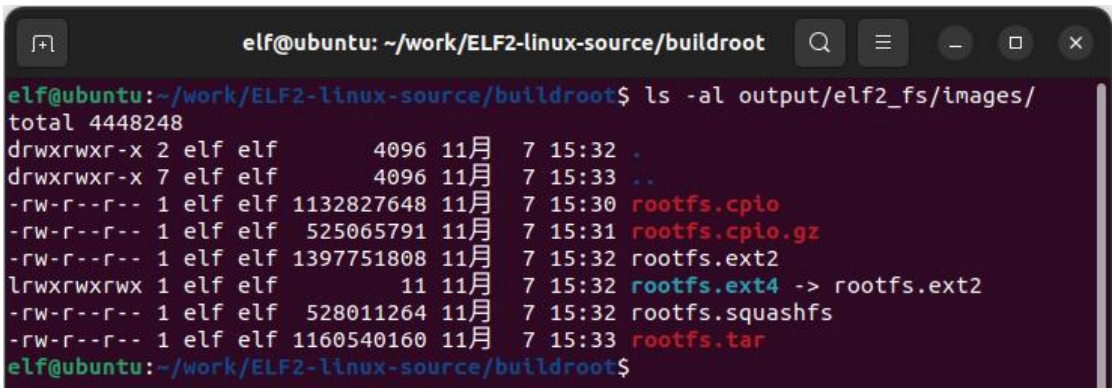
编译 Buildroot 对电脑配置要求较高，而且耗时较长，整体编译 SDK 时默认不会编译 Buildroot。如果想要尝试单独编译 Buildroot 需要把 prebuilts/elfboard/buildroot/rootfs.ext4 和 prebuilts/elfboard/recovery/rootfs.cpio.gz 移动到其他的目录下，因为编译脚本会判断此目录下是否存在 rootfs.ext4 文件，如果存在就不编译。

4.3.1 编译系统

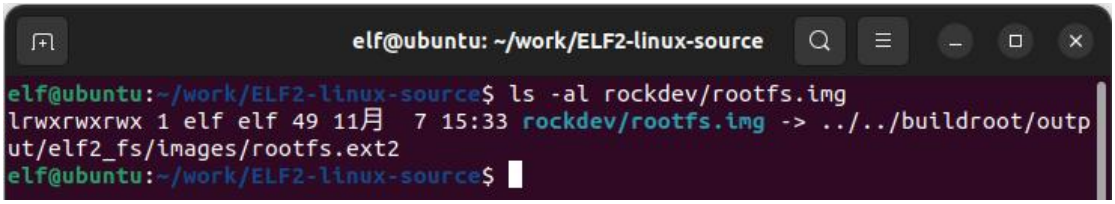
```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh rootfs
```



编译完成后会在 output/elf2_fs/images 目录下生成对应的文件系统镜像。

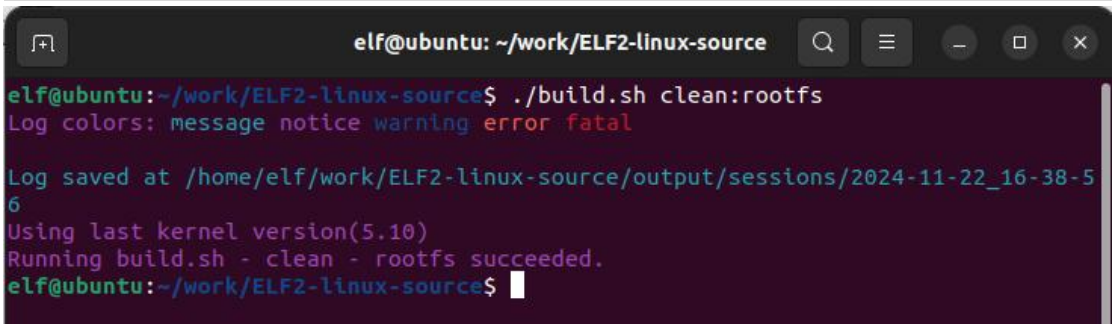


其中 rootfs.ext2 就是需要烧录到开发板中的镜像，rockdev/rootfs.img 链接的也是此镜像。



3.清除编译（谨慎处理，会清除编译的文件系统）

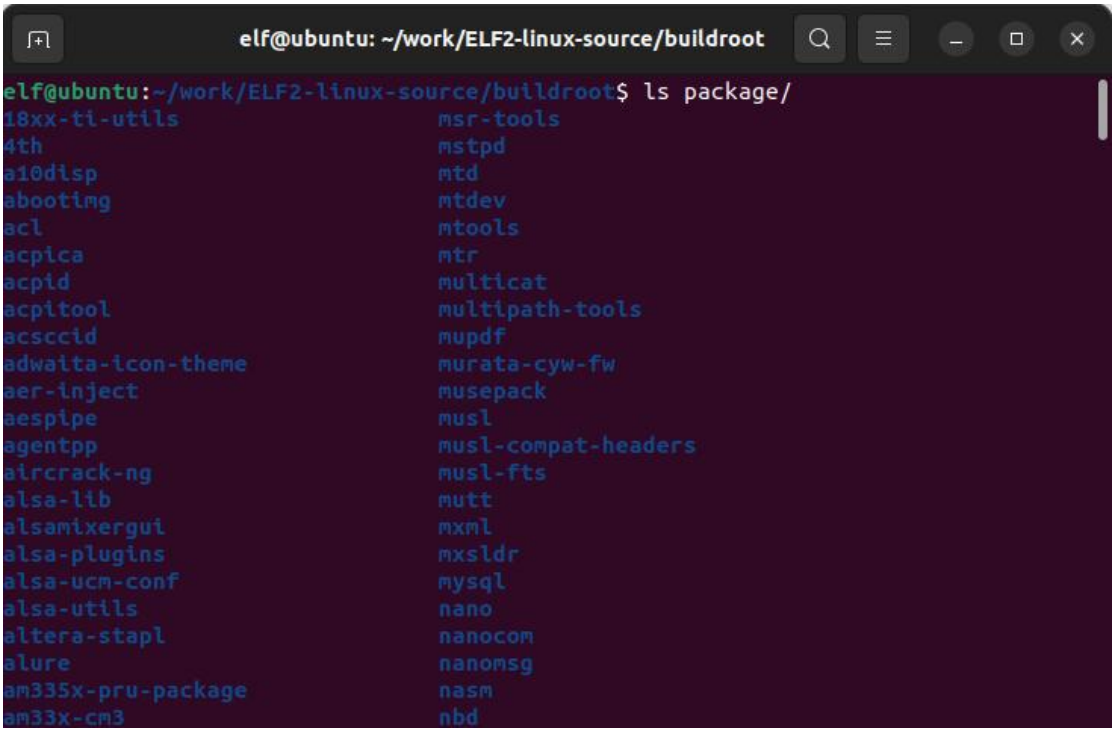
```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh clean:rootfs
```



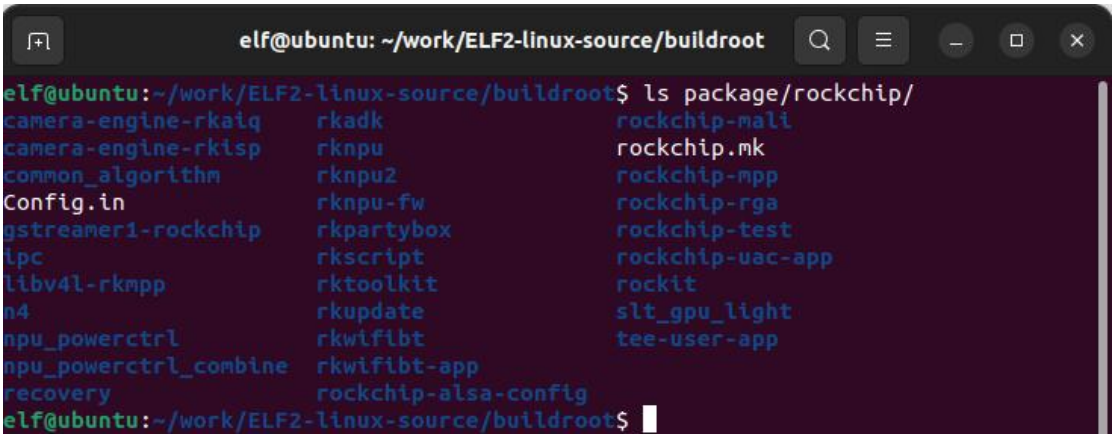
执行 make clean 后会清除构建时生成的文件。

4.3.2 单独编译 package

在 buildroot/package 目录下有很多可供编译的软件包，如果单独修改了某个包，不需要对整个 Buildroot 进行编译，可以选择单独对某一个包进行编译。



其中 package/rockchip/目录下的为瑞芯微提供的一些软件包：



比方说想要单独编译 rockit，进入到 buildroot 目录下执行如下指令：

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ source envsetup.sh
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ make rockit
```

```
elf@ubuntu: ~/work/ELF2-linux-source/buildroot
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ make rockit
RKPARTYBOX_CONF_OPTS is
>>> rockit  Syncing from source dir /home/elf/work/ELF2-linux-source/buildroot/.
./external/rockit
rsync -au --chmod=u=rwX,go=rX  --exclude .svn --exclude .git --exclude .hg --exc
lude .bzr --exclude CVS /home/elf/work/ELF2-linux-source/buildroot/../external/r
ockit/ /home/elf/work/ELF2-linux-source/buildroot/output/elf2_fs/build/rockit
>>> rockit  Configuring
(mkdir -p /home/elf/work/ELF2-linux-source/buildroot/output/elf2_fs/build/rockit
/ && cd /home/elf/work/ELF2-linux-source/buildroot/output/elf2_fs/build/rockit/
&& rm -f CMakeCache.txt && sed -e 's#@@STAGING_SUBDIR@@#aarch64-buildroot-linux-
gnu/sysroot#' -e 's#@@RELOCATED_HOST_DIR@@#/home/elf/work/ELF2-linux-source/buil
droot/output/elf2_fs/host/share/buildroot/#' -e 's#@@TARGET_CFLAGS@@#-D_LARGEFIL
E_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os -g0 -D_FORTIFY_SOURCE=
1#' -e 's#@@TARGET_CXXFLAGS@@#-D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_
OFFSET_BITS=64 -Os -g0 -D_FORTIFY_SOURCE=1#' -e 's#@@TARGET_FCFLAGS@@#-Os -g0#'
-e 's#@@TARGET_LDFLAGS@@##' -e 's#@@TARGET_CC@@#bin/aarch64-buildroot-linux-gnu-
gcc#' -e 's#@@TARGET_CXX@@#bin/aarch64-buildroot-linux-gnu-g++#' -e 's#@@TARGET_
AR@@#bin/aarch64-buildroot-linux-gnu-ar#' -e 's#@@TARGET_AS@@#bin/aarch64-buildr
oot-linux-gnu-as#' -e 's#@@TARGET_LD@@#bin/aarch64-buildroot-linux-gnu-ld#' -e '
s#@@TARGET_NM@@#bin/aarch64-buildroot-linux-gnu-nm#' -e 's#@@TARGET_RANLIB@@#bin
/aarch64-buildroot-linux-gnu-ranlib#' -e 's#@@TARGET_READELF@@#bin/aarch64-build
root-linux-gnu-readelf#' -e 's#@@TARGET_OBJCOPY@@#bin/aarch64-buildroot-linux-gn
u-objcopy#' -e 's#@@TARGET_OBJDUMP@@#bin/aarch64-buildroot-linux-gnu-objdump#' -
```

编译完成后会在 buildroot/output/elf2_fs/build/rockit 目录下生成一个 rockit.tar 压缩包。压缩包中会包含编译出的可执行文件和库文件。

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ ls output/elf2_fs/build/rockit
```

```
elf@ubuntu: ~/work/ELF2-linux-source/buildroot
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ ls output/elf2_fs/build/rockit
CMakeCache.txt       CMakeLists.txt        Makefile    rockit.tar
CMakeFiles           install_manifest.txt  mpi         tgi
cmake_install.cmake  lib                   README.md   toolchainfile.cmake
elf@ubuntu:~/work/ELF2-linux-source/buildroot$
```

使用 make rockchip-dirclean 指令可以清除编译。

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ make rockit-dirclean
```

```
elf@ubuntu: ~/work/ELF2-linux-source/buildroot
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ make rockit-dirclean
RKPARTYBOX_CONF_OPTS is
rm -Rf /home/elf/work/ELF2-linux-source/buildroot/output/elf2_fs/build/rockit
elf@ubuntu:~/work/ELF2-linux-source/buildroot$
```

接下来我们看一下 rockit 的编译规则。

在 ELF2-linux-source/buildroot/package/rockchip/rockit 路径下有两个文件，分别是 Config.in 文件和 rockit.mk 文件。

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ ls package/rockchip/rockit/
```

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ ls package/rockchip/rockit/
Config.in  rockit.mk
elf@ubuntu:~/work/ELF2-linux-source/buildroot$
```

首先来看 Config.in 文件，它的语法与之前讲 Kconfig 一致。表示.config 或者 defconfig 中有定义 BR2_PACKAGE_ROCKIT，就会去编译 rockit 的源码。

```
config BR2_PACKAGE_ROCKIT
        bool "rockit"
        select BR2_PACKAGE_ROCKCHIP_MPP
        select BR2_PACKAGE_ALSA_PLUGINS
        default n
        help
                rockit
~
~
```

rockit.mk 语法类似于 Makefile，规定了源码的路径及编译依赖。

```
################################################################################
#
# rockit project
#
################################################################################

ROCKIT_SITE = $(TOPDIR)/../external/rockit

ROCKIT_SITE_METHOD = local

ROCKIT_INSTALL_STAGING = YES

ROCKIT_DEPENDENCIES = rockchip-mpp rockchip-rga alsa-lib

ifneq ($(BR2_PACKAGE_RK3308),)
ROCKIT_CONF_OPTS += -DRK3308=TRUE
endif

ifneq ($(BR2_PACKAGE_RK3506),)
ROCKIT_CONF_OPTS += -DRK3506=TRUE
endif

$(eval $(cmake-package))
<droot/package/rockchip/rockit/rockit.mk" 23L, 513B          13,1          All
```

ROCKIT_SITE：指定了 rockit 的源码路径，为 external 下的 rockit。

```
elf@ubuntu:~/work/ELF2-linux-source/external/rockit$ ls
CMakeLists.txt  lib  mpi  README.md  tgi
elf@ubuntu:~/work/ELF2-linux-source/external/rockit$
```

ROCKIT_SITE_METHOD：表示获取软件包的方式为本地获取。

ROCKIT_INSTALL_STAGING：用于控制是否将软件包的文件安装到构建的 staging 区域。=YES，表示将软件包编译出的可执行文件安装到 output/staging 目录中。这通常意味着在构建其它软件包时可以访问这些文件，确保它们的依赖关系被正确处理。

ROCKIT_DEPENDENCIES：表示需要依赖的软件包或者库文件。

4.4 Buildroot 定制

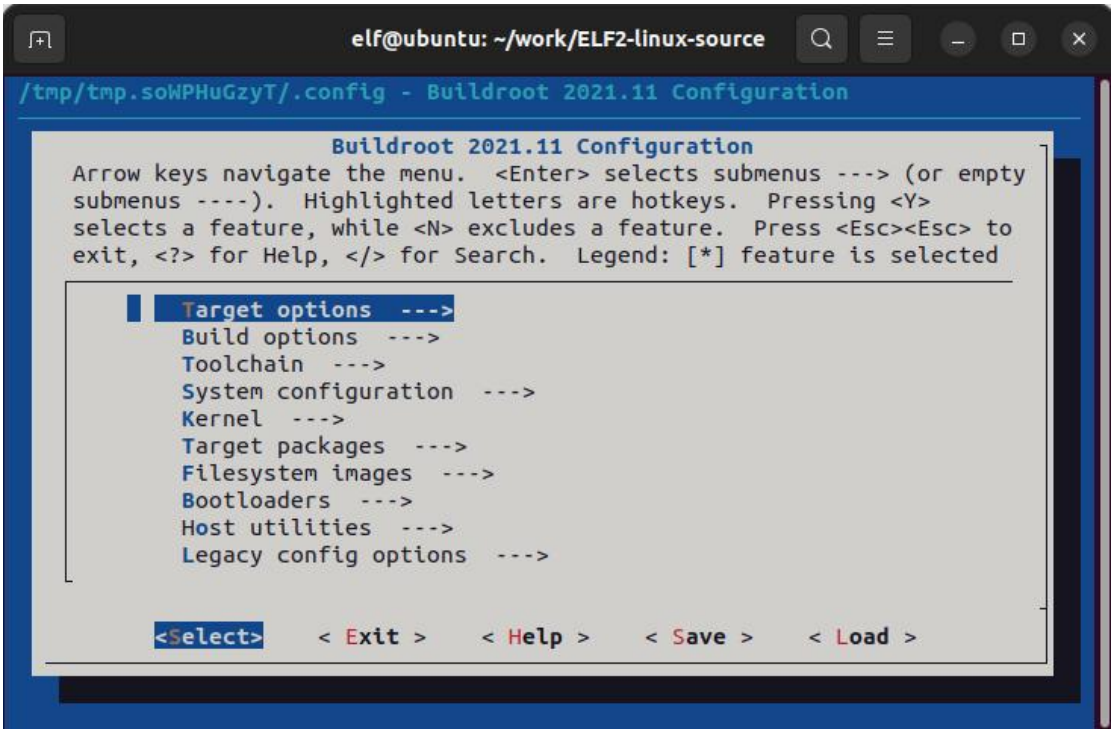
4.4.1 修改编译选项

如果想要修改编译选项，将软件包默认编译到 buildroot 中，需要做如下操作：

1.打开配置选项菜单

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh bconfig
```

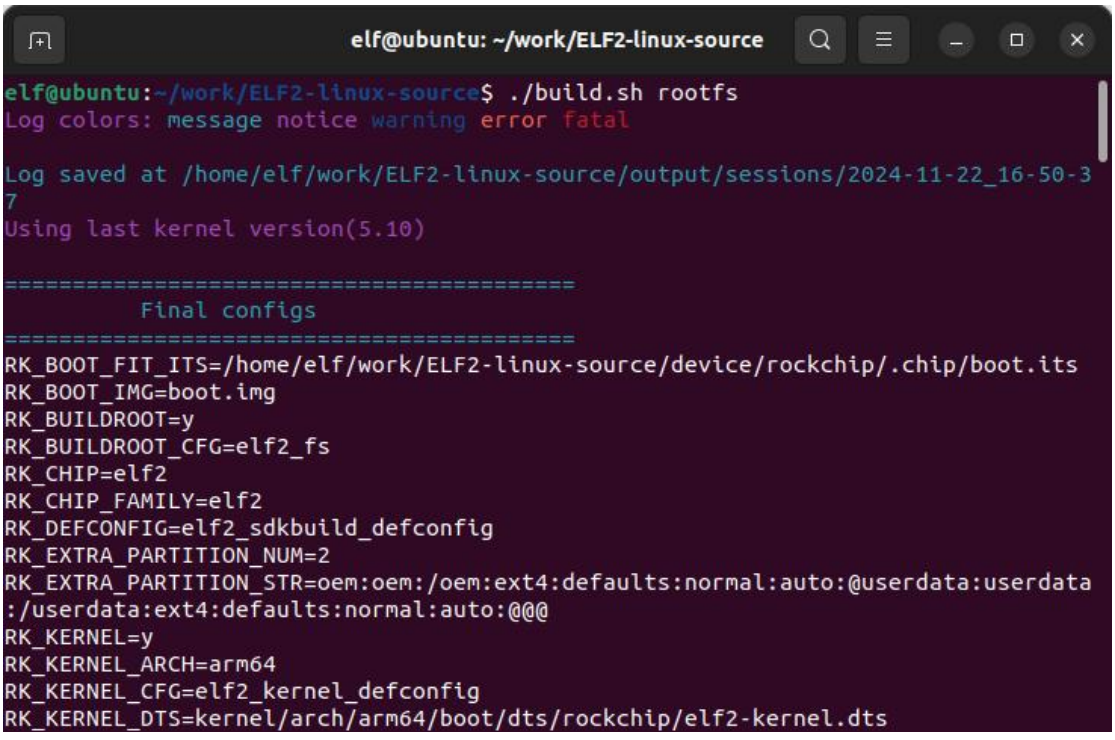
将自己需要的软件包选上，或者将自己不需要的取消掉，保存退出。



2.重新编译

```
elf@ubuntu:~/work/ELF2-linux-source$ ./build.sh rootfs
```

这样就可以将软件包直接编译到文件系统的镜像中了。



4.4.2 添加软件包

上一章节写了如何编译 buildroot 中已有的软件包，本节介绍一下如何在 buildroot 中构建属于自己的软件包。

1.在 ELF2-linux-source/app 目录下新建 elf2 目录。

```
elf@ubuntu:~/work/ELF2-linux-source$ cd app/
elf@ubuntu:~/work/ELF2-linux-source/app$ mkdir elf2
elf@ubuntu:~/work/ELF2-linux-source/app$
```

2.在 elf2 目录下新建 helloworld 程序。

```
elf@ubuntu:~/work/ELF2-linux-source/app$ cd elf2/
elf@ubuntu:~/work/ELF2-linux-source/app/elf2$ touch hello.c Makefile
elf@ubuntu:~/work/ELF2-linux-source/app/elf2$
```

在 hello.c 中打印“Hello World”

```
#include<stdio.h>
int main(void)
{
        printf("Hello World!\n");
        return 0;
}
```

Makefile 中写入如下内容：

```
hello:hello.o
        $(CC) -o hello hello.o
clean:
        rm -f hello
        rm *.o
```

3.添加 package

进入到 buildroot/package 目录下新建一个 hello 的目录：

```
elf@ubuntu:~/work/ELF2-linux-source$ cd buildroot/package/
elf@ubuntu:~/work/ELF2-linux-source/buildroot/package$ mkdir hello
elf@ubuntu:~/work/ELF2-linux-source/buildroot/package$
```

在 hello 目录下新建 Config.in 文件和 hello.mk 文件

```
elf@ubuntu: ~/work/ELF2-linux-source/buildroot/package/h...
elf@ubuntu:~/work/ELF2-linux-source/buildroot/package$ cd hello/
elf@ubuntu:~/work/ELF2-linux-source/buildroot/package/hello$ touch Config.in hel
lo.mk
elf@ubuntu:~/work/ELF2-linux-source/buildroot/package/hello$
```

Config.in 文件中写入如下内容：

```
config BR2_PACKAGE_HELLO
        bool "hello world"
        help
          ELF2 test_demo
```

hello.mk 文件中写入如下内容：

```
##############################################
#
#elf2_hello
#
##############################################
```

```
HELLO_VERSION = 1.0
HELLO_SITE = $(TOPDIR)/../app/elf2
HELLO_SITE_METHOD = local

define HELLO_BUILD_CMDS
        $(MAKE) -C $(@D) CC=$(TARGET_CC)
endef

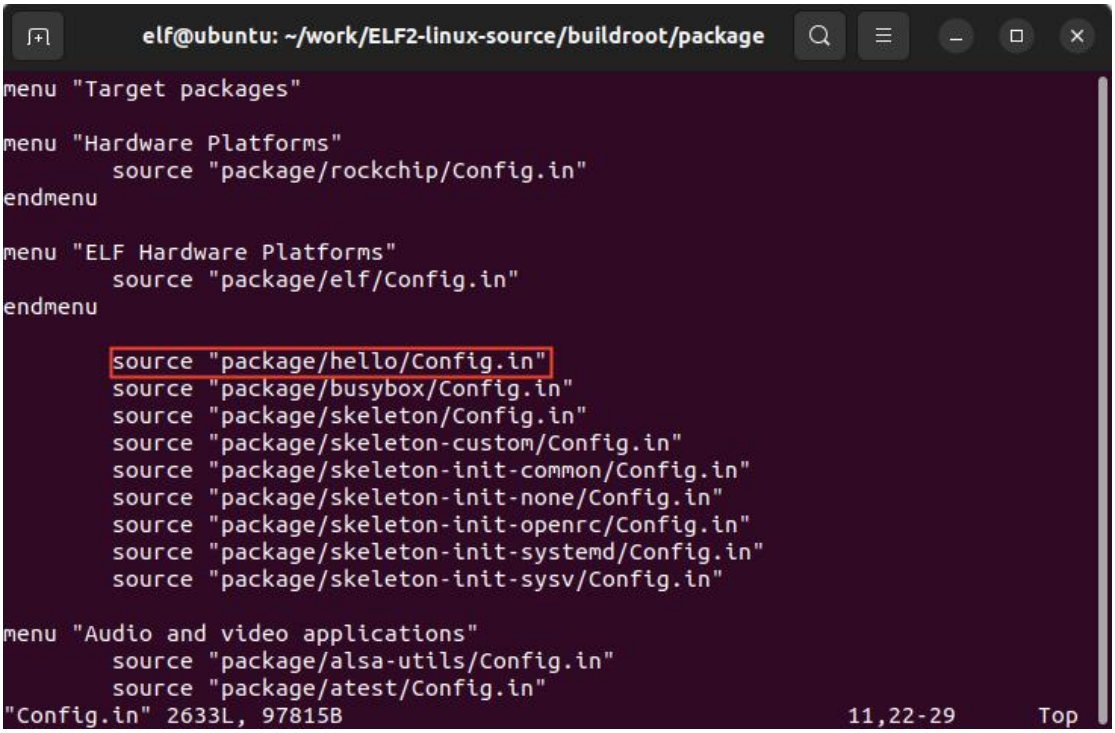
define HELLO_INSTALL_TARGET_CMDS
        $(INSTALL) -D $(@D)/hello $(TARGET_DIR)/usr/bin
endef

$(eval $(generic-package))
```

需要注意的是，上边的宏都是以 HELLO 开头的，这个是与目录名对应的，咱们在 package 中创建的是 hello 目录，所以使用的宏都是以 HELLO 开头的。

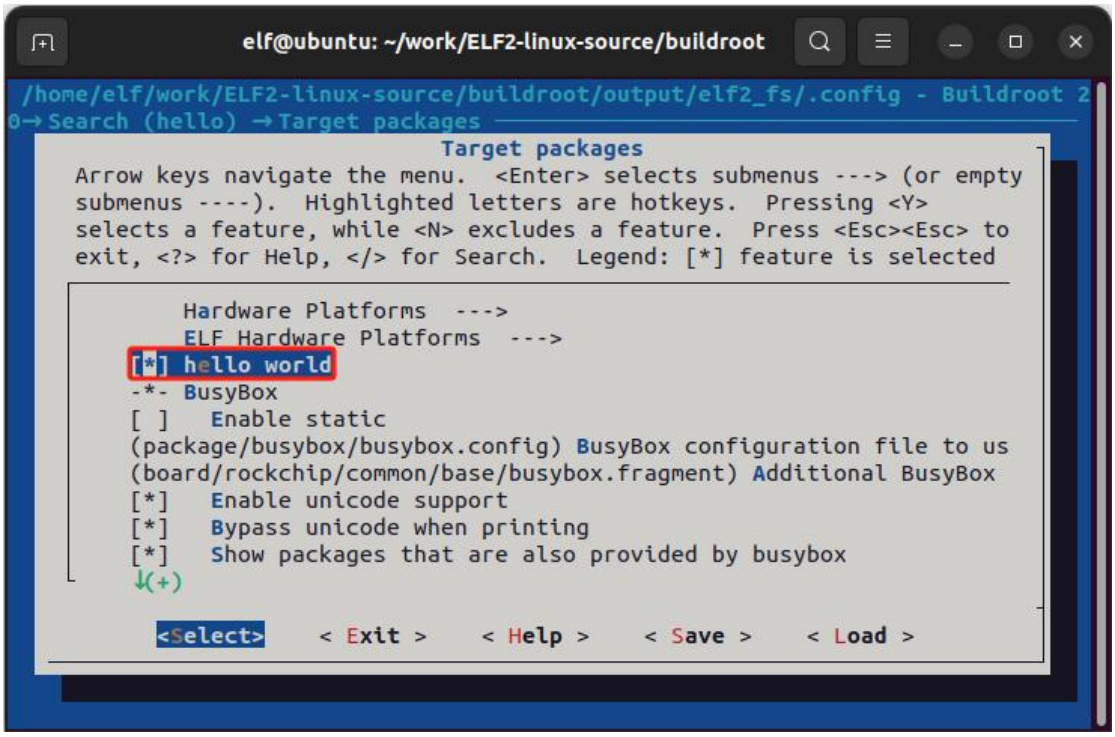
4.修改上层 Config.in

打开 package 目录下的 Config.in，添加 hello 目录下 Config.in 的引用。



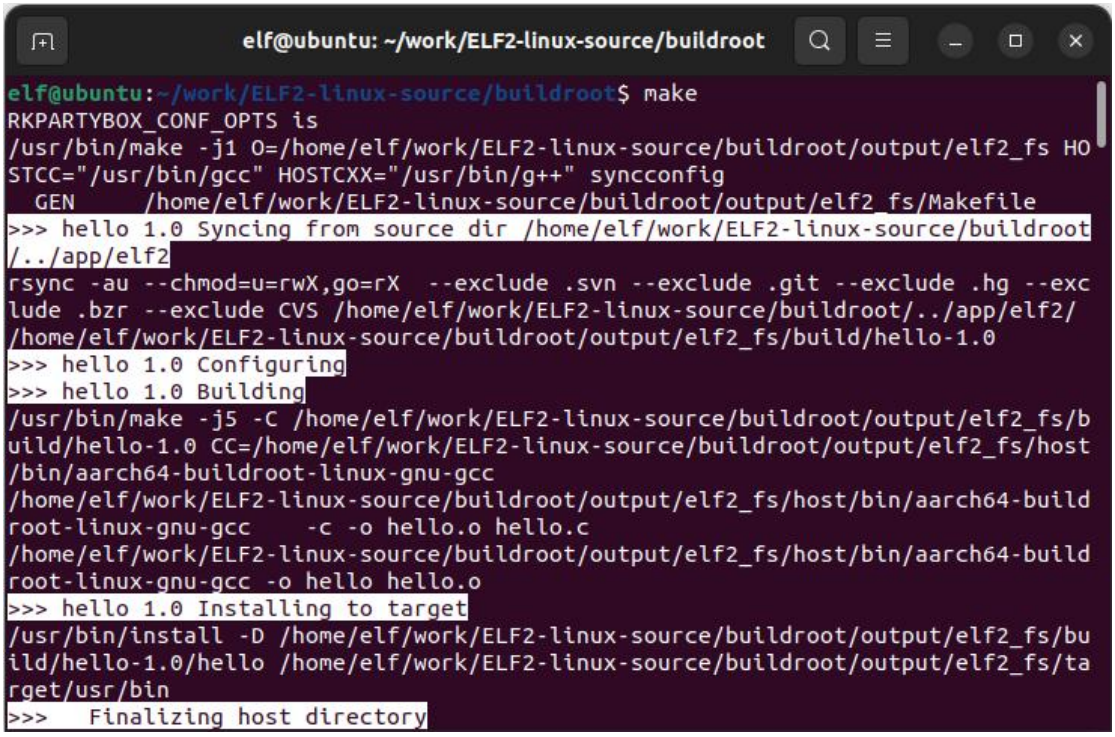
5.使能编译

在 SDK 目录下执行./build.sh bconfig，将新添加的“hello world”选上：



6.编译软件包

```
elf@ubuntu:~/work/ELF2-linux-source/buildroot$ make hello
```



从编译输出日志中可以看出生成的可执行文件安装到了 buildroot/output/elf2_fs/target/usr/bin 目录下。

如果想要重新生成文件系统镜像包，可以直接在 buildroot 目录下执行 make 指令将编译完成的可执行文件打包到文件系统镜像中。

7.验证

将打包好的文件系统烧录到开发板中，执行 hello 进行验证，看到有输出”Hello World!”说明添加成功。



4.5 文件系统常用指令

4.5.1 系统管理与设置常用命令

1.查看系统内核版本

通过查看/proc/version 文件，可获得系统内核版本信息：

```
root@elf2-buildroot:~# cat /proc/version
Linux version 5.10.209 (elf@ubuntu) (aarch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #1 SMP Thu Oct 17 15:17:34 CST 2024
```

2.查看内存使用情况

可使用 free 查看内存使用情况：

```
root@elf2-buildroot:~# free
               total        used        free      shared  buff/cache   available
Mem:         3931136       69548     3767200         852       94388     3820608
Swap:              0           0           0
```

输出结果意义：

第一行 Mem 行解释：

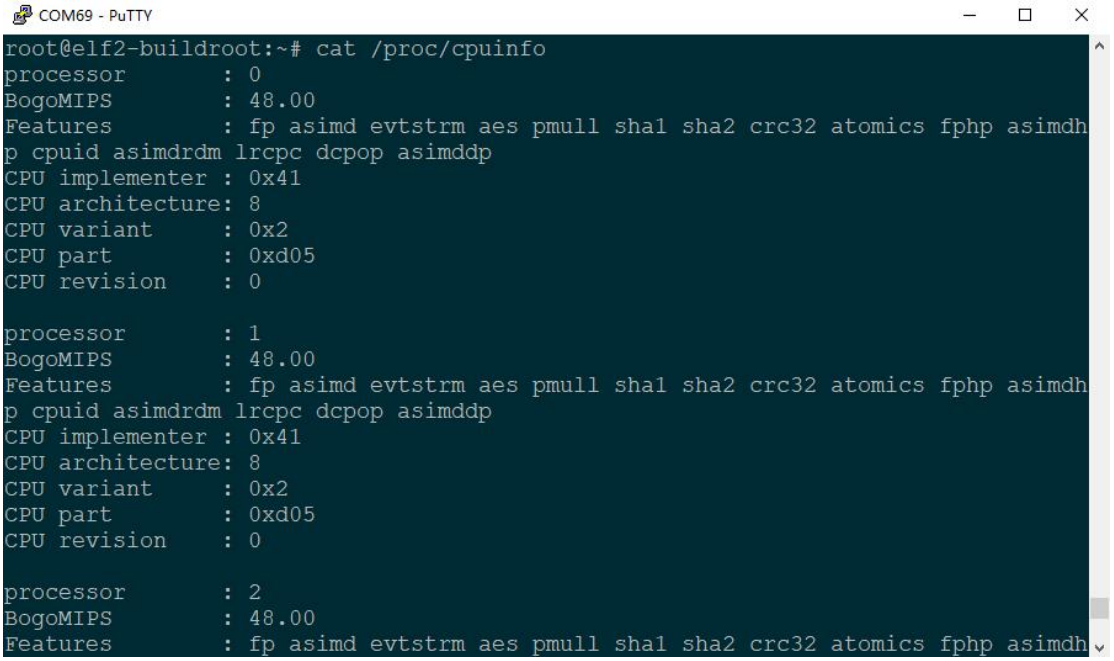
参数	说明
total	内存总数
used	已经使用的内存数
free	空闲的内存数
shared	当前已经废弃不用
buffers Buffer	缓存内存数
cached Page	缓存内存数

关系：total = used + free

第二行是指交换分区。

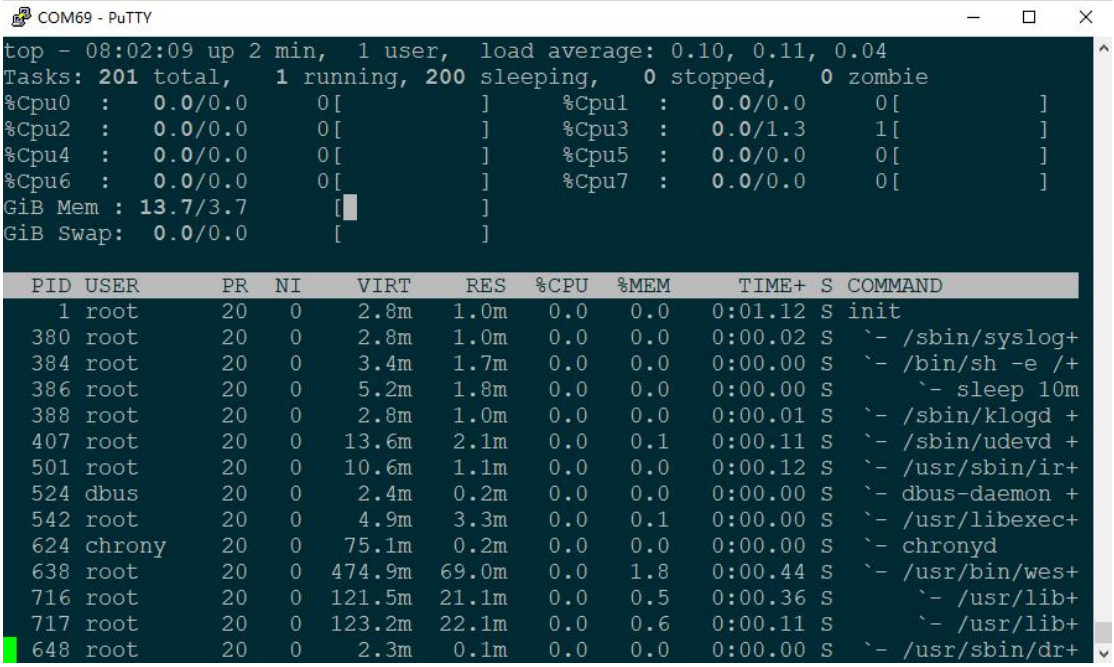
3.查看 CPU 等信息

```
root@elf2-buildroot:~# cat /proc/cpuinfo
```



4.top 用于实时显示 process 的动态

```
root@elf2-buildroot:~# top
```



5.reboot 重新启动系统

当需要关机或重启时，如果有数据存储操作，为了确保数据完全写入，可输入 sync 命令，完成数据同步后关闭电源重启。

也可输入 reboot 命令重启：

```
root@elf2-buildroot:~# reboot
```

4.5.2 磁盘管理与维护常用命令

1.df：查看系统已用空间

df 命令，用于显示 linux 系统中各文件系统的硬盘使用情况。包括文件系统所在硬盘分区的总容量、已使用的容量、剩余容量等。

用法：df [OPTION]... [FILE]...

参数	说明
-k	以 KBytes 显示容量
-m	以 Mbytes 显示容量
-h	以易读的方式自行显示
-T	列出文件系统类型

示例一：显示系统所有 filesystem。

```
root@elf2-buildroot:~# df
```

COM69 - PuTTY

```
root@elf2-buildroot:~# df
Filesystem     1K-blocks    Used Available Use% Mounted on
/dev/root       14426656 1137960  12674908   9% /
devtmpfs         1830380       8   1830372   1% /dev
tmpfs            1964940     248   1964692   1% /tmp
tmpfs            1964940     384   1964556   1% /run
tmpfs            1964940     148   1964792   1% /var/log
tmpfs            1964940   80480   1884460   5% /dev/shm
/dev/mmcblk0p7    125804   11952    112572  10% /oem
/dev/mmcblk0p8  14423440  290720  13515040   3% /userdata
root@elf2-buildroot:~#
```

示例二：将文件系统容量以易读的方式显示。

```
root@elf2-buildroot:~# df –h
```

COM69 - PuTTY

```
root@elf2-buildroot:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        14G  1.1G   13G   9% /
devtmpfs        1.8G  8.0K  1.8G   1% /dev
tmpfs           1.9G  248K  1.9G   1% /tmp
tmpfs           1.9G  384K  1.9G   1% /run
tmpfs           1.9G  148K  1.9G   1% /var/log
tmpfs           1.9G   79M  1.8G   5% /dev/shm
/dev/mmcblk0p7  123M   12M  110M  10% /oem
/dev/mmcblk0p8   14G  284M   13G   3% /userdata
root@elf2-buildroot:~#
```

示例三：将文件系统类型和容量大小以易读的方式显示。

```
root@elf2-buildroot:~# df –hT
```

COM69 - PuTTY

```
root@elf2-buildroot:~# df -hT
Filesystem     Type       Size  Used Avail Use% Mounted on
/dev/root      ext4        14G  1.1G   13G   9% /
devtmpfs       devtmpfs   1.8G  8.0K  1.8G   1% /dev
tmpfs          tmpfs      1.9G  248K  1.9G   1% /tmp
tmpfs          tmpfs      1.9G  384K  1.9G   1% /run
tmpfs          tmpfs      1.9G  148K  1.9G   1% /var/log
tmpfs          tmpfs      1.9G   79M  1.8G   5% /dev/shm
/dev/mmcblk0p7 ext4       123M   12M  110M  10% /oem
/dev/mmcblk0p8 ext4        14G  284M   13G   3% /userdata
root@elf2-buildroot:~#
```

输出结果每项意义：

参数	说明
Filesystem	当前系统在哪个设备的哪个分区。其中 tmpfs 为虚拟文件系统
Type	文件系统类型
Size	当前系统的大小。1K-blocks 对应列的单位是 1KB，可通过-h 或者-m 来改变单位
Used	已经使用的磁盘空间

Avail	剩余可用的磁盘空间大小
Use%	磁盘的使用率。使用率过高时，需要注意，因空间不足造成系统问题。/dev/root 为文件系统根目录。当 Use% 高达 90%以上时，需要注意
Mounted on	磁盘挂载目录

2.du：用于展示磁盘使用量的统计信息

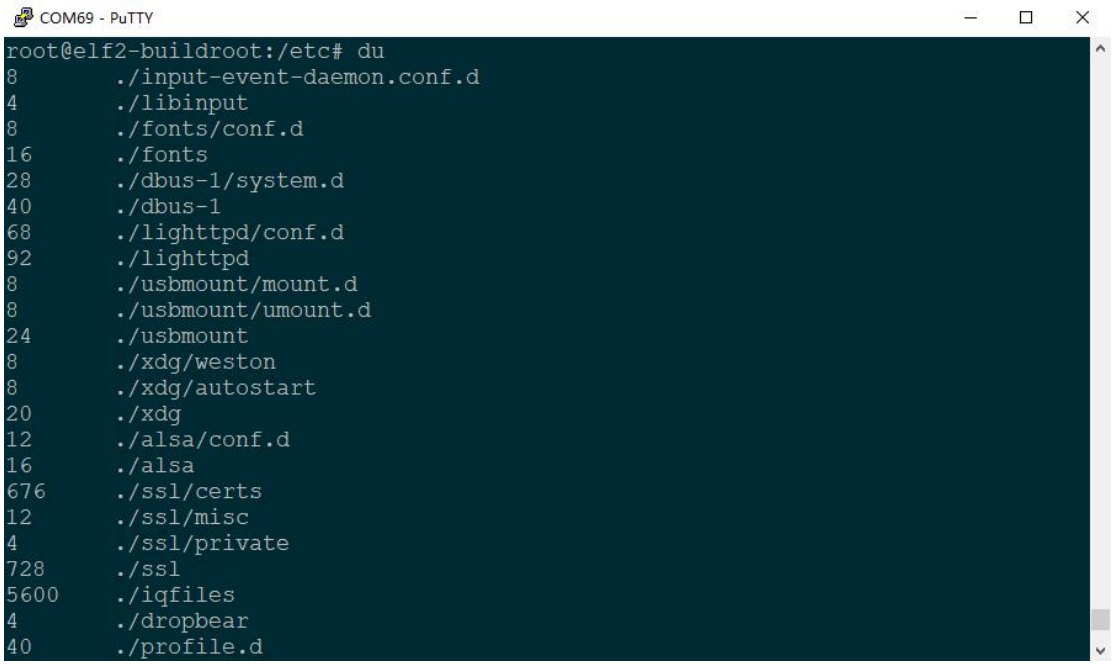
用法：du [OPTION]... [FILE]...

选项与参数：

参数	说明
-a	同时显示每个文件的文件大小
-d N	将输出限制为深度<N 的目录
-c	显示总计
-s	每个参数仅显示总计
-x	跳过不同文件系统上的目录
-h	可读格式的大小（例如 1K 243M 2G）
-m	大小（以兆字节为单位）
-k	大小（以千字节为单位）（默认）

示例一：统计/etc 目录下所有文件所占用的容量。先执行“cd /etc”进入/etc 目录，再执行 du 命令：直接输入 du 在没有加任何参数选项时，du 会显示出当前所在目录的文件与目录所占用的硬盘空间:

```
root@elf2-buildroot:/etc# du
```



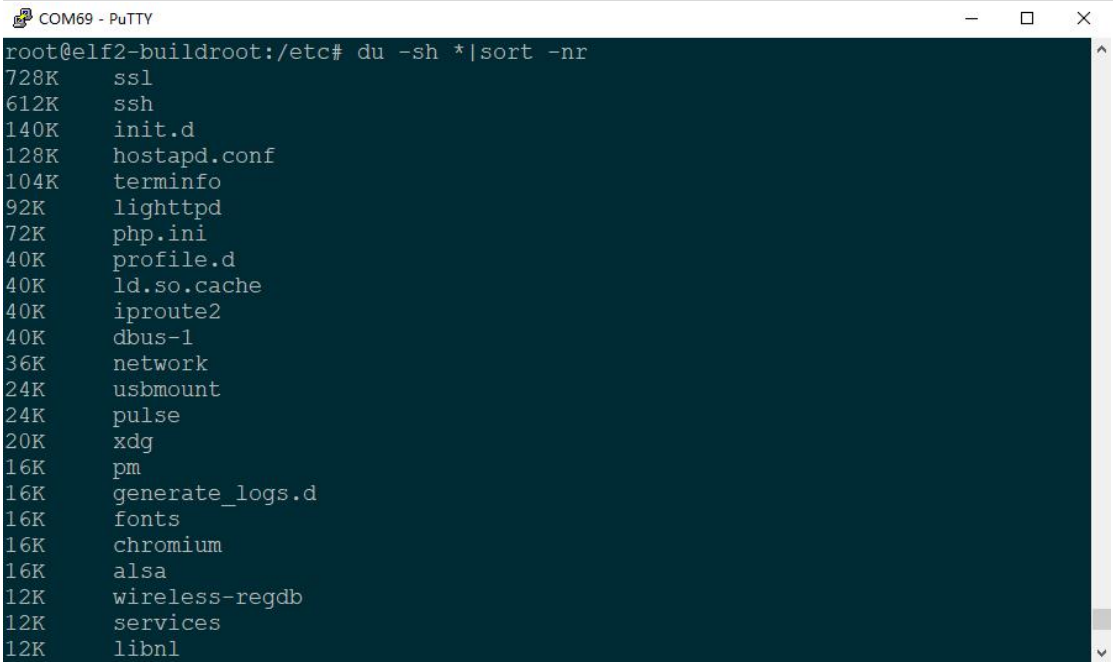
示例二：统计/etc 目录下，每个文件和目录所占用容量的大小，并以易读方式显示：

```
root@elf2-buildroot:/etc# du –ah
```



示例三：按照空间大小排序：

```
root@elf2-buildroot:/etc# du -sh *|sort -nr
```



du 侧重在文件夹和文件的磁盘占用方面，而 df 则侧重在文件系统级别的磁盘占用方面。

声明

本手册版权归保定飞凌嵌入式技术有限公司所有。未经本公司的书面许可，任何单位和个人无权以任何形式复制、传播、转载本手册的任何部分，违者将被追究法律责任。

保定飞凌嵌入式有限公司所提供的所有服务内容旨在协助用户加速产品的研发进度，在服务过程中所提供的任何程序、文档、测试结果、方案、支持等资料和信息，都仅供参考，用户有权不使用或自行参考修改，本公司不提供任何的完整性、可靠性等保证，若在用户使用过程中因任何原因造成的特别的、偶然的或间接的损失，本公司不承担任何责任。

更多帮助

注意事项与维护



- 请勿带电插拔核心板及外围模块！
- 请遵循所有标注在产品上的警示和指引信息。
- 请保持本产品干燥。如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干。
- 使用中注意本产品的通风散热，避免温度过高造成元器件损坏。
- 请勿在多尘、脏乱的环境中使用或存放本产品。
- 请勿将本产品应用在冷热交替环境中，避免结露损坏元器件。
- 请勿粗暴对待本产品，跌落、敲打或剧烈晃动都可能损坏线路及元器件。
- 请勿使用有机溶剂或腐蚀性液体清洗本产品。
- 请勿自行修理、拆卸本公司产品，如产品出现故障请及时联系本公司进行维修。
- 擅自修改或使用未经授权的配件可能损坏本产品，由此造成的损坏将不予以保修。

更新通知与学习资源

ElfBoard 产品相关资料会持续完善和更新，本手册内容也会及时跟进。请在使用这些资料时，确保它们为最新版本；

最新的资料更新通知可通过微信公众号获取，课程视频和嵌入式知识科普则可以在抖音、哔哩哔哩等平台观看，以帮助大家更好地了解和使用 ElfBoard 产品。

官方公众号　官方抖音　哔哩哔哩

资料获取

1.请登录 ElfBoard 官网“www.elfboard.com”，进入“资料”专区进行下载；

2.下载前请阅读《用户资料极速下载必读》。

售后服务政策

1.如产品使用过程中出现硬件故障可根据售后服务政策进行维修；

2.售后服务政策：参见官网“www.elfboard.com”首页->技术服务->售后服务政策。

送修地址

1.地　址：河北省保定市高开区向阳北大街 2699 号飞凌嵌入式技术有限公司新楼五层售后维修部

2.联系人：售后维修部

3.电　话：0312-3102650-952、953

4.邮编：071000

5.邮寄须知：建议使用顺丰、圆通或韵达，且不接收任何到付。

技术支持范围

1.本公司产品的软、硬件资源提供情况咨询；

2.本公司产品的软、硬件手册使用过程中遇到的问题。

技术讨论范围

1.源码的修改以及理解；

2.操作系统如何移植；

3.用户自行修改以及开发中遇到的软硬件问题；

注：以上三点虽不属于技术支持范围，但我公司会尽力为用户提供帮助，如仍然没能解决敬请谅解。

技术支持方式

1. 论坛：bbs.elfboard.com
2. QQ 群：474552704

技术支持时间

1.周一至周五：上午 9:00—11:30；
　　下午 13:30—17:00；

2.公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至论坛 ELF 2 专区或 QQ 群，我们会在工作日尽快给您回复。